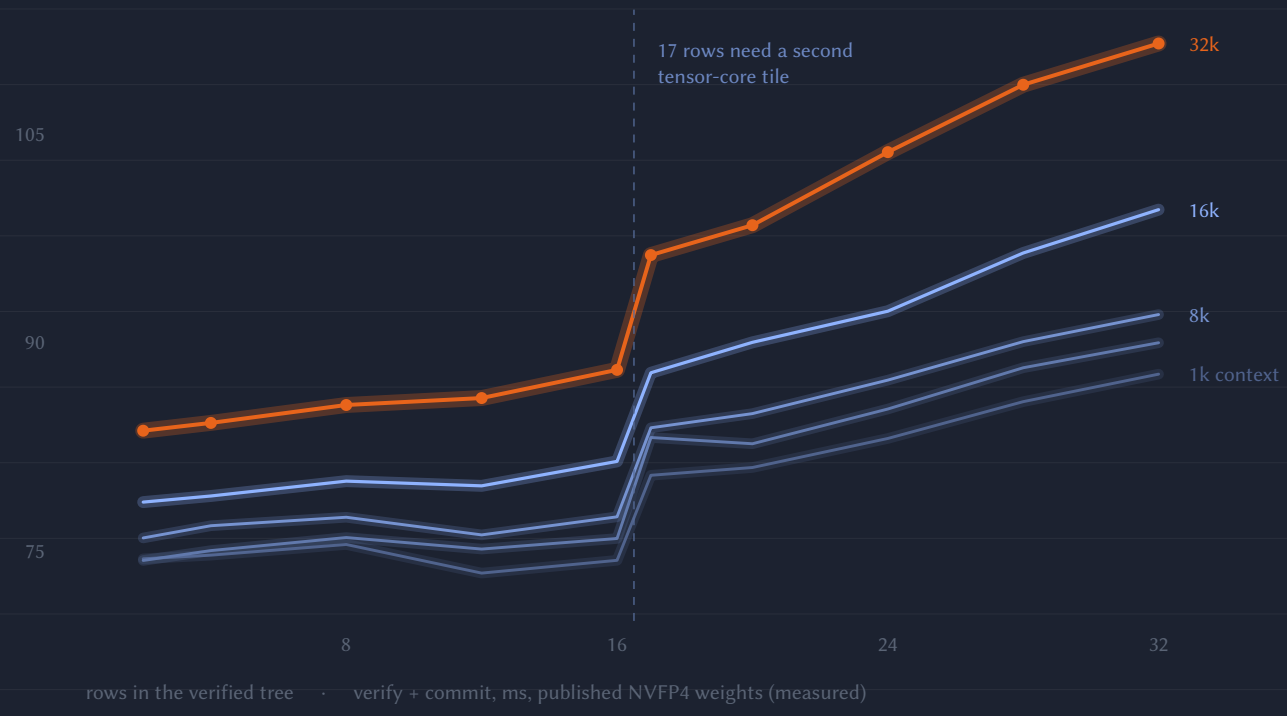
TandemLLM

The manual

An environment for the agents you run on your own machine:
several models working on one request, caches that survive a restart,
memory notes, parallel requests, and one dashboard for this engine,
other engines and cloud APIs



Version 0.8.3 October 2026

The text of this manual is CC BY 4.0; the code it describes is AGPL-3.0-only or a commercial license.

github.com/0xBakeer/TandemLLM

About this manual

This book describes TandemLLM version 0.8.3: an environment that runs the models your agents need, on your own machine and in front of cloud APIs. It covers the engine with the models it runs together and the conductor that plans each turn. It covers the caches and the memory notes, and the dashboard and proxy that watch every backend. It sits beside the Markdown pages in `docs/` and goes further: figures and screenshots show how the parts fit together, and a script writes its reference part from the code.

Part I says what TandemLLM is, names the machine and the model every measurement comes from, and follows one request through it. Part II installs and runs it, alone or together with other engines and cloud APIs. Part III explains how the engine reaches 49.89 tokens a second for one request on its test board, and how the benchmark row measures it. Part IV covers the conductor, the layer that plans each turn and watches it run. Part V walks through the dashboard's views. Part VI is the reference: about 800 rows that a script writes from the code or checks against it, then troubleshooting and the roadmap. A glossary and an index close the book.

How numbers are labelled

Every number in this book is one of three kinds, and the text says which.

- measured: measured on the test machine, a DGX Spark with Qwen3.8-27B (Section 1.1), with the published NVFP4 weights unless the text names another set. The caption or the sentence names the version or the date, and the tool that measured it when one did.
- arithmetic: computed from sizes, such as bytes read divided by memory bandwidth. Nothing ran.
- estimate: a guess with its basis stated. The book uses few of them and marks each one.

The speed numbers of the StairCut paper are the reference for single-request speed. They were measured on the published weights with the persistent suffix store off, on the benchmark row described in Chapter 15. Later releases are held to those numbers by a same-hour comparison before they ship.

Agreement with greedy decoding is a measured property with one documented exception, a near tie between the two best tokens. Chapter 12 states it precisely. Where this book says that resuming from a cache is "bit for bit", a test compared the bytes.

Conventions

Commands, flags, variables, files and HTTP routes are set in `monospace`. Paths under `~/.qwen38-spark-engine/` and `~/.tandemllm/` are the engine's and the proxy's state directories on the machine that runs them. Runtime names still carry the project's old prefix (`QWEN38_`, `QSE_`); a later release renames them together.

The engine runs on Linux. The dashboard and the proxy run on Linux and macOS.

Building this book

The source is in `manual/` of the repository. `make` in that directory regenerates the reference tables from the code (`tools/gen_reference.py`) and runs `latexmk -lualatex`. The repository keeps one built copy, `manual/tandemllm-manual.pdf`, so a reader needs no LaTeX; build it again before a release changes the book. The `manual.yml` workflow also builds it when a release tag is pushed and attaches it to that GitHub release, as `tandemllm-manual-<version>.pdf`.

Citing

Cite the paper for the decoding method and its numbers (Section 1.7), and the repository for the software; `CITATION.cff` has both.

License

The text of this manual is licensed under CC BY 4.0, like the documentation in `docs/`. The code it describes is licensed separately: AGPL-3.0-only, or a commercial license from the copyright holder. The model weights are not part of the repository; the base model and the base drafter are Apache-2.0, and so are the published NVFP4 overlays and fine-tuned drafters derived from them.

Contents

About this manual ii

List of Figures xiii

List of Tables xvi

I Introduction 1

1 What TandemLLM is 2
- 1.1 The test environment . . . 3
- 1.2 Two levers . . . 3
- 1.3 What is in the repository . . . 4
- 1.4 What runs today . . . 4
- 1.5 Two rules . . . 5
- 1.6 The headline numbers . . . 5
- 1.7 The paper . . . 6
- 1.8 Credits . . . 7

2 One request, end to end 8
- 2.1 The client sends a request . . . 8
- 2.2 The proxy routes it . . . 8
- 2.3 The conductor plans the turn . . . 9
- 2.4 Admission decides when it runs . . . 9
- 2.5 The caches decide where the prefill starts . . . 9
- 2.6 The decode loop . . . 10
- 2.7 The monitor watches the output . . . 10
- 2.8 The stream goes back . . . 10
- 2.9 After the request . . . 10

II Installing and running 11

3 Installing TandemLLM 12
- 3.1 Setups, and which ones work . . . 12
- 3.2 What the engine needs . . . 13
- 3.3 A new box: setup mode, then one profile . . . 14
- 3.4 Updating a box in setup mode . . . 15

3.5 The installer . . . 15
3.6 After the install . . . 16
3.7 The first request . . . 17
3.8 Access and sign-in . . . 18
3.9 The small models . . . 20
3.10 Other engines and cloud APIs beside the engine . . . 20
3.11 Running from a checkout . . . 20
3.12 Building the weights yourself . . . 21
3.13 Profiles . . . 22
3.14 Upgrading and uninstalling . . . 22

4 The dashboard and proxy on their own 24
4.1 Installing from a checkout . . . 24
4.2 The tandem-dash command . . . 24
4.3 A first backend . . . 24
4.4 In docker . . . 25
4.5 Where the proxy keeps its files . . . 26
4.6 The proxy inside the engine's server . . . 26
4.7 Access . . . 26

5 Other engines behind the proxy 27
5.1 Backend kinds . . . 27
5.2 Two ways to add a backend . . . 28
5.3 vLLM . . . 29
5.4 SGLang . . . 29
5.5 One command: tandem-dash launch . . . 30
5.6 Any OpenAI-compatible server . . . 30
5.7 OpenAI and the Responses API . . . 30
5.8 DeepSeek . . . 31
5.9 Another TandemLLM . . . 31
5.10 Routing by model id . . . 31
5.11 Keys at rest . . . 32
5.12 Users and their keys . . . 32
5.13 Prices and cost . . . 33
5.14 What the proxy measures . . . 33

6 Operations 34
6.1 One engine per board . . . 34
6.2 Memory rules . . . 34
6.3 Two models, one at a time . . . 35
6.4 Starting and stopping . . . 36
6.5 The watchdog . . . 37
6.6 Holding the board . . . 37
6.7 While the engine is away . . . 38
6.8 Roles beside the engine . . . 38
6.9 Logs . . . 38

6.10 Monitoring . . . 39
6.11 Settings . . . 39
6.12 Deploying and rolling back . . . 39

7 Community weights 41
7.1 The packs in the setup guide . . . 41
7.2 Where the list comes from . . . 41
7.3 Running a pack . . . 42
7.4 Submitting a pack . . . 42
7.5 How verification works . . . 43

III How the engine works 44

8 Architecture 45
8.1 The model . . . 45
8.2 The byte budget . . . 46
8.3 The decode loop . . . 47
8.4 The parts . . . 47
8.5 One sequence, or several . . . 47

9 Quantisation 49
9.1 What the checkpoint stores . . . 49
9.2 NVFP4 . . . 49
9.3 The quantiser . . . 50
9.4 The gate . . . 51
9.5 What stays at full precision . . . 52
9.6 Profiles and bytes . . . 52
9.7 The 27B in EXL3 . . . 52
9.8 Standalone EXL3 packs . . . 53
9.9 Flash-Next's EXL3 packs . . . 53
9.10 Knobs . . . 55

10 Kernels 56
10.1 Row invariance . . . 56
10.2 NVFP4 matrix products . . . 56
10.3 FP8 matrix products . . . 57
10.4 The recurrent layers . . . 57
10.5 Head, norms and attention . . . 58
10.6 CUDA graphs . . . 58
10.7 How a kernel gets in . . . 58
10.8 Knobs . . . 59

11 Drafters, StairCut and tree verify 60
11.1 The round . . . 60
11.2 The block drafter . . . 60
11.3 The lookup drafter . . . 61

11.4 StairCut . . . 62
11.5 Longer contexts . . . 63
11.6 The length router . . . 63
11.7 Building the tree . . . 64
11.8 Tree verify through recurrent layers . . . 64
11.9 Sampling . . . 66
11.10 Reasoning . . . 66
11.11 The same method on Flash-Next . . . 66
11.12 Knobs . . . 67

12 Exactness 68
12.1 The promise . . . 68
12.2 The exception: near ties . . . 69
12.3 Two runs of the same request . . . 69
12.4 Release identity . . . 70
12.5 Sampling . . . 70
12.6 Caches . . . 71
12.7 What changes output on purpose . . . 71
12.8 Flash-Next: no near-tie exception . . . 72
12.9 The checks . . . 72

13 Caches 73
13.1 What a state is . . . 73
13.2 The tiers . . . 74
13.3 One conversation through the tiers . . . 75
13.4 The resident prefix . . . 75
13.5 After a long generation: the end anchor . . . 77
13.6 The state store . . . 78
13.7 The disk tier . . . 78
13.8 Live conversations on the NVMe . . . 79
13.9 Compaction as a continuation . . . 80
13.10 The persistent suffix store . . . 80
13.11 Images in a prompt . . . 80
13.12 Memory limits . . . 81
13.13 Routes and knobs . . . 81

14 Serving 82
14.1 What the API supports . . . 82
14.2 Reasoning . . . 83
14.3 Thinking budgets from the client . . . 83
14.4 Tool calls . . . 84
14.5 Structured outputs . . . 84
14.6 Streaming . . . 85
14.7 A long prefill, and a client that leaves . . . 86
14.8 Images . . . 86
14.9 Admission and limits . . . 87

14.10 The memory gate . . . 87
14.11 The side lane . . . 88
14.12 A turn behind a running turn . . . 89
14.13 Parallel requests . . . 90
14.14 A second model behind the same server . . . 92
14.15 The tool-argument loop guard . . . 92
14.16 Logs and privacy . . . 93

15 Measurement 94
15.1 The row . . . 94
15.2 Two views of every row . . . 94
15.3 Noise, and what a row can decide . . . 95
15.4 Same hour, both sides . . . 95
15.5 The block A/B . . . 95
15.6 The release gate . . . 96
15.7 The teacher-forced bench . . . 96
15.8 Traps . . . 96
15.9 Measuring Flash-Next . . . 97
15.10 Limits . . . 98

16 Adding a model 99
16.1 The adapter path . . . 99
16.2 The seams . . . 100
16.3 The tests a new model must pass . . . 101
16.4 What is still specific to Qwen3.8 . . . 102

17 Qwen3.8-Flash-Next 103
17.1 The model . . . 103
17.2 The packs and the window each one fits . . . 104
17.3 Loading . . . 105
17.4 Kernels . . . 105
17.5 Speculation . . . 106
17.6 Never slower than the plain step . . . 106
17.7 Exactness . . . 107
17.8 Caches . . . 107
17.9 Memory . . . 108
17.10 Serving and operations . . . 108
17.11 What is not measured . . . 109

IV The conductor **110**

18 Profiles and the router 111
18.1 Switching it on and off . . . 111
18.2 Versioned documents . . . 112
18.3 Profiles . . . 112

18.4 The router . . . 113
18.5 Effort and the thinking cap . . . 115
18.6 The note and its replay . . . 115
18.7 Side requests . . . 116
18.8 Compaction . . . 116
18.9 Session addresses . . . 116
18.10 What is journaled . . . 117

19 Packages, the library and the deciders 118
19.1 Packages . . . 118
19.2 The library . . . 119
19.3 The Packages screen and proposals . . . 119
19.4 Roles . . . 120
19.5 Decision models . . . 120
19.6 Laya for the model's scripts . . . 121

20 The session monitor 123
20.1 Modes and policy . . . 123
20.2 Safe points and actions . . . 123
20.3 The tasks . . . 124
20.4 Reasoning loops . . . 124
20.5 Tool-call checks . . . 124
20.6 The claim check: E2 . . . 126
20.7 Turn snapshots and outcomes . . . 126
20.8 Session tasks . . . 126
20.9 Observability . . . 127
20.10 Extending it . . . 127

21 Status lines, the voice and the shadow reader 128
21.1 Status lines . . . 128
21.2 The narrator and the voice . . . 129
21.3 The shadow reader . . . 130

22 Memory notes 133
22.1 Where the files are . . . 133
22.2 How the model writes . . . 133
22.3 How the model reads . . . 134
22.4 The background writer . . . 135
22.5 On the second model . . . 136
22.6 Behind the proxy . . . 136
22.7 The Notes tab . . . 136

23 The conductor in front of any backend 137
23.1 Switching it on . . . 137
23.2 The feature groups . . . 137
23.3 What is different from the engine . . . 138

23.4 The word-level loop detector . . . 139
23.5 Laya, the library and the session address . . . 139
23.6 What was checked . . . 139

V The dashboard **140**

24 The dashboard, view by view 141
24.1 Performance: the live panel . . . 141
24.2 Performance: the history . . . 143
24.3 Usage . . . 144
24.4 Dev . . . 144
24.5 Playground . . . 144
24.6 Conductor . . . 145
24.7 Backends . . . 145
24.8 System . . . 147
24.9 The contract . . . 147
24.10 Building and testing the page . . . 149

25 The Chat tab 151
25.1 What runs where . . . 151
25.2 The sandbox . . . 151
25.3 Permission modes . . . 152
25.4 Rules and “Always allow” . . . 152
25.5 Known limitations . . . 153

26 Watching several engines 154
26.1 Every request at once . . . 154
26.2 Inside vLLM and SGLang: the engine’s page (L1) . . . 156
26.3 Inside vLLM and SGLang: the plugins (L2) . . . 157
26.4 The machine behind each backend . . . 157
26.5 Your own Prometheus . . . 158

VI Reference **159**

27 Command-line reference 160
27.1 The engine’s server . . . 160
27.2 The dashboard and proxy . . . 169
27.3 The service scripts . . . 171
27.4 Measurement and maintenance tools . . . 171

28 Environment and configuration reference 173
28.1 The served profile . . . 173
28.2 Engine switches . . . 176
28.3 Server variables . . . 181
28.4 Proxy, collector and plugin variables . . . 183

28.5 The conductor's documents . . . 185

29 HTTP API reference 187
29.1 Who may call what . . . 187
29.2 The engine's routes . . . 188
29.3 The proxy's own routes . . . 191
29.4 The conductor's and the configuration routes . . . 192
29.5 Headers . . . 196
29.6 Errors . . . 197

30 Metrics and the ledger 200
30.1 Prometheus metrics . . . 200
30.2 The usage ledger . . . 203

31 Troubleshooting 207
31.1 Installing . . . 207
31.2 The engine . . . 208
31.3 The Chat tab . . . 210
31.4 vLLM and SGLang behind the proxy . . . 210
31.5 The proxy . . . 211
31.6 Memory notes . . . 212

32 Questions and answers 213

33 Roadmap 215
33.1 A mixture-of-experts model, and what is left . . . 215
33.2 Parallel requests . . . 215
33.3 The NVMe tier . . . 215
33.4 Profiles as data . . . 216
33.5 Other GPUs, then several . . . 216
33.6 Housekeeping . . . 216

34 Changelog summary 217
34.1 0.8.3, repository hygiene . . . 217
34.2 0.8.2, community weights and permission modes . . . 217
34.3 0.8.1, a new box fetches every file . . . 219
34.4 0.8.0, Qwen3.8-Flash-Next and the Chat tab . . . 220
34.5 What 0.7 added . . . 222
34.6 What 0.6 added . . . 222
34.7 What 0.5 added . . . 222
34.8 What 0.4.8 adds . . . 223
34.9 What 0.4.7 added . . . 223
34.10 What 0.4.0 brought . . . 224
34.11 Measured in the 0.4.0 series . . . 226

Glossary 227

Index 231

List of Figures

1.1 Single-request speed on the benchmark row `serve-single-i256-o256-v1`: StairCut against the same drafter at fixed block widths and against plain greedy decoding. measured: the StairCut paper, published NVFP4 weights, persistent store off. . . . 5

2.1 The request path. A client sends an OpenAI request; the proxy routes it by model id; the conductor's router plans the turn; admission decides when it runs; the caches decide where the prefill starts; then drafters and the target model take turns, round after round, while the conductor's monitor watches the output at fixed safe points. Requests for other backends leave at the proxy. 8

3.1 Which install path 13
3.2 Open WebUI with `TandemLLM` as its model: a demo request on the test board in October 2026, served under the `chat` profile, with the reasoning folded into "Thought for 1 seconds". Only the message pane is shown. 18
3.3 opencode with `TandemLLM` as its provider: the end of a demo task on the test board in October 2026, under the `coding` profile. The model added a function, tested it, asked Laya for a verdict on the test log, removed its scratch files and reported what it did. The screenshot is from a build that sent the reasoning to opencode as text between `<think>` tags; every client but the Hermes agent now gets `reasoning_content` by default (`QWEN38_CLIENT_REASONING`), which opencode shows as folded reasoning parts. 19

5.1 The proxy in front of several backends 27

6.1 Where the engine's memory goes 35

8.1 The layer pattern of Qwen3.8-27B 45
8.2 Bytes a step and the floor they set 46

9.1 The published weights against BF16 51

11.1 One speculative round 61
11.2 Verify cost against tree rows 62
11.3 The teacher-forced bench 63
11.4 Decode speed and verify cost against context length 64
11.5 A draft tree, its accepted path and its mask 65

12.1 Agreement with plain greedy decoding 69

13.1 Time to first token, cold and resumed . . . 73
13.2 The cache tiers . . . 75
13.3 One conversation through the cache tiers . . . 76
13.4 The resident prefix and the end anchor . . . 77
13.5 One compaction three ways . . . 80

14.1 The memory gate . . . 88
14.2 The scheduler: side lane and queue . . . 89

16.1 The engine's layers . . . 100
16.2 The seams a new model plugs into. . . . 100

17.1 The packs at a 262,144-token window . . . 104

18.1 The router's path from a request to a plan . . . 113

20.1 The monitor's safe points . . . 123

21.1 The shadow reader's read rate . . . 131

22.1 The life of a memory note . . . 135

24.1 The dashboard's views . . . 141
24.2 The Performance view, Engine page: the Live panel with a request writing, the decode and prefill rates, and the last requests. Demo data from the dashboard's mock server (`npm run preview:mock`), not a measurement. . . . 142
24.3 The Live panel as an instrument readout . . . 143
24.4 The Usage view: a year of tokens, requests and cost, filtered by model, client, backend and user. Demo data from the dashboard's mock server (`npm run preview:mock`), not a measurement. . . . 144
24.5 The Dev view: the live log, which never holds prompt or answer text, and the recent requests. Demo data from the dashboard's mock server (`npm run preview:mock`), not a measurement. . . . 145
24.6 The Playground before the first message: the chat on the left, the system prompt and the sampling parameters on the right, the engine's own figures for each turn below. Demo data from the dashboard's mock server (`npm run preview:mock`), not a measurement. . . . 146
24.7 The Conductor view, Overview: decisions and the session monitor's acts over 24 hours, the classifier's latency, the roles online and the task classes. Demo data from the dashboard's mock server (`npm run preview:mock`), not a measurement. . . . 147
24.8 The Conductor view, Sessions: each session's client, profile, current class, turns and the hidden notes it carries. Demo data from the dashboard's mock server (`npm run preview:mock`), not a measurement. . . . 148
24.9 The Conductor view, Memory, the Notes tab: the live notes by scope, with kind, status, writer and use. Demo data from the dashboard's mock server (`npm run preview:mock`), not a measurement. . . . 148

24.10 The Backends view with a vLLM, an SGLang and a DeepSeek backend, and the line for this engine's own settings. Demo data from the dashboard's mock server (`npm run preview:mock`), not a measurement. 149
24.11 The System view: the engine, the memory, the GPU, the queue, the caches and the machine. Demo data from the dashboard's mock server (`npm run preview:mock`), not a measurement. 150

26.1 The Performance view, Parallel page: every request in flight on six backends, the aggregate decode rate, and one card per backend. Demo data from the dashboard's mock server (`npm run preview:mock`), not a measurement. 155
26.2 A load test of the Parallel view 156

List of Tables

1.1 The served engine's headline numbers. Source: the StairCut paper, published NVFP4 weights, measured. . . . 6

3.1 Setups and their state in this release. *Live*: run on the test board in the 0.4.0 series. *Built*: the code supports it and its tests pass, but no live run is recorded. . . . 12
3.2 What the two default profiles download on a new box. Free disk: the download plus the 60 GB the script keeps free after it. . . . 14
3.3 install.sh options . . . 16
3.4 The layout of an installed engine. . . . 17
3.5 Pointing a client at the engine. . . . 17
3.6 Who can reach what, by default and with the switches set. . . . 19
3.7 The small models and the line that turns each on. . . . 20
3.8 The three serving profiles. Only the NVFP4 profile's speed is measured on this release; the bytes a step are arithmetic from the tensor sizes. . . . 22

4.1 The subcommands of `tandem-dash`. . . . 25
4.2 The standalone proxy's files. . . . 26

5.1 Backend kinds. . . . 28
5.2 The vLLM flags the proxy needs, and why. . . . 29
5.3 Where each figure of a relayed request comes from. . . . 33

6.1 What the served stack holds . . . 35
6.2 The service scripts in `ops/`. . . . 36

7.1 The first community pack . . . 42

8.1 The two layer kinds and the state each keeps per sequence (arithmetic, from the configuration's sizes). . . . 45
8.2 The bytes one decode step reads, as shipped . . . 46
8.3 A map of the code. . . . 48

9.1 The NVFP4 layout of one projection. . . . 49
9.2 The 27B's EXL3 sizes . . . 52
9.3 What a Flash-Next pack holds, and how the engine holds it. . . . 54
9.4 Quantisation tools and the engine switches that load their output. . . . 55

10.1 The NVFP4 matrix-product kernels by row count. . . . 57
10.2 The recurrent-layer kernels. . . . 58

10.3 The main kernel switches. 59

11.1 Every chain operation has an exact tree counterpart. 65
11.2 The speculative decoding knobs. 67

12.1 Where two runs of one 5,000-token raw greedy turn first wrote a different token. measured, the 0.4.0 code, lookup store off. 70
12.2 The checks that hold the engine to its promise. 72

13.1 The caches and their defaults in the served profile. 74

14.1 Reasoning formats. 83
14.2 The thinking-budget shapes the server reads. 83
14.3 Admission limits. Code defaults; the served values are in parentheses where they differ. 87

15.1 The Flash-Next tools behind the numbers in Chapter 17. 97

17.1 Flash-Next beside the 27B. 103
17.2 The Flash-Next packs 104
17.3 Flash-Next through the server, MTP alone and with the lookup 106
17.4 Speculation and the plain step by context 107
17.5 Flash-Next's parallel limits 109

18.1 The conductor's switches. 111
18.2 The shipped conductor profiles. 112
18.3 The router's shipped classes, with each class's effort and packages. A package marked core goes in on every turn of the class. 114
18.4 Effort levels and their reasoning caps. 115

20.1 Safe points and the actions each allows. 124
20.2 The session monitor's tasks, with their shipped modes. 125

21.1 Kinds of status line. 129
21.2 The narrator's modes. 129
21.3 The router's question asked of the shadow 132

23.1 The conductor's settings per backend. 137
23.2 Feature groups of the conductor over HTTP 138

24.1 The Conductor view's screens. 146

25.1 The Chat tab's permission modes (0.8.2). 152

26.1 The four layers the proxy reads. 154
26.2 The engine figures and the families they come from. 156
26.3 A backend's host setting. 158

27.1 Every flag of the engine's server, server/app.py, with the value the served profile passes where it differs . . . 160
27.2 tandem-dash serve and python -m proxy . . . 169
27.3 tandem-dash launch . . . 170
27.4 tandem-dash hostcollect . . . 170
27.5 The scripts in ops/ and the installed control script. . . . 171
27.6 The tools this manual refers to. . . . 172

28.1 The keys ops/serve.env sets, in file order (paths not shown) . . . 173
28.2 Every QWEN38_* switch in engine/settings.py, with the value ops/serve.env sets . . . 176
28.3 QSE_* variables: the server's settings from the environment or serve.env . . . 181
28.4 TANDEM_* variables: the proxy, the collector, the plugins and the installer . . . 183
28.5 The conductor's versioned documents. . . . 186

29.1 Who may call a route: the labels of the routes' who column . . . 187
29.2 The engine server's routes . . . 188
29.3 The standalone proxy's routes (tandem-dash) . . . 191
29.4 The two small servers beside them . . . 192
29.5 The conductor's routes, /v1/conductor/* . . . 192
29.6 The proxy's configuration routes, /v1/config/* . . . 195
29.7 Every header the engine and the proxy read or write . . . 196
29.8 Every error status and type . . . 198

30.1 Prometheus families registered in the code . . . 200
30.2 The usage ledger's requests table (schema version 9) . . . 204

34.1 Releases of TandemLLM. . . . 218
34.2 Measurements from the 0.4.0 series that this manual cites. All measured, on the published weights on one DGX Spark. . . . 226



Part I

Introduction

Two chapters for every reader. Chapter 1 says what TandemLLM is, names the machine and the model every number in this book comes from, says what the repository contains and what runs today, and states the two rules every change has to pass. Chapter 2 follows one chat request through all of it, from the proxy to the stream going back, and names the chapter that explains each step.

Chapter 1

What TandemLLM is

TandemLLM runs the models your agents need on your own hardware, behind one OpenAI-compatible address. A coding agent such as opencode or a chat client such as Open WebUI sends its request to that address. TandemLLM picks the model that answers, plans the turn, and watches the turn while it runs.

Several models work on one request. Qwen3.8-27B writes the answer. Two small drafters guess its next tokens and a lookup drafter offers text it has seen before, so one step of the big model commits 4.58 tokens on average on the benchmark row (measured, the paper). More models can run beside them in the same process. Laya's decision models answer typed questions such as "is this turn code or prose". A Qwen3.5-2B narrator writes one short line about what the big model is doing, and a Qwen3.5-0.8B shadow reader keeps its own copy of each coding session. A residency planner refuses any model that would not fit in memory (Section 6.8).

You get on your own machine what a cloud provider gives you. Clients use the OpenAI API with streaming and tool calls. A proxy routes each request by its model id, to this engine or to another backend: a vLLM or SGLang server, or a cloud API (Chapter 5). Every request leaves one row in a usage ledger with its tokens and its cost. Keys for cloud APIs sit in the config database as AES-256-GCM ciphertext.

An agent's next turn does not read its whole prompt again. The resident prefix keeps the last prompt's rows with a snapshot of the recurrent state every 1,024 tokens, and the end anchor keeps the end of the last answer. A parked session holds a second conversation while another one runs, and the state store keeps whole snapshots. Two caches live on the NVMe and survive a shutdown. The disk tier keeps prefixes that come back. The session spill writes live conversations at a stop and reads them back at the next start: a 41,587-token conversation came back in 0.94 s (measured, the 0.4.0 series). Last, the persistent suffix store keeps the n-gram tables the lookup drafter copies from. Chapter 13 describes every cache.

With memory notes on, the model keeps notes as files, one fact per file and at most 400 words. Notes about the user and about each project last across sessions. Session notes hold what is true for one conversation only (Chapter 22).

One dashboard watches this engine and the others. It reads the metrics pages of vLLM and SGLang every 15 s, and small plugins inside them report each request's own times. For a cloud API it measures time to first token and decode speed at the proxy, and it shows the machines behind every backend (Chapter 26).

Requests run in parallel. With `--parallel 4` the engine decodes up to four requests in the same rounds (Section 14.13), and the proxy relays any number of requests to other backends at once.

TandemLLM is not tied to one model. Its main model is Qwen3.8-27B, and a second, Qwen3.8-Flash-Next, runs on the same board in its place (Chapter 17). The small models are Qwen3.5 and Laya checkpoints, the proxy serves any model behind an OpenAI-compatible API, and Chapter 16 lists the seams where a main model plugs in.

1.1 The test environment

Every number in this book comes from one machine with one main model: an NVIDIA DGX Spark running Qwen3.8-27B. The Spark has a GB10 GPU and 128 GB of memory that the CPU and the GPU share. That is the project's test hardware and its test model, and the project has tested nothing else so far. Its kernels' tile tables and StairCut's price tables come from that board. Much of the engine follows that model's mix of 48 Gated DeltaNet layers and 16 attention layers (Chapter 8).

The project is growing past both. Other CUDA GPUs run the engine with the one Spark-only kernel turned off; their speed has no measurement yet. The proxy and the dashboard run on any machine with Python 3.11. The roadmap (Chapter 33) names the next models and boards. When this book says "the board" or "the model" without a name, it means the DGX Spark and Qwen3.8-27B.

1.2 Two levers

A decode step of Qwen3.8-27B reads about 27 GB of weights in the vendor's FP8 format. The DGX Spark reads its memory at 273 GB/s on paper and about 240 GB/s in practice, so every step costs at least 100 ms before any code runs (arithmetic). That is a ceiling of 10 tokens a second for one request. The engine is built to get under that floor while writing the model's own greedy text.

With the published NVFP4 weights and StairCut on, it writes 49.89 tokens a second for one request on the benchmark row, against 13.69 for plain greedy decoding on the same box (measured, the StairCut paper, 50 requests each). The rest of this section says how.

The engine pulls two levers on the same cost.

The first lever cuts the bytes a step reads. The served weights store every projection in NVFP4, four bits a weight with a scale per 16 weights, and the output head in FP8. A step then reads 15.01 GB instead of 26.93 GB (arithmetic, from the checkpoint's tensor sizes). Chapter 9 shows how the 4-bit copies were made and what they cost in quality.

The second lever makes each step produce several tokens. Small *drafters* guess the next tokens, and the big model checks all the guesses in one forward pass. A guess the model agrees with is a free token. At the first wrong guess the model puts in its own token, so nothing a drafter proposes reaches the output unless the model would have written it. On the benchmark row a round takes 93.4 ms and commits 4.58 tokens (measured, the paper). That is where the name comes from: two models working in tandem, one guessing and one checking.

The verify is cheap because of the memory floor. A forward pass over 16 tree rows reads the same weights as a pass over one row, so on the published weights at 1k tokens of context it takes 74.4 ms

against 75.5 ms for 8 rows (measured, `ops/stair-tables-nvfp4.json`). Extra guesses in the same pass are nearly free until the row count crosses a kernel's tile boundary. StairCut, the router that sizes each round's draft tree, is built on that staircase (Chapter 11).

1.3 What is in the repository

TandemLLM is several programs that share one repository and one release number.

The engine
: `engine/`, `tools/` (kernels) and `server/app.py`: the model, its kernels, the drafters and routers, the caches, and an OpenAI-compatible HTTP server built on Python's standard library. It needs a CUDA GPU (measured on the DGX Spark only), PyTorch, Triton and the weights. The small models run on its own code: the Qwen3.5 narrator and shadow reader on its Qwen forward, Laya's decision models in the same process.

The conductor
: `server/conductor/`: a layer in front of the model. It classifies each turn, sets the thinking effort, adds hidden notes and skills, watches the generation for loops and broken tool calls, keeps memory notes and shows the client what it does. It is off unless a configuration directory is given.

The dashboard
: `dashboard/`: a web page the server serves at `/dashboard/`. It shows what each request is doing now, a year of usage, the log, a playground, the conductor's settings, the backends and the machine.

The proxy
: `proxy/`: the same dashboard in front of other OpenAI-compatible engines and cloud APIs, routed by model id. It installs with `pip` and needs no GPU.

The plugins
: `plugins/`: two small packages that run inside vLLM and SGLang and report each request's queue, prefill and decode times to the proxy.

The installer
: `install.sh`: one command that sets up the served profile on the test board, and best effort on another CUDA GPU. With `--setup` it starts setup mode instead, and `ops/use-profile.sh` then downloads and starts any weight profile (Section 3.3).

1.4 What runs today

The roadmap (Chapter 33) widens each of these, one step at a time.

- Models: Qwen3.8-27B is the main model the engine serves, a hybrid of 48 Gated DeltaNet layers and 16 attention layers; Chapter 8 explains why much of the engine follows that mix. Qwen3.8-Flash-Next, a mixture of experts on the same hybrid, is the second, served from turboderp's EXL3 packs one model at a time (Chapter 17). Beside it run the two fine-tuned

DFlash2 drafters, the lookup drafter and, when you turn them on, Laya, the narrator and the shadow reader. Behind the proxy, any model a backend serves.

- Hardware: the DGX Spark is the only board with measured kernels and prices (Section 1.1). Other CUDA GPUs are best effort.
- Requests: `--parallel N` gives the engine N sequences, and requests that decode at the same time share one draft pass and one verify pass a round (Section 14.13). The shipped profile keeps one sequence, and the measurements go up to four. With one sequence a request that finds the engine busy waits in a bounded queue, and a short request is served at a safe point inside the running one (the side lane, Section 14.11). The proxy relays any number of requests to other backends at once (Chapter 23).

1.5 Two rules

Every change to the engine keeps two rules, and the release gate checks both before anything ships.

The first rule is agreement with greedy decoding. For every greedy request the engine's output tokens equal what plain decoding on the same weights writes, one token at a time, up to the first near tie where the two best tokens are within one unit in the last place. There the batched verify can round the other way. The paper measured how often: of 30 prompts, 17 outputs equal plain greedy decoding to the end of the answer, and 13 diverge at a near tie with a gap of 0.04 to 0.76 ulp (measured). Chapter 12 has the full statement and the checks.

The second rule is that no measured number gets worse. A candidate runs against the current release in the same hour on the same box, alternated, and ships only if no figure resolves worse: not the mean, the median, the 90th percentile or the maximum, not the time to first token, and not tokens or milliseconds per round. Chapter 15 describes the method and the traps it avoids.

1.6 The headline numbers

Table 1.1 collects the numbers that describe the served engine on the test board. Each comes from the StairCut paper's measurements on the published NVFP4 weights, with the lookup drafter's persistent store off.

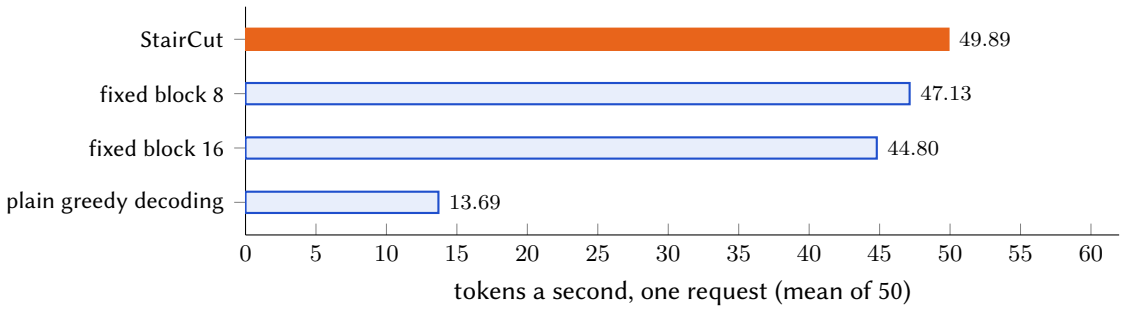


Figure 1.1: Single-request speed on the benchmark row `serve-single-i256-o256-v1`: StairCut against the same drafter at fixed block widths and against plain greedy decoding. measured: the StairCut paper, published NVFP4 weights, persistent store off.

Table 1.1: The served engine's headline numbers. Source: the StairCut paper, published NVFP4 weights, measured.

what	value
single request, StairCut, benchmark row (mean of 50)	49.89 tok/s
the same drafter at a fixed block of 8	47.13 tok/s
the same drafter at a fixed block of 16	44.80 tok/s
plain greedy decoding, speculation off	13.69 tok/s
teacher-forced bench, 25 workloads, StairCut	90.45 tok/s
one round on the benchmark row	93.4 ms, 4.58 tokens
time to first token, cold, at 32k / 64k / 128k tokens	44.4 / 93.7 / 208.3 s
the same prompt sent again, resumed from the cache	0.3 / 0.6 / 1.1 s
held-out loss against Qwen's BF16 release, prose / code	+0.0221 / +0.0426 nats

These are single-request numbers. Parallel requests on the engine are measured in Section 14.13. Long prompts are measured separately in Chapter 13. No task benchmark suite has been run, so this manual makes no claim about task accuracy.

1.7 The paper

Khaled Bakeer's paper describes the engine's decoding method:

> Khaled Bakeer. *StairCut: sizing speculative draft trees on a measured verification-cost staircase.* Zenodo, 29 September 2026. doi:10.5281/zenodo.23045275.

The paper measured the release tagged `v0.2.0-staircut` (commit `c111fa7`) with the published NVFP4 weights on the test board. It backs these parts of this book:

- the draft tree merged from the block drafter and the lookup drafter, and StairCut, which cuts it each round on the measured verify-cost staircase (Chapter 11);
- the commit of an accepted path through the Gated DeltaNet layers in closed form, and the NVFP4 kernels with a fixed reduction order (Chapter 10);
- the NVFP4 quantisation from Qwen's BF16 release and its held-out loss (Chapter 9);
- the speeds of Table 1.1: 49.89 tok/s for StairCut, 47.13 and 44.80 at fixed blocks of 8 and 16, 13.69 for plain greedy decoding, 50 requests each on the benchmark row, and the teacher-forced bench of 25 workloads;
- the agreement with plain greedy decoding: of 30 outputs, 17 identical to the end of the answer and 13 that part at a near tie (Chapter 12);
- the time to first token of a 32k to 128k-token prompt from cold, and again when it resumes from the anchors (Chapter 13).

What came after that release is not in the paper: image input, the conductor, the caches on the NVMe, memory notes, parallel requests, the proxy and the plugins. Their numbers come from the release series, and the text labels them that way. The release gate holds every later release to the paper's single-request numbers and to the agreement rule. `CITATION.cff` in the repository carries the citation; in BibTeX:

```
@misc{bakeer2026staircut,
  author    = {Bakeer, Khaled},
  title     = {StairCut: sizing speculative draft trees on a measured
               verification-cost staircase},
  year      = {2026},
  publisher = {Zenodo},
  doi       = {10.5281/zenodo.23045275},
  url       = {https://doi.org/10.5281/zenodo.23045275}
}
```

1.8 Credits

Qwen made the model weights and their reference implementation. The base block drafter is DFlash2 for Qwen3.8-27B, released by its vendor under Apache-2.0. The engine runs on PyTorch and Triton over CUDA, and its own kernels are written in Triton and CUDA. The lookup corpus that the installer builds comes from public, permissively licensed text. The decision models the conductor can ask are Laya's published checkpoints (Apache-2.0).

Chapter 2

One request, end to end

This chapter follows one chat completion from a coding agent through every part of TandemLLM. Each step names the chapter that explains it. Figure 2.1 is the map.

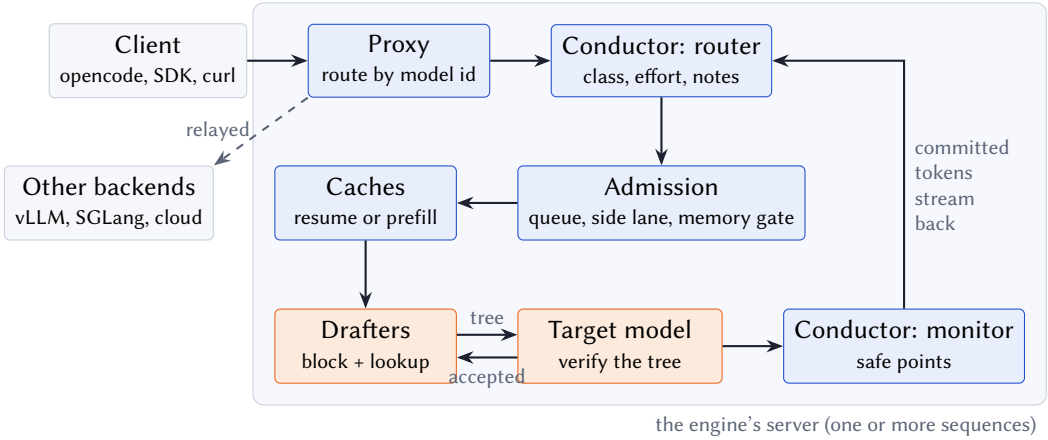


Figure 2.1: The request path. A client sends an OpenAI request; the proxy routes it by model id; the conductor's router plans the turn; admission decides when it runs; the caches decide where the prefill starts; then drafters and the target model take turns, round after round, while the conductor's monitor watches the output at fixed safe points. Requests for other backends leave at the proxy.

2.1 The client sends a request

The client is any program that speaks the OpenAI chat completions API: the OpenAI SDK, opencode, the Hermes agent, Open WebUI or `curl`. It sends `POST /v1/chat/completions` with its messages, its tools and its options, usually streamed. The engine's server reads every OpenAI field it can honour, accepts the ones that have no effect here, and refuses the rest with a 400 that names the field (Section 14.1).

2.2 The proxy routes it

When the server runs with `--backends`, or when the standalone proxy (`tandem-dash`) is in front, the request is routed by its `model` field. A model of another backend (vLLM, SGLang, a cloud API) is

relayed there: the answer's bytes go back to the client unchanged, and the proxy writes one ledger row with the tokens, the time to first token, the decode rate and the cost. A request for `TandemLLM` stays in the engine's server. Chapter 4 sets this up and Chapter 23 covers the conductor in front of other backends.

2.3 The conductor plans the turn

With a conductor configuration the server serves one model name, `TandemLLM`: the weights with a conductor in front. Before the request queues, the conductor resolves a *profile* (by header, by client, or the default), and its router decides what kind of task the turn is. A tool-result turn keeps the session's class; only a new user message asks a classifier, and the default classifier is Qwen itself under a choice grammar. The class sets the thinking budget and adds a hidden note at the end of the last message: the effort sentence, rules and *packages* (skill texts). Later turns replay that note at the same token position, so the cached prefix keeps matching. Chapter 18 explains the router and Chapter 19 the packages.

A client's side request, such as opencode's title call, is recognised by its shape and served small: no classifier, no note, at most 64 output tokens for a title.

2.4 Admission decides when it runs

The engine holds one sequence unless `--parallel N` gives it more (Section 14.13). A request that finds it busy waits in a bounded queue (8 by default). A streamed request that waits more than 10 s gets its stream opened and an SSE comment every 10 s, with an estimate of the wait. A short request (at most 2,048 prompt tokens and 64 output tokens, thinking off) does not wait at all: the running generation serves it at its next safe point, between two decode blocks or two prefill chunks, and gets its state back byte for byte. Before every prefill a memory gate checks that the prompt's workspace fits under the declared peak, giving caches back cheapest first. Chapter 14 covers the queue, the side lane and the memory gate.

2.5 The caches decide where the prefill starts

A prefill is the most expensive thing the engine does: 208.3 s for a 128k-token prompt from cold (measured, the paper). Before it prefills, the engine looks for state it already has. In an agent session each turn's prompt is the previous prompt plus the answer plus a tool result, and the *resident prefix* still holds the previous turn's rows in the engine's own buffers, with recurrent-state *anchors* every 1,024 tokens and an *end anchor* at the end of the last generation. The next turn resumes there and forwards only what is new. Further down are a state store of snapshots, a disk tier on the NVMe for prefixes that come back, and a session spill that keeps whole conversations through a restart. Chapter 13 describes them all.

2.6 The decode loop

Then rounds run until the answer ends. In each round the block drafter proposes up to 15 tokens and the lookup drafter proposes continuations it has seen before; StairCut merges them into one tree and cuts it to the size that maximises expected tokens per millisecond on the measured verify cost. One forward pass of the target model verifies the whole tree through all 64 layers. The engine walks the tree with the model's own tokens, keeps the longest agreeing path plus one token of the model's own, and commits that path's KV rows and recurrent state. Chapter 11 explains the round and Chapter 10 the kernels that make the verify cheap.

2.7 The monitor watches the output

While the model writes, the conductor's *session monitor* reads the output at four safe points: before the prefill, after every committed block, at the end token, and when the answer's tool calls are complete. It can close a reasoning block that loops, continue a turn that ended empty, repair or regenerate a malformed tool call, and stop a tool call whose arguments repeat. Everything it adds is hidden from the client and never counted in `usage`. Chapter 20 lists its tasks.

2.8 The stream goes back

Committed tokens stream to the client as they are verified, several at a time. A tool call streams as argument deltas while the model writes it, with values typed by the request's JSON schema. With a profile's status lines on, the client also sees one short line for each step the conductor takes ("› reading 20,745 tokens, about 18 s left"), which the engine strips again from the next prompt (Chapter 21).

2.9 After the request

The request leaves one log line (counts, finish reason, rate, never text) and one row in the usage ledger. The conductor writes its decisions to a journal, a transcript block for a request with tools, and, when memory notes are on, any `<tandem_memory>` block the model wrote. When the session goes quiet, a background writer may write the session's facts as notes (Chapter 22). The dashboard shows all of it live (Chapter 24).



Part II

Installing and running

How to install and run it. Chapter 3 starts with the setups that work today, then installs the engine with one command, says who can reach it, and adds the small models and other backends. Chapter 4 installs the dashboard and proxy on any machine, Chapter 5 puts other engines and cloud APIs behind it, and Chapter 6 is the operator's chapter: the memory rules, the service scripts, the watchdog, how to hold the board for a measurement, where the logs are, and how to upgrade and roll back. Chapter 7 runs weights other people published, and says how to list your own.

Chapter 3

Installing TandemLLM

TandemLLM has two parts to install. The engine runs the models and needs a CUDA GPU; its test board is the DGX Spark (Section 1.1). The dashboard and its proxy need only Python, run on any machine, and also run inside the engine's own server. Most setups use both. Find your setup in Table 3.1, then follow the sections it names.

3.1 Setups, and which ones work

The parts combine. One engine can sit behind one address together with other engines and cloud APIs, and one dashboard watches all of them. Table 3.1 says which combinations work today, and how far each one is tested.

Table 3.1: Setups and their state in this release. *Live*: run on the test board in the 0.4.0 series. *Built*: the code supports it and its tests pass, but no live run is recorded.

setup	how	state
the engine alone	`install.sh` on the board (this chapter)	live
the engine and cloud APIs, one address	`tandem-dash` on the board or another machine, with the engine and the cloud API as backends (Chapter 4)	live, with DeepSeek
the same, from the engine's own server	`QSE_BACKENDS`: the engine relays to the cloud API, and its conductor can run in front of it (Section 3.10)	built
the engine and other engines on other machines	either of the two above, the other engines as `vllm`, `sglang` or `generic` backends (Chapter 5)	built; vLLM and SGLang live behind the proxy on their own
the dashboard and proxy alone	`pip install` or docker on any machine, in front of the backends you run (Chapter 4)	live
the engine and a second GPU engine on one larger machine	each engine on its own GPU, `tandem-dash launch` for the second	built
the engine and a second GPU engine on one DGX Spark	the engine plans its memory as if it owns the board (up to 77 GiB during a prefill, plus its small models); a board out of memory stops answering until it is power cycled	not supported

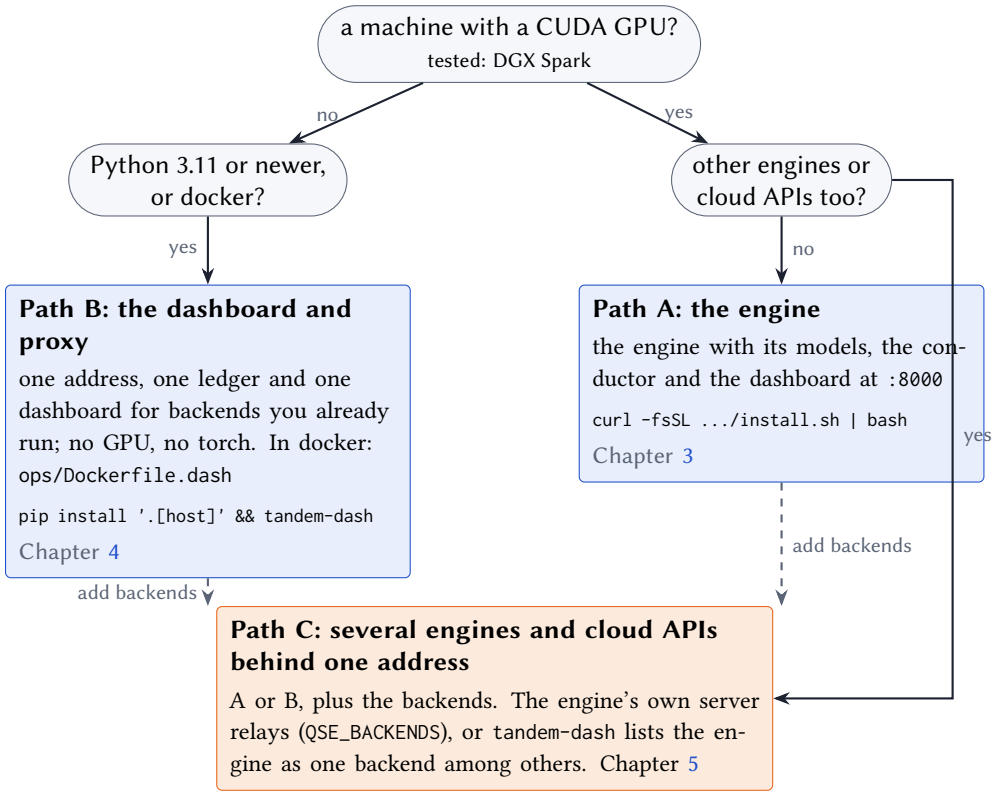


Figure 3.1: Which install path. A machine with the GPU gets the engine (path A); any other machine gets the dashboard and proxy (path B); either grows into path C by adding backends. The engine's own server can relay to other backends, and a standalone proxy can list the engine as one backend among others.

3.2 What the engine needs

The engine is tested on one NVIDIA DGX Spark, a GB10 GPU (`sm_121`) with 128 GB of memory that the CPU and the GPU share, running Linux on aarch64. On another CUDA GPU the installer turns off the one kernel built for the GB10, and the engine runs best effort. The installer checks four things before it changes anything: an NVIDIA GPU and driver, the CUDA compiler (one kernel builds on its first use), about 70 GB of free disk, and 64 GiB of free memory at the start. With `--setup` it downloads only Wren's model, so it asks for about 5 GiB of disk beside the Python environment and 8 GiB of free memory; `use-profile.sh` checks the disk and the memory a profile needs before it downloads or starts it.

The software is pinned. The engine imports PyTorch 2.13 built for CUDA 13.0, torchvision 0.28 (the CUDA 13.0 build that matches that torch), Triton 3.7, transformers 5.12, Pillow 11.3, safetensors and numpy, and nothing else. Python 3.11 is the tested version; under 3.12 one grammar test fails. The installer's lock lists every package with its version, for aarch64 and x86_64.

Two of those packages are there for images. Pillow decodes a request's images, and torchvision is the backend of the image processor that the checkpoint ships. The engine pins that backend (`--image-backend torchvision`) and refuses to start without it, because transformers would otherwise fall back to its Pillow backend, whose patches differ by up to two 8-bit levels after a resize (measured,

0.4 % of the values of one resized image). Every cache keys on those patches, so the backend has to be fixed. `--image-backend pil` is the explicit choice without torchvision, and `--vision off` serves text only.

Run one engine per board. Two engines loading side by side run the board out of memory, and a DGX Spark in that state stops answering until it is power cycled. Section 6.1 explains how the scripts enforce it.

3.3 A new box: setup mode, then one profile

Since 0.8.1 a new DGX Spark needs four commands and no Hugging Face token. Every file a profile reads is public on Hugging Face: the 27B's weights and drafters (Section 9.7) and turboderp's Flash-Next packs (Chapter 17).

```
curl -fsSLO https://raw.githubusercontent.com/0xBakeer/TandemLLM/v0.8.3/install.sh
bash install.sh --setup --yes
~/TandemLLM/run/ops/use-profile.sh flashnext-3.05 --dry-run
~/TandemLLM/run/ops/use-profile.sh flashnext-3.05
```

The first line fetches the installer from the release tag. With `--setup` the installer puts the code and the Python environment in place and downloads only Wren's model (Qwen3.5-2B, 4.3 GB), the setup guide's assistant. It writes the configuration with no weights and starts setup mode on `127.0.0.1:8000`: the dashboard and its `#/setup` page, Wren, and no model of Qwen's.

Run `use-profile.sh` from `~/TandemLLM/run/ops/`, next to the `serve.env` the installer wrote. With `--dry-run` it prints every file's URL and size and changes nothing. Without it, the script downloads what the profile needs and checks every weight file against the sha256 the hub lists. It then leaves setup mode, starts the engine on port 8000 and sends one smoke request. `use-profile.sh --list` prints every profile; `exl3-4.5` is the default for the 27B and `flashnext-3.05` for Flash-Next. Since 0.8.2, `community/<id>` runs a pack someone else published (Chapter 7).

Table 3.2: What the two default profiles download on a new box. Free disk: the download plus the 60 GB the script keeps free after it.

profile	downloads	size	free disk
`flashnext-3.05`	the pack's branch of `turboderp/Qwen3.8-Flash-Next-exl3`	85.1 GB	145 GB
`exl3-4.5`	`0xBakeer/TandemLLM-Qwen3.8-27B-EXL3-4.5`; the K3 drafters and the K4 head (`...-EXL3-drafters`); the BF16 drafters (`...-DFlash2-b8`, `-b16`, into `~/TandemLLM/src/train`); the base checkpoint `Qwen/Qwen3.8-27B-FP8`, without `mtp.safetensors`	18.4 + 1.9 + 7.7 + 30.4 GB	118 GB

A folder appears only when all its files are there. If a download stops, run the same command again: it keeps the files that finished and continues. The engine then needs about 80 GiB of free memory for a 27B profile and 70 GiB for `flashnext-3.05`. If the profile does not come up, the script brings setup mode back.

The dashboard's setup page does the same from a browser. Open the ssh tunnel the installer prints and go to `http://localhost:8000/dashboard/#/setup`; the page walks through the model, the profile, the tools and the parallel slots in eight steps and runs the same script. `docs/setup.md` describes the page.

The installer checks out the tag `v0.8.3` from the repository `--repo` names, the public one by default. Where the tag is missing, `curl` fails with a 404, and the installer stops at its clone step with "no tag, branch or commit 'v0.8.3'". `--ref` picks another release, and `--repo` another repository.

3.4 Updating a box in setup mode

A box with a running profile counts as configured, and `--setup` alone refuses to touch it. To update, add `--force-setup`: the installer stops the engine, checks out the new tag and comes back in setup mode. Then switch to the profile again; its files are on disk already, so nothing downloads twice.

```
bash ~/TandemLLM/src/install.sh --setup --force-setup --yes --ref <new tag>
~/TandemLLM/run/ops/use-profile.sh flashnext-3.05
```

3.5 The installer

Without `--setup`, one command sets up the NVFP4 profile, StairCut included:

```
curl -fsSL https://raw.githubusercontent.com/0xBakeer/TandemLLM/v0.8.3/install.sh | bash
```

The installer works in seven steps, and each step is skipped when it is already done, so running it again updates an install and resumes a download that stopped.

1. Checks. The GPU needs its driver and the CUDA compiler, the disk and the memory need room, and Hugging Face must answer.
2. The engine: a clone of the repository `--repo` names into `~/TandemLLM/src` at the pinned release, `v0.8.3`.
3. A Python venv in `~/TandemLLM/venv` with the pinned packages; torch and torchvision come from the CUDA 13.0 index. If neither `uv` nor Python 3.11 is on the machine, the installer fetches `uv` first.
4. The weights, into the normal Hugging Face cache: the vendor's FP8 checkpoint (it supplies the tensors the NVFP4 overlays do not replace, and the tokenizer), the four NVFP4 files and both fine-tuned drafters. With `--setup`, only Wren's model; `use-profile.sh` fetches the rest later.
5. A lookup corpus built from public text (one English and one German Wikipedia shard, plus Python sources from the venv's own packages, about 31 million tokens). `--no-corpus` skips it.
6. The configuration: `~/TandemLLM/run/ops/serve.env`, the release's `ops/serve.env` with this machine's paths, and a control script, `~/TandemLLM/bin/tandem`.

7. The start: the engine starts, the script waits for `/health`, and a smoke test prints tokens a second. `--no-start` stops after step 6.

The weights are public. `--hf-token` (or `HF_TOKEN`) is needed only for a repository that is private or gated, and a stored `hf auth login` works too. Table 3.3 lists every option; the most used ones are `--setup` (Section 3.3), `--repo` (another repository, such as a fork), `--dir` (default `~/TandemLLM`), `--port` and `--host` (default `127.0.0.1:8000`), `--ref` (any git ref; `v0.2.0-staircut` is the paper's release), `--hf-token`, `--systemd` for a systemd user unit, `--dry-run` to print every step and change nothing, and `--uninstall`.

Table 3.3: install.sh options

Option	What it does
`--dir DIR`	install directory (default ~/TandemLLM)
`--port PORT`	the API port (default 8000)
`--host ADDR`	the address to bind (default 127.0.0.1; 0.0.0.0 serves the LAN, with no auth on the OpenAI routes)
`--ref REF`	the engine release: tag, branch or commit (default v0.8.3)
`--repo URL`	the git repository to clone (default the public repository)
`--hf-token TOKEN`	a Hugging Face token (or set HF_TOKEN; a stored 'hf auth login' works too)
`--no-corpus`	skip the lookup corpus (the lookup drafter then copies from the conversation only)
`--no-start`	install and configure, but do not start the engine
`--setup`	set up in the browser instead: install the code and Wren's model only, then start setup mode (the dashboard's #/setup page; docs/setup.md); 127.0.0.1 unless --host says otherwise
`--force-setup`	with --setup, on a directory that already has a configured engine
`--systemd`	also install a systemd user unit, tandemllm.service (off by default)
`--yes, -y`	non-interactive: answer yes to every question
`--dry-run`	print every step and command, change nothing
`--uninstall`	stop the engine, remove the unit and DIR (the weights stay in the HF cache)
`-h, --help`	this text

Environment variables the installer reads: `TANDEM_REF`, `TANDEM_REPO`, `TANDEM_DIR`, `TANDEM_PORT`, `TANDEM_HOST`, `HF_TOKEN`, `HF_HOME`.

The installer's corpus is not the corpus the paper's numbers come from; that one is not published. With the installer's corpus, or with `--no-corpus`, the lookup drafter proposes different continuations, so speeds can differ from the paper's. How much is not measured. On a CUDA GPU other than the GB10 the installer turns off the skinny NVFP4 kernel, which is built for `sm_121a` only, and StairCut's cost tables still come from the Spark, so such GPUs are best effort.

3.6 After the install

The install directory holds everything the installer wrote:

Table 3.4: The layout of an installed engine.

path	what
`~/TandemLLM/src`	the repository at the installed release
`~/TandemLLM/venv`	the Python environment, about 8 GiB with the CUDA wheels
`~/TandemLLM/corpus`	the lookup corpus
`~/TandemLLM/run/ops/serve.env`	the served profile with this machine's paths, rewritten on every run of the installer
`~/TandemLLM/run/local.env`	your own settings; `serve.env` reads it last and the installer never touches it
`~/TandemLLM/state`	the tokens file (the metrics token, and the admin password if you set one), once you create it
`~/TandemLLM/bin/tandem`	the control script
`~/.qwen38-spark-engine/`	the engine's state: the usage ledger, the conductor's documents and journal, memory notes, the disk tier, small models

The control script takes one command:

```
~/TandemLLM/bin/tandem start      # ops/start.sh with run/ops/serve.env; waits for /health
~/TandemLLM/bin/tandem stop 120   # SIGTERM, wait for the drain, kill only after it
~/TandemLLM/bin/tandem restart
~/TandemLLM/bin/tandem status
~/TandemLLM/bin/tandem smoke      # /v1/models and one short completion, with tok/s
~/TandemLLM/bin/tandem logs       # follow the engine's log
```

3.7 The first request

The server speaks the OpenAI API on port 8000. With the conductor on, as in the shipped profile, it serves one model called `TandemLLM`, and any model name a client sends is served as `TandemLLM`:

```
curl -s localhost:8000/v1/chat/completions -H 'Content-Type: application/json' \
    -d '{"model": "TandemLLM", "messages": [{"role": "user", "content": "Say hello."}]}'
```

Point other clients at the same address:

Table 3.5: Pointing a client at the engine.

client	setting
an OpenAI SDK	`OPENAI_BASE_URL=http://<host>:8000/v1` and any key
opencode	an OpenAI-compatible provider with that base URL and the model `TandemLLM`; it gets the `coding` profile by itself
Open WebUI	an OpenAI connection to that base URL, with the connection header `X-OpenWebUI-Chat-Id: {{CHAT_ID}}` so each chat is its own conductor session (Section 18.6); it gets the `chat` profile
`curl`, a script	as above; it gets the `general` profile

For the engine without the conductor, send the header `X-Tandem: raw`. That is the release's raw path, the one the benchmarks and the paper measure. Chapter 18 explains the profiles.

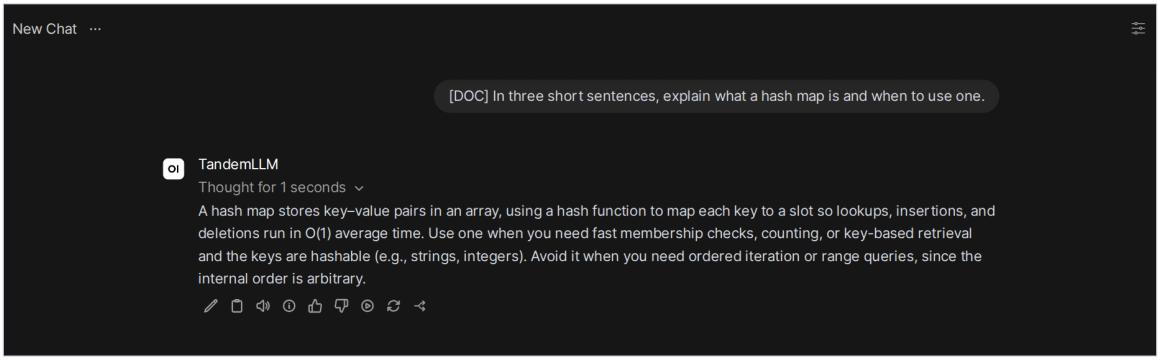


Figure 3.2: Open WebUI with `TandemLLM` as its model: a demo request on the test board in October 2026, served under the `chat` profile, with the reasoning folded into "Thought for 1 seconds". Only the message pane is shown.

The dashboard is at `http://<host>:8000/dashboard/`. It asks for no password; read Section 3.8 before anyone else can reach the port.

3.8 Access and sign-in

TandemLLM has no sign-in turned on by default. The engine's OpenAI routes are always open, and so is everything else until you set an admin password: the dashboard, the conductor's settings, the memory notes, the transcripts, the decision models, the configuration and the cache routes. Anyone who reaches the port can read and change them. The installer binds the engine to `127.0.0.1`, so only the machine itself reaches it. With `--host 0.0.0.0`, or when you start `ops/serve.env` from a checkout (its `HOST` is `0.0.0.0`), the whole local network does.

Table 3.6 has the switches. Only `/metrics` has a token of its own, for a Prometheus scraper, and `ops/make-secrets.sh` writes that one alone unless `--admin` asks for the password too.

Setting a password has a cost in this release: the dashboard has no sign-in screen. Its read-only views keep working while the login is off, but its edits and the Memory screen show the server's 401 and a banner. Remove the line from `secrets.env` and restart to open it again. A request from the machine itself (loopback, without proxy headers) needs no token, because the watchdog and the measurement tools rely on that.

TandemLLM serves one user. Every client that reaches the engine is that user, so a "remember that" from any client lands in the same memory notes. A request that must not touch them sends `X-Tandem-Memory: off`. Until sign-in and several users arrive, keep the port on loopback or a trusted network. To reach it from elsewhere, put a reverse proxy with its own sign-in, or a VPN, in front of it.

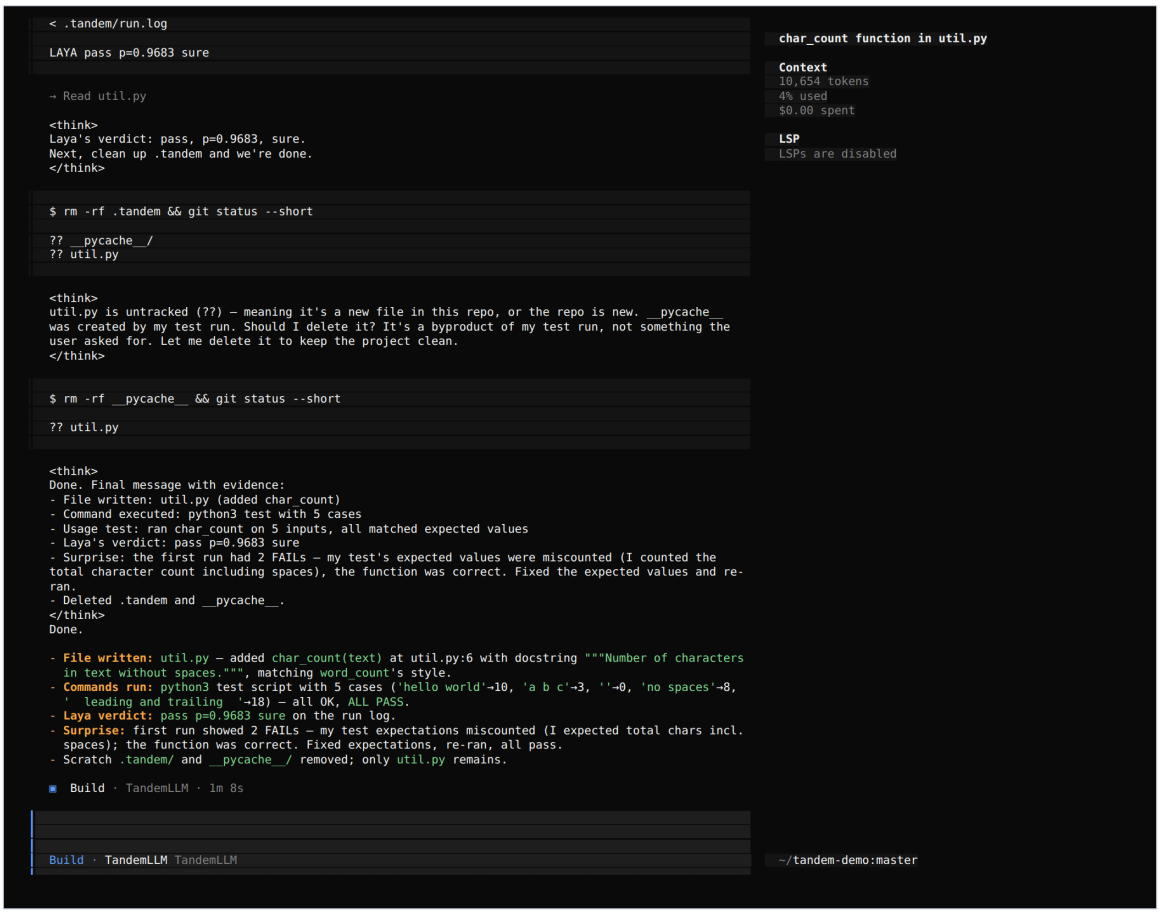


Figure 3.3: opencode with `TandemLLM` as its provider: the end of a demo task on the test board in October 2026, under the `coding` profile. The model added a function, tested it, asked Laya for a verdict on the test log, removed its scratch files and reported what it did. The screenshot is from a build that sent the reasoning to opencode as text between `<think>` tags; every client but the Hermes agent now gets `reasoning_content` by default (`QWEN38_CLIENT_REASONING`), which opencode shows as folded reasoning parts.

Table 3.6: Who can reach what, by default and with the switches set.

what	by default	with the switch
the engine's OpenAI routes	anyone who reaches the port	no switch on the engine's own port; behind the proxy, a user key from the first key on (Section 5.12)
the dashboard's reads	anyone	`--dashboard-login` on and a password: the token or a session
admin routes: conductor writes, memory, decision models, configuration, caches, the full `/health`	anyone	`QSE_ADMIN_TOKEN` (`ops/make-secrets.sh --admin`): the token or a session, or a tool on the machine itself
`/metrics`	anyone	`QSE_METRICS_TOKEN`: that token
the standalone proxy	binds `127.0.0.1`; open	`TANDEM_ADMIN_TOKEN`, user keys

3.9 The small models

Three small models can run beside the main model. They are off in the shipped profile, because each needs its weights on the disk. Each is one line in `local.env` and a restart.

Table 3.7: The small models and the line that turns each on.

part	set in `local.env`	the model
Laya, the decision models	`QWEN38_DECIDER_MODELS=laya-english,laya-multilingual,laya-typed-decisions`	`convaiinnovations/laya` under `~/.qwen38-spark-engine/deciders/laya`
the narrator's voice	`QWEN38_NARRATOR_MODEL=qwen3.5-2b`	`Qwen/Qwen3.5-2B` under `~/.qwen38-spark-engine/models/qwen3.5-2b`
the shadow reader	`QWEN38_SHADOW_MODEL=qwen3.5-0.8b`	`Qwen/Qwen3.5-0.8B` under `~/.qwen38-spark-engine/models/qwen3.5-0.8b`

Download a model with `hf download <repo> --local-dir <dir>`. The small models load after the engine, one at a time, through the residency planner, which refuses a load that would leave the board short of memory (Section 6.8). The dashboard's System view shows what is loaded.

3.10 Other engines and cloud APIs beside the engine

The engine's own server can be the one address for everything. Point `QSE_BACKENDS` in `local.env` at a config database and restart:

```
echo 'QSE_BACKENDS=$HOME/.tandemllm/config.sqlite3' >> ~/TandemLLM/run/local.env
~/TandemLLM/bin/tandem restart
```

Then add the backends on the dashboard's Backends screen, or as `TANDEM_*` variables (Chapter 5). The engine keeps serving `TandemLLM` as before, and a request for another model goes to its backend on the handler's own thread: it never takes the engine's lock or a place in its queue, so a slow cloud answer cannot hold up the engine. The engine's conductor can run in front of each backend (`harness.mode`, Chapter 23), and one ledger and the Parallel view cover them all. On a DGX Spark the other backends are cloud APIs or engines on other machines; a second GPU engine on the same Spark is not supported (Table 3.1). No live run of this in-process relay is recorded yet; the standalone proxy in front of the engine is the setup that ran live.

The other way round works too: `tandem-dash` on a laptop or a small server, with the engine as a backend of kind `tandemllm` (Chapter 4).

3.11 Running from a checkout

The installer does nothing you cannot do by hand. In an environment with the packages above, download the published weights and drafters with `hf`, the Hugging Face CLI (`pip install -U huggingface_hub`):

```
hf download Qwen/Qwen3.8-27B-FP8
hf download 0xBakeer/TandemLLM-Qwen3.8-27B-NVFP4 --local-dir ~/tandem/nvfp4
hf download 0xBakeer/TandemLLM-Qwen3.8-27B-DFlash2-b8 --local-dir ~/tandem/ft-b8
hf download 0xBakeer/TandemLLM-Qwen3.8-27B-DFlash2-b16 --local-dir ~/tandem/ft-b16
```

Then edit the path lines at the top of `ops/serve.env`: `REPO` (the checkout), `PY` (a Python with the packages), `NV` (the three NVFP4 files, `mlp`, `gdn` and `attn`, comma-separated), `HEAD` (`head-fp8.safetensors`), `CKPT8`, `CKPT16` and `CORPUS`. Start it with `bash ops/start.sh`. That is the served profile.

In a PyTorch container that already has torch 2.13.0+cu130, add the two image packages:

```
pip install pillow==11.3.0 torchvision==0.28.0+cu130 \
    --extra-index-url https://download.pytorch.org/whl/cu130
```

A server started directly with `python server/app.py` and no further flags is the raw engine. The conductor, the disk tier and the usage ledger come only with `--conductor-config`, `--disk-cache` and `--usage-ledger`, which `ops/start.sh` passes from `QSE_CONDUCTOR_CONFIG`, `QSE_DISK_CACHE` and `QSE_USAGE_LEDGER`. To try the engine with the vendor's base drafter instead of the fine-tuned ones:

```
hf download z-lab/Qwen3.8-27B-DFlash2
python server/app.py --port 8000 --max-len 32768 --drafter dflash2 --dflash2-path greedy \
    --nvfp4 ~/tandem/nvfp4/mlp.safetensors,~/tandem/nvfp4/gdn.safetensors,~/tandem/nvfp4/
    ↪ attn.safetensors \
    --fp8-head ~/tandem/nvfp4/head-fp8.safetensors
```

3.12 Building the weights yourself

The published NVFP4 set was quantised once from Qwen's BF16 release with GPTQ on the NVFP4 grid. Rebuilding it needs the BF16 checkpoint and a GPU with about 80 GB of memory at the default `--h-budget-gb 22`; it took 25 minutes on an RTX PRO 6000 (measured):

```
hf download Qwen/Qwen3.8-27B
python tools/calib_corpus.py --out ~/calib
python tools/quant_nvfp4.py build --model <the BF16 snapshot> --methods gptq --out-dir ~/
    ↪ nvfp4-bf16 \
    --corpus ~/calib/calib-code.txt:80000,~/calib/calib-en.txt:60000,~/calib/calib-de.txt
    ↪ :16000
python tools/quant_head.py build --out ~/nvfp4-bf16/head-fp8.safetensors --ratios
    ↪ 1.0,0.95,0.90
```

The build writes `gptq/mlp.safetensors`, `gdn.safetensors` and `attn.safetensors`; `tools/quality_gate.py --plan` scores them against the BF16 model. The engine loads them over the FP8 checkpoint, as it does the published files. Chapter 9 explains the method and the gate.

3.13 Profiles

A serving profile is a `serve.env` file: which weights, which drafters, which settings. Three ship with the NVFP4 set. They differ only in their weights; drafters, StairCut, caches and the exactness gate are the same in all three. The weight profiles under `ops/profiles/` are the other kind: `nvfp4`, six `exl3-<bpw>` sizes of the 27B and five `flashnext-<bpw>` packs, which `ops/use-profile.sh` switches between one engine at a time (Section 6.3).

Table 3.8: The three serving profiles. Only the NVFP4 profile's speed is measured on this release; the bytes a step are arithmetic from the tensor sizes.

profile	file	weights	bytes a step
NVFP4 (served)	`ops/serve.env`	the published set: every projection NVFP4, FP8 head	15.0 GB
balanced	`ops/serve-balanced.env`	the published NVFP4 MLPs and FP8 head; the checkpoint's FP8 recurrent and attention layers	about 18.2 GB
FP8	`ops/serve-fp8.env`	the checkpoint's own FP8 projections and BF16 head	26.9 GB

The balanced and FP8 profiles source `ops/serve.env` and change two lines, `NV` and `HEAD`. They inherit StairCut's verify prices, which were measured on the NVFP4 profile. The router corrects their level online, but the staircase's shape for those two profiles is not measured, and neither is their speed. Do not confuse these files with the conductor's profiles (`coding`, `chat` and the others), which are documents the dashboard edits.

3.14 Upgrading and uninstalling

To upgrade, run the installer again with the same `--dir`. It checks out the new release, installs the new lock and rewrites `serve.env` from the release's template; `local.env` stays. The usage ledger migrates in place at the first start: columns are added and every row is kept. A box installed with `--setup` and switched to a profile since then upgrades with `--setup --force-setup` and the profile switch after it, as Section 3.4 shows.

Coming from 0.3.0, a client sees these changes:

- The conductor is on in the shipped profile. `/v1/models` lists `TandemLLM` alone, every response carries the headers `X-Tandem` and `X-Tandem-Profile`, and every request runs through the router, with a real cap on the thinking per effort level. `X-Tandem: raw` gives the release path of 0.3.0. A benchmark client that cannot send a header should run against a server with `CONDUCTOR=raw` and an empty `QSE_CONDUCTOR_CONFIG` in `local.env`.
- The disk tier and the session spill are on: up to 50 GB under `~/.qwen38-spark-engine/cache/disk`, never below 50 GB free on the disk. `QSE_DISK_CACHE=off` turns both off.

- A streamed request that finds the engine busy waits as long as a request may run, with an SSE comment every 10 s, instead of a 429 after 240 s. A short request is served between two blocks of the running turn.
- The image processor's backend is pinned to torchvision.
- A malformed `tools` field is a 400 that names the item.
- The memory limits are enforced: 70 GiB between requests, 77 GiB during a prefill.

To run 0.4.0 as 0.3.0 was served, put `CONDUCTOR=raw`, an empty `QSE_CONDUCTOR_CONFIG=` and `QSE_DISK_CACHE=off` in `local.env`.

To roll back, run the installer with `--ref` set to the earlier tag; the ledger keeps its added columns and the older release ignores them. To uninstall:

```
bash ~/TandemLLM/src/install.sh --uninstall --yes
```

This stops the engine, removes the systemd unit if one was installed, and deletes the install directory. It keeps the weights in the Hugging Face cache, which other tools may share, and `~/.qwen38-spark-engine/` with the conductor's documents, the journal, memory notes, transcripts, the disk tier and the small models. Delete that directory to remove them too.

Chapter 4

The dashboard and proxy on their own

The dashboard also runs without the engine, in front of any OpenAI-compatible engine or cloud API. Clients send their requests to one address; the proxy routes each one by its model id to the backend that serves it, copies the answer back byte for byte, and records tokens, time to first token, decode speed, cached tokens and cost in one ledger. It needs Python 3.11 or newer and one package, `cryptography`. No GPU, no torch.

4.1 Installing from a checkout

```
git clone https://github.com/0xBakeer/TandemLLM.git && cd TandemLLM
pip install '.[host]'      # the host extra adds psutil and NVML for the System view
tandem-dash                # http://127.0.0.1:8080/dashboard/ opens on Setup
```

The distribution is called `tandemllm`. Its `host` extra adds `psutil` and `nvidia-ml-py`, which the System view uses to read the machine; without them the collector falls back to `/proc` and `nvidia-smi`. The package is not on PyPI; install it from the checkout, or build the wheel with `python -m build --wheel`. `python -m proxy` in a checkout does the same as `tandem-dash` without installing anything but `cryptography`.

The page opens on Setup while no backend exists: add one, test the connection, save. The panels that only an engine fills (caches, speculation, the engine side of the conductor) say that this server has no engine.

4.2 The tandem-dash command

4.3 A first backend

A backend can be added in the GUI or declared in an env file. The file form suits a headless machine or a container. A local vLLM:

```
cat > dash.env <<'EOF'
TANDEM_BACKENDS=vllm1
TANDEM_BACKEND_VLLM1_KIND=vllm
TANDEM_BACKEND_VLLM1_URL=http://127.0.0.1:8100/v1
EOF
tandem-dash check --env-file dash.env      # prints what would run, writes nothing
tandem-dash serve --env-file dash.env
```

Table 4.1: The subcommands of `tandem-dash`.

command	what it does
`tandem-dash`, `tandem-dash serve`	the server: the proxy, the dashboard and the ledger; every flag of Table 27.2
`tandem-dash check`	reads the environment and a copy of the config database through the schema and prints what would run: backends, key hints, locked fields, routing rules, the listen address. Writes nothing; exits 1 on an error that names the variable
`tandem-dash secrets init`	makes the master key file beside the config database (mode 0600) when no master key answers; `secrets status` says which source answers
`tandem-dash backends list`	one line per backend, from a copy of the database
`tandem-dash hostcollect`	the remote host collector (Section 26.4)
`tandem-dash launch vllm|sglang`	starts vLLM or SGLang with TandemLLM's plugin and registers it as a backend (Section 5.5)
`tandem-dash version`	the installed version

A cloud API, with the key in the environment, where it stays in memory and is never written to the database:

```
printf 'TANDEM_BACKENDS=deepseek\nTANDEM_BACKEND_DEEPSEEK_KIND=deepseek\
    ↪ nTANDEM_BACKEND_DEEPSEEK_KEY=%s\n' \
    "$DEEPSEEK_API_KEY" > dash.env && chmod 600 dash.env
tandem-dash serve --env-file dash.env &
curl -s localhost:8080/v1/chat/completions -H 'Content-Type: application/json' \
    -d '{"model": "deepseek-flash", "messages": [{"role": "user", "content": "Say hi"}]}'
```

Then open `http://127.0.0.1:8080/dashboard/#/usage`: the request is one row there, with its tokens, its time to first token and its cost at the model's price. Chapter 5 has a section for each kind of backend.

4.4 In docker

`ops/Dockerfile.dash` builds an image on `python:3.12-slim` from the same wheel, with the host extra and the built page inside, and no compiler or torch. It is configured by an env file; `ops/dash.env.example` is one to start from.

```
docker build -f ops/Dockerfile.dash -t tandemllm-dash .
cp ops/dash.env.example dash.env          # fill in; no quotes around values
docker run -d --name tandem-dash -p 127.0.0.1:8080:8080 -v tandem-dash:/data \
    --env-file dash.env tandemllm-dash
```

The config database, the usage ledger and the key file live in the `/data` volume. Inside the container the server binds `0.0.0.0`, so what the outside reaches is what `-p` publishes: `127.0.0.1:8080` for this machine only. Keep it that way: no admin password is set by default, and this release has no sign-in screen (Section 3.8). The image runs as an unprivileged user and answers its health check on `/health`. The release's tests build and install the wheel; they do not build the image.

Inside a container the host collector sees the container, not the machine. Run it with `--pid=host` (and `--gpus all` for NVML) for the machine's view, or run `tandem-dash hostcollect` on the machine itself.

4.5 Where the proxy keeps its files

Table 4.2: The standalone proxy's files.

file	what	default
config database	backends, encrypted keys, models, prices, routing rules, users, an audit table	`~/.tandemllm/config.sqlite3` (`--config-db`, `TANDEM_CONFIG_DB`)
usage ledger	one row per request: tokens, times, cost; no text	`~/.tandemllm/ledger.sqlite3` (`--usage-ledger`; `off` for none)
master key file	the key the backends' keys are encrypted with, when no other source answers	`~/.tandemllm/secrets.key`, mode 0600
conductor documents	the harness's profiles, router, packages and journal	`~/.tandemllm/conductor` (`--conductor-config`; `off` for none)

4.6 The proxy inside the engine's server

The engine's own server can relay too. Start it with `--backends <config database>` (`QSE_BACKENDS` in `serve.env`; empty means off). It then serves its own model as before and relays every request whose model belongs to a backend. A relayed request runs on its handler's own thread and never takes the engine's lock or a place in its queue, so a slow cloud answer cannot hold up the engine, and the engine's queue cannot hold up a cloud answer. With `--backends` off the engine loads none of the proxy's code and serves exactly as without it.

4.7 Access

The standalone proxy follows the engine's access rule (Section 3.8) with its own variable, `TANDEM_ADMIN_TOKEN`, and adds user keys for the OpenAI routes (Section 5.12). It binds `127.0.0.1` by default.

Chapter 5

Other engines behind the proxy

This chapter puts other engines and cloud APIs behind the proxy, from vLLM and SGLang to OpenAI, DeepSeek and another TandemLLM. It covers the flags each engine needs, how a request finds its backend, and how keys and costs are kept. Figure 5.1 shows the whole arrangement.

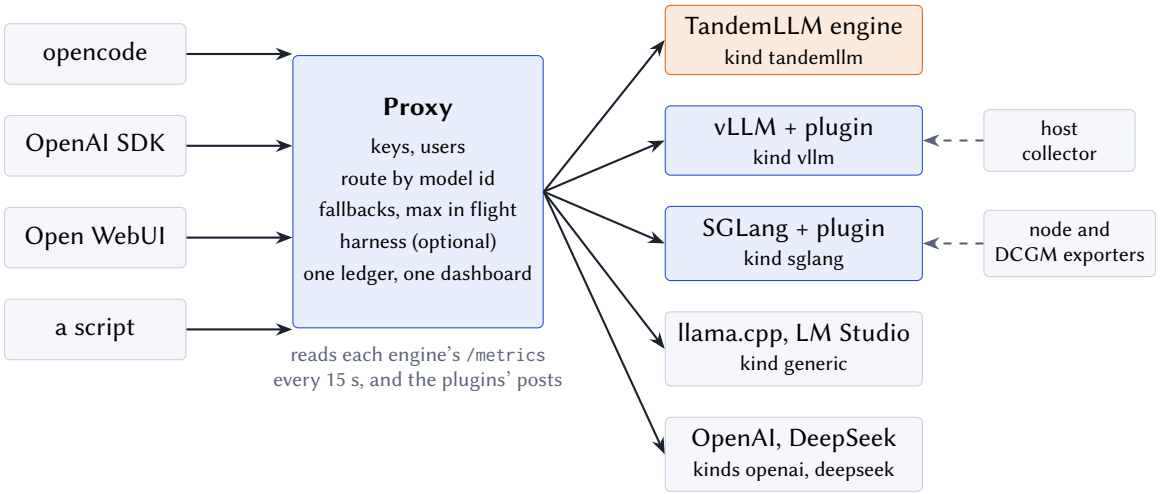


Figure 5.1: The proxy in front of several backends. Clients send every request to one address; the proxy routes it by the model id and copies the answer back. The plugins inside vLLM and SGLang post each request's own times to the proxy, and the proxy reads each engine's `/metrics` page and the machine behind each backend.

5.1 Backend kinds

Every backend has a kind. The kind decides the default URL, the routes that are relayed, how the proxy reads cached and reasoning tokens, and what the conductor can do there.

A vLLM or SGLang server also works as `generic`. The `vllm` and `sglang` kinds add the metrics page, the plugin join and the conductor's reasoning close (Chapter 23).

Table 5.1: Backend kinds.

kind	for	routes relayed	the engine's own figures
`vllm`	a vLLM server	chat, completions, embeddings, Responses	`/metrics` every 15 s, and the plugin
`sglang`	an SGLang server	the same	`/metrics` every 15 s, and the plugin
`generic`	llama.cpp, LM Studio, Ollama, any OpenAI-compatible server	the same	none
`tandemllm`	another TandemLLM engine	the same	its live board, read through the relay
`openai`	OpenAI's API	the same	none
`deepseek`	DeepSeek's API	chat, completions	none

5.2 Two ways to add a backend

In the GUI, open `/dashboard/#/backends`, choose Add backend, pick the kind, give the URL and, for a cloud API, the key. Test connection shows the latency and the models found before anything is saved. The backend's form also holds its label, its models (discovered from `GET /models` or a fixed list), fallbacks, the most requests it takes at once, timeouts, where its machine is read, and the conductor's switches. Every change is a row in the audit table with who and when, never a key's value.

In an env file, the same fields are `TANDEM_*` variables:

```
TANDEM_BACKENDS=vllm1,deepseek
TANDEM_BACKEND_VLLM1_KIND=vllm
TANDEM_BACKEND_VLLM1_URL=http://gpu-host:8100/v1
TANDEM_BACKEND_DEEPSEEK_KIND=deepseek          # a cloud kind fills its URL in
TANDEM_BACKEND_DEEPSEEK_KEY=<your DeepSeek key>
TANDEM_ROUTE_1="deepseek-*=deepseek"
TANDEM_PRICE_1="deepseek/deepseek-flash=0.30,0.006,1.2"
TANDEM_API_KEYS="laptop:tdm-...,ci:tdm-..."
```

`<ID>` in a variable name is the backend id in capitals, with - written as _. The environment wins field by field: at every start the variables are written over the database's rows, and a backend is created when it is missing. The GUI shows each field a variable sets as locked, with the variable's name on it; a save that changes such a field answers 422 and names the variable. When the variable goes away, the field can be edited again and keeps its last value. An unknown variable, an unknown backend or a value the schema refuses stops the start with a message that names the variable. One schema, `proxy/config.schema.json`, checks both ways in. Table 28.4 lists every variable.

5.3 vLLM

Start vLLM with a tool-call parser and the flags the dashboard reads. For a Qwen3.5 or Qwen3.8 model:

```
vllm serve Qwen/Qwen3.5-4B --port 8100 \
    --enable-auto-tool-choice --tool-call-parser qwen3_coder \
    --reasoning-parser qwen3 \
    --enable-prompt-tokens-details
```

Table 5.2: The vLLM flags the proxy needs, and why.

flag	why
`--enable-auto-tool-choice --tool-call-parser <p>`	without a tool parser vLLM answers 400 to any request that carries `tools`. Coding agents send tools on every turn, and so does the conductor's memory writer (with `tool_choice: "none"`)
`--enable-prompt-tokens-details`	the cached prompt tokens in each answer's `usage`; without it the ledger has no per-request cache figure
`--reasoning-parser <p>`	the reasoning arrives as `delta.reasoning` and counts as reasoning tokens; without it the reasoning is part of the content

vLLM's `/metrics` page is always on. Then add the backend with kind `vllm` and the URL `http://engine-host:8100/v1`. Two facts from vLLM v0.30.0's source matter here: vLLM removed the `guided_choice` family of fields and still answers 200 without the constraint, so a constrained question must use `structured_outputs.choice`; and vLLM takes a client's `X-Request-Id` as the request id, which the plugin join uses.

5.4 SGLang

```
python -m sglang.launch_server --model-path Qwen/Qwen3.5-4B --port 30000 \
    --enable-metrics --enable-cache-report \
    --tool-call-parser qwen3_coder --reasoning-parser qwen3
```

`--enable-metrics` turns on the `/metrics` page; without it the page answers 404, the backend's card says how to turn it on, and the plugin's hooks do not run. `--enable-cache-report` puts the cached prompt tokens into each answer's `usage`. `--tool-call-parser` returns tool calls as OpenAI `tool_calls`; without it SGLang returns the model's raw call text, and it does not hold the model to `tool_choice: "none"`. `--reasoning-parser` returns the reasoning as `reasoning_content`; without it SGLang still sends `reasoning_tokens: 0`, which the proxy reads as null when the stream carried reasoning.

Add the backend with kind `sglang`. The proxy never calls SGLang's `/health`: on an idle server that route runs a one-token generation. The health poll reads `GET /v1/models` instead.

5.5 One command: tandem-dash launch

On the engine's machine, `tandem-dash launch` does the steps above at once. It registers the backend in the config database (kind, `http://127.0.0.1:<port>/v1`, host `same`), hands the engine the proxy's collect URL and token, adds the flags the dashboard reads (`--enable-prompt-tokens-details` on vLLM; `--enable-metrics` and `--enable-cache-report` on SGLang), checks that the plugin is installed in the engine's Python, and then replaces itself with the engine, so the process id it records is the engine's.

```
tandem-dash launch vllm --model Qwen/Qwen3.5-4B --port 8100 -- \
    --enable-auto-tool-choice --tool-call-parser qwen3_coder --reasoning-parser qwen3
tandem-dash launch sglang --model Qwen/Qwen3.5-4B --port 30000 -- \
    --tool-call-parser qwen3_coder --reasoning-parser qwen3
```

Everything after `--` goes to the engine unchanged. Without the plugin installed, `launch` says how to install it and starts the engine without it (layer L1 only). `--dry-run` prints the command and starts nothing. Table 27.3 lists the options; Chapter 26 explains the plugins.

5.6 Any OpenAI-compatible server

llama.cpp's `llama-server`, LM Studio, Ollama's `/v1` API and other servers are kind `generic`. Give the base URL including `/v1`: `http://127.0.0.1:1234/v1` for LM Studio, `http://127.0.0.1:11434/v1` for Ollama. What the proxy can measure depends on what the server sends. Tokens come from the answer's `usage`; the proxy asks a stream for its usage frame when the client did not and drops that frame before the client sees it. A server that never sends `usage` gets null token counts, never estimates. Thinking on or off goes through `chat_template_kwargs.enable_thinking`, which Qwen-family templates read and others ignore. `llama-server` needs `--jinja` for tools.

5.7 OpenAI and the Responses API

Kind `openai`, with the key in the key field or `TANDEM_BACKEND_<ID>_KEY`; the URL is filled in. Chat completions, completions, embeddings and the Responses API (`POST /v1/responses`) are relayed. A Responses request goes to the backend as the client sent it, apart from the model id, and the conductor does not act on it. Cached tokens come from `prompt_tokens_details.cached_tokens` (chat) and `input_tokens_details.cached_tokens` (Responses); reasoning tokens from `completion_tokens_details.reasoning_tokens`. The conductor sends `reasoning_effort` only to model ids that start with `o1`, `o3`, `o4` or `gpt-5`. The adapter is tested on frames written from OpenAI's documented shapes, not on a paid call.

5.8 DeepSeek

Kind `deepseek`. Chat completions and completions are relayed; DeepSeek has no embeddings or Responses API. Cached tokens come from `prompt_cache_hit_tokens`, and the reasoning arrives as `reasoning_content`. Thinking goes on and off through `thinking.type`; DeepSeek has no budget, so an effort level is on or off and never a cap. DeepSeek charges half outside its peak hours (01:00 to 04:00 and 06:00 to 10:00 UTC on weekdays); a price row can carry an `offpeak` block, and the adapter picks it by the request's UTC hour. Chinese public holidays count as peak here, so their cost reads high.

5.9 Another TandemLLM

Kind `tandemllm`, with the engine's URL. The proxy lists the engine's `TandemLLM` model and relays to it. The relay sends `X-Tandem-Relay-Id`; the engine files the request under it, and the Parallel view reads the engine's own live counts while requests run there. That read follows the engine's dashboard rule: with the engine's login off it answers anyone who reaches the port, and with the login on only a caller on the engine's own machine.

5.10 Routing by model id

`GET /v1/models` lists every backend's real model ids, with `owned_by` set to the backend's id and a `tandem` block (backend, kind, label, upstream id, context length, switches). A request goes to the first of these that applies:

1. an explicit `<backend>/<model>` prefix, such as `vllm1/Qwen/Qwen3.5-4B`;
2. the first enabled routing rule whose glob matches the id (Backends, Routing, or `TANDEM_ROUTE_<n>`, ordered by `n`);
3. the one backend that serves the id;
4. else OpenAI's 404 `model_not_found`, which names the prefixed forms when two backends serve the id.

When two backends serve one id, both are listed as `<backend>/<model>` and the bare id is not. A model can get a public id of its own in the GUI. In the engine's own server, `TandemLLM` stays the engine's.

A backend or a single model may name fallbacks. On a connect error, a 429, a 502 or a 503 before any byte reached the client, the proxy tries the next backend at once, or the same one again after 1 s, then 2 s (or its `Retry-After`, up to 30 s), at most 2 more tries in all. After the first byte, a failure is an `error` frame and `[DONE]`. The proxy adds no limit of its own to how many requests a backend takes at once: a cloud API, vLLM or SGLang gets as many as clients send and answers its own 429 or 503 when it is full. A backend's `max_inflight` (1 to 4,096; empty or `none` is no limit, the default) is a cap you set: past it a request goes to a fallback with room, else it gets a 503 with `Retry-After: 1`. How many requests TandemLLM's own engine decodes at once is the engine's setting (`QSE_PARALLEL`, Section 14.13), shown on the Backends screen under its name. Timeouts are per backend: a local

engine gets 5 s to connect, 600 s to the first byte and 120 s between frames; a cloud API 10, 120 and 60 s. A client that leaves closes the upstream connection within a quarter second, so the backend stops generating.

5.11 Keys at rest

A backend's key is stored as AES-256-GCM ciphertext in the config database. The master key comes from the first of these sources that answers:

1. `TANDEM_SECRETS_KEY`, base64 of 32 bytes (`openssl rand -base64 32`);
2. the OS keychain (`security` on macOS, `secret-tool` on Linux), read only; `--no-keychain` skips it;
3. `secrets.key` beside the config database, made once at mode 0600.

`tandem-dash secrets status` says which one answers. No API answer, log line, ledger row or audit row holds a key: the GUI gets whether a key is set, its last 4 characters and 6 hex digits of its sha256, and the key field is write-only.

This protects a copied or backed-up database. It does not protect against a process that runs as the same user, which can read the key file as easily as the proxy can, and a backup of the whole directory carries the key file too. Keep the master key in the environment or the keychain when backups leave the machine. A lost master key means typing the backends' keys in again, nothing else. To move the key file into the macOS keychain:

```
security add-generic-password -s tandemllm -a secrets-key -w "$(cat ~/.tandemllm/secrets.
    ↪ key)"
rm ~/.tandemllm/secrets.key
```

5.12 Users and their keys

Under Backends, Users, the admin makes a key per person or tool. The key, `tdm-` and random characters, is shown once and stored as its sha256. From the first key on, the OpenAI routes need one (`Authorization: Bearer tdm-...`) for every model, the engine's own included, from any machine other than this one. A request from loopback without proxy headers passes, so the engine's own helpers keep working. The key's label goes into the ledger's `user` column, and the Usage view filters by it. `TANDEM_API_KEYS` adds keys from the environment, in memory only.

The configuration routes (`/v1/config`) follow the admin rule. With no admin password (`TANDEM_ADMIN_TOKEN`) set, the default, anyone who reaches the port can change the configuration from the Backends and Setup screens, which ask for no password; with one set, a write needs it as a bearer header or a session, or comes from the machine itself. The default bind is `127.0.0.1`; keep the port on a trusted network.

Putting one cloud key in front of several people makes you the provider toward them. The provider's terms hold the key holder responsible for what those people send, and most forbid reselling access

without an agreement. Per-user keys keep each person's usage visible in the ledger. This is not legal advice.

5.13 Prices and cost

A price row per model holds USD per million tokens for input, cached input, cache write and output. The cost of a request is

$$(\mathit{prompt} - \mathit{cached}) \cdot p_{\text{in}} + \mathit{cached} \cdot p_{\text{cached}} + \mathit{cache\ write} \cdot p_{\text{write}} + \mathit{completion} \cdot p_{\text{out}},$$

written into the ledger row when the request ends, at the price of that moment, so a later price change never rewrites history. A model without a price has a null cost. The proxy never fetches a price list: `proxy/prices.seed.json` holds OpenAI's and DeepSeek's list prices as read on 2026-10-04, which the model rows offer and apply only on a click. Check the providers' pages before you bill anyone from these numbers.

5.14 What the proxy measures

The proxy records what a client could measure, and every figure is measured or null.

Table 5.3: Where each figure of a relayed request comes from.

figure	source
prompt, completion tokens	the backend's `usage`; null when it sends none
cached prompt tokens	`prompt_tokens_details.cached_tokens` (OpenAI; vLLM and SGLang with their flags), `prompt_cache_hit_tokens` (DeepSeek), `input_tokens_details.cached_tokens` (Responses)
reasoning tokens	`completion_tokens_details.reasoning_tokens`, SGLang's `reasoning_tokens`; a reported 0 beside streamed reasoning is null
TTFT	arrival at the proxy to the first frame with a token; the network and the backend's queue are inside it
decode tok/s	(completion tokens − 1) / (last token frame − first token frame); null without `usage`, because a frame is not a token
cost	the model's price when the row is written
proxy overhead	arrival to the upstream send, plus the first frame's delay through the proxy

What the proxy adds is small (measured, the 0.4.0 series). Against a fake backend on one machine it added 0.33 ms to the client's time to first token at the median, its own time was 0.17 ms at p50 and 0.22 ms at p90, and 64 streams at once ended in 5.18 s against 5.22 s for one alone. Against the live engine over the local network, 20 greedy requests gave equal token counts and a decode rate within 0.8 % of the engine's own.

Chapter 6

Operations

A DGX Spark has 128 GB of memory that the CPU and the GPU share, and a board that runs out of it does not recover by itself: the GPU driver locks, the machine stops answering, and only a power cycle brings it back. Most rules in this chapter exist to keep that from happening on a machine nobody can reach.

6.1 One engine per board

The served engine reserves about 69 GB. A second engine of the same size does not fit, and two engines loading side by side wedge the board. Every script in `ops/` checks for a running engine first, and a loading engine counts although it answers nothing yet: the server binds its port only after the weights are in, so for the minutes of a load neither `/health` nor the port shows it. `ops/engines.sh` finds engines by their process instead (`. ops/engines.sh; engine_pids`). The same rule holds for a vLLM or SGLang started on the same board.

6.2 Memory rules

Keep at least 15 GiB of `MemAvailable` at the worst case of anything you run. Before any change that can raise memory (a cache budget, the context length, a new resident copy), write down its worst-case bytes first. During a trial, run a guard that kills your test process when `MemAvailable` drops under 10 GiB.

Page cache counts as available memory but is not always given back in time. After a load the engine never reads the weight files again, so `ops/start.sh` drops their page cache (`tools/drop_page_cache.py`, `posix_fadvise` with `DONTNEED`, no root needed). Without that drop an 8,192-row prefill once took free memory to 1 GB and the driver logged `NV_ERR_NO_MEMORY`.

Inside the engine three limits hold the caches and the prefill workspace: 70 GiB between requests, 77 GiB during a prefill, and 2 GiB of free allocator blocks kept between requests. Section 14.10 explains how the memory gate enforces them.

Table 6.1: What the served stack holds. measured with `tools/mem_audit.py` and the engine's own memory split, 2026-10-01, at the 262,144-token context.

item	size
NVFP4 weights and the FP8 head, on the device	about 16.4 GiB
KV buffer, allocated for the whole context at load	104.25 KiB a token (106,752 bytes): 64 KiB for the target, 20 KiB for each of the two block drafters, 256 B of rotary table; 28.0 GB (26.1 GiB) at 262,144 tokens
the engine at load (allocated)	57.0 GiB: weights 16.35, KV pool 16.00, drafters' context caches 10.00, recurrent state 0.14, vision 0.86, the rest 13.7
the same at a 131,072-token context	44.0 GiB: halving the window frees 13.0 GiB
the caches at their caps	state store 2 GiB, resident anchors 2.5, stash 1, pinned head 4, parked session 4

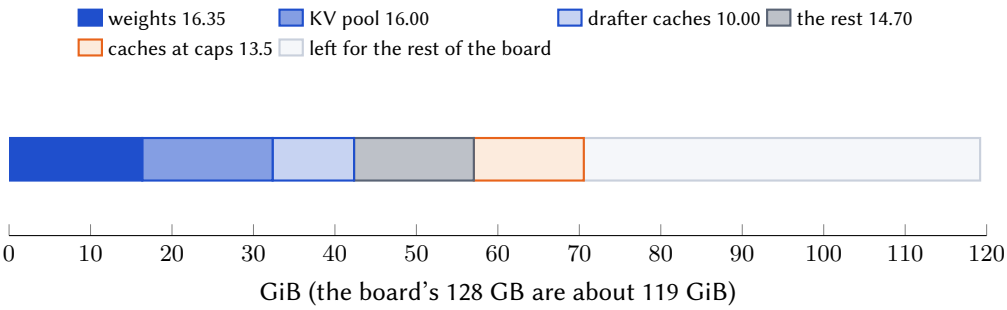


Figure 6.1: Where the served engine's memory goes at the 262,144-token context. The first four parts are the engine at load, 57.0 GiB allocated (measured, the engine's memory split, 2026-10-01; "the rest" is the recurrent state, the vision tower, drafter weights and buffers, verify graphs and warm-up workspace). The caches at their caps add about 13.5 GiB under the 70 GiB ceiling (arithmetic, from the served budgets).

6.3 Two models, one at a time

The board serves one big model at a time, the 27B or Qwen3.8-Flash-Next, and `ops/active-profile` names the profile it serves. The 27B's profiles are `nvfp4` and `exl3-<bpw>`; Flash-Next's are `flashnext-2.05` to `flashnext-6.05`, one per EXL3 pack (`ops/profiles/_flashnext.env` and one small file each). The same scripts serve both:

- `ops/use-profile.sh flashnext-3.05` checks or fetches the pack, stops the engine under the box lock, starts the new profile and runs the smoke check. A failed smoke starts the previous profile again, 27B or Flash-Next. A Flash-Next profile needs no 27B drafter, draft head or base snapshot. Since 0.8.1 a 27B profile's switch fetches those too, when they are missing: the K3 drafters, the K4 draft head, the BF16 drafters into `train/` and the FP8 base checkpoint (Section 3.3).
- The fetch takes one branch of `turboderp/Qwen3.8-Flash-Next-exl3`, pinned to its commit when the download starts. The branch has no checksum list, so `ops/fetch_weights.py`

records the hub's sha256 of every file and checks against it. The 4.05, 5.05 and 6.05 packs share one n-gram table file, which a second pack hard-links instead of downloading 39 GB again.

- `ops/start.sh` passes the profile's `QSE_ENGINE_FLAGS` (`--model <pack> --drafter mtp`) and `--corpus` in place of the 27B's model and drafter flags; without that key the 27B's command line is the one it was.
- The page-cache drop after a start drops the pack's pages and keeps a warm n-gram table, which the engine keeps reading.
- The watchdog and the reboot line run `ops/start.sh` of the tree that serves, so a crash or a reboot brings back the active profile, whichever model it is. The status page names the profile that is loading.
- A start with a `--max-len` above the profile's largest window stops before the load and names both numbers (Table 17.2).

The memory rules hold for Flash-Next with two additions. Under the warm policy the n-gram table is page cache that the engine reads on every token, and MemAvailable counts it as free; every floor check therefore takes the table's resident bytes off first, and the dashboard shows them beside the memory split. Each profile sets its own ceiling and peak, with the arithmetic in the file: 66 and 77 GiB for 3.05, 81 and 92 GiB for 4.05, against 70 and 77 for the 27B. A 3.05 engine held 46.9 GiB after its load, with MemAvailable at 68.0 GiB (measured). Chapter 17 has the windows each pack fits and the runs behind them.

6.4 Starting and stopping

`ops/serve.env` is the served configuration: about 60 settings, from paths and the port to every engine switch, each with the reason for its value in a comment. The scripts read it.

Table 6.2: The service scripts in `ops/`.

script	what it does
`ops/make-secrets.sh`	creates `secrets.env` with the metrics token (mode 600), once; `--admin` adds the admin password, which turns the admin rule on; `--rotate` replaces what is there
`ops/start.sh`	starts the engine on the port from `serve.env` and waits for `/health`; refuses while an engine is alive or the port is held
`ops/stop.sh [seconds]`	sends SIGTERM, waits for the drain (60 s by default), and kills only after that
`ops/watchdog.sh`	one health check, for cron every minute; restarts after 3 failures in a row
`ops/hold.sh <minutes> <cmd>`	runs a command with the board to itself, then restores the service
`ops/gate.sh <label>`	the release gate (Chapter 15)
`ops/roles.sh load|up|down|settle`	loads the roles beside the engine one at a time through the residency planner

`ops/start.sh` is safe to run at any time. If the engine is healthy it says so and exits. Once the server answers, it drops the weights' page cache, asks the conductor to load the decision models named for the start, and loads the narrator's model, each through the residency planner.

Stopping drains. SIGTERM and SIGINT make the server stop taking work: `/health` answers `draining` with a 503, new requests get a 503 with `Retry-After: 30`, the generation in flight finishes with its last chunk and `[DONE]`, and the process exits 0. After the work in flight, the drain writes the resident conversation and the parked one to the NVMe (about 3 s for 130,000 tokens), and the next start reads the resident one back before it serves, so that conversation's next turn resumes at its end instead of reading everything again (Section 13.8).

6.5 The watchdog

The watchdog restarts only what it is sure about. A healthy server is left alone, and so is a draining one, because killing a graceful stop halfway is worse than a slow stop. A loading engine is left alone for up to 900 s. Only a server that fails three checks in a row is restarted, so one slow check during a long verify does not bounce a working server.

At boot two cron lines bring the service up:

```
@reboot sleep 90 && $HOME/<checkout>/ops/start.sh >> $HOME/<checkout>/logs/boot.log 2>&1
* * * * * $HOME/<checkout>/ops/watchdog.sh
```

`ops/qwen38-engine.service` is a systemd user unit for the same job, and the installer can write one (`--systemd`). A user unit survives logout only with `loginctl enable-linger`, which needs root. Install one mechanism, never both: two supervisors for one port start the service twice.

6.6 Holding the board

Measurements need the board to themselves, and so does anything that loads a second copy of the model. `ops/hold.sh` makes the safe sequence the easy one:

```
flock ~/.qwen38-box.flock ops/hold.sh 30 python tools/verify_spec.py ...
```

It arms a pause file so the watchdog stands back, keeps it fresh for the whole hold, stops the service and runs the command. Afterwards it restarts the service and waits for `/health`. On every exit path, a killed command included, it removes the pause. The lock file makes two holds wait for each other, and a pause file older than 2,400 s is ignored, so a crashed hold cannot leave the service unsupervised for long. Background jobs of the engine itself (the summary written ahead, the shadow reader, the memory writer) take the same lock without waiting and give it back when a hold asks for it. Taking the service down stops everyone who uses it: hold the board when nobody needs the service, and keep holds short.

6.7 While the engine is away

A stop, a hold or a start takes the engine away for minutes. `ops/start.sh`, `ops/stop.sh` and `ops/hold.sh` write what they do to `logs/engine-status.json`: the state (`stopping`, `held`, `starting`, `up`, `failed`, `down`), since when, a hold's reason (`HOLD_REASON`, or the first line of the file `HOLD_NOTE` names) and its budget, and the last five start times, whose median is the next estimate. With `QSE_STATUS_PORT` in serve.env the scripts keep a small responder on 127.0.0.1 (`ops/engine_status.py`). It answers what a reverse proxy sends it when the engine does not: a page for `/dashboard/` that names the state, the reason and about how long is left, checks `/health` every 5 s and opens the dashboard when the engine is back; and for every other request the OpenAI error shape, status 503, `Retry-After`, with the same facts under `engine`. An open dashboard shows them in its offline banner. `GET /v1/models` is the exception: it answers 200 with the engine's last model list, which `set up` keeps in the status file, so Open WebUI keeps TandemLLM in its list through a restart and a chat shows the engine's sentence instead of "Model not found". The nginx lines are in `docs/operations.md`.

6.8 Roles beside the engine

The engine loads first and holds most of the board. Anything else that runs beside it (a decision model, the narrator, the shadow reader) loads after it, one at a time, through the residency planner (`server/conductor/residency.py`). A role may load when

$$MemAvailable - \underbrace{(peak - held\ now)}_{\text{the engine's growth}} - claim \;\geq\; floor + margin,$$

with the engine's declared peak at 77 GiB and the floor and margin at 15 and 6 GiB by default. It never loads during a hold, and never while another load runs. A refusal names its reason (`refused_hold`, `refused_load_in_progress`, `refused_memory`, `refused_unknown_role`) and is journaled in `residency.jsonl` under the conductor's directory.

A claim is the role's peak, not its idle size, and it is what the role takes from `MemAvailable`, not what `nvidia-smi` shows. The floor, the margin, the settle time, the load order and the claims are the conductor's `residency` document, versioned like the others and edited in the dashboard. The System view shows the plan: the engine's own memory split, `MemAvailable`, every role with its claim and whether it fits now, and the last refusal. Nothing unloads on its own when `MemAvailable` falls after a role loaded; the plan shows it.

`ops/roles.sh` loads external roles. `ROLES_AUTOLOAD` in `serve.env` names the roles `start.sh` brings up after the engine (none in the shipped profile). A role served by vLLM runs BF16 weights with an unquantised KV cache, and its `--gpu-memory-utilization` is the planner's byte math, never vLLM's default of 0.9, which on this board would take the engine's memory too.

6.9 Logs

Each request leaves one log line with its token counts, its finish reason and its rate, and no line carries the text of a prompt or an answer. `--log-content` lets exception messages that quote

a request into the log, and `--log-request-keys` logs parameter names and scalar values, never messages. The installed engine's log is `~/TandemLLM/src/logs/engine.log` (`bin/tandem logs` follows it), and the dashboard's Dev view shows it live.

6.10 Monitoring

`/health` returns the status, the queue counters (running, waiting, served, refused, errors, timeouts, abandoned), the cache report and the allocator's memory figures. On this board `nvidia-smi` reports no used memory, so `torch.cuda.memory_allocated` and `memory_reserved` are the numbers to watch. Allocated memory that climbs across requests is a leak; reserved memory that climbs while allocated memory stays flat is fragmentation.

`/metrics` serves Prometheus metrics (Chapter 30). `ops/monitoring/` has a scrape configuration, alert rules with their tests, and a Grafana dashboard that `tools/grafana_dashboard.py` generates. The conductor's stall watchdog can send a push notification when an unattended session's client has gone; the URL comes from `TANDEM_NOTIFY_URL` or the policy, and is empty in the shipped policy.

6.11 Settings

`engine/settings.py` is the one registry of every `QWEN38_*` switch the engine reads, with its default and the module that reads it; `tests/test_settings.py` fails if any module reads a switch another way. To see what is in effect:

```
python -c "from engine.settings import SETTINGS; import json; print(json.dumps(SETTINGS.
    ↪ describe(), indent=1))"
```

Server behaviour (ports, limits, caches, reasoning) is set by command-line flags, which `ops/start.sh` builds from `serve.env`. Chapter 27 lists every flag and Chapter 28 every variable.

6.12 Deploying and rolling back

An installed engine upgrades by running the installer again (Chapter 3). A checkout that serves the service upgrades in four steps:

1. Copy the serving directory to a dated backup.
2. Copy the new files in, keeping the local `serve.env` values that are site-specific (paths, port).
3. Restart through `ops/hold.sh`, so the watchdog does not race the restart.
4. Check that `/health` says `ok` and that `VERSION` is the new one, then send a short request.

Run the gate and the client smoke set before step 1 (Chapter 15). A deploy that fails step 4 goes back to the backup. The usage ledger and the config database migrate forward in place and keep every row, and an older release ignores the columns a newer one added, so a rollback needs no data step. The disk tier sweeps files of another code fingerprint at the start; a rollback therefore starts

that tier empty, and the session spill keeps its conversations when only the files that keep state changed.

Chapter 7

Community weights

A community pack is a set of weights someone else published on Hugging Face that TandemLLM can load: a smaller quantisation, or a fine-tune of a model we serve. Since 0.8.2 the setup guide lists these packs under their model, after our own profiles, and `ops/use-profile.sh` runs one like any other profile. Listing a pack takes one JSON file in the repository. The first entry is ThinkingCap Qwen3.8-27B Kearuga by 0xWhiteMage, an EXL3 pack at 4.0 bits a weight.

7.1 The packs in the setup guide

The weights step of the setup guide (`#/setup`) shows a Community weights section below our profiles, one card per pack. A card shows the maker's GitHub avatar and `@login`, the networks the entry lists, the fine-tune it comes from, the format, the bits a weight and the download size. It also lists the modalities the pack holds, in grey with the reason when TandemLLM cannot serve one yet, and which of our drafters it runs with. Once a maintainer has run the pack, its card shows the measured speed too. A badge gives the license, another the status, and a link leads to the Hugging Face repository.

You can search over names, authors, handles, tags and the fine-tune's source, and sort by five keys, newest first by default. "Use" selects a pack the way a radio card selects one of our profiles, as `community/<id>`. Every later step takes that name, Wren included. A pack that cannot run images turns the Vision tool off, as the server would. "Add yours" opens the issue form of Section 7.4 with the model filled in.

7.2 Where the list comes from

The list is `models/community-index.json`, one entry per file under `models/<model>/community/`. A box asks GitHub for that file on the public repository's `main` branch at two moments only: when the weights step opens and when you press Refresh (`POST /v1/setup/community/sync`). Nothing fetches it on a timer. The box sends the ETag of its last copy, so an unchanged list costs one 304 answer.

When the fetch fails, the page shows the box's last copy (`~/.tandemllm/community/index.json`). A box that never fetched one shows the list that came with its release. Look at the section's head to see which source the list came from and which entries it dropped. A 404 from GitHub is not an error: the repository has published no list yet, and the page shows a quiet note, "no published list

yet", beside the bundled list. A real failure is one short line without the URL, such as "GitHub could not be reached"; the server's log has the URL and the detail. `QSE_COMMUNITY_URL` points the box at another index, and `QSE_COMMUNITY_URL=off` never fetches.

A fetched index is untrusted. One with the wrong shape never replaces the cached copy, and an entry reaches the page only after it passes the registry's checks, those of Section 7.5.

7.3 Running a pack

```
~/TandemLLM/run/ops/use-profile.sh community/thinkingcap-27b-kearuga-exl3-4.00 --dry-run
~/TandemLLM/run/ops/use-profile.sh community/thinkingcap-27b-kearuga-exl3-4.00
```

The script writes the pack's profile from its entry to `run/community/<id>.env`, which `serve.env` reads. It downloads the pack at the commit the entry pins into `~/.tandemllm/community/<id>`, checks every weight file against the sha256 Hugging Face publishes, and fetches our draft models when they are missing. Then it stops the engine, starts it on the pack and sends one test chat. The checks are those of our own profiles: it refuses a download that would leave less than 60 GB of disk free, and it rolls back to the previous profile when the start or the test chat fails. `--dry-run` prints every URL and size and writes the profile into a folder of its own.

The memory estimate for a pack comes from our own profile of the same format and the nearest size: `exl3-4.0` for a 4.0 bpw EXL3 pack. It stays an estimate until a maintainer has loaded the pack.

Table 7.1: The first community pack. Speed: one run per kind of text on a DGX Spark with 0.8.2, as the entry's `verified` block records it (measured).

field	value
id	`thinkingcap-27b-kearuga-exl3-4.00`
author	0xWhiteMage
fine-tune of	`bottlecapai/ThinkingCap-Qwen3.8-27B`
pack	`0xWhiteMage/ThinkingCap-Qwen3.8-27B-Kearuga-EXL3-4.00bpw`, EXL3 at 4.0 bits a weight, 16.38 GB, standalone
modalities	text; the pack also holds a 6-bit vision tower, which the standalone path does not serve yet
drafters	ours, the EXL3 K3 drafters and K4 draft head
speed	39.6 tok/s prose, 95.9 code, 51.3 chat; 15.2 without speculation
license	PolyForm-Small-Business-1.0.0, with Apache-2.0 for Qwen's part

7.4 Submitting a pack

Send a pull request that adds one file: `models/<model>/community/<id>.json`. The file name is the id. The first entry, `models/qwen3.8-27b/community/thinkingcap-27b-kearuga-exl3-4.00.json`, is a good one to copy, and the JSON Schema is `models/schema/community.schema.json`. If you would rather not write JSON, open an issue with the "Add a community weight pack" form (`.github/ISSUE_TEMPLATE/community-quant.yml`) and a maintainer writes the file.

The fields that matter most:

- `hf.repo` and `hf.revision`: a public repository, and the full 40-character commit. A branch name is refused, because a branch can change after the review.
- `format` and `bpw`: only `exl3` passes today. The schema knows three more formats, which the check refuses until TandemLLM can load them.
- `load`: `standalone` when the pack is the whole model (Section 9.8), or `overlay` when it holds EXL3 projections that go over the base checkpoint.
- `modalities`: what works in TandemLLM today. A standalone pack is text only for now, even with a vision tower inside; list `image` in `modalities_in_pack` instead. An overlay keeps the base checkpoint's vision tower.
- `drafters`: `tandem-exl3` for our EXL3 drafters; `docs/community-weights.md` lists the other two values.
- `author.github`, your GitHub user name, whose avatar the card shows. Leave `status` at `submitted`.

`docs/community-weights.md` lists every field. The registry refuses a pack that is private or gated, a branch in place of a commit, a format TandemLLM cannot load, a base model other than `qwen3.8-27b`, a modality the load path cannot serve, a link that is not https, markup in a text field, and a license that does not let others download and run the weights.

7.5 How verification works

Verification has two parts. On the pull request, the `community` check runs `python -m server.setup.registry --check` and `python -m server.setup.community --index --check`, which hold the file to the schema and to the rules above. Then a maintainer downloads the pack at the pinned commit and runs it on a DGX Spark. If it loads and answers, the maintainer fills `verified` (the build, the device, the date, tok/s per kind of text, tokens per round, a note) and sets `status` to `verified`. A pack that does not load stays `submitted`, or becomes `broken` with a note.

After the merge, a maintainer regenerates the index (`python -m server.setup.community --index`). Every box sees the pack the next time someone opens the weights step or presses Refresh.



Part III

How the engine works

Why the engine is fast, and why its output is still the model's own text. The chapters go from the model's layers and the byte budget (8) through the weights (9), the kernels (10), the drafters and StairCut (11), the exactness contract (12), the caches (13), the server (14) and the measurement method (15) to the seams a new model plugs into (16), and ends with the second model served through them, Qwen3.8-Flash-Next (17). Every number is labelled measured or arithmetic.

Chapter 8

Architecture

This part explains how the engine works, from the model's layers to the measurement method. This first chapter is the tour: the model, the byte budget that sets the speed limit, the decode loop, and a map of the code.

8.1 The model

`engine/config.py` reads the checkpoint's `config.json` and nothing else. The text model has 64 decoder layers, a hidden size of 5,120, a dense SiLU MLP of width 17,408 in every layer, and a vocabulary of 248,320 tokens. It has no mixture of experts. A 0.92 GB vision tower sits in the same checkpoint; the language-model loader skips it by name, and `engine/vision.py` loads it on its own for image input.

Every fourth layer (3, 7, 11 and so on up to 63) is full attention, 16 layers in all. Gated DeltaNet fills the other 48. It is a recurrent layer: it keeps a fixed-size state per head instead of a KV cache that grows with every token.

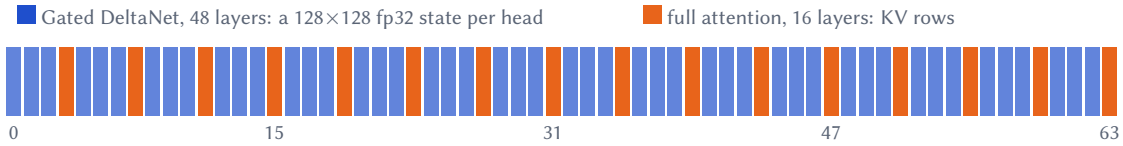


Figure 8.1: The layer pattern of Qwen3.8-27B: three Gated DeltaNet layers, then one full-attention layer, sixteen times.

Table 8.1: The two layer kinds and the state each keeps per sequence (arithmetic, from the configuration's sizes).

layer kind	count	state per sequence
Gated DeltaNet	48	an fp32 state of 48 heads, each 128 by 128 (3.15 MB), and a 4-wide convolution tail (80 KB), per layer
full attention	16	BF16 keys and values, 4 KV heads of 256: 4 KB a token per layer

That split shapes most of the code. Published work on speculation and caching assumes attention, where undoing a token means moving a pointer back by one row. A recurrent state has absorbed

every token and has no inverse. The engine needs exact ways to undo it, to branch it for a tree of guesses, and to restore it from a cache. Chapter 11 shows the undo and the branch, and Chapter 13 the restore.

8.2 The byte budget

A decode step reads every weight once. Table 8.2 sums the bytes from the checkpoint's safetensors headers.

Table 8.2: The bytes one decode step reads, as the vendor ships the model (arithmetic, from the safetensors headers).

group	bytes a step
48 Gated DeltaNet layers	18.43 GB
16 attention layers	5.96 GB
the output head, BF16, 248,320 rows by 5,120	2.54 GB
total, FP8 as shipped	26.93 GB

The three MLP projections are 63.6 % of that. Quantising them first, then the rest, moved the budget down in steps, and each step had to pass the quality gate (Chapter 9). Figure 8.2 shows the steps and the floor each puts under a step at the board's paper bandwidth of 273 GB/s.

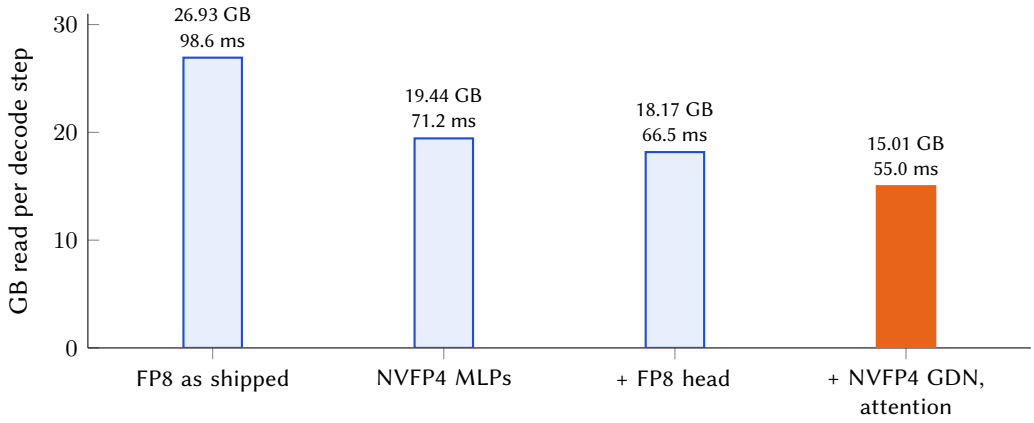


Figure 8.2: Bytes a step for each weight set, and the floor each sets under one step at 273 GB/s (arithmetic). The served set is the last bar. The board reads about 240 GB/s in practice, so the real floors are about 14 % higher.

State is small beside the weights. The 48 recurrent states are 151 MB together. The KV rows cost 64 KB a token across the 16 attention layers: 2.1 GB at 32k tokens and about 17 GB at the full 262,144-token context.

8.3 The decode loop

Speed is one fraction:

$$\text{tok/s} = \frac{\text{tokens committed a round}}{\text{draft ms} + \text{verify ms} + \text{commit ms}}.$$

A round starts from the last committed token. The drafters propose a tree of guesses after it. One forward pass runs over the whole tree through all 64 layers. The engine reads the model's own choice at every node and keeps the longest path where the model agrees with the guesses, plus one more token, the model's own choice after the last accepted guess, so a round yields at least one token. Then it commits the state of that path and starts again.

On the served profile a round on the benchmark row takes 93.4 ms and commits 4.58 tokens, and the row's rate is 49.89 tok/s (measured, the paper). The rate is the mean of the 50 requests' own rates, not the quotient of the two means (4.58 tokens in 93.4 ms is 49.0 tok/s). Without speculation the same box writes 13.69 tok/s at 72.8 ms a token. Chapter 11 follows a round step by step.

8.4 The parts

Table 8.3 maps each part of the engine to the files that implement it and the chapter that explains it.

8.5 One sequence, or several

The numbers in this part describe the loop of one sequence, which is what the shipped profile runs. With `--parallel N` the engine holds N sequences over the one copy of the weights, and requests that decode at the same time verify their trees in one forward pass, so the weights are read once for all of them (Section 14.13). With one sequence, requests wait for the engine's lock in a bounded queue, the side lane serves a short request inside the running one (Section 14.11), and the proxy relays other backends' requests without touching the engine's lock (Chapter 4).

Table 8.3: A map of the code.

part	where	chapter
checkpoint layout and loading	`engine/loader.py`, `engine/config.py`	16
the quantiser and its gate	`tools/quant_nvfp4.py`, `tools/quant_head.py`, `tools/quality_gate.py`	9
matrix-product kernels	`tools/nvfp4_*.py`, `tools/fp8_linear.py`, `tools/head_gemv.py`	10
recurrent-layer kernels	`tools/gdn_*_kernels.py`, `engine/gdn.py`	10
target model, verify and commit	`engine/model.py`, `engine/tree.py`	11
drafters	`engine/drafters/`	11
routers: StairCut, length router, tree	`engine/lenrouter.py`, `engine/router.py`, `engine/prices.py`	11
sampling and penalties	`engine/sample.py`, `engine/penalty.py`	12
structured outputs	`engine/grammar.py`	14
caches	`engine/cache.py`, `engine/disk_cache.py`, `engine/spill.py`	13
settings registry	`engine/settings.py`	6
the server	`server/app.py`, `server/stream.py`, `server/toolcall.py`	14
the conductor	`server/conductor/`	18
live activity, metrics, ledger	`server/live.py`, `server/activity.py`, `server/metrics.py`, `server/ledger.py`	24
the dashboard	`dashboard/`, `docs/contract/dashboard-v1/`	24
the proxy and the collector	`proxy/`	5
service scripts and the gate	`ops/`	6
measurement tools	`tools/row3.py`, `tools/block_ab.py`, `tools/verify_spec.py`	15

Chapter 9

Quantisation

Four-bit weights take a decode step from 26.93 GB to 15.01 GB (arithmetic), and on this board bytes are time. This chapter covers the formats the engine reads, the quantiser that wrote the published 4-bit set, the gate that decides whether a weight set ships, and what stays at full precision.

The served weights are the published set, `0xBakeer/TandemLLM-Qwen3.8-27B-NVFP4` on Hugging Face: every projection in NVFP4, quantised once from Qwen's BF16 release with GPTQ, plus an output head in FP8 (e4m3). The files load over the vendor's FP8 checkpoint, which supplies every tensor the overlays do not replace. The 27B also runs from six public EXL3 sizes (Section 9.7). The second model, Qwen3.8-Flash-Next, is not quantised here: it runs from turboderp's published EXL3 packs, read as they are (Section 9.9).

9.1 What the checkpoint stores

Qwen ships the model as block-scaled FP8. Every large projection is a pair of tensors: `X.weight` holds e4m3 codes of shape $[N, K]$, and `X.weight_scale_inv` holds one BF16 scale per 128 by 128 block. A weight is

$$w = \mathit{code} \cdot \mathit{scale}[\lfloor n/128 \rfloor, \lfloor k/128 \rfloor].$$

`engine/loader.py` keeps that pair as an `FP8Block` and never expands it in memory, because a decode step pays for exactly these bytes. Some tensors stay BF16 in the checkpoint: the output head, the embedding, every norm, the convolution weights and two small projections of the recurrent layers.

9.2 NVFP4

NVFP4 stores a weight in three tensors:

Table 9.1: The NVFP4 layout of one projection.

tensor	type	shape	meaning
`weight`	uint8	$[N, K/2]$	two e2m1 codes a byte, low nibble first
`weight_scale`	e4m3	$[N, K/16]$	one scale per 16 weights of a row
`weight_scale_2`	fp32	one number	the scale of the scales

A weight is e2m1(*code*) · *weight_scale*$[n, \lfloor k/16 \rfloor]$ · *weight_scale_2*. The e2m1 grid has eight magnitudes (0, 0.5, 1, 1.5, 2, 3, 4 and 6) and a sign. A weight costs 0.5625 bytes against 1.0 in FP8, so one layer's MLP drops from 267.4 MB to 150.4 MB (arithmetic).

The group of 16 is what makes 4 bits usable. An FP8 block shares one scale across 16,384 weights; an NVFP4 group shares it across 16, so the grid follows a row's local range.

The engine's kernel is W4A16: it unpacks the 4-bit codes to 16-bit numbers in registers and multiplies on the ordinary tensor cores. On this board NVFP4 is a storage format, not a compute format. It saves bytes, and the arithmetic stays 16-bit (Chapter 10).

9.3 The quantiser

`tools/quant_nvfp4.py build` loads the BF16 model once (about 50 GB) and does everything in that process.

1. It runs a calibration text through the engine. `tools/calib_corpus.py` writes it from `bench/calib.txt`, Python standard-library modules, English Wikipedia and German Wikipedia: 149,682 tokens in the build that was published. Any document that shares a run of 12 words with a held-out file is dropped.
2. It records each projection's mean square input, and the full second moment of each input, $H = X^\top X$. At fp32 the second moments are 101 GB for 64 layers, so the build takes them in passes of about 13 layers (`--h-budget-gb 22`).
3. It quantises every projection with GPTQ on the NVFP4 grid. GPTQ rounds one input column at a time and pushes each column's rounding error onto the columns not yet rounded, weighted by the inverse of H. At the first column of each group of 16, a clip search picks the group's scale from the weights as they stand.

The clip search tries five scale multipliers per group (1.00, 0.95, 0.90, 0.85 and 0.80). A smaller multiplier clips the group's largest weight and gives the rest a finer grid. It keeps the multiplier with the smallest error as the model sees it,

$$\sum_k act_ms[k] \cdot (w_k - \mathrm{dequant}(\mathrm{quant}(w_k)))^2,$$

where $act_ms[k]$ is the mean square of input channel k over the calibration text. A channel the model never uses can take a coarse grid at no cost, and a channel it leans on cannot. GPTQ left a lower output error than the clip search alone on the calibration inputs, and a lower loss on held-out text, so the published set uses it. The format and the kernel do not change between the two; only the stored codes do.

The output head is quantised separately. `tools/quant_head.py build` writes it as e4m3 with one fp32 scale per vocabulary row (1.27 GB instead of 2.54 GB), searching the multipliers 1.0, 0.95 and 0.90 of each row's largest weight. One scale per row, because rows are tokens, and their sizes differ more than a block scale shared by 128 neighbouring ids can follow. The head decides what the engine writes, so it took the full gate. Its kernel returns fp32 logits at every row count.

9.4 The gate

`tools/quality_gate.py` reports three things, and a weight set ships only when all three hold.

1. Held-out loss, teacher-forced, as a change against a reference on the same tokens in the same process. With `--plan` the reference is a model loaded from another checkpoint, which is how the published set is scored against BF16 while it loads over the FP8 release as it does in serving. The gate files are `bench/heldout_code.txt`, a Python file the calibration does not contain, and `bench/heldout_prose.txt`, English prose on unrelated topics. The bar is +0.05 nats on the worse of the two.
2. Argmax agreement with the reference, overall and at positions where the reference is confident (its top choice ahead of the second by at least 1.0 in the logits). On a flat distribution over 248,320 tokens the top choice is decided by rounding, so the confident number is the one to watch.
3. A free generation of 900 greedy tokens on five prompts the calibration never saw, with repetition statistics.

Test 3 matters most. Teacher-forced loss never lets an error grow, so it cannot see a model that drifts off its own path. On an earlier engine on this board, loss rated a broken configuration better than a healthy one.

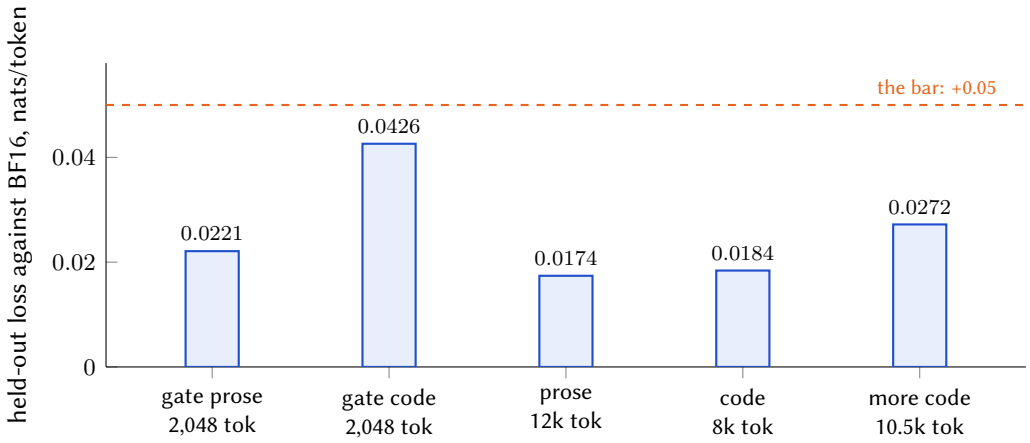


Figure 9.1: The published NVFP4 set against Qwen's BF16 release: the change in held-out loss on the two gate files and on three wider texts that neither the calibration nor anything else reads. measured: the StairCut paper's quality section.

The published set passes on both gate files: +0.0221 nats on prose and +0.0426 on code, inside the +0.05 bar (Figure 9.1). Code costs more than prose. Re-run the gate before quantising anything further, and before trusting a weight set from elsewhere: a weight set passes or fails on its own measurement, not on its format.

9.5 What stays at full precision

The 48 recurrent states stay fp32. They are 151 MB, read and written once a step, about 1.5 ms of traffic. A recurrent state is a sum over the whole sequence, carried across every step, so its rounding error grows with every step; a weight's does not. The convolution weights, the decay and bias tensors of the recurrent layers, every norm and the drafters' own layers keep their shipped precision too.

9.6 Profiles and bytes

Three `serve.env` profiles ship with the NVFP4 set (Table 3.8); the EXL3 weight profiles of Section 9.7 and Flash-Next's packs are switched with `ops/use-profile.sh`. The FP8 profile reads nothing but the vendor's files, so its quality is the checkpoint's. The balanced profile keeps the recurrent and attention layers on the vendor's weights and takes the published set's NVFP4 MLPs and FP8 head; the MLPs carry 70 % of a verify's projection bytes. Neither profile's speed is measured on this release.

Bytes and accepted tokens partly substitute for each other. At one token a step every byte saved is time saved. On a round that commits several tokens the weight read is already spread over them, so the gain from fewer bytes shrinks as acceptance grows.

9.7 The 27B in EXL3

Since 0.5.0 the 27B also runs from EXL3 weights at six sizes, one weight profile each (`ops/profiles/exl3-<bpw>.env`, switched with `ops/use-profile.sh`, Section 6.3). We quantised them with exllamav3 from Qwen's BF16 release, and all six are public on Hugging Face as `0xBakeer/TandemLLM-Qwen3.8-27B-EXL3-<bpw>`. The drafters for these profiles are in `0xBakeer/TandemLLM-Qwen3.8-27B-EXL3-drafters`: both DFlash2 drafters with their projections in EXL3 at 3 bits a weight (the K3 files) and a draft head at 4 bits (K4). The six sizes and the drafters form one collection, TandemLLM Qwen3.8-27B EXL3.

Table 9.2: The 27B's EXL3 sizes against the BF16 model on wikitext-2, where BF16's perplexity is 6.849. measured: copied from the model cards on Hugging Face.

profile	disk	KL vs BF16	top-1 agreement	note
`exl3-2.5`	12.0 GB	0.1112	86.3 %	the fastest
`exl3-3.0`	13.5 GB	0.0621	89.8 %	
`exl3-3.5`	15.2 GB	0.0298	93.1 %	
`exl3-4.0`	16.9 GB	0.0171	94.8 %	
`exl3-4.5`	18.4 GB	0.0096	96.1 %	the default of a new box
`exl3-6.0`	23.3 GB	0.0018	98.5 %	the closest to BF16

Each card also has a speed table for every size on one DGX Spark (TandemLLM 0.6.2, one greedy request of 512 tokens, speculation on). An EXL3 profile needs four more downloads beside its

weights: the K3 drafters and the K4 head (1.9 GB), the BF16 drafters `0xBakeer/TandemLLM-Qwen3.8-27B-DFlash2-b8` and `-b16` (7.7 GB, into the repository's `train/`), and the base checkpoint `Qwen/Qwen3.8-27B-FP8` (30.4 GB, without `mtp.safetensors`). Since 0.8.1 `ops/use-profile.sh` fetches all of them, checks each weight file against the hub's sha256, and needs no token. Each card's `calibration/` folder has the recipe; the calibration repository itself stays private, and no profile reads it.

9.8 Standalone EXL3 packs

Our EXL3 profiles are overlays: their files hold the coded projections, and the vendor's FP8 checkpoint supplies the rest. Since 0.8.2 a complete EXL3 pack of the 27B family also loads on its own, passed as `--model`, with no BF16 or FP8 base to download. 0xWhiteMage wrote this path in pull request #1, and the first community pack (Chapter 7) runs on it. The loader recognises `quantization_config.quant_method: exl3` in the pack's configuration; the pack must hold the embedding, the norms and the recurrent layers' small tensors too.

```
python server/app.py --model /path/to/exl3-pack --drafter none --vision off
```

The projections and the output head keep their EXL3 codes, channel scales and mixed bit widths, also when a projection's parts span several shards. When the pack declares a BF16 source (`dtype` or `torch_dtype`), the loader turns its FP16 plain tensors into BF16 only where every stored value survives the round trip exactly. Missing or non-BF16 source metadata, a value that is not finite, or a lossy conversion stops the load with a named error. The pack's stored values are kept; precision lost while the pack was made cannot come back, and only the original checkpoint can show that a pack equals its source.

The path adds no codebook, no architecture and no MTP format: the `mul1` reader and its checks are the ones above. It has three limits. It is text only, even for a pack that carries a vision tower, and the registry refuses a community entry that claims images on it (ENG-271). Quantised MTP layers are not qualified. A stock DFlash2 drafter runs one sequence (`--drafter dflash2 --dflash2-ckpt <dir>`), and speculative parallel serving still needs the `lenrouter` configuration with our drafters, which `use-profile.sh community/<id>` fetches for a pack whose entry names them. Our own profiles, an EXL3 overlay over the base or NVFP4, load exactly as before.

9.9 Flash-Next's EXL3 packs

turboderp publishes Qwen3.8-Flash-Next in five sizes, one branch each of `turboderp/Qwen3.8-Flash-Next-exl3`, and the engine reads them as they are: no quantiser of ours runs on them and no format is added. The branch name says what is inside. `3.05bpw_h5_ng5` is 3.05 bits a weight on average, the output head at 5 bits and the n-gram table at 5 bits a value. Table 17.2 lists the five with the window each fits. A pack is lossy against the BF16 release by design, and speculation is lossless against the pack that is served (Section 17.7).

`engine/models/flashnext/pack.py` reads every shard header and sorts each tensor into one kind (Table 9.3). An EXL3 projection is four tensors: `X.trellis` holds the codes at K bits a weight, `X.suh` and `X.svh` scale and sign the rotated input and output sides, and `X.mul1` is the codebook's

multiplier. K is read from each tensor's own shape. On the 3.05 pack the audit counted 303,121 tensors, every routed expert at $K = 3$, and the dense projections and the head at $K = 5$ except the sparse-attention indexer's projection ($K = 3$) and the MTP layer's (K from 3 to 5). By those headers a plain decode token reads about 4.23 GB in all at 1k of context (arithmetic).

Table 9.3: What a Flash-Next pack holds, and how the engine holds it.

kind	what	on the device
`expert`	the routed experts' projections, 98 % of the bytes	EXL3, one image a layer
`exl3`	every other coded projection: GDN, attention, the indexer, the shared expert, the MTP layer's, the head	EXL3
`mixer`	the hyper-connection mixers	int8 with an fp32 scale a group of 128 weights (fp16 in the pack)
`plain`	norms, the router, small projections, conv weights, the embedding	as stored
`table`	the n-gram table, its own file	never; read a row at a time
`vision`	the vision tower	not loaded

`tools/fn_exl3_xcheck.py` holds our reading against exllamav3's own on the 3.05 pack: the trellis decode equal bit for bit on 16 tensors, the dequantised weight within 0.36 % of the largest weight (our BF16 rounding of the fp32 rotations), and 4,096 n-gram rows equal bit for bit (measured).

The mixers are read on every token, so the engine keeps them as int8 with one fp32 scale for each group of 128 weights of a row (`QWEN38_FN_MIX`, `int8`; `int8rows` is one scale a row, `fp16` the pack's values); a weight is off by at most its group's largest weight over 254. Against the pack's fp16 mixers on the same kernels, with 8,192 tokens on 3.05, the groups gave a KL of 0.0031 on code and 0.0188 on prose, inside the arithmetic noise between the reference forward and the kernels (0.0038 and 0.0255). One scale a row gave 0.267 on code and 0.047 on prose, because rows whose largest weight is 10 to 420 times their median left the small weights coarse. Int4 gave 0.38 on prose and changed the continuation at its first token, so the mixers stay int8 (measured, `tools/fn_mix_quality.py`). The groups cost no decode speed and 0.02 GiB. The prefill reads exact fp32 copies of the mixers, the router and the small projections (`QWEN38_FN_PREFILL_MM`); with fp16 copies it failed its agreement gate on a 3k-token code prompt.

The n-gram table codes each 160-value row as a ring of trellis codes at 4 to 6 bits a value, with one fp16 scale a row. It is 32.6 GB in the 3.05 pack and never sits in device memory: each token reads 16 rows and decodes them on the device. A profile reads it warm, through the page cache, or cold, with `O_DIRECT` through a 2 GiB row cache (`QWEN38_FN_TABLE`); Chapter 17 says which pack uses which and why.

Nobody has measured a pack's quality against the BF16 model. The BF16 release is 360 GB and does not fit the board, so the registry's quality fields are empty. What is measured is our arithmetic against the reference on the same pack: all 48 layers equal the official `transformers` layer within BF16 noise (relative error 0.19 to 1.49 %).

9.10 Knobs

Table 9.4: Quantisation tools and the engine switches that load their output.

knob	what
`--nvfp4 a,b,c` or `QWEN38_NVFP4`	the NVFP4 files to load, one per target; empty means the checkpoint's FP8
`--fp8-head FILE`, `--fp8-head build`, `QWEN38_FP8_HEAD`	the e4m3 head; `build` makes it at load; empty means the BF16 head
`tools/quant_nvfp4.py build --methods gptq`	statistics and GPTQ from one load of the BF16 model
`tools/quant_head.py build --ratios`	the per-row scale search for the head
`tools/calib_corpus.py --out DIR --eval-chars`	the calibration text and the wider held-out texts
`tools/quant_sensitivity.py measure`, `mix --keep-gb`	per-piece sensitivity, and a mix that keeps the most sensitive pieces in FP8
`tools/quality_gate.py --plan plan.json`	weight sets against a reference checkpoint
`QWEN38_FN_MIX`, `QWEN38_FN_TABLE`, `QWEN38_FN_PREFILL_MM`	Flash-Next: the mixers' form, the n-gram table's policy, the prefill's weights
`tools/fn_exl3_xcheck.py`, `tools/fn_mix_quality.py`	Flash-Next: our reading against exllamav3's; a mixer form against the pack's

The gate files are one held-out file per domain, 2,048 tokens each, and the wider texts add 8k to 12k tokens per domain. No task benchmark suite has run against the quantised engine (Chapter 15, Limits).

Chapter 10

Kernels

Decode on this board is a memory-read problem. Every round must read the projection weights once, about 13.7 GB of the 15 GB at NVFP4, and the GB10 reads memory at about 235 to 240 GB/s in practice. A kernel is good here when it streams those bytes near that rate at the row counts a verify uses, 1 to 32 rows. This chapter covers the matrix products, the recurrent-layer kernels, the head and the norms, CUDA graphs, and the one property every kernel must keep.

10.1 Row invariance

Take a row of a 1-row product and the same row as row 5 of a 16-row product. Its result must be the same bits in both cases. The agreement gate compares speculative output against one-token-at-a-time output, and that comparison can hold only if the arithmetic of a row ignores its neighbours.

The engine gets this by fixing the reduction order. Every tile table is keyed by the weight's shape (N, K), never by the row count, so the sum over K runs in the same order at 1 row and at 32. No kernel uses atomics, whose order would depend on the scheduler. Where K is split across warps, the partial sums are added in warp order. `tools/fp8_probe.py --sweep` keeps only FP8 tile choices that are bit-identical to the default at 1 to 32 rows, and the NVFP4 tables follow the same rule.

Row invariance keeps a tree verify close to the one-token decode, but it does not make them identical. The output head uses a different kernel for one row than for several, and the verify path of the recurrent and attention layers orders the one-token arithmetic differently. A near tie between the two best tokens can go the other way. Chapter 12 has the full contract and its measured exception.

10.2 NVFP4 matrix products

The weights are 4-bit and the activations 16-bit. The engine never uses the FP4 tensor-core path: on this chip an FP4 operand still has to travel through shared memory and a decode step before a tensor core sees it, and the chip has 99 KB of shared memory per SM. A 4-bit activation path would also need its own quantiser and quality gate, to save at most the 8 % of traffic the activations are.

Three kernels share one stored format and pick by row count:

Table 10.1: The NVFP4 matrix-product kernels by row count.

rows	kernel	file
1 to 32 (decode, verify)	the skinny kernel, CUDA	`tools/nvfp4_skinny.py`
33 to about 1,024 (prefill chunks)	Triton, v2	`tools/nvfp4_linear_v2.py`
above that	expand to BF16 once, then the library GEMM	`tools/nvfp4_linear.py`

The skinny kernel rests on one fact about the hardware. The instruction `cvt.rn.f16x2.e2m1x2` turns one packed byte into one register holding two 16-bit values, and that register is exactly the operand layout of an `mma.m16n8k16` tensor-core instruction. A thread loads 16 bytes of a weight row with one vector load and decodes each byte straight into an operand register, so nothing goes through shared memory. The group scale is decoded by `cvt.rn.f16x2.e4m3x2` and multiplied onto the weight in fp16. That product is exact: e2m1 times e4m3 needs six significant bits, and fp16 has eleven. The weights are the B operand (8 rows an instruction) and the activations the A operand (16 rows an instruction), so a 16-row verify costs one instruction per weight tile, and 17 to 32 rows cost two. That is the step in the verify cost at 17 rows that StairCut prices (Figure 11.2).

It builds on first use with `torch.utils.cpp_extension` for `sm_121a`, because the e2m1 converter is specific to that chip. A tile table per shape (`ops/skinny-tiles.json` up to 16 rows, `ops/skinny-tiles-wide.json` for 17 to 32) holds the launch settings a sweep picked.

10.3 FP8 matrix products

`tools/fp8_linear.py` reads the checkpoint's block-scaled FP8 as stored. The tile is 128 wide along K, so each step covers one scale block and the scale is one number applied to the fp32 sum. For the FP8 profile, `ops/fp8-tiles.json` holds one launch setting per shape for every row count up to 32, so the profile decodes and verifies with a single program shape per weight.

10.4 The recurrent layers

A Gated DeltaNet layer updates a 128 by 128 state per head at every token:

$$S \leftarrow S \cdot e^{g}, \qquad \delta \leftarrow (v - S^{\top}k) \cdot \beta, \qquad S \leftarrow S + k \otimes \delta, \qquad o \leftarrow S^{\top}q.$$

Each shape of this work has its own kernel:

Trees rely on depth-first order. When the walk reaches a node at depth d, the last node it visited at each shallower depth is that node's ancestor. A stack of one set of factors per depth therefore holds exactly the node's own path. In the served build the commit happens inside the next verify: the accepted rows are recorded, and the next verify applies them to each state tile as it loads it. That halves the state traffic of a round, from 604 MB to 302 MB (arithmetic).

Table 10.2: The recurrent-layer kernels.

work	kernel	what it does
one token	`tools/gdn_kernels.py`	loads a state tile once, applies the whole step in registers, stores once
a chain	`tools/gdn_verify_kernels.py`	walks up to 16 tokens over one loaded tile and returns the factors a rollback needs
a tree	`tools/gdn_tree_kernels.py`, `tools/gdn_verify_kernels.py`	walks the tree in depth-first order, rebuilding each node's parent state from a stack of factors along its path
a commit	`tools/gdn_commit_kernels.py`	writes the accepted path's state as one low-rank update
a prefill	`tools/gdn_prefill_kernels.py`	the chunked form, with the chunk-local work in registers

10.5 Head, norms and attention

Two kernels serve the output head: `tools/head_gemv.py` for one row, a tensor-core GEMM for more. At one row a GEMV reads fastest. At eight rows the same GEMV does eight times the scalar work without a tensor core, and a verify once ran 2.5 times slower because of that. Both return fp32 logits.

Each RMS norm is one kernel instead of eight small ones, written with the reference's own rounding points (`tools/norm_kernels.py`). The gated norm of the recurrent layers rounds to BF16 between the weight multiply and the gate, and doing that multiply in fp32 changes the answer. Attention uses the library's fused attention with a causal flag, a lower-right causal mask for chunks that start past position 0, and a decode kernel for short query counts.

10.6 CUDA graphs

Every verify runs from CUDA graphs (`engine/verify_graph.py`), one per shape of round: chain or tree, row count, context class, flags. Positions and lengths live in device buffers, so one captured graph serves every round of that shape. Graphs took the launch gaps inside a verify from about 4.5 ms to about 1.5 ms. The drafter's forward runs from graphs too. Graphed and eager runs agree bit for bit, and the tests check that. Every graph capture holds a process-wide lock (`engine/capture_lock.py`), because a CUDA call from another thread during a capture fails the capture; the disk tier's writer holds the same lock around its own CUDA calls.

10.7 How a kernel gets in

Every kernel has a `check()` against the PyTorch function it replaces, and it answers to that before anyone times it. Only the engine's own round time decides whether it ships, measured with the flag flipped inside one process (Chapter 15). A standalone kernel timing is a ranking at best: the same NVFP4 configuration once read 0.254 ms in one process and 0.627 ms in another, which

differed only in what they had allocated before. Time a kernel over a working set larger than the chip's caches; `tools/cold_bw.py` does. A warm probe once reported 97.6 % of peak bandwidth on a GEMM, which is not possible.

10.8 Knobs

Each kernel path has a `QWEN38_*` switch, and `ops/serve.env` sets the served combination. Table 28.2 lists every switch with its default and its served value. The main ones:

Table 10.3: The main kernel switches.

switch	what
`QWEN38_NVFP4_SKINNY`, `QWEN38_SKINNY_TILES`, `_WIDE`	the skinny kernel and its tile tables
`QWEN38_FP8_TILES`	the FP8 per-shape launch table
`QWEN38_FUSED_GDNVERIFY`, `QWEN38_GDNV_WY*`	the fused verify recurrence and its variants
`QWEN38_COMMIT_IN_VERIFY`	fold the commit into the next verify
`QWEN38_VERIFY_GRAPH`, `QWEN38_DRAFT_GRAPH`	CUDA graphs for the verify and the drafter
`QWEN38_FUSED_GDNPREFILL`, `QWEN38_NVFP4_PREFILL_V2`	the prefill paths
`QWEN38_DECODE_ATTN`	decode and verify attention that reads the KV once

Chapter 11

Drafters, StairCut and tree verify

A verify of 16 tree rows costs about the same as a verify of 8. On the published NVFP4 set at 1k tokens of context, a tree verify with its commit takes 75.5 ms at 8 rows, 74.4 ms at 16, 83.1 ms at 24 and 87.8 ms at 32 (measured, `ops/stair-tables-nvfp4.json`, median of 8). That flat stretch is why this engine speculates: the model's forward pass is bound by reading weights, and extra rows in the same pass are nearly free. TandemLLM therefore lets cheap drafters guess many tokens and checks them all at once. This chapter covers the drafters, how StairCut decides what to spend each round, how the guesses become one tree, and how the tree is verified through recurrent layers.

11.1 The round

Each round has four steps (Figure 11.1).

1. Draft. The block drafter proposes a block of tokens, and the lookup drafter proposes continuations it has seen before. The router merges them into one draft tree of at most 32 nodes.
2. Verify. One forward pass of the target model runs over the whole tree and reads the model's own next token at every node.
3. Accept. Walk down from the root. At each node take the model's token and move to the child that carries it. Stop where no child carries it. The model's token at the stopping point is kept too, so a round yields at least one token.
4. Commit. Keep the KV rows and the recurrent state of the accepted path, and drop the rest.

On the benchmark row a StairCut round takes 93.4 ms and commits 4.58 tokens (measured, the paper).

11.2 The block drafter

`engine/drafters/dflash2.py` runs DFlash2, a small drafter that proposes a whole block of tokens at once. It has five decoder layers of its own (hidden size 5,120) and reads five hidden states of the target, the residual stream entering layers 5, 19, 33, 47 and 61, fused into one input. It has no

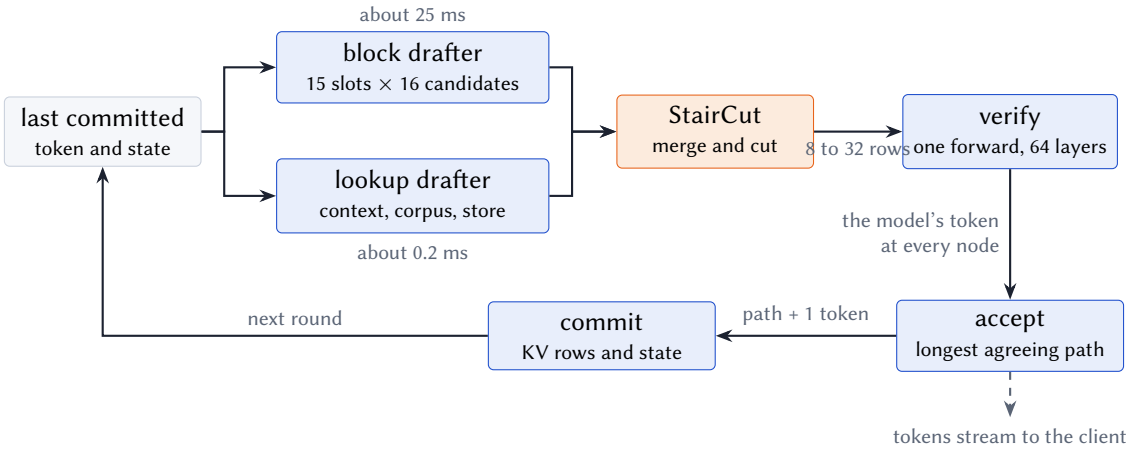


Figure 11.1: One speculative round. Both drafters propose; StairCut merges their proposals and cuts the tree to the node count with the most expected tokens per millisecond; one verify pass checks every node; the longest agreeing path and one token of the model's own are committed.

embedding and no head of its own: it borrows the target's. Inside a block every row attends to every other row, which is what lets it guess 15 tokens in one pass instead of 15 chained steps.

The engine holds two fine-tunes of the vendor's drafter, trained on the target's own output and published on Hugging Face: one trained at block 8 (the *narrow* drafter: it proposes 7 tokens a block) and one at block 16 (the *wide* drafter, 15 tokens). StairCut, the served behaviour, drafts with the block 16 one only. The served profile loads the block 8 one too and releases it at the first round; it drafts only with StairCut off. A draft moves about 6 GB (3.6 GB of drafter layers plus one read of the head for all rows), about 25 ms.

The drafter's head gives the top 16 candidates per slot, and a small bigram selector scores how each candidate follows the one before. That is a lattice of 15 slots by 16 candidates. The greedy path through it becomes the spine of the tree, and the next-best candidates become branches.

11.3 The lookup drafter

`engine/drafters/ngram.py` copies text instead of predicting it. It looks for the longest match of the last few tokens in three places:

- the request itself, prompt and output so far (orders 2 to 8);
- a corpus of public text, memory-mapped with a suffix array;
- the persistent suffix store, a log of what this engine has read and written before (Section 13.10).

Every continuation found is a vote, and the votes form a tree whose node scores are path probabilities. A lookup costs about 0.2 ms against about 25 ms for the block drafter. On fresh prose it rarely fires. On an edit of a file already in the context, or on a quotation, it can commit 15 tokens in a round. It fires only when the tree it would propose is worth its verify cost, because a drafter that proposes

on every step and is usually wrong costs the round. After two wide rounds in a row that commit their whole width, the lookup drafter may propose one deep chain of up to 32 rows (`QWEN38_DEEP`).

11.4 StairCut

StairCut is the served behaviour: `QWEN38_LEN_SWITCH=1` and `QWEN38_LEN_MODE=wide`, off in the code and on in `ops/serve.env`. Every round drafts with the wide drafter and builds four candidates of up to 31 nodes: its lattice tree, its chain, the lookup tree, and their merge. Each candidate is cut to the node count that maximises its expected committed tokens per millisecond,

$$\frac{1 + \min(\sum_{\text{kept}} q,\ n)}{\text{verify}(\text{rows}) + \text{draft} + \text{commit}},$$

and the round runs the best of the four.

A node's q is its path probability from the drafter's lattice (read at temperature 1.4), or the lookup's vote share, scaled by a calibration the router learns online and capped at 1. Nodes are admitted in the order of the drafter's own score, each with the ancestors it needs. The price of a tree is verify plus draft plus commit; a chain pays the rollback instead of the commit, weighted by the chance it is rejected. The verify price is a staircase, not a line, so the router tries every count instead of a search that assumes a smooth cost. A cut ends at 7, 15, 23 or 31 nodes (8, 16, 24 or 32 rows), or at the whole candidate when it is smaller.

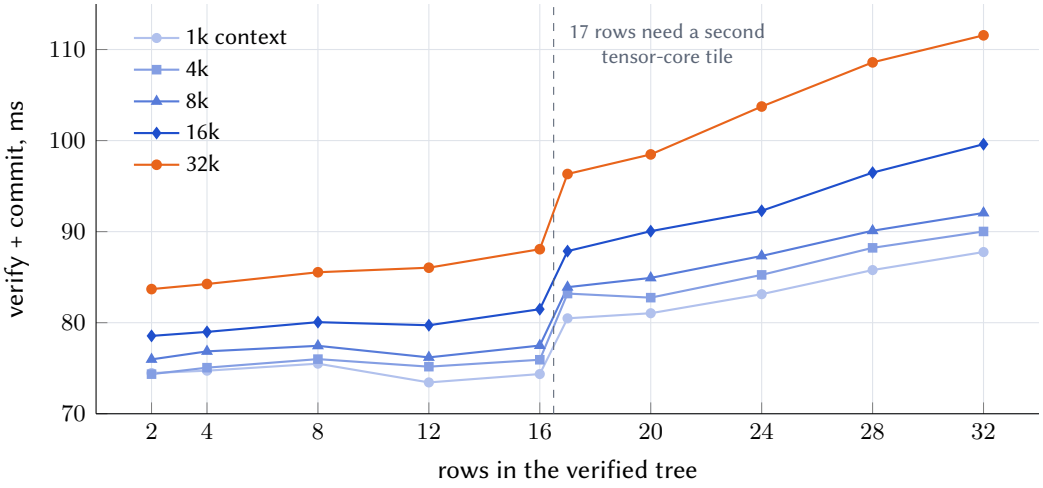


Figure 11.2: The verify-cost staircase: a tree verify with its commit, by rows, at five context lengths. Flat to 16 rows, then steps. measured: `ops/stair-tables-nvfp4.json`, published NVFP4 set, the medians StairCut prices its cuts on; this chart is drawn from that file.

`QWEN38_STAIR_TABLES` names the JSON file of verify prices by context length, chain and tree apart (`tools/verify_curve.py` measures them). Figure 11.2 draws the served file. At 32k a 24-row tree costs about 16 ms more than a 16-row one, against 9 ms at 1k, and a tree costs 2 to 4 ms more than a chain of the same rows. `QWEN38_STAIR_RHO` names a prior for the lookup's continuation rate on 8-token matches (`ops/lookup-rho.json`), which lets a long verbatim copy keep its whole line; the router updates it online against every committed block. `QWEN38_STAIR_SKIP=1` decides on the same

prices whether to draft the head at all once a copy run has started: on a copy round the lookup line alone often wins, and at 32k a draft costs more than at 1k.

The draft cost is the drafter's own measured time at the current context length, learned online per context class. What the router learns (the draft cost per context class, the calibration, the copy counts) belongs to the router object, and the server keeps one router for its whole life, so what one request teaches reaches the next. `--no-learn-cost` keeps the served price table instead, so what earlier requests took no longer moves a block's choice and a speed measurement repeats much more closely (Chapter 15). The timings inside the request still move it, so two runs can still part at a near tie (Section 12.3).

With the node count calculated, the wide drafter's lattice cut to 16 nodes is a bush near the root on fresh text and a long line on copies, and the narrow drafter has nothing left to add. The router releases it at the first round.

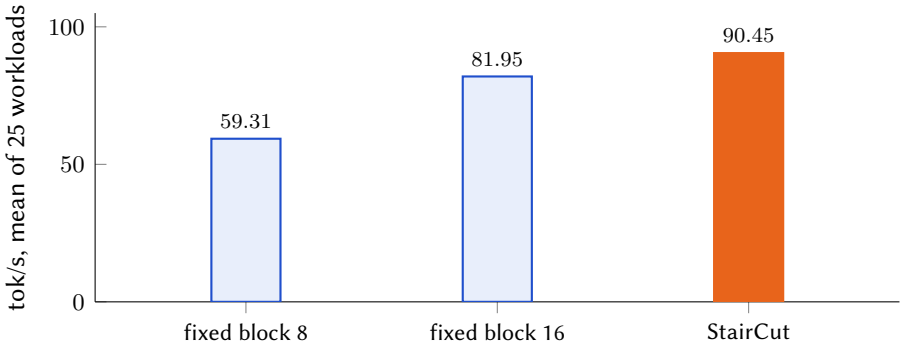


Figure 11.3: The teacher-forced bench: 25 workloads where every configuration commits the same reference text, so only rounds and their time differ. StairCut is above block 16 on 23 of the 25 workloads and above block 8 on 22. measured: the StairCut paper.

11.5 Longer contexts

Every token of context adds 64 KB of KV rows that each round reads, and the drafter reads the whole context too, so a round costs more as the conversation grows. The verify itself shows it: a 16-row tree with its commit takes 74.4 ms at 1k tokens of context and 81.5 ms at 16k, and 88.1 ms at 32k (measured, the price table). Figure 11.4 puts the two measured decode rates the public record holds beside that curve and beside the bandwidth bound of plain decoding.

Two points are all the public record holds for decode speed against prompt length on the published weights, and they come from two builds of the same series. A sweep over prompt lengths, under the rules of Chapter 15, is not published yet.

11.6 The length router

Before StairCut, the served profile picked the block width once per request, and with the StairCut flags off `engine/lenrouter.py` still does. It runs four wide rounds first. A wide round prices both widths at once, because a narrow block is a prefix of a wide one and the target's token at row i

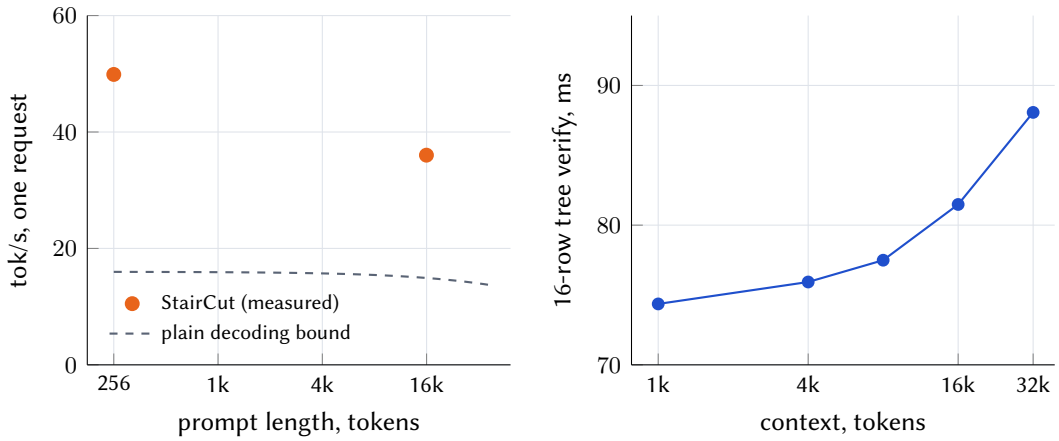


Figure 11.4: Left: single-request decode rate with StairCut at two prompt lengths (measured: 256 tokens, the paper's benchmark row; 16,350 tokens, the shadow reader's A/B with the shadow off, the 0.4.0 series), and the bandwidth bound of plain one-token decoding when every step reads 15.01 GB of weights plus the context's KV at 240 GB/s (arithmetic). Right: the 16-row tree verify by context length (measured, `ops/stair-tables-nvfp4.json`).

does not depend on rows after i. Where that says the narrow width still has room, it runs up to four narrow probes. Then it decides, keeps the decision for the rest of the request, and releases the losing drafter (`--drop-idle`), which then stops syncing its cache. It decides once because switching costs acceptance: a drafter keeps its own cache of the context, and a switched-in drafter starts behind. `--len-fixed 8` or `16` pins one width; those are the fixed baselines the paper compares StairCut with.

11.7 Building the tree

`engine/router.py` merges the block drafter's lattice and the lookup tree into one `DraftTree` (`engine/tree.py`), where a shared prefix becomes a shared node. With StairCut on, the merge is built at up to 31 nodes and the staircase cut sets its size. With StairCut off, the router prunes it to a node budget: a node stays only while its expected gain in accepted tokens pays for its share of the verify cost, priced on a verify curve (`QWEN38_TREE_MS`, or a price table). The budget is 16 nodes for either drafter, and the wide drafter gets 24 once the request has committed 32 tokens, so a short answer never pays for branches it cannot use.

The router's cost estimates are learned in the loop. It scores the lookup tree against the tokens the target actually wrote on every round, chosen or not, so its calibration settles before the lookup drafter has had a turn.

11.8 Tree verify through recurrent layers

Verifying a tree through attention layers is known work: give each node an attention mask of its ancestors and a position equal to its depth. The 48 Gated DeltaNet layers are harder. Their state

after node t is the state after t's parent, updated by t, and a plain left-to-right walk overwrites the parent's state before t's sibling needs it.

`engine/tree.py` stores a tree in depth-first pre-order: every node's parent has a lower index, and every subtree is a contiguous range (Figure 11.5). In that order the ancestor relation is part of the strict lower triangle, and every prefix operation of a chain verify has an exact tree counterpart.

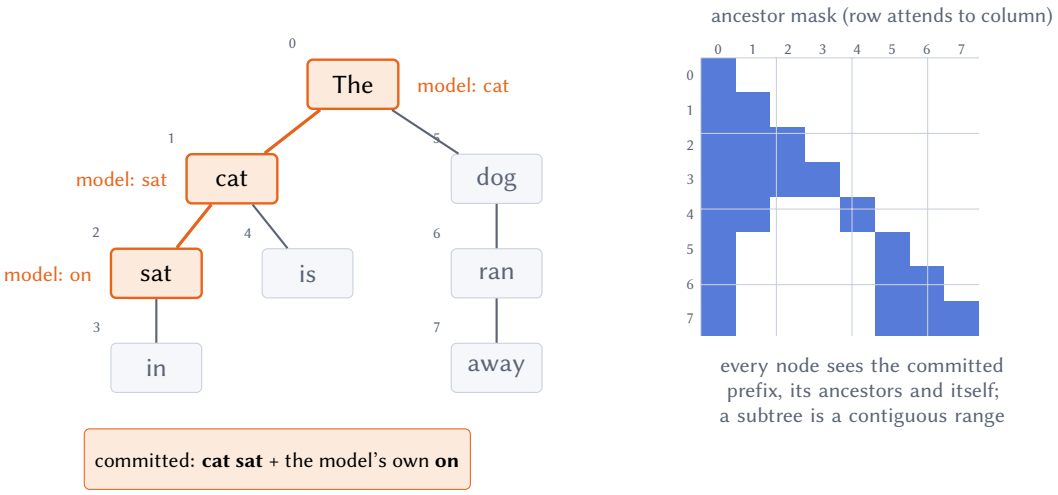


Figure 11.5: A draft tree of 8 nodes in depth-first pre-order. The model's own token at node 0 is "cat", which node 1 carries; at node 1 it is "sat", which node 2 carries; at node 2 it is "on", which no child carries. The round commits "cat sat" and the model's "on". Right: the ancestor mask the verify applies in the attention layers, and the relation the recurrent layers walk.

Table 11.1: Every chain operation has an exact tree counterpart.

chain	tree
causal mask	the committed prefix plus the node's ancestors
position = index	position = depth, so two siblings share a position
the convolution slides over neighbours	it gathers each node's 3 nearest ancestors
cumulative gate along the sequence	the sum of gates along the node's path
the delta rule's lower-triangular solve	the same solve on the ancestor relation
accept a prefix, roll back to a length	accept a path, commit it by a gather

The state after any accepted path can be written from quantities the verify already computed. Let S_0 be the state before the round, and for each node t on the path let k_t be its key, u_t its pseudo-value and g_t its cumulative gate. Then

$$S_{\text{path}} = e^{g_{\text{last}}} S_0 + \sum_{t \in \text{path}} e^{g_{\text{last}} - g_t} \, k_t \otimes u_t .$$

u_t depends only on t's own path, because the inverse of an ancestor-masked matrix is ancestor-masked too. The formula holds for every root-to-node path, not only for prefixes. The commit is therefore one read of the state and one small matrix product per layer, and it reads no weights. For a chain the same identity is the rollback: keeping the first n tokens of a block is the path of length

n. The KV side is simpler: the cache holds every node at its depth-first slot, and a commit gathers the accepted rows down into place.

11.9 Sampling

Greedy requests (temperature 0) take the path above. A sampled request stays exact in distribution: the target draws its own token at each node, and a drafted token survives exactly while the draw lands on it. For a drafter that proposes one fixed token this is the standard rejection sampler, with no second draw. With `--sampled-tree det` (served) a sampled request keeps the tree. A request with a `seed` draws with noise keyed by position (the Gumbel-max trick), so the same seed gives the same text whatever the drafters and the router did, up to the same near-tie rule as greedy decoding. Chapter 12 covers both.

11.10 Reasoning

The model thinks inside `<think>` tags. A reasoning budget closes the block when it is spent: the engine appends the vendor's closing sentence and `</think>`, runs those tokens through the model at their real positions, and the model answers from there. This changes the answer, by design. The budget comes from the server's `--think-budget`, from the client in any of the shapes clients send (Section 14.3), or from the conductor's effort level. With the conductor on, the closing phrase is hidden from the client, which receives `</think>` and the answer; the conductor's loop guards close a reasoning block that goes in circles (Chapter 20).

11.11 The same method on Flash-Next

Flash-Next drafts with the multi-token-prediction layer its checkpoint ships, and its StairCut is written once for every model (`engine/methods/staircut.py`). Each round the MTP head drafts a chain of 4 tokens with its top two a depth, from one captured graph. The cut keeps the spine's prefix and, with siblings, the second choices that maximise

$$\frac{1 + \sum_{\text{kept}} q}{V(\text{rows}) + \text{draft}},$$

where q comes from `ProbDeciles`, the head's probability mapped to the acceptance measured per tenth of probability and learned from every verified round. With `QWEN38_FN_LOOKUP=1` the lookup's best continuation joins the same tree: where its tokens match the head's chain the two share nodes, where they differ it hangs under the last shared node, and the cut admits nodes best-first up to 32 rows. A round still runs one verify, and on a copy the lookup lifted the rate from 102.3 to 132.9 tok/s (Table 17.3).

$V(\text{rows})$ comes from `PriceModel`: a prior from a price file in the format of the 27B's stair tables, `ops/prices/<model>/<profile>.<device>.json`, followed online and never cheaper for more rows. The device is `dgx-spark` for the GB10 (`engine/methods/device.py`); a profile without a file for its device learns every price from the first rounds. On this model the verify is a ramp, not a staircase:

every row can route to other experts, and at about 1k of context 2 rows cost 27.8 ms and 16 rows 68.8 ms (measured, learned online).

The cut also decides whether to draft at all. Prices, the plain step's cost and a drafted round's cost a token are kept per context class, by wall time; a class whose drafts cost more a token than its plain steps takes the plain step, with probe rounds at a doubling interval, and a turn drafts only when the saving pays for the prompt rows the head has not read yet. Section 17.6 has the rule and the runs that set it. A verified row on Flash-Next is the decode step's row bit for bit, so none of this moves the text.

11.12 Knobs

Table 11.2: The speculative decoding knobs.

knob	what
`--drafter lenrouter`, `--dflash2-ckpt`, `--dflash2-ckpt16`	the block drafters (both loaded; StairCut drafts with block 16)
`--narrow-arm on|off`	build the block 8 drafter at all
`QWEN38_LEN_SWITCH=1`, `QWEN38_LEN_MODE=wide`	StairCut
`QWEN38_STAIR_TABLES`, `_RHO`, `_SKIP`	its prices, the lookup prior, the head-skip rule
`--len-fixed 8|16`, `--len-latch`, `--drop-idle`	with StairCut off: pin a width, decide once a request, release the loser
`--tree`, `--budget`, `QWEN38_TREE_NODES*`, `QWEN38_TREE_MS`	the tree and its node budgets (StairCut off)
`--corpus`, `--suffix-store`	the lookup drafter's corpus and persistent store
`QWEN38_DEEP`, `QWEN38_DEEP_AFTER`	the deep chain after long accepted paths
`--sampled-tree det|mixed`, `QWEN38_DRAFT_TEMP`	sampled requests on the tree, and the drafter's temperature
`--learn-cost`, `--no-learn-cost`	learn verify and draft costs online, or keep the price table
`--think-budget`, `--reasoning-effort`	the server's reasoning defaults
`--drafter mtp`, `QWEN38_FN_DRAFT_HOT`	Flash-Next: its MTP layer as the drafter, through a head of the 32,768 most frequent ids
`QWEN38_FN_LOOKUP`	Flash-Next: the lookup in the MTP round's tree (on in its profiles)
`ops/prices/<model>/<profile>.<device>.json`	the shared StairCut's price file

Chapter 12

Exactness

Under greedy decoding, TandemLLM writes the text its target model writes one token at a time, with one exception: at a near tie between the two best tokens, a batched verify can round the other way. Drafters, the lookup store and the caches change how fast the text arrives, not what it says. The length router picks the tree's shape from timings, so two runs of the same request can also part at a near tie (Section 12.3). This chapter states the promise, names the exception, and lists the checks that hold the engine to it.

12.1 The promise

The reference is plain greedy decoding on the same weights: one forward pass per token, take the highest-scoring token, repeat. For every greedy request, the engine's output tokens equal the reference's tokens up to the first near tie where the two paths choose differently.

"The same weights" matters. A profile chooses the weights, and the NVFP4 profile is held to NVFP4 plain decoding, not to the BF16 release or the FP8 checkpoint. The difference between weight sets is a quality question, and the quality gate measures it separately (Section 9.4). Speculation never adds to that difference.

The promise, with its exception, covers:

- any drafter, including one that is wrong on purpose on every token;
- chains and trees of any shape the router builds, and both block widths;
- a request resumed from the prefix cache or the resident prefix;
- request options that change the scores (penalties, `logit_bias`, structured-output masks), because the engine applies them to every verified row exactly as plain decoding applies them to its one row.

12.2 The exception: near ties

Two tokens can score the same up to one rounding step of the arithmetic. Which one wins an argmax then depends on the order of a floating-point sum. A batched verify and a one-token step do not always sum in the same order, so at such a position the two can pick different tokens. After that position the texts differ, and both are correct greedy continuations of their own prefixes.

The engine keeps this rare by design. The head returns fp32 logits, and every matrix-product kernel fixes its reduction order by the weight's shape and never by the row count (Chapter 10). Two differences remain between a batched verify and a one-token step: the output head uses one kernel for a single row and another for several, and the verify path of the recurrent and attention layers does the one-token arithmetic in a different order. A small fp32 difference that falls on a BF16 rounding boundary changes one value; the recurrent state and the KV cache carry it forward, so over a long output the verify's logits drift from the one-token decode's. Where the two best tokens are closer than that drift, the two paths can choose differently.

The agreement gate, `tools/verify_spec.py`, prints the gap between the top two logits in BF16 units in the last place (ulps) wherever outputs differ. A difference passes only when the gap at that position is within one ulp; anything larger is a bug. The gate decodes a fixed number of tokens and does not stop at the end-of-turn token, so a difference after that token, which no answer served with the default stop tokens contains, is judged separately.

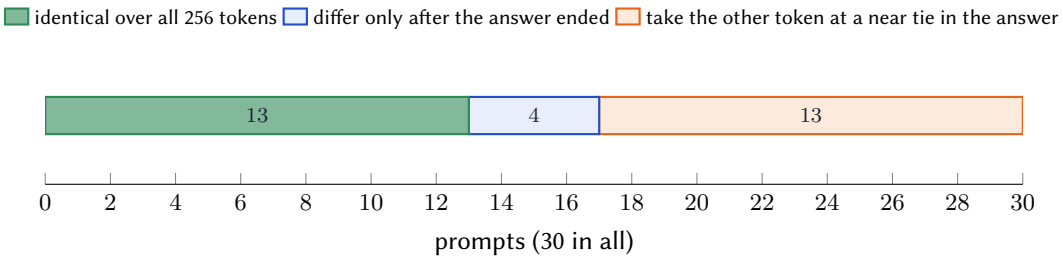


Figure 12.1: StairCut in its served configuration against plain greedy decoding on 30 prompts of held-out prose, code or chat (256 new tokens). 17 outputs are identical to the end of the answer. The 13 that diverge inside the answer do so at a gap of 0.04 to 0.76 ulp; no output differs inside the answer at a larger gap. measured: the StairCut paper, published NVFP4 weights.

The paper measured how often this happens (Figure 12.1). Of 30 prompts, 17 outputs are identical to plain greedy decoding to the end of the answer: 13 over all 256 tokens, and 4 that differ only after the answer ended (largest gap 1.46 ulp). The other 13 follow plain decoding up to a near tie inside the answer, with a gap of 0.04 to 0.76 ulp, take the other token, and continue from there; in one of them the token taken is the end-of-answer token, so that answer ends early. The one-ulp rule names the condition under which the two decoders can differ. It does not show that the difference is harmless.

12.3 Two runs of the same request

The length router sizes each block from timings: the draft and verify times it measures inside the request and, unless the server runs with `--no-learn-cost`, the costs it learned from earlier requests.

Two runs of the same prompt can therefore build different trees. Identical tree shapes give identical bits, but different shapes can move a near tie, so the text of two runs is the same only while their shapes are. Two runs alone can part this way, and so can a run with side-lane guests and one without.

Table 12.1 has a measurement on the 0.4.0 code (lookup store off, 2026-10-05): one raw greedy turn with a prompt of about 5,000 tokens at temperature 0, sent alone twice and once with 10 side-lane guests. Each parting is a near tie. Where the runs alone parted, the gap between the two best tokens was 0.022 and 0.029 nats at token 12, and 0.052 and 0.021 nats at token 504. Each text after the parting is a correct greedy continuation of its own prefix. `--no-learn-cost` takes out what earlier requests taught the router, not the timings inside the request.

Table 12.1: Where two runs of one 5,000-token raw greedy turn first wrote a different token. measured, the 0.4.0 code, lookup store off.

runs compared	`--no-learn-cost`	learned costs
alone, twice	token 12	token 504
alone, and with 10 side-lane guests	token 61	token 573

The checks below therefore hold arithmetic to bit equality, and text only where the shapes are fixed. A text check on the served engine (`tools/api_text_check.py`, `tools/lane_check.py`) can show a difference with no bug behind it. Such a difference is judged by the near-tie rule, as `tools/verify_spec.py` judges one.

12.4 Release identity

Every release candidate is also held to the release before it. The served configuration with the new flags off must produce bit-identical logits and state to the build it replaces (`tools/flagoff_identity.py`). That tool runs one fixed scenario: a prefill, chain verifies of 16 and 8 rows, a branching tree verify and a one-token step. The shapes are fixed, so it compares the arithmetic, not a router's choices; given the same tree shapes, the candidate writes the bits the release writes. A speed change that alters one bit of that scenario does not ship. The text check sends 13 plain requests (the five benchmark workloads, streamed and not, one with thinking, one with tools, one under penalties, one seeded sampled request) to both builds and compares them byte for byte, pass for pass (`tools/api_text_check.py`). It runs the served router, so a difference there is read with the near-tie rule, and the same build's second pass is the control. StairCut is the one documented exception: its verify shapes differ from the release before it, so at near ties its text can differ. It is held to plain greedy decoding instead, under the near-tie rule. Every release since has passed the identity gate against the one before it, on the raw path.

12.5 Sampling

A sampled request (temperature above 0) is exact in distribution. At each verified node the engine draws the target's own token, and a drafted token survives exactly while the draw lands on it; when

the draw lands elsewhere, that draw is the token. For a drafter that proposes one fixed token this is the standard rejection sampler, accepting with probability $\min(1, p/q)$, with no second draw. All sampling filters (`temperature`, `top_p`, `top_k`, `min_p`) run after the penalties, through one function in `engine/sample.py`, so drafted rows and plain rows see the same distribution.

A request with a seed draws with noise keyed by its position: the token at index t is the argmax of $\log p_t + G_t$, where G_t is Gumbel noise from a generator seeded by (seed, t). That is an exact sample from p_t, and it does not depend on how many draws the drafters made before, so the same seed gives the same text whatever the drafters proposed, up to the near-tie rule: another tree shape can move the last bits of $\log p_t$, and where two values of $\log p_t + G_t$ are that close, the draw can go the other way. `tests/test_sample.py` checks the samplers statistically on the CPU, and `tools/sampled_dist.py` checks the served engine's sampled answers against the target sampled alone.

12.6 Caches

A cache restore gives back bytes, not an approximation. The prefix cache, the resident prefix and the disk tier take their snapshots on the prefill chunk grid, and a resumed request forwards the rest of its prompt on the same grid, so it computes the same arithmetic as a cold prefill on that grid. Tests on a small random model check that a resumed request's logits and state equal a cold prefill's bit for bit. The disk tier's files carry a fingerprint of the weights, the engine's code and the arithmetic flags, and a file of another fingerprint is never read. On the served engine, `tools/disk_restore_check.py` checked that a second process's cold prefill equals the first's bit for bit and that the first process's disk files restore to it (measured, a build of the 0.4.0 series, a 10,701-token prompt).

Session resumes are one step weaker. A session resume continues a conversation from the state its last turn ended in, which verify rounds wrote rather than prefill chunks. That state produced the previous answer, but it is not the state a cold re-read would compute. It is held to the greedy gate (same tokens), not to bit equality. Resuming at the resident prefix's end anchor is of that kind, and so is everything a prefill that resumed there writes. The engine tracks the lowest such row (`taint`): the state store takes no prefix snapshot of such a prompt, the disk tier writes nothing of it, and resuming at a grid anchor below it gives a cold grid prefill's state again, bit for bit. Chapter 13 has the details.

An opt-in response cache (`--response-cache`) answers an identical greedy request from memory. It is off by default, so a benchmark can never be pointed at a dictionary.

12.7 What changes output on purpose

Some features change the text because the client or the operator asked for it. They are not exceptions to the promise: plain decoding with the same setting writes the same text.

- Penalties (`repetition_penalty`, `presence_penalty`, `frequency_penalty`) and `no_repeat_ngram_size`.
- Structured outputs and forced tool calls, which mask the tokens the constraint does not allow.

- A reasoning budget, and the conductor's hidden closes and notes, which change what the model reads.

Two relaxed accept rules exist (`--relax-tau`, `--relax-rank`) that accept a drafted token when it is close to the target's choice. They break the promise, exist only for measurement, and are off in every profile; the server prints a warning when either is set.

12.8 Flash-Next: no near-tie exception

Qwen3.8-Flash-Next is held to a stricter promise. Its verify kernels give every row the decode step's bits, whatever the number of rows, so a greedy speculative run equals plain greedy decoding token for token, with no near-tie exception, and a seeded sampled request draws what plain sampled decoding draws. The load-time self-check proves the row property on the device before speculation turns on. Two places are held to less. A request resumed from an end anchor follows the 27B's greedy gate, because the decode kernels wrote that state and the prefill kernels round differently. A think budget at low effort closes at a block boundary, so a speculative and a plain run close the reasoning a few tokens apart. Section 17.7 has the checks.

12.9 The checks

Table 12.2: The checks that hold the engine to its promise.

check	what it holds
`tools/verify_spec.py`	greedy output with no drafter, an adversarial drafter, a half-right drafter and the served drafters must match, up to the one-ulp rule
`tools/flagoff_identity.py`	the served configuration with new flags off is bit-identical to the build it replaces, on a fixed scenario of fixed shapes
`tools/api_text_check.py`	13 plain requests compared byte for byte with the current release, pass for pass; a near tie can part them, as it can two runs of one build
`tools/cache_gate.py`	warm and cold runs agree: bit-identical on the chunk grid, the same tokens for a session
`tests/test_forward_tree.py`, `test_tree.py`	tree verify equals chain verify on a tiny model, on the CPU
`tests/test_cache.py`, `test_resident.py`, `test_disk_cache.py`	restores are bit-exact, the disk tier's through a reopened tier and after a failed restore
`tests/test_end_anchor.py`	resuming at the end of the last generation writes the same greedy tokens as one at the prompt's line
`tests/test_side_lane.py`	the engine is bit for bit an uninterrupted run's after a guest
`tools/lane_check.py`	a long request's tokens with guests against runs alone: the same up to the first fork, and the fork a near tie (the runs took each other's runner-up, top-2 gap within 0.125 nats, printed)
`tests/test_sample.py`, `tools/sampled_dist.py`	sampled output follows the target's distribution

Chapter 13

Caches

A long prose prompt sent twice took 44.4, 93.7 and 208.3 s to its first token at 32k, 64k and 128k tokens from cold, and 0.3, 0.6 and 1.1 s the second time, resumed from the resident prefix (measured, the paper, one request each). A prefill is the most expensive thing the engine does, and the caches exist so that it runs once per token of a conversation instead of once per turn. Every cache here restores exact state, and none of them is meant to change the output (Chapter 12).

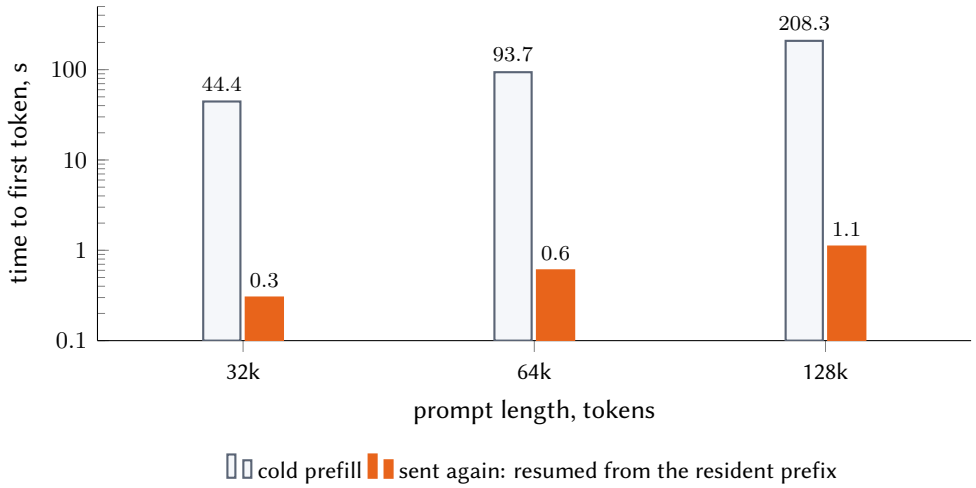


Figure 13.1: Time to first token for a long prose prompt, cold and sent a second time (log scale). measured: the StairCut paper, published NVFP4 weights, one request each.

13.1 What a state is

To continue a sequence at token L the engine needs three things:

- the KV rows of the 16 attention layers for tokens 0 to L, 64 KB a token;
- the 48 recurrent states and their convolution tails, about 147 MiB whatever L is;
- the drafters' own position-indexed rows, about 40 KB a token for both drafters.

The recurrent part is the hard one. A KV row depends only on the tokens up to it, so rows written for an earlier request stay valid. The recurrent state has absorbed every token and cannot be rewound, so a cache must have taken a copy at L.

The caches know no model. Every tier reads a sequence through the engine's adapter (`eng.cache_state()`, `engine/models/state.py`): its recurrent *sections* by name, copied whole, and its position-indexed *views* with their token axis, sliced to L. For the 27B the sections are `gdn.S` and `gdn.conv` and the views K and V. Flash-Next has more parts, and the tiers carry all of them with no code of their own for it:

- sections: 36 GDN states and convolution tails, the n-gram embedding's convolution and token context, and `mtp.h_last`, the last committed row's streams that its draft layer reads next, 115,662,864 bytes (110.3 MiB) whatever L is;
- views: K, V and the sparse-attention indexer's keys of its 12 attention layers, and the draft layer's own, 29,952 bytes a token.

Flash-Next's grid is 4,096 tokens, its prefill chunk, so a resumed prefill runs the chunks a cold one runs and restores from grid anchors, the state store, the disk tier and the spill equal a cold prefill bit for bit. A disk anchor of format 2 names its sections; the 27B's older files of format 1 are read as `gdn.S` and `gdn.conv`. Section 17.8 has the measured restores.

13.2 The tiers

Figure 13.2 shows the caches in the order the engine looks them up before a prefill. A deeper tier is taken only when it starts later than every tier above it, and the NVMe tiers only when their read costs less than the prefill it saves.

Table 13.1: The caches and their defaults in the served profile.

cache	what it keeps	where	default
resident prefix	the KV the last prefill and generation wrote, in place, with anchors every 1,024 tokens and one at the end of the last generation	device	on
state store	whole state snapshots at prompt boundaries (session cache) and chunk boundaries (prefix cache)	device	on, 2 GiB
persistent suffix store	token ids the engine read and wrote, with a suffix array	disk	on, 192 MiB
response cache	the tokens of an identical greedy request	memory	off
disk tier	rows and anchors of prefixes prefilled more than once, and of a session's first long request	NVMe	on, 50 GB
session spill	whole live conversations, at a planned stop and before an eviction	NVMe	on, 40 GB



Figure 13.2: The cache tiers, in lookup order, with the budgets of the served profile. Device memory first, then the NVMe, then a cold prefill.

13.3 One conversation through the tiers

Figure 13.3 follows one opencode session through a day and names the cache that answers each turn. The pattern holds for any agent client whose conversation only grows: the engine's own buffers answer the next turn, the NVMe answers after a restart or an eviction, and a cold prefill reads only what no tier holds. Each case is described in the sections that follow; the numbers in the figure are the cases those sections measured.

13.4 The resident prefix

An agent client such as opencode sends one conversation that only grows: each turn's prompt is the previous prompt, plus the previous answer, plus a tool result. At 190k tokens a whole snapshot would be about 20 GB, so the state store cannot keep one.

`engine/cache.py::ResidentPrefix` uses the fact that the previous prompt's KV is still in the engine's own buffer, row for row. It leaves the KV in place and clones only the recurrent state and convolution tails (146.8 MiB) at every 1,024-token boundary of the prefill, the *grid*. These copies are the anchors. A new request that shares the first c tokens of the resident prompt resumes at the largest anchor at or below c: the engine copies the recurrent state back, sets the KV length, tells the drafters their caches are valid up to there, and prefills the rest.

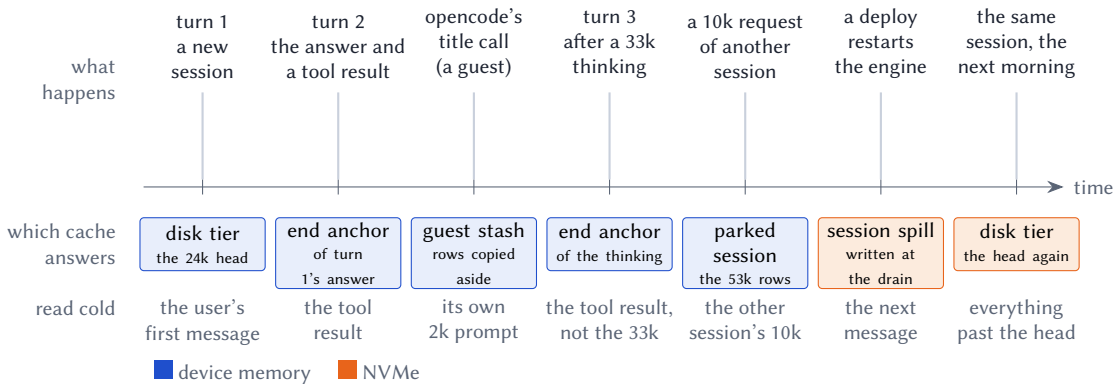


Figure 13.3: One agent conversation through the cache tiers. Top: what the client does. Middle: the cache the prefill resumes from. Bottom: what it still reads cold. A turn after a long thinking resumes at the end anchor and reads only the tool result; a title call is a guest whose rows are stashed and put back; another session's request parks the conversation; a restart brings it back from the session spill; the next morning only the common head is still on the NVMe. The token counts are the measured cases of Sections 13.4 to 13.8.

Resuming this way computes the same state as a cold prefill on the same grid. Anchors come only from prefill chunks, never from rows a verify round wrote, and the resumed request forwards the rest of its prompt on the same 1,024-token grid a cold prefill uses. `tests/test_resident.py` checks on a small random model that the resumed logits, recurrent state, convolution tails and KV rows equal a cold prefill's bit for bit. The anchor budget (`--resident-gb`, 2.5 GiB) holds 17 anchors, and the newest 4 are never evicted, because the next turn resumes near the end.

Resuming at a finer line than 1,024 was built and measured: a forward that starts inside a grid chunk does not reproduce the rows of the cold forward over that chunk on the served kernels, so it is not in the engine.

Guests. A short unrelated request between two turns, such as a title request, would overwrite rows the conversation needs. Such a guest (at most `--resident-guest-max` tokens, 8,192, and under a quarter of the resident prompt: both must hold) gets the rows it can reach copied aside first, into a stash (`--resident-stash-gb`, 1 GiB), and they are copied back before the next request. The stash holds what the guest can write: its prompt, its `max_tokens`, 2,048 rows for the monitor's hidden tokens when the monitor runs, and two margins of 512 rows, not every row the 1 GiB holds (a 410-token question used to ask the memory policy for 1 GiB, and a refused stash cuts the resident rows at the guest's start). With `--parallel N` a guest takes an idle side slot first and needs no stash (Section 14.13). A guest keeps no end anchor of its own: a short conversation that starts while a longer one is resident re-reads its first answer at its second turn.

The pinned head. A request too long to be a guest replaces the resident prompt. Before it cuts the rows, the head of the old prompt is pinned: its KV rows and up to 8 anchors spread over it are copied aside, as long a head as `--resident-pin-gb` holds (4 GiB, about 33,000 tokens). A later request that shares more with the pinned head than with the rows in place gets them copied back and resumes there. opencode's compaction request is the case this was built for.

The parked session. Two sessions used in turn (A, then B, then A) share their head and nothing after it. When a request of another session replaces the resident prompt, the rows it is about to overwrite, from where the two part, are parked with the newest 4 anchors (`--resident-park-gb`, 4 GiB, about 35,000 tokens past the head). A later request that shares more with the parked prompt gets them back: the parked rows and the rows in place are swapped a block at a time, so B is parked while A comes back. In a hold on 2026-10-01, A came back after B with 41,984 of 53,209 tokens reused and 11,225 re-read in 16.4 s, where it would have re-read about 45,000 (measured). `tests/test_resident_park.py` checks every swap against a cold prefill bit for bit.

13.5 After a long generation: the end anchor

The rows a generation writes are never anchored on the prefill's grid. Until the end anchor existed, the next turn shared at most the previous prompt with the resident rows and resumed at its last anchor, so it re-read the whole previous output. On 2026-10-02 one opencode turn prefilled 34,038 tokens and thought for 32,773; the next turn re-read 33,172 tokens in about 44 s (measured).

The end anchor (`--end-anchor`, on) fixes that. At the end of every generation the resident prefix keeps one more anchor: the recurrent states and convolution tails at the last position the engine forwarded (147 MiB, the same two buffers turn after turn), with the rows in place marked valid up to there. A generation that stopped part way through an accepted block has its pending commit cut back to the committed tokens first. The next request that starts with every one of those tokens (the prompt and the answer as the client sends them back) resumes there and forwards only what is new: the turn after a 33k thinking forwards its tool result and the template's glue. The ledger's cache source is then `end`. Figure 13.4 shows the idea.

An answer's reasoning the client does not send back. Open WebUI sends a finished answer back without its reasoning, and the Qwen3.8 template renders such a message with an empty `<think>` block, so the copy parts from the rows right after `<think>`. The end alignment keeps where each generation's reasoning ends and aligns such a copy with the reasoning taken out; the prompt is served with the rows' tokens up to the end anchor, the reasoning included, and resumes there (`QWEN38_END_ALIGN_REASONING`). Before it, every Open WebUI chat whose first answer thought re-read that answer on its first follow-up (cached 0.71 to 0.89, 2.5 to 5.0 s to the first token); with it the first follow-ups took 0.97 to 0.998 of the client's tokens from the cache (measured).



Figure 13.4: The resident prefix in an agent conversation. Turn 1's prompt is anchored every 1,024 tokens; its answer was written by verify rounds and gets one anchor at its end. Turn 2 sends back the prompt and the answer and adds a tool result: it resumes at the end anchor and forwards only the new part.

Verify blocks wrote that state, so resuming there is a session resume, held to the greedy gate rather than to bit equality (Chapter 12). The engine tracks the lowest such row (`taint`) and keeps the rest of the caches exact around it.

Two details make the client's copy match. The tokens the conductor hid in an answer (a hidden close's phrase, a note) are in the rows and not in the client's copy, so the server puts them back where they sit before the prefill. And the client's copy is text the chat template renders again, whose whitespace next to a hidden span can differ from the rows'; the replay aligns runs of whitespace-only tokens there (`--end-align`). In the served engine's suffix store on 2026-10-02 (4,607 turns that followed another), the next prompt held the engine's visible output token for token to its end in 3,844, 83 % (measured); where it did not, it parted late, and such a turn resumes at the prompt's last grid line as before.

13.6 The state store

`engine/cache.py::StateStore` keeps whole snapshots, keyed by the tokens that produced them, for two uses. The session cache keeps a conversation's state after its turn ends, and the next turn forwards only the new message. The prefix cache takes a snapshot at the longest chunk boundary of a prefill that fits the per-entry cap, so a shared system prompt costs nothing from the second request on.

A snapshot is about 151 MB of recurrent state plus about 104 KB a token of target and drafter KV. The budget therefore counts conversations long before it counts tokens. The served budget is 2 GiB (`CACHE_GB`) and one snapshot may take at most 1 GiB (`--store-entry-gb`, about 7,000 tokens): under opencode the store serves short chats, and longer sessions are the resident prefix's. Room is made before a snapshot is cloned, not after. Every hit re-checks the stored tokens element by element before restoring anything: a 64-bit hash is a hint, never an answer.

13.7 The disk tier

Every opencode session starts with the same 24,000 to 30,000 tokens: the reasoning header, opencode's system text and the tool block. The resident prefix serves that head only while its rows are in the engine's buffer; a restart, a deploy or a long unrelated request throws them away. `engine/disk_cache.py::DiskTier` keeps them on the NVMe.

It stores what a restore needs in two kinds of file on the prefill's 1,024-token grid. A chunk file holds rows $b - 1024$ to b of every position-indexed buffer (the attention KV and the drafters') and those 1,024 token ids, keyed by the hash of the whole prefix up to b: 104 MiB a chunk on the served stack. Two prompts that share 30,000 tokens share 30 chunk files. An anchor file holds the 48 recurrent states and their convolution tails at one line (147 MiB), with the hashes of the chunks below it; it is usable when every chunk below it is on disk.

What is written is decided before the prefill runs. Every grid line a prefill forwards cold is counted by its hash, and the longest line of the prompt counted twice (`admit_after`) gets an anchor. The first long request of a session gets anchors at the last 4 lines below where its user message starts, because the next session in the same folder parts there. A line where an earlier prompt read this far

and went on differently (a branch) gets one too. Prompts under 4,096 tokens are not written, and neither is anything a session resume wrote.

With `--parallel N` the side slots read the tier too: a side slot's prefill restores an anchor into its own buffers, the same bytes as on slot 0, and writes, counts and fences nothing of slot 0's, so a new chat sent to a side slot starts from its client's head on the NVMe (`QWEN38_SIDE_DISK`). The writes never hold a request. An anchor's state is copied to a staging slot as the prefill passes its line; a writer thread copies the rows out after the prefill and writes them while the request decodes. Each file goes to a temporary name first and is renamed after its fsync, with a crc32 per section. A read compares every stored token id with the prompt's before a byte reaches the engine, checks every section's crc32, and makes the restored prompt the resident one with the disk anchor as its anchor. A file that fails its check is deleted with every longer chunk of its chain, and the request resumes from what is in place or starts cold. Files of another fingerprint (weights, engine code, arithmetic flags, grid, library versions) are swept at the start.

On the board, O_DIRECT writes ran at 4.3 GB/s and reads at 7.35 GB/s (measured, `tools/disk_cache_bench.py`). On 2026-10-02 a 15,360-token head was restored in 334 ms and the server's time to first token went from 23.1 s cold to 2.7 s (measured, `tools/disk_cache_live.py`).

The dashboard's Memory screen, Disk cache tab, sets the budget (50 GB), how long an unused entry is kept (7 days) and the free space the tier leaves the file system, never under 50 GB. When it is full, the least valuable entry goes first: its hits plus one, times the tokens it saves per byte it holds, halved for every day unused.

13.8 Live conversations on the NVMe

The disk tier keeps what a cold grid prefill wrote, which in an agent conversation is the head. Everything past the first answer is session state. Two re-reads on 2026-10-03 show the gap: a deploy restarted the engine and the next turn of a 130,000-token conversation read 120,102 tokens again (158 s); minutes later a 10,216-token request of another client replaced the resident prompt and the conversation's next turn re-read everything above 32,768 tokens (165 s).

`engine/spill.py::SessionSpill` keeps whole conversations on the NVMe: the rows of every position-indexed buffer, the end anchor with its hidden spans, the newest 4 grid anchors, the token ids, `taint` and the session key. These are the bytes that were in the engine, so a restore is held to the gate the rows were held to in RAM. It writes at two moments. A planned stop (SIGTERM's drain) writes the resident and the parked conversation, and the next start reads the resident one back before its first request. A request of another session that is not a guest writes the resident conversation first when it has at least 16,384 tokens and the park cannot hold it. The spill writes by content, in blocks of 1,024 rows with two 64-bit sums per buffer computed on the device, so a block that did not change is not written again.

Measured on the board (the 0.4.0 series): a 41,587-token conversation evicted by another session was written in 1.03 s (5.20 GB) and read back in 0.94 s. After a restart through the drain, the next turn of a 41,719-token conversation resumed at its end anchor with a time to first token of 635 ms (measured). The NVMe's own write cache sets the bound: a sustained write ran at 4 to 5.6 GB/s for the first 16 to 24 GB and then at 0.3 to 1.3 GB/s until the drive rested.

13.9 Compaction as a continuation

opencode's `/compact` sends its compaction agent's prompt and one user message that holds the whole history as text. That text shares no token with what the engine read, so a compaction read tens of thousands of tokens cold while the conversation sat resident. `server/compaction.py` serves it instead as the continuation of the conversation the engine holds for the session: the tokens up to the end anchor of its last generation (in RAM, or its entry on the NVMe), then one user turn with the client's system text and instructions, thinking off. The prefill reads only that turn. A guard checks that the first and last user lines of the client's history are in the record; any miss serves the request as sent.

A summary still has to be written, about 2,000 tokens, and the server writes it ahead: when no request has come for 30 s and the resident conversation is an opencode session of at least 32,768 tokens, it runs the continuation in scratch rows, under the board's lock taken without waiting. A request that comes for the engine, or a hold that asks for the lock, stops it at the next chunk or block, and the conversation's rows and anchors stay as they were. A compaction request of that session at that end anchor is then answered from the summary without the engine.

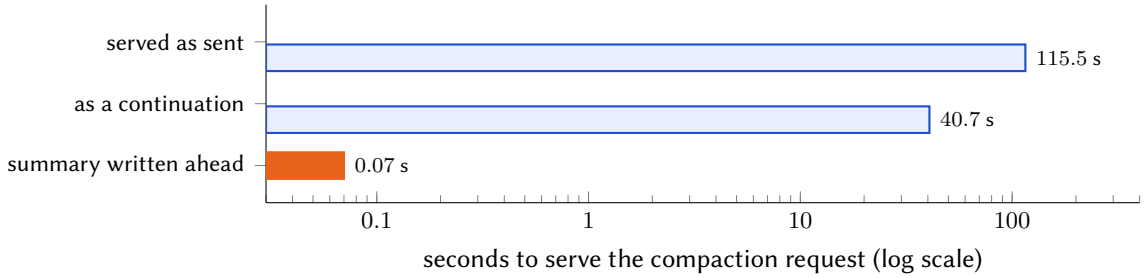


Figure 13.5: One captured 57,008-token compaction request of a 104,313-token opencode turn, served three ways. As a continuation the prefill took 1.04 s; the rest was writing the 1,406-token summary. measured, 2026-10-04.

13.10 The persistent suffix store

`engine/cache.py::PersistentSuffixStore` is an append-only log of the token ids this engine has read and written, with a suffix array over it. The lookup drafter reads it beside the corpus. It holds token ids and never text, in a directory of mode 0700 outside the repository, capped by `--suffix-store-mb` (192 MiB, about 48 million tokens); over the cap it forgets the oldest half. A request adds only what one of the last four requests did not already add, so an agent conversation that resends its growing prompt is not written 200 times. A store that has seen a benchmark's prompts inflates that benchmark, so benchmarks run with the store off or clean (Chapter 15).

13.11 Images in a prompt

An image's prompt rows are placeholder tokens, and the placeholder is the same token for every image, so a cache keyed on tokens would answer a request with another image's state. The caches therefore compare key ids in which each image's rows are replaced by an id taken from a digest of

its preprocessed patches. The same URL with new pixels misses; the same pixels under another URL hit. The vision tower's output is cached by the same digest (`--image-cache-mb`, 512), so a conversation that sends an image on every turn encodes it once.

13.12 Memory limits

All caches share the board with the weights and the KV buffer. `engine/cache.py::Budget` holds them to three limits, and Section 14.10 describes the gate that enforces them before every prefill. Every limit decides only where a prefill starts; a cache given back is the cold path, never a different token.

13.13 Routes and knobs

`GET /v1/cache/stats` reports every cache's counters, and `POST /v1/cache/clear` empties them (the admin rule; `?tier=disk` the disk tier, `?tier=all` both). The flags are in Table 27.1; the main ones are `--resident-gb`, `--resident-stash-gb`, `--resident-guest-max`, `--resident-pin-gb`, `--resident-park-gb`, `--end-anchor`, `--cache-budget-gb`, `--store-entry-gb`, `--prefix-chunk`, `--disk-cache`, `--session-spill` and the `--compaction` family.

Chapter 14

Serving

`server/app.py` speaks the OpenAI HTTP API, so any OpenAI client uses the engine by changing its base URL. It is built on the Python standard library (`ThreadingHTTPServer`), with no web framework. This chapter covers the request fields, reasoning and thinking budgets, tool calls, structured outputs, streaming, images, and how the server behaves under load: the queue, the side lane, the memory gate and the loop guard. Chapter 29 lists every route.

14.1 What the API supports

`server/compat.py` gives every OpenAI request field one of three answers, and `tests/test_compat.py` pins the table.

- Honoured: `messages`, `model`, `stream`, `stream_options`, `max_tokens`, `max_completion_tokens`, `stop`, `temperature`, `top_p`, `seed`, `presence_penalty`, `frequency_penalty`, `logit_bias`, `logprobs`, `top_logprobs` (up to 20), `n` (up to 16, not streamed), `tools`, `tool_choice`, `parallel_tool_calls`, `reasoning_effort`, `response_format`, `modalities` (text output only). Messages may carry `image_url` parts.
- Accepted, without effect by their own definition: `user`, `metadata`, `store`, `service_tier`, `prompt_cache_key`, `safety_identifier`, `prediction`.
- Refused with a 400 that names the field: `audio`, `functions` and `function_call` (use `tools`), `web_search_options`, `verbosity`, and for completions `suffix` and `echo`. A malformed `tools` (not a list, an item without a function name, `parameters` that is not an object) is a 400 that names the item.

The request's own shape is checked first, and a wrong one is a 400 that names the field before anything reads it: no `messages` or an empty list, an unknown role (`messages[0].role`), a system prompt alone or only tool results (no user message to answer), a system message after the conversation began, `max_tokens` or `max_completion_tokens` below 1, an empty completion `prompt`, a body that is not a JSON object, and a prompt that renders to no token. OpenAI's `developer` role is the system prompt; a system and a developer message at the start become one system prompt, because the chat template reads one.

Fields that are not OpenAI's pass through without a 400, because clients send extensions meant for other servers. The engine reads its own: `top_k`, `min_p`, `repetition_penalty`, `no_repeat_ngram_`

`size`, `max_reasoning_tokens`, `reasoning_format`, `draft_temperature`, `chat_template_kwargs`, and the other servers' thinking-budget shapes.

14.2 Reasoning

The model thinks inside `<think>` tags. The chat template opens the block in the prompt, so the model only ever writes `</think>`; the server re-emits `<think>` as the first delta so that clients which fold on a matched pair see one. `--reasoning-format`, or `reasoning_format` in the request, picks how the thinking reaches the client:

Table 14.1: Reasoning formats.

value	content	reasoning_content
`tags` (default)	the thinking inside `<think>...</think>`, then the answer	empty
`reasoning_content`	the answer only	the thinking, as deltas
`both`	as `tags`	as `reasoning_content`

14.3 Thinking budgets from the client

A client may cap the thinking in any of the shapes clients send, and `server/thinking.py` reads them all into one budget.

Table 14.2: The thinking-budget shapes the server reads.

shape	read as
`max_reasoning_tokens`, `thinking_budget`	a cap in tokens
`chat_template_kwargs.thinking_budget`, `.max_thinking_tokens`, `.reasoning_budget`, `.thinking_token_budget`	a cap in tokens
`reasoning: {max_tokens, effort, enabled}`	`max_tokens` a cap, `effort` a word, `enabled: true` alone medium, `enabled: false` off
`thinking: {type, budget_tokens}`	`enabled` with `budget_tokens` a cap, `enabled` or `adaptive` alone on without a cap, `disabled` off
`reasoning_budget`, `max_thinking_tokens`, `thinking_token_budget`, `thinking_budget_tokens`	a cap in tokens
`reasoning_effort`, `reasoning.effort`, `chat_template_kwargs.reasoning_effort`	the level's cap: `minimal` and `low` 1,024, `medium` 8,192, `high` 16,384, `xhigh` and `max` none; `none` is thinking off
`enable_thinking: false` (top level or in `chat_template_kwargs`)	thinking off

Whether the model thinks is decided first: by `chat_template_kwargs.enable_thinking` when it is set, else by the top-level `enable_thinking`, else by any off form. The cap is then the first number

in the table's order, else the first effort word's cap. A number beats a word, and 0 is no cap. An unknown word or a negative number is a 400 that names the field.

With the conductor deciding the effort, a client's number always wins, and its effort word is a ceiling on the router's level (Chapter 18). When a client asked for a budget in any shape and the reasoning would otherwise run into the output limit, the reasoning closes early enough to leave the answer `--answer-reserve` tokens: `auto`, the default, is the smaller of 2,048 and a quarter of the limit. A request with none of these fields is served as before, token for token.

14.4 Tool calls

The request's `tools` go into the chat template, and the model answers a call in its own XML form:

```
<tool_call>
<function=read>
<parameter=offset>
150
</parameter>
</function>
</tool_call>
```

`server/toolcall.py` turns that into OpenAI `tool_calls` with `finish_reason: "tool_calls"`, on the streamed and the JSON path. It reads the strict form, a lenient form and a JSON call object, and only in the answer, never in the thinking. On a stream, a call goes out as argument deltas while the model writes it: the id and the name first, then each string value in pieces, so a whole-file write shows progress instead of minutes of silence.

Argument values carry their schema's types. The model writes every value as text, so `offset` arrives as the characters `150`. The parser looks up the request's own schema, and a parameter whose schema does not allow a string is returned as JSON: `150`, `true`, a list. A typed value is held back on the stream until it is complete, then sent whole. On an 8-task agent run, a strict client that validates arguments against the schema refused 58 of 80 calls before this, and 0 of 26 after (measured).

`tool_choice` works as OpenAI defines it. `none` renders the prompt without tools. `required` and a named function become a constraint on the answer, the same token masks structured outputs use. `parallel_tool_calls: false` keeps one call.

14.5 Structured outputs

`response_format` with `json_object` or `json_schema`, and `structured_outputs` with a `regex`, a `choice` list or a JSON schema, constrain the answer to a regular language. `engine/grammar.py` compiles each form to a finite automaton over bytes, because the tokenizer is byte-level and a token can end halfway through a character. A token is allowed in a state when walking its bytes never reaches the dead state; the engine computes that for the whole vocabulary at once per state and caches it, so a state costs a few milliseconds once. Each mask applies to every verified row of a draft tree, not only to one row per step, so constrained output under speculation is the text plain

constrained decoding writes. JSON schemas with recursion are refused, and `json_object` allows a fixed nesting depth.

A `</think>` the model writes again in its answer, alone on a line, is not sent under a conductor profile: Open WebUI closes a reasoning block at the first one and showed the second as text between two tool rows. The tag in a sentence, in code, in a tool call or closing a block the answer opened itself stays, and the raw path sends what the model wrote.

Open WebUI's terminal tools end `run_command`'s description with the terminal's live working directory, inside the system prompt's tool block, so an agent's first `cd` changed the prompt about 4,800 tokens in and the next user message read the whole conversation again (measured: 41,749 tokens, 56 s). Under a conductor plan that is on, the sentence moves into a note at the turn where the directory changed and replays there, so the tool block stays the same for the session (`QWEN38_CLIENT_CWD`, `note`; `off`: the tools as sent).

A client kind can have a format of its own when it names none and the server's is `tags`, under a conductor plan (`QWEN38_CLIENT_REASONING`, `kind=format,...`, `*` for every kind not named). The default, `*=reasoning_content,hermes=tags`, gives every client but the Hermes agent `reasoning_content`: an OpenAI SDK or curl client printed `message.content` and showed the model's thinking as its answer, and opencode with the installer's provider configuration showed it as literal `<think>` text. A client that wants the tags inline sends `reasoning_format: tags`. For Open WebUI the reason is its live view: Open WebUI 0.11.3 folds a `<think>` block out of the content after the fact and shows the stream piece that carries `</think>` as the text of a new message until the round's stream ends, so a long agent chat showed a literal `</think>` in 15 of 70 page scans; with `reasoning_content` three agent chats showed it in none of 683 scans (measured). For that client the template's blank lines go out as one content piece where the reasoning ends, so Open WebUI closes and times each tool round's block even when the round goes straight to a tool call; a blank message is not shown. For the other clients the answer starts with its first character. A request's own `reasoning_format` wins, and the raw path keeps the server's.

Under a conductor plan a request's `stop` strings apply to the answer, never inside the reasoning, and the blank lines that start the answer are not matched: `stop: ["\textbackslash n\textbackslash n"]` used to end a response at the first blank line of the thinking, with no answer. A plain API client (curl, an OpenAI SDK, an unnamed client) gets no status line in its content. A request that offered no `tools` never gets `tool_calls`: a call the model writes anyway is read out of the answer and dropped, and the finish stays `stop`.

A JSON answer (`json_object`, `json_schema`) is parsed whole by its client, so a request that names no `reasoning_format` gets `reasoning_content` for it: the thinking goes to its own field and `message.content` is the JSON alone. A client that asks for `tags` keeps them. Such a request also gets the answer reserve a client's own thinking budget gets (Section 14.3), so the reasoning cannot spend `max_tokens` and cut the JSON, and it carries no status lines.

14.6 Streaming

A streamed response sends one chunk per token, carries `usage` on the last chunk when the client asks for it (and by default, `--usage-default on`), then a chunk with `finish_reason` and `data: [DONE]`.

A verify round accepts several tokens at once, so they reach the socket together: half the gaps between tokens are microseconds and the other half a whole round.

usage counts what the client sent and what it received. `prompt_tokens` is the client's messages as the chat template renders them; the tokens the conductor adds (its new and replayed notes, and the closed think block of a turn it turned thinking off) are `usage.prompt_tokens_details.hidden_tokens`, absent when there are none, and `cached_tokens` is the client's tokens served from a cache: the cached span less the hidden tokens inside it, which the server tracks by position (an opencode turn that re-read 3,827 of its 24,706 client tokens used to report all 24,706 cached; it now reports 20,879). The engine's own counts of the whole prompt are in `timings`, and the ledger keeps them as `engine_cached` and `forwarded_tokens`.

`finish_reason` is `stop`, `length`, `tool_calls`, `timeout` (the request's wall-clock cap) or `error`. On `error` the partial text has already been sent and an `error` object sits beside the finish reason. A client that hung up is logged as `abandoned`, not as an error. `server/stream.py` turns tokens into characters and holds back a trailing partial character until the next token completes it, so every character it sends is final and no replacement character reaches the client.

14.7 A long prefill, and a client that leaves

A long prompt prefills for minutes: 208.3 s at 128k tokens (measured, the paper). After every prefill chunk the handler checks its socket. If the client has gone, the prefill stops there, the request ends `abandoned`, and the engine is free at once; the rows already prefilled stay resident, so a retry continues from there. A streamed prefill that has run for 5 s also sends an SSE comment after each chunk, at most once a second (`: prefill 12288/95131`). Clients ignore comment lines; proxies see a live stream and do not time it out.

14.8 Images

A user, assistant or tool message may carry `image_url` parts with an `https://` or `` URL. The server fetches and decodes them before the request queues and runs the checkpoint's own image processor: 16-pixel patches, 2 by 2 merge, 65,536 to 16,777,216 pixels. Each merged patch is one prompt row, so a 640 by 480 image is 300 rows, and those rows count as prompt tokens in `usage`. The vision tower (27 blocks, 0.92 GB, BF16) runs inside the prefill, when the first chunk that holds the image runs; the live view reads "Encoding image 1 of 2" meanwhile.

A bad image is a 400 that names the part: a scheme other than https or data, a failed download, broken base64, a format the server does not read (it reads five: PNG, JPEG, WEBP, GIF, BMP), more than 20 MB, a header that declares more than 64 million pixels, more than 16 images in the newest message, more than 65,536 image rows in the newest message, an image in a system message, a video part, or any image when the server runs with `--vision off`. `--image-act-gb` (6) refuses an image whose encode would need more memory than that, before it queues.

An https URL is a request the server makes for the client, so it is fenced. The host must resolve to public addresses only: loopback, private, link-local, shared, multicast and reserved ranges are refused, IPv4 inside IPv6 included. The connection goes to the address that was checked,

with the certificate checked against the host name, so a second DNS answer cannot move it. A redirect is followed only to another https URL that passes the same check, at most 3 times. `--image-https-private on` lets URLs reach private addresses; `--image-https off` accepts data URLs only.

A long conversation is not refused for the images it has gathered. A coding agent that reads a screenshot every few turns sends every earlier image again on every turn, so once it has read 17, every later request would be over the cap of 16. When the images are over `--max-images` or `--max-image-rows` and the excess is in earlier messages, the oldest images are left out instead, each replaced in the prompt by a fixed text. They go in steps of half the cap (8 by default), counted from the conversation's start, so the prompt's head that the caches resume stays the same for several turns. The response header `X-Tandem-Images-Left-Out` says how many.

14.9 Admission and limits

A sequence serves one request at a time, so requests wait for a free one in a bounded queue. The shipped profile has one sequence; Section 14.13 adds more.

Table 14.3: Admission limits. Code defaults; the served values are in parentheses where they differ.

limit	value	what happens
`--max-queue`	8	the next request gets a 503 with `Retry-After: 5`
`--queue-timeout`	120 s (240)	a request without a stream that waited that long gets a 429; a streamed chat request waits up to the request timeout
`--request-timeout`	900 s (3,600)	a generation that runs that long ends with `finish_reason: "timeout"`
`--max-len`	8,192 (262,144)	a prompt that does not fit gets a 400; `max_tokens` is clamped to what fits
`--default-max-tokens`	8,192 (32,768)	used when the request sends no `max_tokens`

At load the server allocates the KV buffer for the whole context, so the context length is a memory decision taken before the first request (Table 6.1).

14.10 The memory gate

Three limits hold the engine's memory. `--mem-ceiling-gib` (70) is what the engine may hold between requests, allocated plus the allocator's reserve; a cache grows only while the engine stays under it. `--mem-peak-gib` (77) is the ceiling plus the workspace of a 262,144-token prefill, what the engine may hold during a prefill. `--reserve-slack-gib` (2) is how many free blocks the allocator may keep between requests; over it they go back to the driver after the request, never inside one.

Before every prefill the gate brings the engine back under the ceiling and then checks that the prompt's estimated workspace fits under the peak. When it does not, it gives caches back, cheapest first (Figure 14.1). The last level releases the small models' own memory pools. Before a refusal the gate collects and empties the allocator once more, and a prompt whose workspace fits in the

allocator's own free reserve is admitted. When nothing is enough, the request gets a 503 with `Retry-After` instead of running the board into its out-of-memory wedge, and the Dev view shows an error line with the allocated and reserved figures.

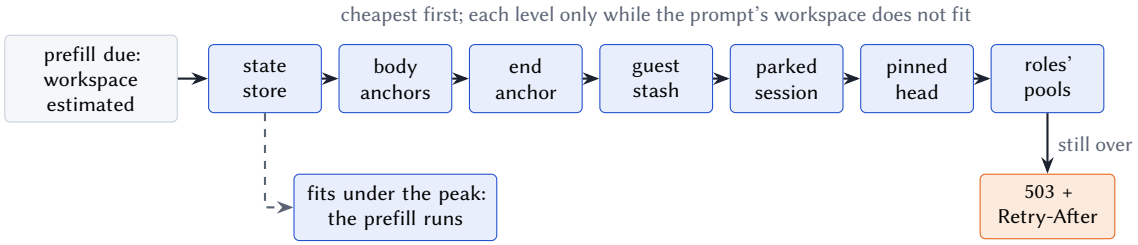


Figure 14.1: The memory gate before every prefill. Each level is given back only while the prompt's workspace does not fit under the declared peak (77 GiB); a cache given back costs a later cold read, never a different token.

14.11 The side lane

A short request does not wait for a long turn to end. When a request finds the engine busy and is short (at most 2,048 prompt tokens and 64 output tokens by default, one choice, no images, thinking off) and is either a raw request or a client's side request such as a title call, it becomes a job. The generation that holds the engine serves it at its next safe point, between two decode blocks or between two prefill chunks (`--side-lane decode` leaves the prefill out, `off` turns the lane off). The guest is prefilled from position 0 on the prefill's grid and decoded one token a forward with no drafter, so its answer is plain greedy decoding of its prompt.

The generation gets the engine back as it was. `engine/cache.py` `Borrow` copies what the guest can write: the KV rows the guest can reach, the recurrent state, the convolution tails, a walked chain state, and the record of a pending commit, copied unsettled so the next verify folds the same record into the same bytes. The drafter is never touched. A copy costs about 0.3 GB for a 2,000-token guest and is freed after it. `tests/test_side_lane.py` checks on the random model that the next block's logits, the state and every KV row are bit for bit an uninterrupted run's. The turn's text equals an uninterrupted run's only when its tree shapes are the same: the length router sizes blocks from timings measured inside the request (and, without `--no-learn-cost`, learned across requests), so a guest can change a later tree and a near tie can go the other way, as it can between two runs alone (Section 12.3).

Measured on the engine (the 0.4.0 series, `tools/lane_check.py`, a 5,000-token prompt with 768 output tokens): with 10 short requests fired into it, the long request's reasoning and answer matched two runs alone in that hold. Nine of the ten were served in the lane, 1.2 s at the median and 5.1 s at most; the tenth asked for more output than the lane's cap and waited. The long request took 7.4 s longer, for 4.7 s the guests held the engine (measured). A later measurement on the release's code parted at token 61 with guests, and at token 12 between two runs alone, both at near ties (Table 12.1).

Three guards apply. A job waits for a later safe point when the engine's memory plus the guest's copy and workspace would pass the declared peak. A disk-tier write that has not copied the rows

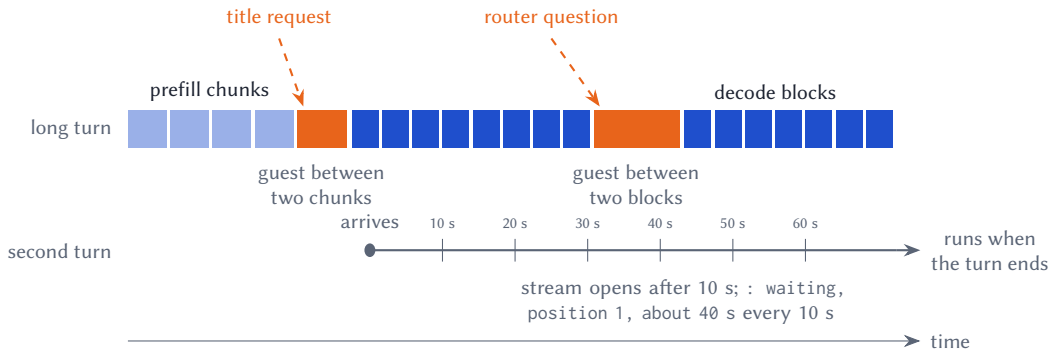


Figure 14.2: The scheduler. A long turn holds the engine. Short guests (a title request, the router's own question) are served at the turn's next safe point and the turn gets its state back bit for bit. A second full turn waits in the queue; its stream opens after 10 s and carries a comment every 10 s.

below the guest yet stops where it is. A job no holder takes runs on its own thread under the lock. Side-lane rows are no speed sample: a short plain decode beside a turn says nothing about the engine's decode rate.

14.12 A turn behind a running turn

The side lane serves short requests only. A second session's full turn still waits for the running turn to end. A streamed chat request waits as long as one request may run (`--request-timeout`). After 10 s in the queue its stream opens and an SSE comment goes out every 10 s:

```
: waiting, position 1, about 40 s, 30 s so far
```

A comment is not an event, so no client reads it as a token, and a client or proxy with an idle-read timeout keeps the connection. With the profile's progress lines on, the waiting line carries the same estimate (Chapter 21). The estimate (`server/queue_wait.py`) reads the work of the request that holds the engine: the prefill it still has to read at its live rate, plus the tokens it is likely to still write, the 75th percentile of what earlier requests of its kind (thinking or not) wrote past the tokens it has written, never past its `max_tokens`, at its own decode rate. Each waiter ahead adds its typical time. Replayed over 3,716 requests of the reference box, with the waiter arriving at 10 to 90 % of each hold, the estimate for waits of 10 to 60 s was a median 0.52 of the real wait when it used only the hold times of earlier requests, and is 1.06 now; for waits over a minute 0.17 before and 0.59 now (measured). Without a view of the holder, or with too few finished requests of its kind, the earlier rule stays: the median remaining hold time of requests that ran as long, and with too few of those the position only. A request without a stream cannot show that it is alive, so it keeps `--queue-timeout` and its 429. A full queue is still refused at once with a 503. A request that has waited less than 2 s gets no line. When TandemLLM's own work holds the engine (the shadow reader, a summary prepared ahead), the line says so in plain words, and since that work yields at its next chunk, its estimate is the median of the waits requests really had behind it; the ledger's `wait_own_ms` records each request's time behind it.

Two full turns are not interleaved on one sequence. Serving a second turn between the first one's blocks would mean prefilling its whole prompt and copying every KV row it can reach, each time it

gets a slice; for a full turn that would cost more than it saves and hold two conversations' state at once. The client is told how long the wait is instead, unless the engine holds more than one sequence (Section 14.13).

14.13 Parallel requests

With `--parallel N` (`QSE_PARALLEL`, off by default) the engine builds N - 1 side slots beside itself. Each is one sequence's complete state over the one copy of the weights: its own KV, recurrent state, verify graphs and drafter stack, `--side-max-len` tokens long (65,536). Slot 0 keeps the full window and every cache. A request takes a side slot when slot 0 is busy, its prompt and `max_tokens` fit, it has no images, and it does not continue slot 0's resident conversation; everything else about it is served by the same code. Whether a request continues that conversation is read off the client's own tokens: the server keeps the last slot-0 prompt whose rows stayed resident as the client's messages render it (no router note, no generated row), and a request whose rendered prompt starts with all of it, within one prefill chunk, waits for slot 0 and resumes in place. The resident rows themselves would not do: under a conductor plan they hold the router's notes and the end of the last generation, so an agent's next turn would never match and would read its whole conversation again on a side slot. TandemLLM's own work on slot 0 (the shadow reader, a summary being prepared, the memory writer, the router's question) gives way to a waiting request, so it never sends one to a side slot (a request waits for it up to 15 s). Two long agent sessions at once mostly take turns on slot 0, each resuming its own conversation (one in place, the other from the park or the disk): a side slot keeps no conversation, so a long agent turn there would read its whole prompt again. Measured with two opencode sessions started together (measured, single runs): both done in 301 s taking turns, against 490 s with one of them on the side slot and 193 s one after the other. A side slot pays off for a new conversation, a short chat or a script. Sharing a client's head is not continuing a conversation: Open WebUI's system prompt and tool block (about 6,700 tokens) and opencode's (about 7,400) are the same for every chat, so a request continues slot 0's conversation only when the part it shares reaches past the end of its own first user message (`QWEN38_SIDE_CONV`). Without that rule a second Open WebUI tab asking a one-line question beside a short essay chat shared 6,675 of slot 0's 7,096 rows and waited 35.8 s with the side slot idle; with it, it ran on the side slot at once (TTFT 2.6 s, measured, n = 1). A side slot reads slot 0's disk tier, so that tab started from 6,144 tokens of the client's head on the NVMe (Section 13.7). One kind of request takes an idle side slot even when slot 0 is free: a guest of slot 0's conversation, short (at most `--resident-guest-max` tokens and under a quarter of it), sharing less than one prefill chunk with it, while slot 0 holds two chunks or more. The router's own questions before a new user message (its class and its package, 150 to 450 tokens) and a client's title, tags and follow-up tasks are such requests: on a side slot they cost the same prefill and leave slot 0's rows alone (`QWEN38_SIDE_GUESTS`, on). The side slots' buffers stay in the engine's memory limits whatever value a residency document gives the ceiling and the peak (`/health` `cache.memory.fixed`); in the test builds before this release the first request's document set them back and the caches lost the slot's 7.65 GiB, and the router's questions, refused their stash, cut the conversation's rows: replayed with one build (measured, n = 1 each), two new user messages after a 32,875-token conversation read 0 % from the cache (67.5 and 70.7 s) with the bytes lost and 99.9 and 99.3 % (6.4 and 9.4 s) with them. The `[parallel]` log line names the slot and why; the usage ledger's `slot` column records it.

The GPU is shared in turns. Two or more decoding requests draft in one packed pass over the drafter and verify their trees in one joint pass: every projection runs once over all their rows, so the weights are read once, and each request's tree is cut to its share so the pass stays at or under 32 rows, the range where a row's result does not depend on its neighbours. A request alone keeps its own graphed verify. Admission by bytes keeps MemAvailable at or above `--parallel-floor-gb` (15): a request waits while its own prefill workspace, the other requests' and what the caches may still grow into, together at most the engine's room to `--mem-peak-gib`, would take it under the floor. Every cache and every prefill passes the memory policy, so the engine never takes more than that room. The wait is queue time (`wait_mem_ms` in the ledger) and the client gets a waiting line. A 26,236- and a 17,582-token prompt prefilled and decoded at once with MemAvailable at 28.7 GiB or more (measured).

Measured on one DGX Spark (measured, single runs, four long-answer prompts of 384 tokens, store off): one request at a time 48.8 tok/s; two at once 67.4 in total (1.38×), four 69.2 (1.42×; 76.5 with `--parallel-cut joint`). With the flag on, one request at a time costs nothing. A side slot of 8,192 tokens takes 6.9 to 7.5 GiB beyond its window, most of it the pools of its 121 verify graphs.

Beside the small models. A box that also runs Laya and the narrator has less room. On a 128 GB DGX Spark at the 262,144-token window with Laya-english, the narrator (Qwen3.5-2B) and the shadow reader, MemAvailable at the worst moment with one sequence is about 22.7 GiB (measured). Four 8,192-token slots (about 25 GiB) do not fit; two do, with one side slot of 65,536 tokens and `--side-graphs off`: the slot then holds no verify graphs and verifies eager when it decodes alone (a joint pass is eager either way), about 1.5 GiB beyond its window instead of 7.5. The window matters more than the number of slots: a request takes a side slot only when its prompt, its `max_tokens` and the conductor's margin (8,192 tokens for a request it shapes) fit, and a chat client's `max_tokens` of 32,768 alone rules out an 8,192-token window. Side slots are built while the engine loads, before the residency planner loads Laya and the narrator, so the `residency` document's claims must be the models' measured sizes; with larger declared ones the planner refuses them. While two requests run, Laya answers only when slot 0 is idle (a decision forward takes the engine lock), and the narrator serves both requests, one line at a time.

A request beside others writes its alone text up to a near tie (Chapter 12): its tree is cut to its share, another shape than alone, and the shape can move a near tie. On 30 prompts 17 texts were identical to their alone text at two requests and 15 at four, every first difference at a top-2 gap under 0.12 nats. `tools/parallel_bench.py` measures the totals and the agreement and proves from `/health` that the requests decoded together.

Waiting for slot 0. A request that continues slot 0's conversation prefers slot 0, but since 0.8.2 it waits there at most `QWEN38_SIDE_SLOT0_WAIT_S` seconds (3), then takes a free side slot its prompt fits in and prefills there what the disk tier does not hold. A request too long for every side slot still waits for slot 0. A match on the NVMe now counts as the request's conversation only when it reaches past the first user message; a match at the client's head, such as Open WebUI's system prompt and tool list, is logged as `spill_head=` and the request is a new conversation. On v0.8.0 such a head match held a new chat for 437 s while both side slots were free. `/health` `parallel` says how many requests may run at once (`max_running` with `max_running_reason`) and counts `spill_waits`

and `slot0_gave_up`; the Lanes view's queued line says why a request waits, for example "Queued: all 3 slots are busy".

14.14 A second model behind the same server

`--model` with an EXL3 pack of Qwen3.8-Flash-Next serves that model through this server. The server reads the checkpoint's own configuration, finds its architecture in the registry (`engine/models`), and calls that entry's builder; it has no branch on the model's name (`tests/test_load_shared.py` fails on one). Everything in this chapter is the same code for both models: the request fields, reasoning formats and budgets, tool calls and grammar masks, penalties and `logit_bias`, logprobs, seeds, stops, streaming, the queue and the memory gate, the live view and the usage rows. Through the server, 19 of the 20 requests of `tools/fn_server_check.py` gave the same text, tool calls and logprobs with and without speculation; the 20th, a think budget at low effort, closes at a block boundary and differs after it (Section 17.7).

A model's capabilities say what the server may ask beyond the request loop (`caps`). Flash-Next holds every cache, the KV pool and side slots, so parallel requests run on it without changing any answer, and its profiles turn them on with three slots (Chapter 17). Images go through the same path: Flash-Next's builder hands the server its own tower on the EXL3 kernels (VIS-97), and its profiles run with `VISION=on`. One thing is off on Flash-Next: the conductor's shadow reader, because its outputs were validated on the 27B's reasoning only; its status says why. `/health` names the pack, its draft layer, the load time and the self-check.

Since 0.8.2 the reference box serves Flash-Next with the conductor on, as it serves the 27B. Asked who it is, the model then answers that it is TandemLLM running the served model on the user's own machine: the conductor's identity note names the model from the pack's card. With the conductor off, v0.8.0 named another assistant in 20 of 20 answers. The Flash-Next profiles also load Laya, because the memory features ask a decision model whether a message means remember, forget or recall (about 20 ms); without it memory did not work on Flash-Next.

14.15 The tool-argument loop guard

A model can loop inside a tool call, where the reasoning guards do not look. In one recorded case the model wrote the same 97-character row of a tile map 12,866 times inside a `write` call: 65,536 tokens, 327 s, nothing written, and the cut text made the next prompt 157,806 tokens. The lookup drafter copied the loop: all 31 draft tokens were accepted in 2,007 of its 2,209 blocks. The guard (`--argloop on|log|off`, `server/argloop.py`) reads the open call's arguments every 64 tokens once the block has 1,024 and fires on either of two rules. The periodic rule: the end of the block is one unit of at most 4,096 characters repeated exactly, 6 times over at least 1,024 tokens, or, for a unit of 256 characters or more, 3 times over at least 384 tokens. The size rule: the call passed 32,768 tokens and three times the request's longest message, and its last 16 KB compress 16 times or more.

On the tile-map incident it fires 2,176 tokens into the call, 1,055 after the loop began. On a recorded Open WebUI call that looped on a 405-character piece of a sprite, it fires 448 tokens after the loop began, at three copies; the six-copy rule alone needed 1,024. On 4,780 real writes and edits from an opencode database it fires on none, with the long-unit rule or without it (measured,

`tools/argloop_fp.py`). Three identical frames of a sprite written in a row do not fire; a literal block of one line repeated over more than 1,024 tokens (600 zeros written out) does, with either rule. When it fires, the generation stops, the client gets one sentence and `finish_reason: "stop"`, and the session's next request with tools carries a hidden note naming the tool and the repeated piece: write it without the repetition, generate repeated rows with code, write a large file in parts. A call that already streamed part of its arguments keeps what was sent, and the cut text is never sent again as content.

A stopped call sent back. Open WebUI's tool loop answers a call whose arguments never closed with a tool error and sends the conversation again, the stopped arguments included. In the recorded case the model copied its own loop five times, byte for byte, about 55 s each, the prompt growing from 17,692 to 46,419 tokens. Now an assistant call in the history whose arguments do not parse (or a value of opencode's `invalid` stand-in) and end in a periodic stretch is cut to its first 1,500 characters and one marker before anything reads the messages; a call that parsed is never touched. The markers after the last user message are the retry streak: the retry's note says when the same loop came again, the router thinks one level higher on it (at least medium) unless the client turned thinking off, and after two identical stops in a row, or three of one tool, the engine does not generate again. The client gets one sentence and no call, and its tool loop ends.

14.16 Logs and privacy

Each request leaves one log line (Chapter 6), and no line carries the text of a prompt or an answer. The usage ledger (Chapter 30) stores counts and timings, never text. The conductor's journal keeps the hidden texts the conductor itself wrote, for 30 days, and its transcripts keep a session's tool turns with secrets redacted, under the engine's home at mode 0600.

Chapter 15

Measurement

Speed claims for inference engines are easy to inflate by accident. The suffix store once held the benchmark's own prompts and answers, and it added about 21 % to the benchmark row without the engine getting any faster. This chapter describes how TandemLLM measures itself, from the benchmark row and the teacher-forced bench to the release gate, and the traps each rule answers.

15.1 The row

The benchmark row is `serve-single-i256-o256-v1`: one request at a time, 256 prompt tokens, 256 generated tokens, thinking off, temperature 0, seed 42, 50 requests after 3 warm-ups. Every configuration runs with the same runner and the same prompt set. Each request's speed comes from its own record:

$$\text{tok/s} = \frac{\mathit{completion_tokens} - 1}{\text{end-to-end seconds} - \text{time to first token}}.$$

The first token belongs to the prefill, so it is subtracted, and the decode time is the request's wall clock after its first token. A row reports the mean, p50, p90 and maximum over the 50 requests, with the median time to first token.

Two readings of a row are easy to get wrong. The gaps between tokens say nothing about round time: a verified round writes several tokens into the socket at once, so half the gaps are microseconds and the other half a whole round. And the mean is not the median: some prompts ask the model to repeat text already in the context, the drafters commit 15 tokens a round on them, and that lifts the mean. The median reflects fresh text.

15.2 Two views of every row

A row runs in two views of the suffix store. Store off: the engine's persistent suffix store is off, so nothing the engine has seen before can help it. Clean store: the store is on but holds no text of the benchmark. A row with the benchmark's own text in the store measures the store, not the engine. Benchmarks that read a store of real traffic run with `--suffix-store-readonly`, so they cannot write themselves into it.

With the conductor on, a benchmark also needs the raw path: `X-Tandem: raw`, or the locked `bench` profile. `tools/row3.py --no-server` refuses a server that lists `TandemLLM`, because a runner that sends no headers would measure the router, the effort cap and the notes along with the engine.

15.3 Noise, and what a row can decide

Two server processes of the same configuration can differ by several per cent. StairCut learns its costs and copy rates online, so two runs can make different decisions on the same prompts. `tools/row3.py` runs N rows per configuration through one server, recomputes each from its own records, and reports the median with the spread of the runs. `--compare` calls a difference resolved only when it is larger than the noise and the run ranges of the two sides do not overlap. A change worth less than about 3 % of the median cannot be settled by the row at any number of repeats.

Speed is a quotient: tokens a round over time a round. A change can trade one factor for the other and leave the quotient flat. The served loop therefore counts its own rounds, and every row reports tokens per round and milliseconds per round next to tok/s. Both are gated: a kernel change must not move tokens per round, and a drafter change must move it up.

A server run can hold the costs still. `--no-learn-cost` keeps the router's served price table, so what earlier requests took no longer moves the choices. The router still times the blocks inside the request, so two runs can still choose differently and, at a near tie, write different text (Section 12.3). With learning on, one prompt took 366, 468 and 504 blocks for the same 2,000 greedy tokens in three runs, and the runs spread 37 %; with the flag, six runs wrote one text in 514 blocks each and spread 0.19 % (measured, the 0.4.0 series). An A/B on a server therefore uses the flag, two warm-ups an arm, and compares only runs that wrote the same text.

Every report names the code it measured with `code_sha256`, a hash over every file of the four code folders. The same commit hashes the same everywhere, so a report can be matched to a commit.

15.4 Same hour, both sides

Nobody compares a candidate with a number from another day. It runs against the current release in the same session on the same box, alternated: base, candidate, base, candidate. A board drifts over hours (its temperature, its memory, whatever else runs), and alternation spreads that drift over both sides. The adoption rule is "never worse": a candidate ships only if nothing resolves worse against the same-hour base, in both store views. Not the mean, p50, p90 or maximum, not the time to first token, and not tokens or milliseconds per round.

15.5 The block A/B

Most kernel changes are worth 0.5 to 3 ms of a round of about 93 ms, below what the row can resolve. For a change whose output is bit-identical, only time can differ. `tools/block_ab.py` loads the served engine once, switches each state's flags inside the process, warms each state, freezes the router's learned costs, and alternates base and candidate over three workloads for N pairs. Every run's tokens must equal the base's first run, and the rule is applied to milliseconds per round. Two states of the same code read "not resolved", with a spread of 0.35 to 0.87 %.

15.6 The release gate

`ops/gate.sh` is the whole protocol in one command, run on the board inside a hold. It stops at the first failing step:

1. every CPU test, `tests/test_*.py`, in a clean environment;
2. every GPU test, `tests/gpu/test_*.py`;
3. identity: the served configuration with the new flags off is bit-identical to the build it replaces, on a fixed scenario of fixed shapes (`tools/flagoff_identity.py`);
4. agreement: `tools/verify_spec.py` with the served flags and the candidate's, under the near-tie rule;
5. optionally, the block A/B;
6. rows: two store-off rows and one clean-store row with the candidate's flags;
7. compare: `tools/gatecheck.py` against the base reports, with the never-worse rule.

It writes a dated report with every command and its output. Rows are not enough on their own: a release candidate once passed every row and broke a real client's request shape. Before a deploy the gate is followed by `tools/client_smoke.py` (the request shapes real clients send, through the official `openai` client: tools validated against their schemas, every `tool_choice` mode, streamed tool-call deltas, prompts from 8k to over 100k tokens), by `tools/api_text_check.py`, and by a real agent session whose tool calls must all go through.

15.7 The teacher-forced bench

A change of the verified tree's shape moves near ties, so two configurations rarely write the same text on the same prompt, and their rows measure different texts. `tools/forced_bench.py` removes that. Each workload has one fixed reference text, and every configuration drafts and verifies with the real kernels at the real context but accepts along the reference, so all of them commit the same tokens and differ only in rounds and time per round. The paper's run covers 25 workloads: prose, chat, code, twelve prompts of the benchmark row, three edit tasks, a quotation and a copy, and fresh text, a copy and a code edit at 32k tokens of context. StairCut averages 90.45 tok/s against 81.95 for the fixed block 16 and 59.31 for the fixed block 8 (Figure 11.3).

15.8 Traps

Each of these cost a wrong number once.

- A standalone kernel timing is a ranking, never a budget. The same configuration read 0.254 ms in one process and 0.627 ms in another.
- The first configuration in a fresh process pays for autotuning. It once read a verify of 260.2 ms against 115.5 ms for every later one, and the policy under test "won" by 35 %.

- A bandwidth-bound engine measured beside another engine measures the other engine. Two engines halve the bandwidth each gets. Take the board's lock.
- A speculative decoder judged on a short generation hides bugs. One drafter bug read 81 % acceptance at 48 tokens and 24 % at 128.
- Acceptance with the reasoning block open differs from acceptance with it closed. Measure in the serving regime.
- A kernel probe whose working set fits in cache measures the cache. 266 GB/s on a 273 GB/s board is not a real reading.
- Two runs that agree to six decimals have not agreed; they are the same run. Once, a test server had bound its port beside another.
- A quiet log is not a hung process: Python block-buffers into a pipe, so use `python -u`.

15.9 Measuring Flash-Next

Flash-Next's numbers in this manual come from tools of their own (`tools/fn_*.py`), each answering one question, and every one ran under the board's lock with one model loaded. Most are single runs of one request, labelled so where they appear; none is a benchmark row yet.

Table 15.1: The Flash-Next tools behind the numbers in Chapter 17.

tool	the question it answers
`fn_server_check.py --compare`	do 20 request shapes give the same text, tool calls and logprobs with and without speculation; `--sampled`: do seeded sampled answers
`fn_spec_check.py`	does speculative greedy equal plain greedy in the engine, with the state bit-equal after every run
`fn_fit_run.py`	does a pack hold its largest window: one long request, MemAvailable at 10 Hz, every limit knob the server reports
`fn_cache_check.py`	does a request restored from each cache tier answer as on a cold server
`fn_resume_check.py`, `fn_guest_check.py`	does a resumed prefill equal a cold one; does a short guest request leave the resident conversation's rows intact
`fn_lookup_ab.py`	one greedy request a class through a server, for an A/B of the drafters
`fn_memory_check.py`	memory after the load, the prefill workspace, the table's resident bytes under each policy
`fn_exl3_xcheck.py`, `fn_refcheck.py`	our reading of a pack against exllamav3's; our forward against `transformers`'

Every speculative rate in Chapter 17 comes with the statement that its texts equalled plain decoding, because a rate whose text changed measures another answer. The runs by context length started a fresh server and report its second round, so the first round's one-time graph captures do not count in them. What Flash-Next has not had yet is the release gate: no row of `serve-single-i256-o256-v1`, no block A/B and no task suite.

15.10 Limits

The published numbers come from the test board and the test model (Section 1.1), one request at a time. Flash-Next's come from single requests on the same board (Section 15.9). Long prompts are measured separately (Chapter 13), and parallel requests in single runs with `tools/parallel_bench.py` (Section 14.13). The quality gates are held-out loss, confident argmax agreement and free generation; no task benchmark suite has been run, so this manual makes no claim about task accuracy.

Chapter 16

Adding a model

The engine serves two big models: Qwen3.8-27B, a hybrid of Gated DeltaNet and attention layers, and Qwen3.8-Flash-Next, a mixture of experts on the same hybrid (Chapter 17). The small models beside them (the Qwen3.5 narrator and shadow reader) run on the 27B's forward code. Flash-Next was the first model added through the adapter path below. This chapter lists the seams where a new model plugs in, what each one expects, and the tests a new model must pass before its numbers mean anything. Some of the engine is still specific to this family, and the last section says which parts.

16.1 The adapter path

A new architecture brings a package under `engine/models/<name>/`, one registry entry (`engine/models/__init__.py`) and data files, and no edit anywhere else. The methods read a model only through its adapter. `tests/test_load_shared.py` fails if code under `server/` names a model or asks an engine what it is. Five layers depend only downward (Figure 16.1). The upper three exist, the formats as today's `Linear` protocol, and so does the server's one loader. Four pieces are still design: format plugins (ENG-253), a kernel registry (ENG-251), device profiles (ENG-255) and the mesh (ENG-259).

An adapter is the engine object with two sets of verbs. The hot verbs run every round and are the engine's own bound methods, never a wrapper, so graphs capture the same launches: `forward`, `forward_tree`, `forward_block`, `commit_tree`, `rollback_to`, `picks`, `hidden_post_norm`, `kv.length` and `reset`. The cold verbs run around a cache copy or at load: `caps` (what the server may ask: caches, side slots, the cache grid, the shadow reader, joint verify), `cost()` (state bytes and the prefill workspace), `cache_state()`, `resume_at(n)`, `settle()`, `discard_pending()`, `trim_pending(n)`, `manifest()` (what its stored rows depend on, for the disk tier's fingerprint), `memory_parts()`, `resident_files()` and `precapture()`. The 27B's cold verbs are installed on its class by `engine/models/qwen3_5/adapter.py`, so `engine/model.py` keeps its bytes.

`cache_state()` returns a `ModelState`: *sections*, the recurrent state by name and copied whole, and *views*, the position-indexed buffers with their token axis and sliced by length. Every tier keeps a sequence in that form, and a signature over names, shapes and dtypes refuses a restore across models.

A new model then needs, in order: a config reader that refuses unknown fields; a reader for its checkpoint and a `Linear` object for each weight; a reference forward checked layer by layer

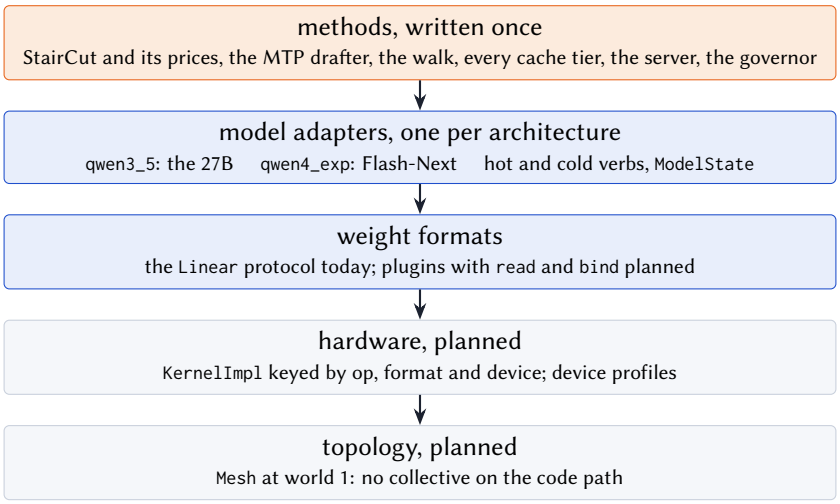


Figure 16.1: The engine's layers. A layer reads only the layers under it, and per-model numbers are data the adapter returns or files under `models/` and `ops/`. Filled: built; outlined: planned.

against the published implementation; the engine with the hot verbs, every verified row equal to the decode step's row; the cold verbs; a draft head if the checkpoint ships one (the shared MTP drafter asks for `depth`, `reset`, `prime`, `capture_all`, `launch`, `read`, `views` and `resume`); a server builder `serve(a, ctx)`; the registry entry; and its data files: a StairCut price file per profile and device, the weight profiles (`ops/profiles/<name>.env`) and the Setup Guide's registry entry (`models/<id>/model.yaml`, described in `docs/setup.md`).

16.2 The seams

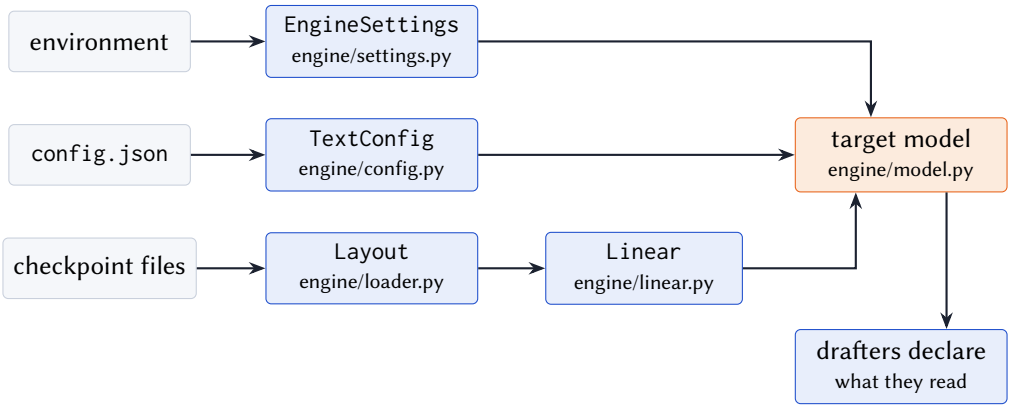


Figure 16.2: The seams a new model plugs into.

Config. `engine/config.py::load_config` reads `config.json` into a `TextConfig` with every size the engine needs, from the layer count to the rotary settings, and answers which layers are recurrent. It accepts a vision-language config (the text model under `text_config`) and a text-only one. A new

family either fits `TextConfig` or extends it; the rest of the engine asks `TextConfig`, never the raw JSON.

Layout. `engine/loader.py::Layout` says how a checkpoint names its tensors, in one place. The engine addresses every tensor by a canonical name, the language model's own naming with the checkpoint's wrapper prefix taken off. The layout lists the prefixes to strip, the tensors the language model never reads (such as a vision tower), where the checkpoint keeps its prediction head, and the projections that may be stored as FP8 pairs or as plain matrices. `tests/test_layout.py` pins the served checkpoint's key set: 1,250 language-model keys read, 333 vision keys skipped.

Weight formats. Every weight the engine multiplies by implements one small interface (`engine/linear.py`): a shape, the bytes read per use, and `matmul`. `FP8Block`, `NVFP4Block`, `FP8Head` and `BF16Block` implement it. The model calls `w.matmul(x)` and never asks which format it holds. A new format is a new class with those three members, plus a kernel that keeps the row-invariance rule (Chapter 10).

Settings. `engine/settings.py` is the registry of every `QWEN38_*` switch, read only through `SETTINGS.get(name)`. `EngineSettings(env, prefix)` gives the same reads over another mapping or another prefix.

What a drafter reads. A drafter reads parts of the target: hidden states at some layers, the embedding, the head. Each drafter declares these through `Drafter.requires()`: the hidden size, the tap layers, the target tensors it reads by name, and the vocabulary size. At load the engine compares them with the target and refuses a mismatch by name, so a DFlash2 drafter trained for another model fails there instead of drafting garbage. A new model needs its own block drafter; the lookup drafter works for any model once its corpus is rebuilt with the new tokenizer (`tools/build_corpus.py`). The lookup stores record the tokenizer's sha256 and refuse to open under another tokenizer.

The tokenizer and the chat template. The server renders prompts with the model's own chat template and reads the model's own tool-call and reasoning formats. A model with another format needs its own parser behind the same OpenAI output.

16.3 The tests a new model must pass

In this order, each on the new model:

1. `tools/refcheck.py`: argmax agreement with the published reference implementation on teacher-forced positions, the reference in its own process because only one model fits on the board at a time. On Qwen3.8 the engine agrees on 99.25 % of 400 positions and on all 242 confident ones (measured).
2. `tools/quality_gate.py` for every quantised weight set (Chapter 9).

3. `tools/verify_spec.py`, the agreement gate with its near-tie rule, with every drafter (Chapter 12).
4. The CPU suite, including `tests/test_refactor_gate.py`. That test records the exact bytes of a tiny model's run (every logit and every byte of state after a prefill, a decode, a block verify, a rollback, a tree verify and a commit) and fails if any byte moves. A change that is supposed to change arithmetic re-records the fixture and says why.
5. `ops/gate.sh` and the client smoke set before any number is published (Chapter 15).

Flash-Next took its own form of these gates: the one loader (`tests/test_load_shared.py`), every cache tier against a cold prefill (`tests/test_cache_any_model.py` and its server check), a fingerprint that moves with its kernels and tables and with nothing else (`tests/test_fp_v3.py`), the load-time self-check of every verify width (3,936 rows bit for bit on the 3.05 pack), and speculative against plain decoding in the engine and through the server (`tools/fn_spec_check.py`, `tools/fn_server_check.py --compare`).

16.4 What is still specific to Qwen3.8

- `engine/model.py` implements the 27B's forward and `engine/models/flashnext/` Flash-Next's. A new family needs its own layer code behind the same verify, commit and cache interfaces; Flash-Next routes its experts inside the verify, and every row reads its own experts.
- The recurrent-layer kernels assume Gated DeltaNet's state shape, heads of 128 by 128.
- Tile tables and router prices are measured for each model's shapes on the GB10. A new model or a new board needs its own sweep and its own price file; the kernel registry and device profiles that would make a board data are planned, not built.
- Some modules read their switches into globals at import, so two engines with different settings cannot share one process yet.
- Runtime names still carry the old project name, from the `QWEN38_` and `QSE_` prefixes to the `Qwen38Engine` class.

The Qwen3.5 small models already run on the engine's own forward: the narrator and the shadow reader load them through `engine/generative/qwen35.py`, the same layer code, because they share Qwen3.8's architecture family and its tokenizer (Chapter 21).

Chapter 17

Qwen3.8-Flash-Next

A plain decode token of Qwen3.8-Flash-Next on its 3.05 pack reads about 4.23 GB (arithmetic, from the pack's headers), and the engine decodes it at 45.8 tok/s at 1k tokens of context with no drafter (measured, 2026-10-11). Flash-Next is the second model TandemLLM serves. It runs on the same board as the 27B, one model at a time, through the same server, the same caches and the same speculation methods. This chapter collects what is different, from the model and its packs to the parts that are not measured yet. Chapter 16 describes the adapter path it came in through.

17.1 The model

Flash-Next keeps the 27B's hybrid of Gated DeltaNet layers with an attention layer every fourth, and changes most of what sits around it (Table 17.1). `engine/models/flashnext/config.py` reads its configuration and refuses any field it does not know, naming it, so a later release that adds an arithmetic switch is not served as if it were this one.

Table 17.1: Flash-Next beside the 27B.

part	Flash-Next	the 27B
layers	48: 36 Gated DeltaNet, 12 attention	64: 48 Gated DeltaNet, 16 attention
MLP	512 experts a layer; 10 routed and 1 shared run a token	dense, width 17,408
residual stream	4 streams of 2,560, joined by hyper-connection mixers	one stream of 5,120
attention	sparse past 2,051 tokens: an indexer scores blocks of keys and each row reads the 512 best	full
extra input	a hashed n-gram embedding: 16 table rows a token, in front of layer 1	none
draft layer	one multi-token-prediction (MTP) layer in the checkpoint	none; DFlash2 drafters beside it
state of a sequence	110.3 MiB of sections plus 29,952 bytes a token	about 147 MiB plus 64 KB a token
weights served	turboderp's EXL3 packs, 2.05 to 6.05 bits a weight	NVFP4 with an e4m3 head, or EXL3

The tokenizer is the 27B's (248,320 ids), so the lookup drafter's corpus and the persistent suffix store serve both models unchanged; both carry the tokenizer's fingerprint and are checked against it at load.

17.2 The packs and the window each one fits

turboderp publishes Flash-Next in five sizes, one branch each of `turboderp/Qwen3.8-Flash-Next-exl3`, and each size is a weight profile (`ops/profiles/flashnext-<bpw>.env`). The engine reads the packs as they are; Chapter 9 says what is inside one. The largest window of each pack is in its registry entry (`models/qwen3.8-flash-next/model.yaml`), and a start with a longer `--max-len` stops before the load and names both numbers.

Table 17.2: The Flash-Next packs on one DGX Spark. Weights: the GiB on the device, the draft layer and the int8 mixers included; measured for 3.05, estimated from the file sizes for the others.

profile	disk	weights	n-gram table	largest window
`flashnext-2.05`	61.5 GB	32.85 GiB	24.44 GiB, warm	262,144
`flashnext-3.05`	85.1 GB	47.87 GiB	30.4 GiB, cold	262,144 (recommended profile)
`flashnext-4.05`	106.1 GB	62.47 GiB	36.36 GiB, cold	208,896
`flashnext-5.05`	121.9 GB	77.16 GiB	36.36 GiB, cold	does not load yet
`flashnext-6.05`	137.7 GB	91.87 GiB	36.36 GiB, cold	none: it does not fit

One request of 260,800 tokens through the server, on each of the three packs that load, decided those windows (Figure 17.1). The Setup Guide's estimate is calibrated on these three runs and predicts their lowest MemAvailable within 0.7 GiB. On the box the other programs took 4.6 GiB; the estimate's frame keeps 10 GiB for them, which is why 4.05 comes out at 208,896 tokens with the vision tower on (212,992 without) although its run at 260,800 kept MemAvailable above the floor. 5.05 would come out at 53,248 with the tower and 65,536 without, once it loads.

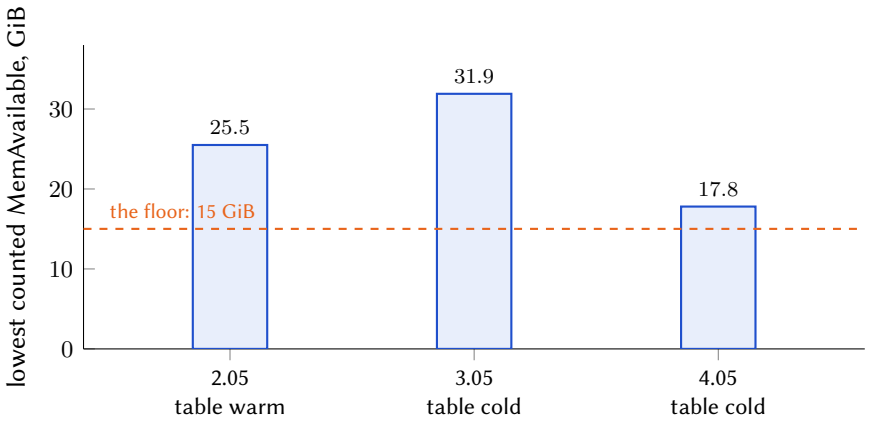


Figure 17.1: The lowest MemAvailable, less a warm table's resident bytes, during one 260,800-token request at a 262,144-token window, sampled at 10 Hz. measured, 2026-10-12, one run a pack, conductor on, with 4.6 GiB of other programs on the box.

The run also measured what a long prefill takes: 571 tok/s for the 260,800-token request on 3.05, 261,549 rows with the conductor's notes (585 on 2.05 and 581 on 4.05), and a prefill workspace of 10.47 GiB above the engine, which stops growing at about 140k tokens. Decoding at that length, with the prompt from the caches, 3.05 made 22.5 tok/s with the drafter at 2.0 tokens a block; 2.05 made 27.0 and 4.05 20.5 (measured, one run each). A server without a drafter decoded 36.6 tok/s at that length (one request), and Section 17.6 says what changed.

Two packs do not run. 5.05 codes its routed experts at 5 and 7 bits, and the MoE kernels take 2 to 6 and 8, so its load stops at the expert bank. 6.05 needs about 92 GiB for its weights, which leaves too little under the 15 GiB floor at any window. 3.05 reads its n-gram table cold. Warm, with the whole table resident, MemAvailable less the table fell to 8.4 GiB during a 146,400-token request at a 147,456-token window, under the floor.

17.3 Loading

`engine/models/flashnext/pack.py` audits a pack before anything loads: every tensor the configuration expects, every shard's size against its header, and the hub's sha256 of every file. A missing tensor or a wrong size stops the load with its name. The loader reads each layer's experts as one file range into a 16-byte-aligned image and copies it to the device in one go, then drops the shards' page cache (52.3 GB on 3.05) and keeps the n-gram table's. On 3.05 a cold load took 56.2 s and a warm one 24.0 s (measured); through the server, 53.8 s.

The self-check runs at every load. It decodes 32 tokens one at a time, replays the recorded steps as chains of 2, 4, 8, 12, 16 and 32 rows and as an 8-row tree, and compares every row bit for bit: 3,936 rows on 3.05. A single differing bit turns speculation off for the process, and the log names the layer and the row. An injected one-bit difference failed it at row 5 of an 8-row chain.

17.4 Kernels

Every Flash-Next kernel is row-invariant: a row's instruction sequence does not depend on how many rows the call has or where its neighbours sit (Chapter 10). The MoE expert kernel (`tools/fn_kernels.py`) gives each (row, expert) pair its own work, rotates the pair's input with that expert's scales, and sums each row's slices in a fixed order; unselected experts are never read. At one row it moves 186 GB/s, 21.7 MB a layer in 0.117 ms (measured). One decode token is one replay of a CUDA graph, about 21.8 ms at 1k of context (measured).

A prompt runs through other kernels (`tools/fn_prefill_kernels.py`): a chunk of 4,096 rows sorts its (row, expert) pairs by expert and runs one matrix product over each expert's rows. On 3.05 the prefill made 1,185, 1,243 and 1,161 tok/s at 2k, 8k and 32k tokens (measured). The prefill and the decode path differ by BF16 rounding. Teacher-forced on prose and code at 3k and 8k tokens, their top tokens agreed at 96.2 to 98.1 % of positions (measured). The gate the prefill passed asks that it agree with the decode path at least as well as the decode path agrees with the reference forward, less one point.

17.5 Speculation

The MTP layer is the drafter. Each round it drafts a chain of 4 tokens with its top two tokens a depth, from one captured graph, through a draft head cut to the 32,768 most frequent ids (`QWEN38_FN_DRAFT_HOT`). The shared StairCut (`engine/methods/staircut.py`) keeps the part of that draft with the most expected tokens a millisecond, and with `QWEN38_FN_LOOKUP=1` (on in every Flash-Next profile) the lookup's best continuation joins the same tree. One verify a round checks it. A verified row is the decode step's row bit for bit, so greedy speculation writes the plain greedy text token for token, and a seeded sampled request draws what plain sampled decoding draws (Section 17.7).

Table 17.3: One greedy request of up to 512 tokens a class (prose stopped at 421) through the server on 3.05, MTP alone and with the lookup. Every text equalled plain decoding, which made 41.5 to 45.7 tok/s. measured, 2026-10-12, one request each.

class	MTP, tok/s	with lookup	tokens a block	with lookup
edit	106.2	131.7	4.56	10.43
copy	102.3	132.9	4.37	11.88
prose	49.4	51.1	2.16	2.15
chat	62.1	64.0	2.47	2.49
code	84.9	82.7	3.43	3.48

Copies gain less in speed than in tokens a block because a verify costs more with every row on this model. Each row routes to its own experts, and a block reads the union: about 19 experts a layer at 2 rows, 26 to 33 at 4 and 72 to 82 at 16, across prose, code and chat at 1k and 8k (measured). The learned prices were 27.8 ms for 2 rows, 38.6 for 5, 52.2 for 10 and 68.8 for 16 at about 1k of context.

Sampled requests take the same tree (`--sampled-tree det`): at each node the target draws its own token with the 27B's sampler, keyed by seed and position. At temperature 0.7, `top_p` 0.9, `top_k` 20 and `min_p` 0.05 on 3.05, every seeded answer was the same with and without speculation, each seed repeated, and two unseeded draws differed (measured, one run each). Speculation made 43.1 and 40.2 tok/s on prose after 1k and 8k tokens of held-out text, 76.2 and 73.6 on code and 59.0 and 60.8 on chat, against 38.4 to 41.7 plain.

17.6 Never slower than the plain step

A drafted round at long context can cost more a token than the plain step: on the first runs at 260,800 tokens the drafter decoded 22.5 tok/s and a server without one 36.6. StairCut on Flash-Next therefore prices every round by its wall time per context class. The class is 0 while the sparse attention is still dense, and above that the next of $4{,}096 \cdot 2^k$ and $6{,}144 \cdot 2^k$ at or above the context, up to 262,144. Per class it keeps the plain step's cost and a drafted round's cost a token, both starting from the price file (`ops/prices/qwen3.8-flash-next/flashnext-3.05.dgx-spark.json`, provisional).

- A class whose drafted rounds cost more a token than its plain steps takes the plain step, and drafts one probe round at an interval that doubles while drafting keeps losing (16 rounds, up to 256).

- A turn drafts only when what speculation saves over the expected answer pays for the draft head's backlog, the prompt rows it has not read (0.28 ms a row at 260,800 tokens).
- What a class learned of its drafted rounds ages back to the prior while its turns take the plain step.
- One second after the last request, the server captures the graphs the current class lacks, 25 to 30 in about 4.3 s. Before that, the first drafted request in a new class paid 3.4 to 5.1 s of captures inside its answer.

Table 17.4: One greedy request a length and the same request again (its prompt from the caches), the second round on one fresh 3.05 server, and the same pair on a server with no drafter. measured, 2026-10-12. All 16 texts equalled plain decoding.

context	speculation, tok/s	plain, tok/s
1k	56.6 and 57.3	41.0 and 45.4
32k	34.9 and 55.6	33.0 and 42.5
140k	42.1 and 52.4	33.9 and 41.4
260,800	41.0 and 41.0, plain turns	39.6 and 40.5

With the idle captures in place, the first request at a new length took the plain step and the next one drafted: 48.2 against 42.5 tok/s at 32k and 46.0 against 41.4 at 140k, every answer equal to plain decoding (measured, one fresh server a length).

17.7 Exactness

On the 27B a batched verify can round a near tie the other way (Chapter 12). On Flash-Next it cannot: its verify kernels give every row the decode step's bits, and the load-time self-check proves it on the device before speculation turns on. `tools/fn_server_check.py --compare` sends 20 requests to a `--drafter none` server and a `--drafter mtp` server (greedy, thinking, streaming, every `tool_choice` mode, JSON, penalties with `logit_bias`, logprobs, seeds, a stop string): 19 gave the same answers and tool calls, and the largest logprob difference was 0.0. The 20th is a think budget at low effort, which closes at the first block boundary past the cap, so the two runs close the reasoning block a few tokens apart (1,049 and 1,052 reasoning tokens) and differ after it.

Two places hold the 27B's rule instead. The prefill and the decode path differ by BF16 rounding, so a conversation resumed from an end anchor, a state the decode kernels wrote, is held to the greedy gate and not to bit equality (Section 17.8). And the weights themselves differ from the BF16 release by design; how much is not measured.

17.8 Caches

Every cache tier keeps Flash-Next's state through its adapter (`eng.cache_state()`). Its sections are the 36 GDN states and convolution tails, the n-gram convolution and its token context, and `mtp.h_last`, the last committed row's streams that the MTP layer reads next: 115,662,864 bytes. Its views are K, V and the indexer keys of the 12 attention layers, and the MTP layer's own: 29,952

bytes a token. The pooled indexer keys are not stored; a restore rebuilds them. The grid is 4,096 tokens, the prefill's chunk, so a resumed prefill runs the same chunks as a cold one: a 4,096-token chunk file is 122.7 MB and an anchor file 115.7 MB.

`tools/fn_cache_check.py` sent a request sequence to a server with its caches on and the same requests to a cold server. Restores from grid anchors, the state store, the session spill after a restart and the disk tier after a restart answered bit for bit (logprobs equal). Two restores from end anchors answered the same first token: one the same text, logprobs within 0.044, and one a longer list from its 13th token at a near tie. A reset keeps the KV rows in place; before that fix, a short request run between the turns of a long conversation zeroed rows the conversation resumed over.

17.9 Memory

MemAvailable counts page cache as free. Under the warm policy the n-gram table is page cache the engine keeps on purpose, so every floor check (the residency planner, the roles' governor, the side slots' admission, the KV pool) takes the table's resident bytes off first. After a load of 3.05, MemAvailable read 68.0 GiB with 46.9 GiB allocated (measured).

Each profile sets its engine's ceiling and peak with the arithmetic in the file: the weights, 4.15 GiB of runtime measured on 3.05, slot 0's KV at 262,144 tokens (7.58 GiB) and 6 GiB of caches, and on top the prefill workspace (10.75 GiB, where it stops growing). 3.05 sets 66 and 77 GiB, 4.05 81 and 92. The adapter states the workspace: a floor of 2.75 GiB, 76,309 bytes a context token, and that cap.

17.10 Serving and operations

`ops/use-profile.sh flashnext-3.05` switches the box to Flash-Next as it switches between the 27B's profiles: it fetches the pack's branch when it is missing (pinned to its commit, checked against the hub's sha256, the shared n-gram table hard-linked), stops the engine under the box lock, starts the new profile and rolls back to the previous one, 27B or Flash-Next, when the smoke check fails. `ops/start.sh` passes the profile's `QSE_ENGINE_FLAGS` (`--model <pack> --drafter mtp`) and the server picks the engine from the checkpoint's own configuration. The watchdog and the reboot line start the active profile, whichever model it is. Chapter 6 has the rest.

Behind the server everything is the 27B's code: every route, reasoning formats and budgets, tool calls and grammar masks, penalties, logprobs, seeds, stops, the live view and the usage rows. The conductor and its roles run in front of it; the shadow reader does not, because its outputs were validated on the 27B's reasoning only, and the status says so. The dashboard's system view names the pack and its branch, shows `max_len` against the pack's largest window, and reports the table under its policy. Flash-Next reads images through the pack's own vision tower on the EXL3 kernels, with the image rows' three-axis rotary positions (VIS-97, `VISION=on` in its profiles). On 3.05 the image chats answered the same with the MTP drafter and without (5 of 5: two single-image chats, a two-image chat of 2,089 prompt tokens and a second turn from the caches), and a text-only answer is the same with the tower loaded and without. Against transformers' `Qwen4ExpVisionModel` on the dequantised weights the tower's outputs sit within the official model's own BF16 noise (0.048 to 0.090 from the fp32 model, against 0.046 to 0.104). In the language model the text rows after an image keep the text's own agreement (KL 0.019 after 140 image rows and 0.0025 after 2,184 with

the sparse attention on, top-1 97.9 %; the text floor 0.022), and every verify pick equals the decode (`tools/fn_vision_check.py`, VIS-97).

Parallel requests work on Flash-Next without changing any answer, and its profiles turn them on: three slots over one engine with the KV pool (`QSE_PARALLEL=3`, `QSE_KV_POOL=on`). On fresh servers three requests arriving together took 27.18 s against 27.03 s one after the other, the same total throughput, and the first tokens came at 2.2 to 6.1 s instead of up to about 20 s for the last of three. One request alone decodes at 45.78 tok/s with three slots, 45.79 with one (measured). At a 262,144-token window with two 65,536-token side slots, a 260,800-token prompt kept MemAvailable at 31.6 GiB or more. Table 17.5 lists the limits.

setting	value
sequence slots (`QSE_PARALLEL`)	3
side window (`QSE_SIDE_MAX_LEN`)	65,536, capped by `MAX_LEN`
admission floor (`QSE_PARALLEL_FLOOR_GB`)	15 GiB
joint verify	up to 32 rows
turn quantum, batch timeout	50 ms, 2 ms
KV pool	hard floor 10 GiB, margin 256 rows, page 2 MiB
memory ceiling, peak, reserve slack	66, 77, 2 GiB

Table 17.5: Flash-Next's parallel limits in its profiles (ENG-241). A side slot took 0.20 to 0.22 GiB allocated at load.

17.11 What is not measured

- Quality against the BF16 model, per pack. The BF16 release is 360 GB and does not fit the board; the registry's quality fields are empty.
- Any task benchmark, and the benchmark row of Chapter 15. The rates in this chapter are single requests, mostly one run each.
- A sweep of StairCut's prices. The price file is provisional: class 0 holds online-learned prices, and the other classes single requests.
- The 5.05 pack, which needs a 7-bit expert kernel, and the 6.05 pack, which needs more than one board.
- Whether parallel requests pay on Flash-Next: one run of three, and no timing of the joint pass on its own.



Part IV

The conductor

The conductor: the layer in front of the model that plans each turn and watches it run. Profiles and the router (18), packages, the skill library and the small decision models (19), the session monitor (20), status lines, the voice and the shadow reader (21), memory notes (22), and the same conductor in front of other backends (23). Nothing here changes the model's arithmetic; all of it changes what the model reads.

Chapter 18

Profiles and the router

With the conductor on, the engine serves one model, `TandemLLM`: the weights with a conductor in front. For every turn the conductor decides what kind of task it is, how hard the model should think, which instructions and skills go into the prompt, and what to do when the model loops, stops empty or writes a broken tool call. It tells the user what it is doing while the model works, keeps notes between sessions, and keeps long conversations resumable.

Nothing of the conductor changes the model's own arithmetic. A turn's tokens come from the same greedy decoding; the conductor changes what goes into the prompt (a hidden note, a thinking cap) and acts at fixed safe points. Every decision, acted on or not, goes to one journal, and the dashboard's Conductor view shows it. This part of the book covers the conductor: this chapter the profiles and the router, then the packages and decision models (Chapter 19), the session monitor (Chapter 20), the status lines and the small models (Chapter 21), memory notes (Chapter 22) and the conductor in front of other backends (Chapter 23).

18.1 Switching it on and off

Table 18.1: The conductor's switches.

what	how
the whole conductor	`--conductor-config DIR` on the engine (`QSE_CONDUCTOR_CONFIG`, set in the shipped profile); `off` or empty: the engine serves its own model name and none of this runs
the monitor's acts	`--conductor raw|shadow|on` (`CONDUCTOR`, `on` in the shipped profile), with its policy file `ops/supervisor.yaml`
one request, raw	header `X-Tandem: raw`: the release's raw path, no class, no note, no effort change, no monitor
one request, shadow	header `X-Tandem: shadow`: everything decides and journals, nothing acts
one request, a profile	header `X-Tandem-Profile: <name>`; an unknown name is a 400
in front of another backend	the backend's `harness.mode` (Chapter 23)

A header can lower the mode, never raise it. Every response says what ran in its `X-Tandem` and `X-Tandem-Profile` headers. `/v1/models` lists `TandemLLM` alone. Any other model name a client

sends, the engine's own name included, is served as `TandemLLM`, so no client breaks on a name. The usage ledger records `model_served = TandemLLM` beside the name the client sent.

18.2 Versioned documents

The configuration lives in versioned JSON documents under the conductor's directory, outside any repository: the profiles, the router's classes and thresholds, the roles, the packages, the settings, the deciders, the residency plan and the monitor's policy. The dashboard's Conductor view edits every one of them. Each save is checked first and shows its diff, then becomes a new version with a required note; any old version can be restored, and the next request uses it, with no restart. A file that fails to parse or validate keeps the last good version. The API is `/v1/conductor/*`; writes follow the admin rule: open while no admin password is set, else the admin token, a dashboard session, or a request from the machine itself. With `--conductor-defaults shipped`, a new directory starts with the shipped documents as version 1; with `builtin` it starts with minimal built-ins.

18.3 Profiles

A request resolves to a profile: the `X-Tandem-Profile` header, else the profile mapped to the client, else the default (`general`). The client is known by its requests (headers, tools, its fixed system texts), never by a word list.

Table 18.2: The shipped conductor profiles.

profile	for	what it turns on
`raw`	locked	nothing: the raw engine, as the release serves it
`bench`	locked	the same, for benchmarks and the paper
`chat`	Open WebUI, the Playground	class-aware effort, the reasoning-loop guards, the status lines in the reasoning
`coding`	opencode	every routing decision, packages and skills, the loop guards, the empty-turn continue, the tool-call checks, turn snapshots, the library in shadow, the status lines as titled reasoning parts, the voice, the shadow reader when its model is set
`hermes`	the Hermes agent	coding's routing, tasks and packages, the status lines in the reasoning
`overnight`	unattended opencode runs, by header	coding plus the session tasks acting, raised caps, a push notification when the client has gone
`general`	any other client	class-aware effort and packages, the status lines in the reasoning

A profile's mode is `raw`, `shadow` or `on`, and it caps the session monitor's mode. Its sections hold the decisions it acts on (class, effort, packages), the tasks' modes and caps, the thinking settings, what the user sees (`ui`), the outcomes, the library, memory and the shadow reader. The Profiles screen edits each one. Memory notes are off in every shipped profile; the shadow reader runs in the coding profile once its model is set.

18.4 The router

Before a turn queues, the router decides which kind of task it is. Only a new user message asks a classifier. A turn that carries only tool results keeps the session's class, and so do a tool's media that a client sends as a user message right after the results, and the first message after a compaction. Figure 18.1 shows the path.

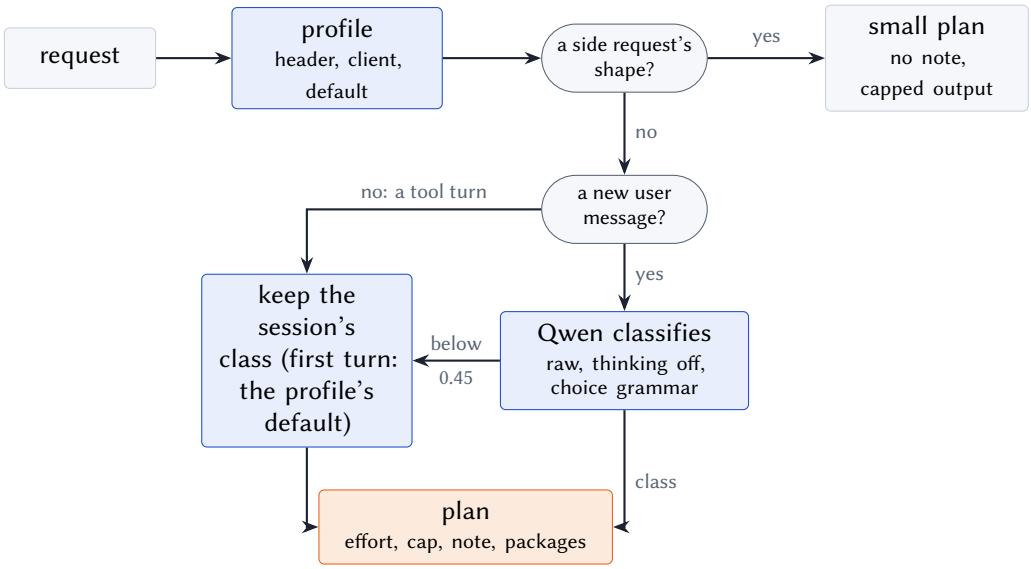


Figure 18.1: The router's path from a request to a plan. A client's side request gets a small plan of its own. A tool turn keeps the session's class. A new user message asks Qwen; below 0.45 the turn keeps the session's class, or the profile's default class on a first turn.

In the shipped router, which `ops/start.sh` stores as version 1 on a new box (`--conductor-defaults shipped`), one classifier decides:

1. Qwen itself, raw, thinking off, its answer constrained to the class ids by a choice grammar, taken at a confidence of 0.45 or more; every class's probability comes from the answer's first token and its alternatives, and a code class under 0.7 for a request that names a piece of prose (an essay, a letter, "700 words") and nothing of code gives way to the best class outside the code classes (`guard` in the journal; on 374 labelled prompts it changed two essays and nothing else);

2. below that, the turn keeps the session's class, or on a session's first turn the profile's default class, unless the top answer is a class that builds something (its policy names packages) and leads the second by 0.1: then the first turn takes it (`classifier.below_accept`). On 341 labelled prompts that rule turned two answers right and none wrong; one of the two was "Create a new folder and create there a pagoda garden" at three_d 0.378, which had gone to chat (measured).

Asked who or what it is, the model names its model and that it runs in TandemLLM ("I'm Qwen3.8-27B, running in TandemLLM") in the user's language: the router adds one hidden note on the turn whose message asks (English, German, French, Spanish, Italian, Dutch), replayed at its place on

every later turn like the other notes, under every profile but `raw` and `bench`. `QWEN38_MODEL_NAME` names another checkpoint.

The decider stage, a decision model asked a `choice` question (Laya, Chapter 19), is off in the shipped router. On 216 labelled prompts Qwen's answer was right 0.949 of the time against Laya's 0.782 (measured, `tools/conductor_eval.py`). A server started with `--conductor-defaults builtin` gets the minimal built-in router instead: the decider first, used at 0.80 or more, then Qwen at 0.55.

No word list decides anything. The classifiers read a digest of the new user message, its head and its tail, sized to what each stage reads. Two more stages exist and are off by default in the router document: Qwen classifying inside the conversation the engine already holds (`classifier.context_stage`, for follow-ups and long prompts; every class is a single-letter token, so one forward's softmax is the answer), and a linear probe on Qwen's own hidden rows (`classifier.probe_stage`).

An unsure deliverable is built and stated, not asked. A class a classifier picks on the turn the class starts, below 0.8, with another class at 0.15 or more, both with a `reading` phrase in the router document (three_d "a 3D three.js scene", design_ui "a flat web page", drawing "a 2D drawing"), gets one sentence in its note: build it as the picked class and open the answer with one short line such as "Building it as a 3D three.js scene. Say if you wanted a flat web page.", never a blocking question (`classifier.reading`, under the profile's `classify` decision; `mode: shadow` journals the sentence only). On 308 labelled prompts it fired on none, on 13 ambiguous ones on 7 (measured, `tools/class_core_eval.py`).

Table 18.3: The router's shipped classes, with each class's effort and packages. A package marked core goes in on every turn of the class.

class	what	effort	packages
`code_edit`	write, change, refactor or extend code	xhigh	coding (core), testing, Laya's test skills, test-web, test-cli, test-api, test-data, test-lib
`debug`	find and fix a bug, a failing test, an error, a crash	xhigh	the same
`design_ui`	the look of a page or an app	high	testing, Laya's, ui-design
`drawing`	draw, paint or animate with code	high	testing, Laya's, three drawing skills
`three_d`	a 3D scene or model	high	testing, Laya's, threejs-scene (core), photo-to-3d, voxel-seasons
`writing`	prose: an essay, an email, documentation	medium	none
`plan`	plan an approach before building	high	none
`research`	look up, compare, explain	medium	none
`summarize`	summarize given text, or a compaction	off	none
`chat`	small talk or a short question	medium	none

18.5 Effort and the thinking cap

The class sets the turn's effort, and every level except xhigh is a real cap on the reasoning that the engine enforces.

Table 18.4: Effort levels and their reasoning caps.

effort	reasoning cap
off	thinking off
low	1,024 tokens
medium	8,192
high	16,384
xhigh	none below the 32,768-token backstop (D2)

When the reasoning reaches its cap, the session monitor closes it with the vendor's closing phrase, hidden: the model reads the phrase, the client receives `</think>` and the answer. A profile's `thinking.budgets` overrides a level's cap. Thinking off is done in the prompt's tail: the generation prompt's open `<think>` becomes the template's closed empty block, so nothing before the tail changes.

The client's own wishes count. A number of reasoning tokens in any shape clients send (Section 14.3) always wins, and so does thinking off or on. Thinking off (`reasoning_effort: none` and every other off form) holds under every profile: the router's journal row then says `thinking: off_client` ("client turned thinking off"; the dashboard shows "thinking off (client)"), not the class's level, and the turn gets no effort sentence, since it cannot think. An effort word such as `reasoning_effort: medium` is a ceiling on the router's level by default. The coding and overnight profiles ignore it, because opencode sends `medium` on every turn whatever the user picked (`thinking.client_word`: `ceiling` or `ignore`).

Two refinements are switches of their own. Effort by task size (`thinking.by_size`, shadow by default) gives a class's levels per task size from four structural signals: the tokens the session added, the files its tool calls named, the gates a checker names, and the turn index. A tool-result turn's own level is `structural.tool_turn_effort` in the router document, `inherit` by default.

18.6 The note and its replay

The class's plan goes into the prompt as a hidden note: the effort sentence, the class's rules, and the packages that fit. The note goes into the last message just before its end token, and later turns of the session replay it at the same token position, so the resident prefix and every cache keep matching. It is never streamed, never counted in `usage`, never stored in the suffix store, and a request with a note skips the response cache.

A later user message's note carries only the parts that no live note of the session holds with the same text (`QWEN38_NOTE_DEDUP`, `parts`): a coding message whose task skill differs from the last one's adds that skill, a few hundred tokens, and not the general packages again (about 4,800 tokens). The replayed notes have a budget (`injection.max_replay_tokens`, 16,000). Over it the oldest note goes, which changes the prompt at that note's position, and the turn reads everything after it again. In

the test builds before this release every new task skill brought the whole note back, and the third copy crossed the budget: in an Open WebUI agent chat a turn read 35,442 tokens again (49 s). `block` keeps the old rule for an A/B.

The router's session is the client's conversation id (opencode's `x-session-id`; Open WebUI's `X-OpenWebUI-Chat-Id` when its connection sends the chat id as a header), else a hash of the conversation's head: its messages up to and including the first user message, which every turn sends again. Every chat that opens with the same message has that hash, so a head holds a list of conversations: a conversation's very first turn starts a new one when the newest of that head has had a turn, and a later turn finds its own by a hash of its whole first turn as the client sends it back. Without this, a new agent chat was measured starting with 27 turns and 21,216 session tokens of earlier chats and their notes replayed. A first turn regenerated without an id cannot be told from a new chat and starts afresh, which costs it nothing. The decision row's `extra.session` says how the session was found (`id`, `new`, `continued`, `alias`). Prefer the connection header to `ENABLE_FORWARD_USER_INFO_HEADERS`, which sends the same id but also the user's name and email to every connection.

18.7 Side requests

opencode sends its title and summary calls beside the session's turns, and the Hermes agent sends a title call. The router knows such a request by its shape, not by its words: no `tools`, and the client's own fixed instruction opening the system message. A side request asks no classifier, gets no note, and touches nothing of the session; thinking is off unless the client asked for a number of reasoning tokens; its output is capped at 64 tokens for a title and 256 for opencode's summary. In the queue a side request of at most 8,192 prompt tokens goes ahead of every waiting turn, and with the engine busy it is served in the side lane. The router document's `structural.side_shapes` adds other clients' shapes.

18.8 Compaction

opencode's compaction request is the class `summarize`: structural, thinking off, no package, nothing replayed. The first user message after a compaction keeps the session's class and asks no classifier, because the compaction carried the task over and its few words ("continue") are not a new task. The class's note goes in again at the compacted history's insertion point. How the engine serves a compaction from the conversation it already holds is in Section 13.9.

Only opencode's request is a compaction: its system-prompt shape from any client, and the bare shape (no tools, no system text, the history as one long user message) only when the client is opencode. Another client's pasted document with "summarize" in a fresh chat has the bare shape too, and is an ordinary turn whose class the classifier decides.

18.9 Session addresses

The notes give the model `{tandem_url}`: the request's own address plus `/s/<token>`, 12 random characters kept 48 hours. A library read, a Laya question or a memory call from the model's scripts

under that address is filed to its session in the journal. An unknown token is served and attributed to nobody; other routes under `/s/` are a 404.

`QWEN38_TANDEM_URL` sets the address the notes give instead of the request's own, for clients whose scripts reach the engine under another name. The testing packages also name `{tandem_scratch}`, the place for their logs, helpers and kits: `.tandem` in the project by default, which the model excludes from git when it creates it and deletes before it reports done; `QWEN38_TANDEM_SCRATCH` names another place (`{token}` in it is the session token). A directory outside the project does not work under opencode, which asks before any path outside it.

18.10 What is journaled

The router's decisions and the monitor's go to one journal, the table `conductor_events` in the usage ledger's file, kept 30 days: the class, the confidence of every stage, the effort and its source, the note's text and its token count, every act with its hidden text (stored once by its hash in `conductor_texts`). `GET /v1/conductor/sessions/<key>/export?format=md` lists what the model saw that the client did not. With the conductor on, every request with tools is also written as a transcript block under `~/.qwen38-spark-engine/memory/sessions/`, mode 0600, kept 30 days, with secrets redacted first: a private key, a bearer or basic header, a service token, a JWT, credentials in a URL, a password assigned to a string all become `[redacted:<kind>]`.

Chapter 19

Packages, the library and the deciders

The router's note carries more than an effort sentence. It carries packages: texts that tell the model how the work at hand is done well. This chapter covers the packages, the library of longer skills the model can search itself, the roles that sign one contract, and the small decision models the engine can serve beside the big one.

19.1 Packages

A package is a text the router may put into a turn's note: a task skill, a test skill, a card that describes a client's tools, or a coding guide. The `packages` document lists them with their source (an inline text, or a skill file on disk), a switch, a `max_tokens` limit, and optional flags. The shipped texts are in `packages/` of the repository and are stored as the first version of the document on a new box. The testing package sets an order and a budget: the deliverable before any test tooling, a verification tool that fails twice dropped with one sentence, servers in the background with a time limit and stopped before "done", everything the model creates kept inside the project (never `/tmp`), a scratch directory a cut run left cleaned first, enough verification named (it loads, no console error, one run with real input), never a product change made only for the model's own test, and a final report of what was built, verified and left untested. Laya's playtest is a check with a stop rule: a spot only the bot fails is not a bug, and two runs that end at the same spot end it. `tools/packages_migrate.py` brings a box's stored texts up to date.

A class lists the packages that fit it, at most 12. One marked `general` goes in whole on every turn of the class, before the task skill, and so does one the class names in its policy's `core`, the class's defining skill (`threejs-scene` for three_d, `coding` for code_edit and debug): left to the package question, the 3D skill had been answered none on a three_d turn that never said 3D, and the model built a flat page. The router's package question picks one among the rest, asked of the same classifiers as the class question. A package's text is cut at its `max_tokens` (3.5 characters a token) with a fixed note that says so. The packages fit the request's tools: the router reads what the tools can do off their schemas (a shell is a tool with a `command` argument, a file write one with a path and a content argument), and a package names what its steps need in `requires_tools` (absent: a shell when its text calls TandemLLM through `\{tandem_url\}`). A package the tools cannot carry out stays out of the turn or gives way to its `fallback` (`coding` to `coding-answer`), and the note says which tools the chat has and to deliver the files in the answer. A plain Open WebUI chat asking for a landing page used to get the testing packages, call tools it did not have and take 518 s; it now

answers in one request. Once in a session, a package stays live in the session's notes and replays at its position; it is not offered again.

Two cards are kept live the same way on the coding classes with tools. The tool card lists the request's own tools (names, required arguments starred, which take a path) with the client's pitfalls package (`tools-opencode`, `tools-hermes`). The `tandem-harness` card says what the runtime around the model does and where its addresses are.

19.2 The library

The library is a set of 44 longer skills in opencode's `SKILL.md` form: frontmatter with a `name` and a `description`, the text that goes in, then a `## Reference` section with snippets and commands. An `INDEX.md` names the classes each skill fits. Each skill is a package with `library: true`; its injected text is the part before the reference, which the model reads only on request.

`server/conductor/libsearch.py` keeps a BM25 index (`k1` 1.2, `b` 0.75) over every enabled library package's name (weighted three times), description (twice) and text. No embeddings. It is rebuilt when the packages version, a skill file or the index moves; 44 skills build in about 2 ms and a 900-character query searches in about 40 µs on the board's CPU (measured).

The model searches it itself, from the scripts it writes, through routes that are open like the chat API:

```
curl -s "$TANDEM_URL/v1/library/search?q=playwright+screenshot+test&k=5"
curl -s "$TANDEM_URL/v1/library/test-web"       # body and reference
```

`GET /v1/library` lists every enabled skill. Errors are OpenAI-shaped (422, 404, 429 over 240 calls a minute). Every call is a journal row with the query as a hash and a length, never its text.

The router can retrieve too, under a profile's `library` switch (`off`, `shadow`, `on`). On a new user message, and on a tool turn that went badly (a low-quality or stuck outcome), it searches the digest of the message, keeps the skills that fit the turn's class, and asks the package question over the best 4. With `on` the pick becomes a note at the prompt's tail; with `shadow` the journal says what would have gone in, and the prompt is the one `off` serves. The shipped coding and hermes profiles have it in shadow. The `tandem-library` card (191 tokens: the library exists, how to search it and read a skill) stays live on the coding classes with tools.

19.3 The Packages screen and proposals

The dashboard's Conductor view, Packages screen, lists every package with its kind, source, switch, general flag, the tokens of the text that goes in on the engine's own tokenizer against its limit (a library skill's limit is 350 tokens), the classes that list it, and its use in the last 24 hours and 7 days. A search asks the library's own index. A package's detail shows its text rendered or raw, its frontmatter and reference, its classes and its recent rows; an inline package edits in a Markdown editor with a live token count. Every save is checked first, shows its diff, then becomes a new version with a note.

The model may propose a change to a package, from a memory block with `op=propose` (Chapter 22). The proposal is a file with its evidence; the Proposals tab shows its diff against the live text, and Apply stores it as a new version of the packages document. The model never applies a proposal itself.

19.4 Roles

Everything beside the big model that the conductor asks is a role in the `roles` document, and every role signs one contract:

- its `kind`, one of six (reasoner, decider, probe, clerk, librarian, tool); a task asks for a kind, never a model;
- its `memory_claim_gib`, the declared peak the residency planner checks;
- its `window`, when it may use the GPU: `idle` between requests, `in_decode` beside a decoding model (a decider or a probe only), or `any`;
- whether it is `preemptible`, its `latency_budget_ms` and its `fallback` past the budget (pass, stale, skip, the next role);
- what it `writes`: only the reasoner may write tokens the client sees; the others write hidden notes, memory files or nothing;
- its `tokenizer_sha256` and the forms of `sharing` it may use: `text` always, `tokens` only with the engine's own tokenizer hash, `states` only for a probe inside the engine.

A role without a claim or a window is refused, as is any unknown value. The built-in roles include the `decider` (the router's decider stage, off in the shipped router), the `tester` (the test judge), the `narrator` and the `shadow` reader (Chapter 21). A role may name the engine itself (`self`), a model the engine serves, or, behind the proxy, another backend (`backend:<id>/<model>`).

19.5 Decision models

The engine can serve small decision models itself (`engine/decision/`): models that answer typed questions over a text in one forward pass with calibrated probabilities. A question is a `choice` (one of N options), a `score` (an expected level) or a yes-no probability. Laya's three published checkpoints (Apache-2.0) are the models it serves: `laya-english` (ModernBERT-large, 512 tokens of state and question), `laya-multilingual` and `laya-typed-decisions` (1,024 tokens each). `QWEN38_DECIDER_MODELS` names the ones a server loads; empty, nothing of it is imported and its routes are absent.

The arithmetic is the vendor's own, through transformers' ModernBERT, so the answers equal the vendor's SDK's (`tools/decider_equivalence.py`). Nothing is shared with Qwen: each decision model's tensors live in a memory pool of their own, its forwards run on a stream of their own, and a forward runs only while no request holds the engine. A completion that arrives during a decision waits at most one decision forward; a question asked while a completion runs waits for it up to its

deadline and is dropped after. A model loads through the residency planner (its claim is 8.2 GiB), at the start for the `deciders` document's `on_start` models or on first need; the Deciders screen loads and unloads by hand.

What asks a decider:

- the router's first classifier stage, through the `decider` role, asked in process, when the router document turns that stage on;
- the memory writer's gate: does the session hold something worth keeping;
- `POST /v1/systemone` (and `/v1/predict`), typed decisions under the admin rule;
- the model's own scripts, through `POST /v1/laya/ask`;
- the test judge, in shadow: on a run's output with no exit code or test counts, Laya is asked after the request whether it shows a failure, and the answer is journaled beside the structural twin. A question may act only after `tools/judge_report.py` shows a confident precision of 0.95 at a coverage of 0.3.

19.6 Laya for the model's scripts

`POST /v1/laya/ask` is the small, stable shape a script needs, open like the chat API beside it. A request names a question, its `choices` (2 to 20 labels, or labels with meanings), a `scale` of 2 to 10 levels, or neither for a yes-no question, and a `text` or several `texts`:

The packages that teach it name the engine only as `{tandem_url}`; a package whose text calls `/v1/laya`, `/v1/library` or `/v1/memory` at a literal address is refused at save, because a literal `https://` address fails wherever the machine running the model's scripts does not trust the server's certificate authority. The helper they give the model exits 3 with one line when the call fails ("the call failed on this machine, your code is not the cause"), and the model goes on without the verdict. The same packages keep their scratch files in `{tandem_scratch}`, have the model remove its state hooks (`tandem_state`, `/__state`, `--dump-state`) before it reports done unless the user asked for one, and forbid downloading a runtime or an executable. `tools/packages_migrate.py` applies these texts to a box's saved packages document as a new version.

```
POST /v1/laya/ask
{"question": "Does this test log show a failure?",
 "choices": {"failure": "a test failed or crashed", "clean": "every test passed"},
 "text": "<the tail of the log>"}
-> {"label": "failure", "p": 0.97, "sure": true, "probs": {...}, "model": "laya-english",
    ↪ "ms": 31}
```

Laya reads a window of a long text, its head and its tail, cut by code to what the model reads (1,120 characters for `laya-english`). A call runs only in the idle window: a caller that meets a running completion waits up to its deadline (15 s) and then gets a 503 `busy`. The route allows 240 texts a minute by default, answers CORS so a test inside a page can call it, and journals each call with the question as a hash, never the text.

A playtest kit builds on it. `GET /v1/laya/kit/laya_drive.py` and `laya-drive.js` serve a driver the model copies into its project: it reads the program's state each step (a page's `window.tandem_state()`, a headless program's JSON line, a state command), asks Laya which action of a fixed set comes next, applies it as real input, and logs deaths, wins and stuck spots. A driver session has its own rate (10 texts a second) and one journal row at its end.

The same kit checks a page without Laya playing it: `laya_drive.py page URL --step ...` starts the kit's headless Chrome, runs the steps with real input (`click:`, `key:`, `type:`, `wait:`, `read:`, `expect:`, `shot:`), takes screenshots at 1280 and 390 px with an overflow reading, lists console errors, uncaught exceptions and failed loads, asks Laya once for a verdict, closes Chrome on every path and prints one report, in about 5 s. The testing packages make it the browser for a page: never a driver of the model's own, one check with every main control, fix, check again, then the final message, and no Laya playtest for a one-screen tool. The final message is for the user: what was built, where it is and how to open it, how to use it, one plain sentence on how it was checked, what is untested. Laya's labels and probabilities, the kit, state hooks, the scratch folder, draw calls and fps stay out of it, and a readout added only for the check is removed before "done" or kept behind `?debug`. An opencode run that built a one-file page found no Chrome on its `PATH`, only Firefox, and spent the rest of its 600 s on a Marionette driver of its own; with the page check, five runs of the same task ended with a checked page and a report in 221 to 260 s (measured). The kit starts Chrome with `--password-store=basic` and `--use-mock-keychain`: without them every start waited about 25 s for the desktop keyring.

Chapter 20

The session monitor

The session monitor is the conductor's second half. The router plans a turn before it runs; the monitor watches the turn while it runs and acts when the model goes wrong in a way it can recognise without guessing: a reasoning block that goes in circles, an answer that ends empty, a tool call that does not parse, the same call a third time, a claim of done without the evidence. This chapter lists where it may act, what it may do, and each task.

20.1 Modes and policy

`--conductor raw|shadow|on` switches it. `raw`, the code default, runs none of it, and nothing closes a reasoning block but a budget. `shadow` runs every rule and journals what it would do, and executes nothing. `on`, the shipped profile's value, executes each task in the mode its policy gives it. A request's `X-Tandem` header can lower the mode, never raise it, and a request with `n` above 1 never acts.

The policy is a file, `ops/supervisor.yaml`, validated at start against `ops/supervisor.schema.json`. It holds every task's mode (`off`, `shadow`, `act`), every threshold and every text the monitor may write. The conductor keeps it as its `monitor` document, versioned and edited like the others, and a profile's `tasks` section overrides a task's mode per profile.

20.2 Safe points and actions

The monitor acts only at four safe points, from a fixed set of actions (Figure 20.1).

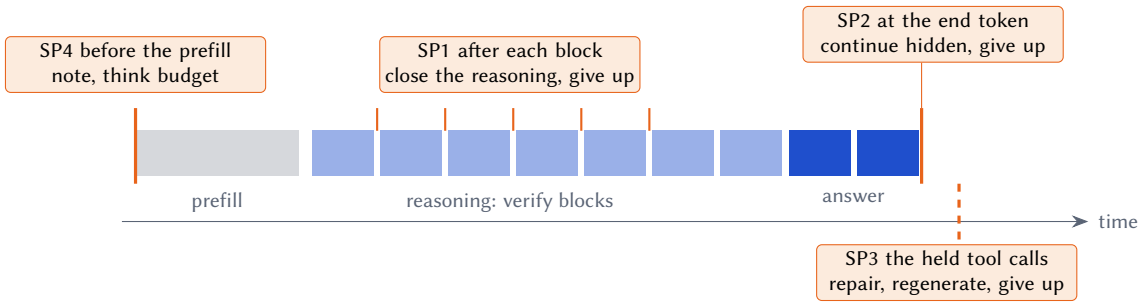


Figure 20.1: The session monitor's four safe points on one request: before the prefill, after each committed block, at the end token before it reaches the client, and when the answer's held tool calls are complete.

Table 20.1: Safe points and the actions each allows.

point	when	actions
SP4	the prompt is rendered, before the prefill	`PROMPT_TAIL_NOTE`, `SET_THINK_BUDGET`
SP1	a verify block is committed	`CLOSE_REASONING`, `GIVE_UP`
SP2	the model wrote its end token, before the client sees it	`CONTINUE_HIDDEN`, `GIVE_UP`
SP3	the answer ended and its tool calls are held	`REPAIR_CALL`, `REGENERATE_CALL`, `GIVE_UP`

What the client sees is little. A hidden close forwards the policy's phrase through the engine and streams only from the closer on (`</think>` in the `tags` format). A hidden continue drops the end token and forwards a nudge, and the answer follows in the same response. `GIVE_UP` is one visible sentence with finish `stop`. Hidden tokens are never in `usage`, never in the suffix store, and a response the monitor acted on is not stored in the response cache. The KV holds them, and the end anchor puts them back into the next prompt where the rows hold them (Section 13.5).

Caps hold the monitor back: at most 2 acts a request and 6 a session hour in the shipped policy, and never the same hidden action twice in a row on one turn. Beyond a cap a decision is journaled and not executed.

20.3 The tasks

Each profile sets its tasks' modes on top of the policy. S5 runs under every profile that runs the monitor and needs no state file. The session tasks S1, S2 and S4 act only in the `overnight` profile, and only in a project whose system prompt names a `PROGRESS.yaml`; the tool checks run where the profile holds calls (the coding and overnight profiles).

20.4 Reasoning loops

D1 watches the reasoning with three arms: a pattern of 1 to 16 tokens repeated at least 4 times over 128 tokens or more; the share of the last 256 tokens' 16-grams already seen earlier in the block, checked every 64 tokens once the block has 2,048; and novelty collapse, the share of new 8-grams in the last 128 tokens falling under 0.1. When an arm fires in act mode, the monitor closes the block with a hidden phrase and the model answers from what it has. D2 is the backstop: no reasoning block runs past 32,768 tokens, or past the output limit less 2,048, so an answer always has room. With the conductor raw, nothing closes a block but a budget.

20.5 Tool-call checks

With a tool-call task able to act, every call of an answer is held until the answer ends; a block that does not parse is held too, not let out as content. At the end token every call is checked by every

Table 20.2: The session monitor's tasks, with their shipped modes.

task	rule	what it catches, and what it does	mode
D1	`reasoning_loop`	a reasoning block that goes in circles (below); closes it, hidden	act
D2	`reasoning_backstop`	reasoning that reaches 32,768 tokens, or the output limit less 2,048; closes it	act
E1	`empty_turn`	an end token inside the reasoning, or an empty answer to a request that offered tools; tool calls the model wrote inside its reasoning go out as its answer when they parse and name an offered tool, else continues hidden, then gives up after two tries	act
E2	`claim_check`	a coding turn that ends claiming the work is done without the tool evidence (below)	shadow
C4	`malformed_call`	a tool-call block that does not parse; repairs it when the fix is not a guess, else has the model write it again	act
C3	`repeated_call`	the same call a third time after identical results; asks for something different	act
C1	`risk_floor`	`rm -rf` outside the project root, `git push`, a write outside the root, a download piped into a shell	shadow
C2	`project_rules`	what text alone can decide of the project's `AGENTS.md`: a numbered copy of a file, a path under `/tmp/`, a written file over 400 lines	shadow
S1	`gate_stagnation`	a checker gate that failed 3 times in a row without getting closer; one hidden note	act
S2	`context_health`	after a compaction, two tool steps that read none of the state files; one hidden note	act
S4	`stall_watchdog`	no file changed, nothing committed and no checker number better for 20 minutes; one hidden note, and a push when the client has gone	act
S5	`turn_budget`	on every client: the same problem stated three times; a build turn three minutes and five tool steps in with nothing written; an Open WebUI turn past 12 minutes without a new product file, or 18 in any case; one hidden note	act

SP3 task before any is released. If all pass, the calls go out whole, each in one delta with an id made at release. If any act fails, none goes out: the end token is dropped, a hidden turn shaped as the calls' tool result carries the reminders, and the model writes the set again in the same response. An answer that ended without its end token (the token limit) cannot be written again, so there a failure gives up. A block the client's own stop string cut is let out as the text it was.

C4's repairs are deterministic: JSON with trailing commas or Python literals, an unquoted name, a closing tag that never came. It checks arguments against the tool's JSON schema (required keys, types) in shadow. The tool-argument loop guard (Section 14.15) runs beside these checks and also stops a call that streams live.

20.6 The claim check: E2

A model can end a coding turn with "done, verified in the browser" after a test that set a key's state in code instead of pressing it, or that read a lost game as a working one. E2 checks such a claim before the end token reaches the client. The trigger is structural: the turn's class is one of the building classes, the request offered tools, and the model wrote its end token after a non-empty answer with no tool call. Then Qwen itself answers, thinking off, one letter a question: does the message tell the user the work is done; and if so, what the tool evidence since the last write says (supported, no run after the last write, errors in the evidence, an unexplained surprise, a faked test). The questions are asked in place, inside the conversation the engine holds, because a request to the server's own port would wait for the lock the turn holds. In act mode an unsupported claim gets a hidden note with the turn's evidence digest, and the model continues.

20.7 Turn snapshots and outcomes

A profile's `outcomes` section drives a structural reading of each tool-result turn, every switch off when absent.

- The snapshot: from the request body alone, what the session did since the last one: the calls, the files written, each screenshot's hash and size, a console's level counts and error lines, checker lines, exit codes, a test runner's counts, a state dump's diff. It is stored beside the transcript block, at most 32 KB, and takes about 0.023 ms a request (measured).
- The outcome: the first label whose facts matched, in order: no visible change (the same screenshot after a write), regression, error, low quality, surprise, stuck, tool misuse, a faked test (a script that assigns to a member of an object it did not declare), an environment error, lost context; else progress or none. Every fact is structural; no word decides what anybody meant.
- In act mode an outcome raises the turn's effort (no visible change and regression to xhigh, error to high) for two tool turns or until progress, and the acting outcomes add their note once per signature.
- Retrieval: a low-quality or stuck outcome asks the package question over the test skills, and the package that fits becomes the note.

20.8 Session tasks

Three tasks look across turns. They read what a request carries about the turns before it, each call taken once per session by its id, so a retried request counts nothing twice. Each acts with one hidden note at the end of the prompt, and a request carries at most one of their notes. S1 watches a checker gate (a line such as `RESULT: FAIL <- ...` from a project's checker) that fails three times in a row without getting closer to its limit. S2 watches the first two tool steps after a compaction for a read of the state files. S4 watches for 20 minutes without a change, a commit or a better number. When no request comes for S4's minutes after a turn ended, the client has gone, and with the

policy's `notify` on the watch sends a push to `TANDEM_NOTIFY_URL`; the URL is empty in the shipped policy.

S5, the turn budget, watches the turn: the requests that carry the same last user message. Its first arm reads the model's own visible text between its tool calls: when the first sentence states the same problem three times among the last eight problem sentences (shared word triples, a number in common), the note tells it to stop tuning, deliver what works and say what is untested. An Open WebUI agent chat had a working game after 25 minutes and then wrote "The bot is stuck at x=436" seven times while it tuned its own playtest bot; it never answered. The second arm watches a build turn: three minutes and five tool steps in, with nothing the user asked for written (a write outside the scratch directory, `/tmp`, logs and pid files), the note says to write the deliverable first. The third is a wall clock for a chat client someone waits in front of (Open WebUI): after 12 minutes of tool calls with no product file in the last 4, or after 18 in any case, the note says to finish and answer, and every 4 minutes after it a shorter one. Replayed on that chat, the notes come at 3.1, 18.0 and 36.0 minutes; on four clean opencode runs it adds none (measured). The fourth arm, `verify`, watches what follows the deliverable: when the steps since the last product write wrote only scratch files, two of them test tools (scripts or pages, not logs or data), and 90 s have passed since that write, the note says to stop building test tools, check once with a tool that works here and answer. An opencode page run wrote its page at 176 s, its last product edit at 257 s, then spent the run's remaining five minutes on a browser driver of its own; replayed, the note comes at 347 s, and four clean opencode runs get none (measured). Time alone is not a reason: opencode has no clock, and a chat client has the wall clock's quiet rule. A box whose policy is a stored document gets S5, or the keys its stored S5 lacks, with `tools/monitor_migrate.py`.

20.9 Observability

Every decision, passes included, is one journal row and one `[conductor]` log line with the task, the decision, the reason and counts, never a text. The ledger's `requests` row gets `conductor_acts`, `conductor_hidden_tokens` and `conductor_acted`, and the dashboard's speed figures leave out the requests the conductor acted in. Prometheus counts every decision and action (`qse_conductor_*`). `GET /v1/conductor/events` lists the decisions, and `GET /v1/conductor/watch` what the session tasks hold per session.

20.10 Extending it

The interface the policy engine builds on is kept stable. A rule is `tier0.register(name, safe_point, params, required, doc)` on a function from an event to an action; `params` is the JSON Schema of the task's parameters, which the policy schema picks up. The events are `RequestEvent`, `BlockEvent`, `FinishEvent` and `ToolCallEvent`. An action is one of the fixed set. `python tools/conductor_policy.py --schema` writes the schema from the rule registry, and `python tools/conductor_policy.py <file>` checks a policy file.

Chapter 21

Status lines, the voice and the shadow reader

A client shows the model's reasoning and its answer. What the conductor does around them, the engine's queue, a long prefill, a hidden close, was visible only on the dashboard. This chapter covers the lines that show it in the client, the small model that can write them, and a second use of small models: the shadow reader that keeps its own exact copy of a session.

21.1 Status lines

With a profile's `ui.states: on` (the shipped profiles), each step the conductor takes is one short line the client renders, at the moment it happens:

```
›
 code edit, maximum effort›
 reading 20,745 tokens, about 18 s left›
 waiting: another request is running, position 1, about 40 s›
 thinking 25 s: designing a 140x15 tile map with pits, coins, enemies and platforms›
 writing render.js, 211 lines so far›
 checking the tool call
```

`ui.states_channel` says where the lines go. `reasoning` puts them inside the reasoning the client renders. `content` sends each as a content delta of its own, before, between and after the reasoning's parts. `inline`, the coding profile's value, is built for opencode: each line opens a reasoning part of its own, titled by the line, in time order, and the answer's text part opens after them. It works because opencode's provider ends an open reasoning part when an empty `tool_calls` delta arrives and opens a new one at the next reasoning delta, which the server sends as `**› line**`. That was checked by running recorded streams through opencode's own parser, taken out of its binary.

The model never reads a line. Every line is framed so that the server can find it exactly, and whenever the conductor is configured the server takes every framed line out of whatever a client sends back, before the router, the monitor or the template read it. The prompt, the resident prefix it resumes and the notes' replay positions are exactly what they would be without the lines. The lines are not tokens: never in usage, the suffix store or the response cache's key. The journal row keeps each one with its kind and its time.

Table 21.1: Kinds of status line.

kind	template	when
`route`	`deciding: code_edit, high effort`	a classifier answered this turn
`package`	`retrieving: skill photo-to-3d`	the note carries a skill
`cap`	`thinking closed at the 16,384 cap`	the turn's cap closes the reasoning
`backstop`	`thinking closed at the 32,768 backstop`	D2 closes it
`loop`	`loop detected, steering`	D1 closes a reasoning loop
`continue`	`empty answer, continuing`	E1's hidden continue
`check`	`checking the tool call`	SP3 checks the held calls
`repair`	`repairing the tool call`	C4 repairs a call
`queue`	`waiting: ..., position 1`	the request waits for the engine
`prefill`	`reading 35,189 tokens at ~750 tok/s, about 47 s left`	a long prefill, every 10 s
`thinking`	`thinking 4 min, 9,800 tokens`	every 2 minutes or 4,096 tokens of thinking

Open WebUI times a reasoning block from its first reasoning piece to its first answer piece and shows that as "Thought for N seconds", so the waiting and reading lines counted as thinking: a summary with thinking off that read 12,874 tokens showed "Thought for 14 seconds". For the client kinds in `QWEN38_STATUS_CLIENTS` (default `open-webui`) every line said before the model's first generated token goes out as an Open WebUI status event, the line under the model's name that its tools use, and one last event at the first token says what the wait was ("Read 12,874 tokens in 14 s"). The reasoning opens with the model's own first piece, and a turn that does not think shows no "Thought for". Lines after the first token keep the profile's channel.

The progress lines (`queue`, `prefill`, `thinking`) are templates with numbers: the queue position and an estimate of the wait, a long prefill's rate and time left, a long thinking's time and tokens (`ui.progress`). A request that waits for the engine opens its stream at its first waiting line, so a refusal after that ends the stream with an `error` object and `[DONE]`.

21.2 The narrator and the voice

`ui.narrator` picks who writes the lines that are not progress templates.

Table 21.2: The narrator's modes.

value	lines
`off`	the templates
`on`	each line rewritten by a small model, in at most 150 ms, else the template
`thoughts`	one summary per stretch of thinking of at least 128 tokens: "Thought 1m 43s: planned the level layout and decided on a fixed timestep"
`live`	the voice: what the model is doing right now

The voice (`live`, the coding and chat profiles) separates facts from words. The engine writes the facts: the timer, a tool call's action and target read from the open call block as the model writes it, a write's line count, the queue's place, the prefill's tokens. The small model writes only the short phrase after them, from a window of the model's newest tokens: what the thinking is about now (every 25 s from 15 s), what a thought of at least 10 s decided, what a write is for. Every phrase passes a check before it is said: a file name or number it holds must be in the text the small model read, and so must most of its content words; no first person, no markup, no copy of an example, nothing like the last four lines, and the user's language (English or German). A phrase that fails is not said. Nothing is said for a short turn: no thinking line before 15 s and no tool line for a call done within 1.5 s.

The small model is `QWEN38_NARRATOR_MODEL`, a Qwen3.5 model (`qwen3.5-2b` for the voice). It runs on the engine's own Qwen3.5 forward (`engine/generative/qwen35.py`) and reads the model's own token ids without a second tokenisation, because the Qwen3.5 small models share Qwen3.8's vocabulary and merges; the server checks that at load against the served checkpoint's tokenizer. Its decode step runs from a CUDA graph captured at load, in a memory pool and on a stream of its own. It loads at the start through the residency planner (role `narrator`, a 6 GiB claim) and stays loaded.

Its time is rationed. The phrases are written in slices of at most 150 ms on the request's own thread, between two of the model's rounds, with the device synchronised first, so nothing of the small model runs beside the big one. A request's small-model time is one share: 3 % of the time since the model's first token (`ui.live.share`; 2.5 % in the coding profile, which also runs the shadow reader from the same share).

Measured on the engine with Qwen3.5-2B (the 0.4.0 series, the same 2,000-token text and 504 blocks in every run, 3 runs an arm, `--no-learn-cost`): 63.47 tok/s with the voice off, 62.26 on, -1.90 % (measured). On four recorded opencode sessions the 2B wrote a phrase in 308 to 369 ms at the median, in slices of at most 150 ms (measured, `tools/voice_replay.py`).

21.3 The shadow reader

A small Qwen3.5 model keeps its own state of each coding session: its KV rows and its Gated DeltaNet state for every token the session holds, the served prompt and the model's output. It reads each turn's new tokens and nothing else, so a small model can answer a question about a 100,000-token session in one forward of the question. It is off until `QWEN38_SHADOW_MODEL` names a model (`qwen3.5-0.8b`).

Two cheaper ways to give a small model the big model's context were tried first and failed: a few soft tokens made from the 27B's hidden rows answered at chance, and a per-layer map from the 27B's state into a small model's reached a held-out R^2 of about 0.5, far from what agreement needs. The small model's own state of the same tokens is exact, and that is what the shadow keeps.

Exact on a grid. The state advances only in whole chunks of 512 rows at absolute positions $k \cdot 512$. A fresh prefill of the same ids on the same grid runs the same chunks on the same inputs, so the two states are equal bit for bit, however many turns it took. Measured on the GPU on two real

conversations of 41,802 and 58,278 tokens: every KV row, the recurrent state and the convolution state equal a fresh prefill on the same grid (measured, the 0.4.0 series). When a new prompt differs from what was read (a template drops an earlier turn's reasoning), the reader goes back to the newest checkpoint at or below the difference, one kept at each turn's prompt end.

When it reads. While the big model decodes, only it and its drafters use the GPU. The reader has two windows. In the idle window it looks every 0.2 s, and when no request runs or waits it reads for at most 2 s, checking before every chunk whether a request waits; a request that arrives waits at most one chunk, 36 to 50 ms on the 0.8B. Inside a turn it may use 2 % of the time since the model's first token, in slices between two rounds. The idle window alone is not enough for an agent: in the ledger's opencode sessions the median gap between one request's end and the next one's start is 0.31 s.

What it costs. With the shadow on, the big model's decode on a 16,350-token prompt went from 36.02 to 35.28 tok/s, -2.05 %, and the time to first token did not move (measured, `tools/shadow_ab.py`, 6 runs an arm, the 0.4.0 series). That is the 2 % share, as designed.

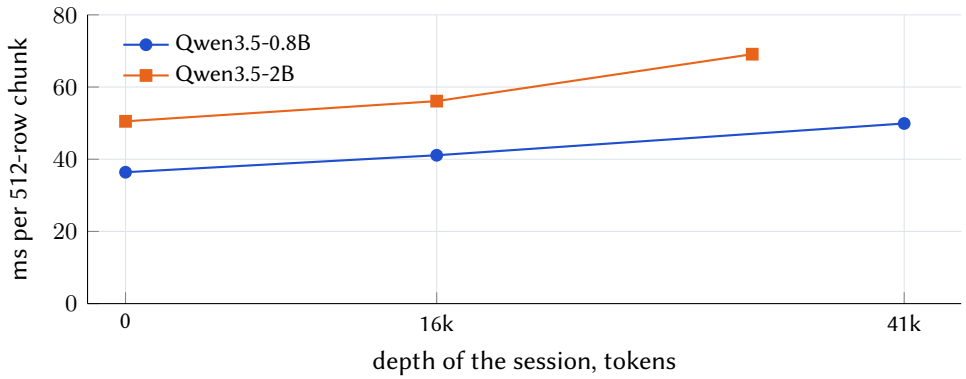


Figure 21.1: The shadow reader's read rate: milliseconds per 512-row chunk at three depths of the session (the 2B's last point at 33k). measured: `tools/shadow_measure.py` with the served process's arithmetic flags, median of 3, the 0.4.0 series.

Who asks. `ask(session, question, choices)`, also `POST /v1/conductor/shadow/ask`, is a typed decision of the small model over the whole session: each choice is a letter, and one forward of the question gives the letters' softmax. At the end of a 41k session the router's 9-class question took 68 ms on the 0.8B (measured). In the coding profile the router's class question is asked of the shadow in shadow mode on turns of at least 8,192 tokens and journaled beside the router's own class; nothing of the turn changes.

What that journal showed is a result in its own right. On 58 journaled turns, the small models agreed with the router's class no better than a constant answer does:

The shadow therefore keeps the 0.8B (1.5 GiB of weights, the fastest read) and the question stays in shadow. A question the small models can answer, such as two or three coarse classes asked only where the router decides fresh, is the next thing to try.

Table 21.3: The router's question asked of the shadow on 58 journaled turns of 6 opencode sessions. A constant `code_edit` scores 42 of 58. measured, `tools/shadow_router_eval.py`, 2026-10-05.

model	classes with descriptions	short labels only
Qwen3.5-0.8B	8 of 58	2 of 58
Qwen3.5-2B	25 of 58	8 of 58
Qwen3.5-4B	19 of 58	44 of 58

Memory. Resident states are capped at `QWEN38_SHADOW_GB` (4 GiB). Past the cap, the least recently used session goes to the NVMe (`QWEN38_SHADOW_DISK`, one file a session, 48 h) and comes back exact. A 58k session is 0.88 GiB on the 0.8B. The model loads through the residency planner in an idle window soon after the start, and its own pool, up to its claim, is left out of the engine's memory ceiling. A live memory pool never gives its free segments back, so after an idle burst that leaves more than `QWEN38_SHADOW_POOL_SLACK_GIB` (2) free in the pool, the shadow parks its sessions on the disk and unloads the model; the next turn loads it again and reads on from the disk.

Chapter 22

Memory notes

The model can keep notes about the user and the work, as files, and read them back in later turns and sessions. One note is one Markdown file with a frontmatter: one fact, a title of at most 120 characters, a body of at most 400 words. The files are the truth, and the dashboard's Memory screen, Notes tab, reads and edits them. Memory notes are off in every shipped profile; this chapter says what they do when you switch them on.

22.1 Where the files are

```
~/.qwen38-spark-engine/memory/notes/<user>/
  INDEX.md                              generated each night: one line per live note
  user/<kind>-<slug>--<id>.md           about the person
  projects/<name>-<hash8>/<kind>-...md  about one working directory
  sessions/<session>/<kind>-...md       true for one conversation only
  revisions/<id>/<n>.md                 every earlier text of a note
  archive/<yyyy-mm>/...                 archived notes, deleted after 30 days
  usage.json                            how often each note went into a request
```

<user> is one fixed name for the engine's own requests. A request the proxy relayed carries the proxy user's name (the key's label, or `X-Tandem-User` from a trusted address); a relayed request without one reads and writes nothing. Files are mode 0600, and the store refuses a root inside a git work tree.

TandemLLM serves one user: every client of the engine is that user, so a "remember this" from any client is a note in the same store, shown as the line `› remembered: <title>` and archived or deleted in the Notes tab. A request that must not touch the notes sends `X-Tandem-Memory: off` (nothing read, nothing written, no card; the journal says why) or `read` (the reads only; a block the model writes anyway is dropped). Tests and scripts against a live engine send `off`.

A note has one of eight kinds: `rule` (how the user wants things done), `ruling` (a decision), `gotcha` (what went wrong once), `runbook`, `program_state` (with a `verify_by` date, stale after it and never injected), `reference`, `registry` and `archive`. And one of three scopes: the user, a project (one working directory), or a session.

22.2 How the model writes

The model writes a block in its answer, usually after its text:

```
<tandem_memory op="add" scope="project" kind="gotcha">
title: the build needs node 20
tags: build

npm run build fails on node 18 with "Unexpected token 'with'".
</tandem_memory>
```

`op` is `add`, `update` (with an `id`) or `archive` (with an `id` and a reason). The server holds the block back from the client the way it holds a tool call (`server/memblock.py`), writes the note, and the client sees one status line, "› remembered: the build needs node 20", in the content after the answer whatever the request's `reasoning_format` (opencode shows it as a reasoning part's title), so Open WebUI and the OpenAI SDK show the write too; the server takes the line out of the history the client sends back. A block that is not taken goes out as the text it was: one that never closed, one over 2,000 characters, the fifth of an answer, or one the store refused. A block inside the reasoning is left alone. The block's tokens are recorded as hidden spans, so the next turn's prefix still matches the rows.

The rules are code, not prompt text (`server/conductor/notes.py`):

- A write over a cap is refused, never cut: title 120 characters, body 400 words, 20 notes a session, 500 live notes a user.
- Every title and body passes the secret-pattern refusal of the transcripts, and a body that is mostly a secret is refused.
- A `rule` or `ruling` about the user needs the user's own message in the turn: a tool turn or a script cannot write one.
- The model writes provisional notes. Only the dashboard ratifies, pins or deletes a note, and a model's revision of a ratified note makes it provisional again. The model's archive of one does the same: the note stays, with the model's reason in its changelog, until the user archives it or ratifies it again.
- Before an `add` lands, the same scope is searched; the same normalised title, or a close BM25 match, turns the add into a revision of that note.

The model's scripts have the same write as a route, under the session's address only: `POST {tandem_url}/v1/memory`, with search and read beside it. Without a known session token the routes answer 403. Figure 22.1 follows a note from its write to its archive.

22.3 How the model reads

A profile's `memory` switch is `off` (the default: the request is the release's), `shadow` (the card goes in and blocks are written, nothing is read back) or `on`. With `on`, three tail notes, each replayed at its place on later turns so each costs its prefill once:

- the memory card, about 150 tokens: the block's shape and when to write one;

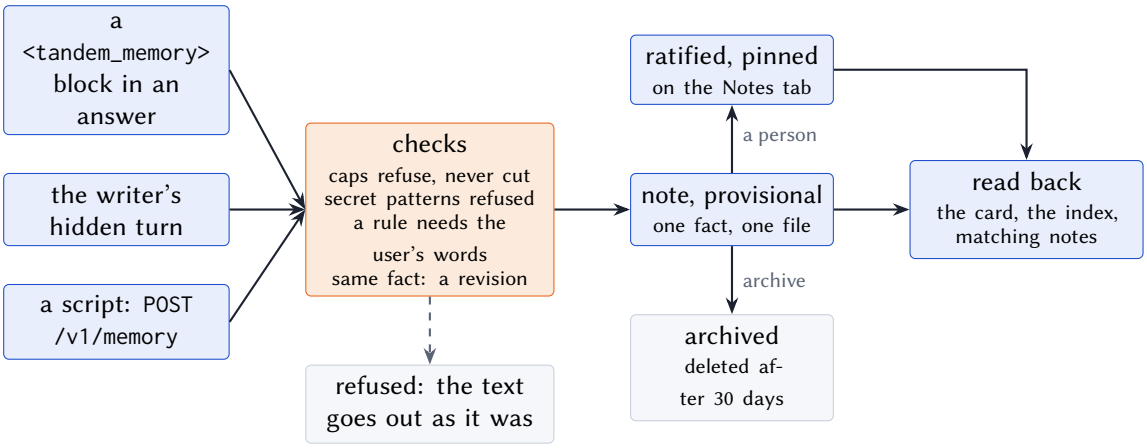


Figure 22.1: The life of a memory note. Three ways in, one set of checks, a provisional note that a person may ratify or pin, reads back into later turns, and an archive.

- the index, on the first turn of a session and again after a compaction: the pinned ratified rules, then one line per note the request may see, within 1,500 tokens;
- the matching notes, on a new user message (never a tool turn): BM25 over the message, at most 3 notes that share at least 2 of its words, within 600 tokens. On a session's first turn they go in after the index. Their note has its own frame: the other hidden notes say "do not mention it", but these are the user's notes, so the frame and its head tell the model to answer from a note that answers the message, to say it comes from its notes (a ratified note is confirmed by the user), and to tell the user both when the files disagree.

22.4 The background writer

Besides the blocks in its answers, a session's facts can be written by a hidden turn of the model itself, thinking off, on the conversation the engine already holds (`server/conductor/memwriter.py`). Its switch is `settings.memory.notes.writer.mode`: `off` (the default), `shadow` (the turn runs, its blocks are counted and logged to a shadow file, nothing is stored) or `on`.

- Triggers: the session's end (no request for 180 s and at least 1,500 new tokens since its last pass), a give-up, and a compaction.
- The record: the session's last generation (its end anchor), else the last prompt in the rows up to its last message's end.
- The gate: Laya's question "do these messages contain something to remember" on the session's last user messages; a confident no skips the turn.
- The turn's prompt carries the notes that already match the session, so it updates instead of adding. Its output is parsed as the inline blocks are, and notes land provisional.
- The tool-call token gets a logit bias of -100 in the writer's turn (`ban_tool_call`, on), so the turn cannot start a call; any other call it writes anyway ends the turn at its function's name, and the run records that it wrote nothing.

- A request that arrives stops the turn at its next chunk; the conversation's rows and anchors stay as they were.

Measured on a test engine (the 0.4.0 series, a synthetic 10-turn coding session, low effort, writer on): the writer at the session's end wrote 2 notes in 7.6 s for 282 tokens from a 5,374-token record. A new session that asked what was remembered got 1,781 prompt tokens with memory on (the index and 3 notes) against 136 off, and its first token in 0.97 s against 0.39 s; the answer named both rules the user had asked to keep. Decode was 99.7 ms a block with memory on and 99.6 off (measured).

22.5 On the second model

Nothing in the notes depends on which model serves: they reach the model as prompt text, and the server parses the blocks it writes. In front of Qwen3.8-Flash-Next (Chapter 17) the same code runs. Nobody has checked yet whether Flash-Next writes useful blocks, because no session with memory notes on has run against it. Its profiles keep the notes off, like every shipped profile.

22.6 Behind the proxy

For a backend behind the proxy (vLLM, SGLang, any OpenAI-compatible server, a cloud API), the writer's turn is one more request to the backend the session used (`proxy/memwriter.py`): the messages of the session's last request as they were sent, so the backend's prefix cache matches, its answer, and the writer's user turn. Thinking is off, and the session's tools stay in the body with `tool_choice: "none"`. The answer goes through the engine writer's own parser and store, and the request is a ledger row of the client `harness:writer`.

It runs in the backend's idle time: one writer request at a time, only while the backend has no client request in flight; on a local backend a client request that arrives closes the writer's stream. Per backend: `writer` `auto` (on, but off on a paid cloud model), `off`, `shadow` or `on`; a spend cap a day on a paid model (0.25 USD, checked against an estimate before each run); 48 runs a day; a prompt of at most 32,000 estimated tokens. A paid model without a price is refused. Measured on 2026-10-05 with Qwen3.5-0.8B behind vLLM and SGLang, one run each: the writer started nothing while a client request was in flight, and with an answer prefill it wrote 2 notes on vLLM and 1 on SGLang (measured).

22.7 The Notes tab

The Memory screen's Notes tab shows the tree by scope, the list with each note's uses in 7 days, a note as a file (edit as a ratified revision, ratify, reject, archive, pin a ratified rule, delete), its history as diffs, the search with scores, the requests the notes went into, the writes and refusals with their reasons, the writer's runs, the notes settings, an export as a zip, and the user's delete behind a typed confirmation. Its routes are `/v1/conductor/memory/notes...`, under the admin rule for reads too: open while no admin password is set.

What is journaled: each write (operation, id, scope, kind, why it was refused, redaction counts), each injection (the note ids), the dashboard's edits and the nightly job. Never a title or a body.

Chapter 23

The conductor in front of any backend

The conductor runs in front of other backends too: the router's task class, the effort, the packages and skills as notes, the side requests' caps, the loop guards, the empty-turn continue and the tool checks, the compaction section, the status lines and memory notes. In the engine's server with `--backends` it is the engine's own conductor. The standalone proxy keeps one of its own under `--conductor-config` (default `~/.tandemllm/conductor`, `off` for none), with its journal in the usage ledger's file, as on the engine.

23.1 Switching it on

Table 23.1: The conductor's settings per backend.

setting	where	values
`harness.mode`	the backend's form, `TANDEM_BACKEND_<ID>_HARNESS`	`off` (the default): the relay, byte for byte; `shadow`: journal what it would do; `on`
`harness.profile`	the form, `TANDEM_BACKEND_<ID>_HARNESS_PROFILE`	the conductor profile for this backend's requests; empty picks by client
a feature group per model	the model's row	`inherit`, `off`, `shadow`, `on`; never above the mode

Start with `shadow`, read what it would have done in the Conductor view's journal and the Compare view's precision per backend, then switch to `on`. With the conductor off for every backend, a relayed request is copied byte for byte apart from the model id and the usage frame.

23.2 The feature groups

Each feature group can be switched per model. What `inherit` means depends on the side: a local engine gets everything, while a cloud API gets the groups that cost tokens on every turn, or a second request, off or in shadow until a model is switched on.

The Backends screen shows, next to each switch, what it resolves to, one sentence on what it costs or loses on that backend, and what its extra requests cost over the last 24 hours. The class questions are ledger rows of the client `harness`, and a second request's tokens and cost are in its journal row.

Table 23.2: Feature groups of the conductor over HTTP, what `inherit` resolves to, and what each costs.

group	what it does	local	cloud	cost
`class`	the router's question, asked of the backend with its own constraint field	on	on	one short extra request per new user message
`effort`	the backend's thinking switch: `chat_template_kwargs.enable_thinking`, `reasoning_effort`, DeepSeek's `thinking.type`	on	on	nothing; a level is a word or on/off, never a hard cap
`packages`	the router's note at the end of the newest user message, replayed at the same message index	on	off	input tokens every turn; a prefix cache keeps its match
`side`	a client's title or summary call capped, thinking off	on	on	saves tokens
`loops`	D1 and D2 on the streamed words; the tool-argument guard	on	on	a reasoning close is a second request (vLLM and SGLang only)
`e1`	an empty answer to a request with tools gets a second request	on	off	one more full request
`tools`	the tool-call checks; a regenerate is a second request	on	shadow	the checks are free, a regenerate is one more request
`compaction`	the TandemLLM section appended to a client's compaction request	on	on	nothing extra
`states`	the status lines in the stream	on	on	nothing
`memory`	the memory card and notes; the model's blocks written; the background writer	on	shadow	the card's tokens; the writer's request

23.3 What is different from the engine

The note is text. On the engine the note is token ids placed before the turn's end token. Over HTTP it is text at the end of the user's message, inside the `<runtime-note>` frame the router uses, so a model could quote it. The replay keeps the bytes of the conversation's head the same from turn to turn, so a backend with a prefix cache (vLLM, SGLang, OpenAI's and DeepSeek's prompt caches) keeps its match: a hash covers each message's role, content, name and tool calls up to the noted one. A client that rewrites its history breaks the hash; the note is gone, and the router adds its block again.

The class question is a request. On the engine the class question costs nothing beside the turn. Over HTTP it is a request of its own, a few hundred prompt tokens, with the backend's own way to constrain the answer: vLLM's `structured_outputs.choice`, SGLang's `regex`, an OpenAI JSON-schema enum, TandemLLM's own grammar, else plain text. A new user message can ask up to three questions (the class, the package, a library skill); a tool turn asks none. With the conductor on, the proxy's own time in the ledger includes the plan.

Second requests continue the first. E1, the regenerate and the reasoning close are second requests. The client still gets one stream: the second answer goes out as the continuation of the first, with the first chunk's id, no second role delta and one `[DONE]`. To make that possible the proxy holds the finish frame, the usage frame and `[DONE]` until the turn is settled, and holds the tool-call frames while a check can act. When the client asked for usage, the usage it gets is the sum of every request, and so is the ledger row. A reasoning close needs `continue_final_message`, which the `vllm` and `sglang` kinds have; on any other backend D1 and D2 are journaled in shadow. A request without a stream gets the plan and none of the monitor's acts, which read the stream.

23.4 The word-level loop detector

D1's three arms run over words instead of tokens over HTTP (`server/conductor/monitor/tier0_words.py`), with thresholds measured for words, never the token ones copied. On the reasoning of an opencode database (2,674 finished turns and 20 loops), the strictest of 32 settings fired on 6 of the finished turns and caught 11 of the 20 loops (measured, `tools/loop_words_fp.py`, 2026-10-04). The engine's own D1 acted on that traffic, so some of those 6 may be loops it closed; the text alone does not say which. The shipped words block therefore says `calibrated: false`, and every D1 verdict over the proxy stays in shadow until the Compare view's labels show its precision.

23.5 Laya, the library and the session address

The standalone proxy serves the library (`GET /v1/library`) and `POST /v1/laya/ask` when the conductor's `decider` role names a decision model over HTTP. Both answer under `/s/<token>/`, the address the notes give the model, and are journaled to that session. The `classifier` role may name a backend (`backend:<id>/<model>`) to answer the class question instead of the request's own backend, and the `narrator` role the same for the status lines; without one, the lines are the templates.

23.6 What was checked

Against the live engine as a backend of kind `tandemllm`: the class question was answered by the engine through its grammar, the note was added and replayed on the next turn at the same message, and a title was capped at 64 tokens with thinking off (`tools/proxy_harness_check.py`). The close by `continue_final_message` and the reasoning parsers' output after it are tested on a fake backend. The cloud providers' answers to the class question and to the effort switch are tested on their documented shapes, not on a paid call.



Part V

The dashboard

The dashboard, view by view (24), its Chat tab with the sandbox and the permission modes (25), and how it watches several engines at once: the Parallel page, the engines' own metrics pages, the plugins inside vLLM and SGLang, and the host collector (26). Every figure on it is measured or null, never estimated.

Chapter 24

The dashboard, view by view

The engine serves its own web page at `/dashboard/`, and the standalone proxy serves the same page at `:8080/dashboard/`. It answers what a log cannot answer at a glance: what the engine is doing right now, for whom, and how fast. It is written in TypeScript with Lit components, its charts are hand-drawn SVG, it loads nothing from a CDN, and the server serves it from `dashboard/dist/` with no Node on the machine that serves it. The parts every view loads are about 45 KB of gzip. Figure 24.1 shows the seven views and what each reads.

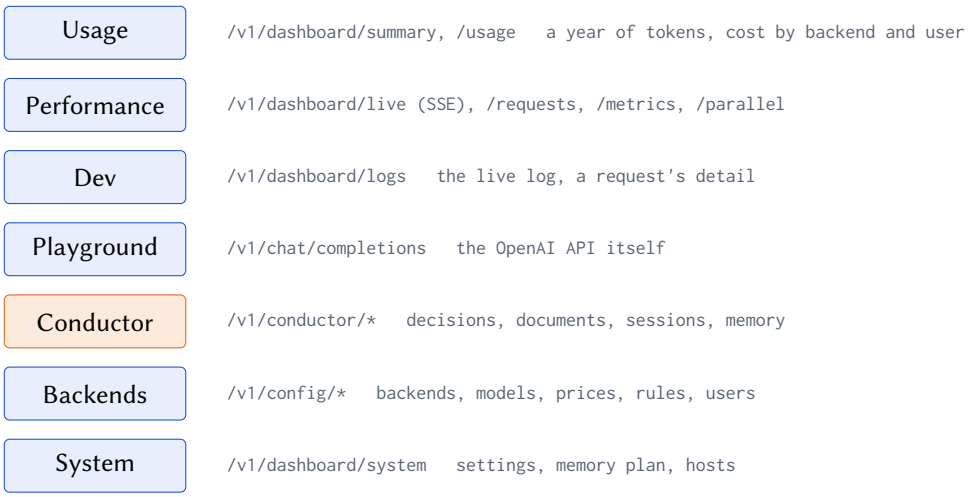


Figure 24.1: The dashboard's seven views, in the order of its navigation rail, and the server routes each reads. Every view only reads, except where an edit makes a new version of a document or a configuration row.

24.1 Performance: the live panel

The Performance view has two pages. Engine shows the engine's own requests; Parallel shows every request on every backend (Section 26.1).

The Live panel on top streams `GET /v1/dashboard/live` over server-sent events: four events a second while a request runs, one otherwise. A line at the top says what the engine is doing now, in the server's own words:

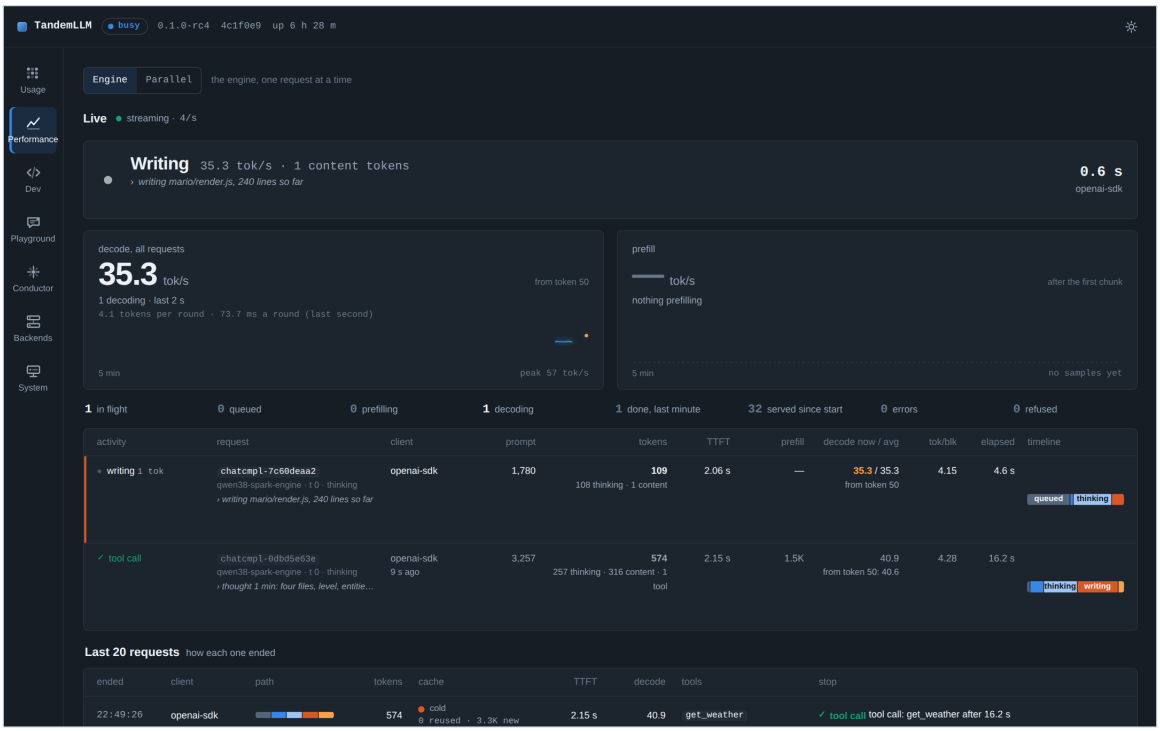


Figure 24.2: The Performance view, Engine page: the Live panel with a request writing, the decode and prefill rates, and the last requests. Demo data from the dashboard's mock server (`npm run preview:mock`), not a measurement.

- `Prefilling 36,864 of 48,210 (76%)`, with the prefill rate and the time left;
- "Thinking" or "Writing", with the decode rate and the tokens so far;
- "Calling tool write_file", with the size of the arguments written so far;
- "Waiting for client: running tool bash", between two turns of an agent;
- "Idle" or "Draining".

When the client's socket closes while the engine still works for it, the line says so in red: "client disconnected 12 s ago". Under the two big figures (decode and prefill tok/s, each with a 5-minute sparkline) the panel shows tokens per round and milliseconds per round over the last second. Each request in flight gets a row: its state, its client, tokens split into thinking and content, time to first token, the rates, the newest status line, and a timeline strip with one segment per state. When an agent sends its next turn, the row says which request it continues and what the client did in between ("continues chatcmpl-7f3a after 8.4 s, client ran bash").

The live rates leave each request's warm-up out (the first 50 decode tokens and the first prefill chunk) and show a dash when nothing runs, so a figure is always a rate of work being done now. A list of the last 20 requests says how each one ended in one sentence ("abandoned by the client after 73 s of silent prefill, 0 tokens sent"); abandoned, failed, timed-out and cancelled requests get a red mark, refused and rejected ones an amber one. The panel also reports on its own stream

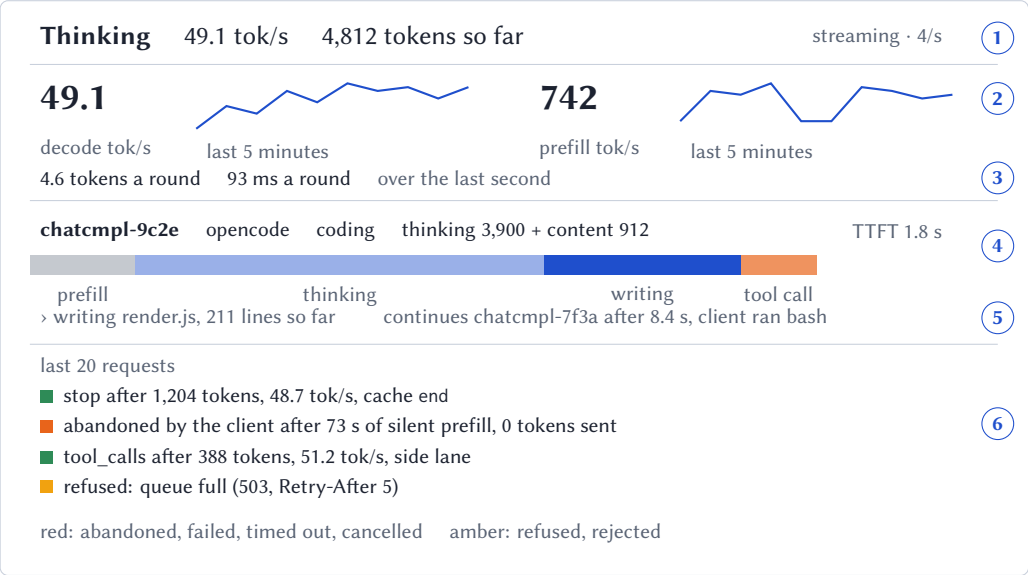


Figure 24.3: The Live panel, read as an instrument. (1) The state line, in the server's own words, and the panel's report on its own stream. (2) The two big figures with 5-minute sparklines; each leaves the request's warm-up out and reads a dash when nothing runs. (3) Tokens a round and milliseconds a round, the two factors of the engine's speed. (4) One lane per request in flight: state, client, profile, tokens split into thinking and content, time to first token, and a strip with one segment per state. (5) The newest status line, and what the agent did between two turns. (6) How each of the last 20 requests ended, in one sentence; a row opens its detail in the Dev view. The figures shown are examples, not measurements.

("streaming · 4/s", "no update for 3.0 s", "reconnecting since 4 s"), so a silent page is never mistaken for an idle engine.

Every state and every sentence comes from the server (`server/activity.py`); the page only formats numbers. The engine's loop keeps the state it needs anyway, and a sampler reads it once per tick, so no statement runs per token for the dashboard's sake. A tick costs 410 to 470 µs of CPU at 4 Hz (measured), and with the activity on and a stream open, the release gate found the text byte-identical and the round time unchanged within noise. `--live-activity off` turns the activity fields off.

24.2 Performance: the history

Below the Live panel: the time to first token, tokens per round and acceptance over the last 5 minutes, then speed history from the usage ledger. The Recent requests chart plots decode tok/s against prompt length, one dot per request. A replay (a response-cache hit, a summary written ahead) decoded nothing and is never a dot there, nor in any average, percentile or live rate; a switch lists the replays with their time to first token. Side-lane requests are not plotted either, because a short plain decode beside a turn says nothing about the engine's decode rate. The y axis ends at twice the 95th percentile, and a point above it is a triangle at the top edge with its value in the tooltip. Each request's cache source (`end`, `resident`, `disk`, `spill`, `continuation` and the others) is shown with it, so a slow first token can be read at once.

24.3 Usage

A year of tokens as a heatmap, with totals for today, 7 days, 30 days and the year, split four ways: input, cached input, output, reasoning. Filters by model, client, backend (`engine` is the engine's own rows) and user, and the cost of priced models. The view reads the usage ledger, which keeps at least 400 days and never stores prompt or answer text (Section 30.2). TandemLLM's own calls to itself (the router's question to the engine, client kind `internal`) are left out of the totals, the filters and Dev's recent requests unless the "show internal" switch is on; each names the request it served. In the Live panel the cached tokens never read above the client's prompt, and TandemLLM's notes show as their own figure, "+5,235 hidden".

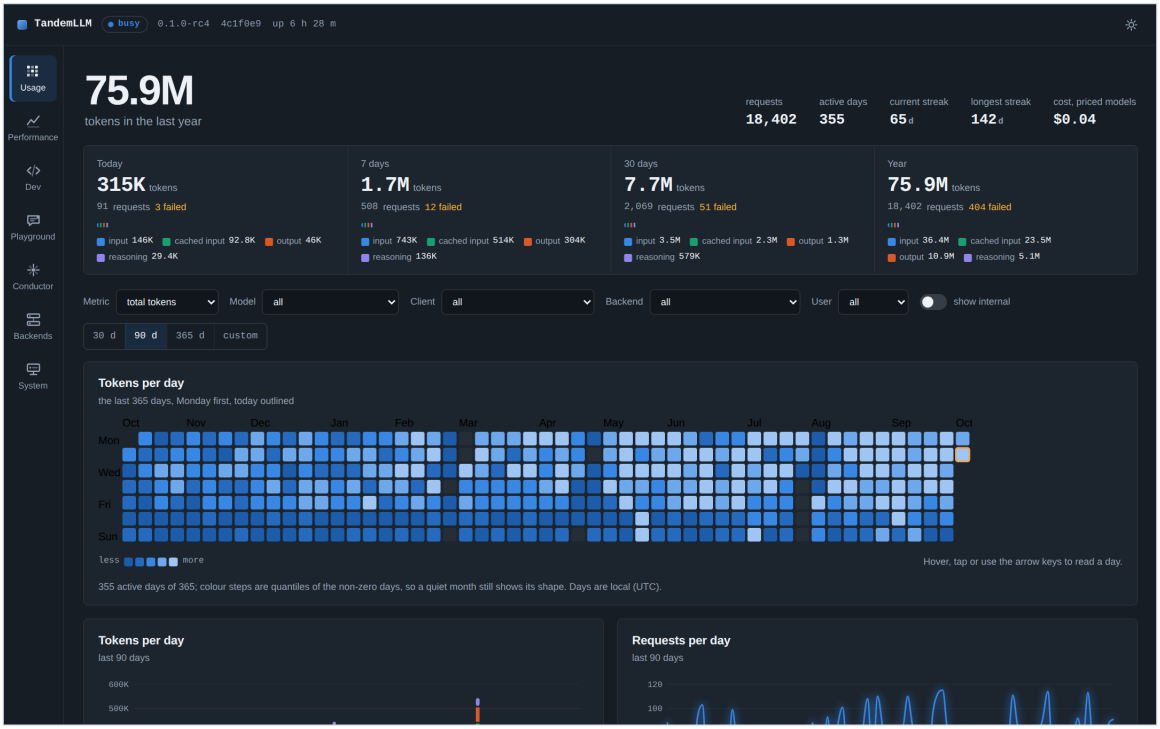


Figure 24.4: The Usage view: a year of tokens, requests and cost, filtered by model, client, backend and user. Demo data from the dashboard's mock server (`npm run preview:mock`), not a measurement.

24.4 Dev

The live server log, filterable, with a detail view for each request: its timings, its cache source, its tokens, the conductor's decisions for it and the reason it ended. A memory-gate refusal is an error line here with the allocated and reserved figures.

24.5 Playground

A streaming chat against the server's own OpenAI API, with tools and parameters, and the figures of each answer. It uses the same routes any client uses, so it works with the login on or off.

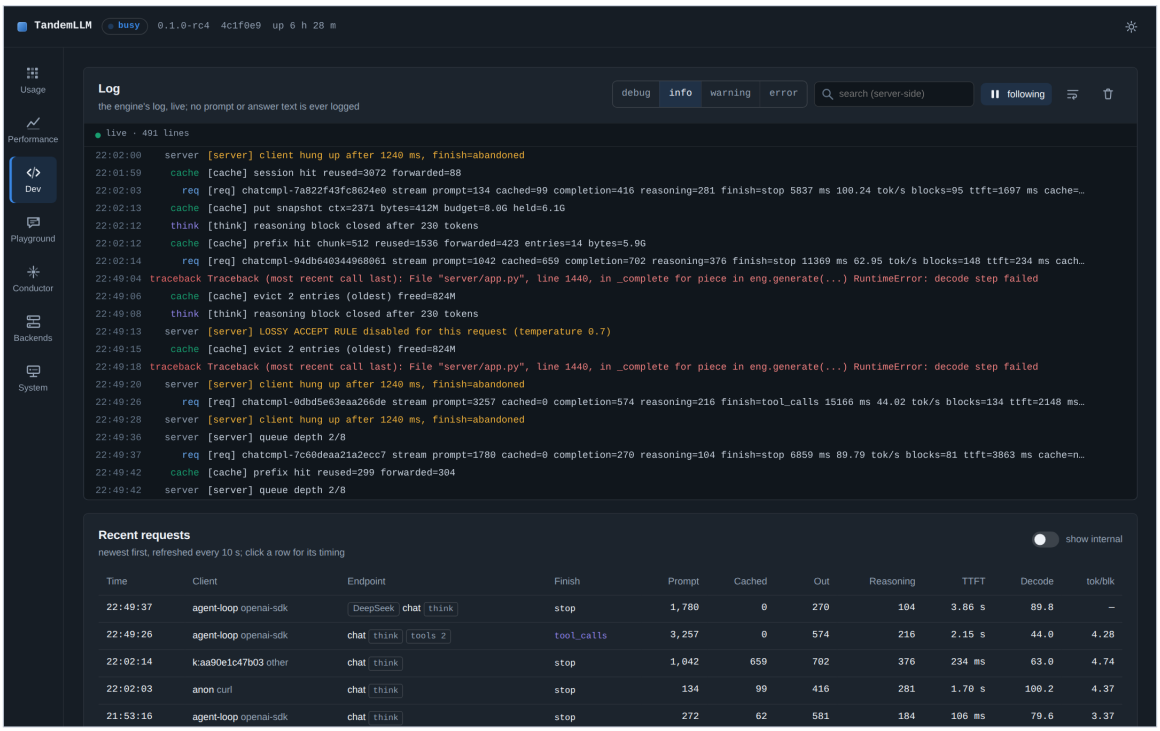


Figure 24.5: The Dev view: the live log, which never holds prompt or answer text, and the recent requests. Demo data from the dashboard's mock server (`npm run preview:mock`), not a measurement.

24.6 Conductor

The Conductor view shows how `TandemLLM` decides, and edits every knob of it. Every edit is checked first, shows its diff, and becomes a new version with a note; nothing asks for a password. Its screens:

24.7 Backends

On an engine without `--backends` the System answer says `proxy.mode: off`, and the view says the proxy is off without asking the configuration routes. The dashboard has no token dialog and no sign-in screen (Section 3.8).

The proxy's registry: each backend's health, models, requests in flight and cost today. Add one after a test connection, edit, delete. The key field is write-only and shows the key's last 4 characters; a field a `TANDEM_*` variable sets is locked and names it. The models have public ids, prices (USD per million tokens, the release's list prices one click away) and the conductor's feature switches, each with what it resolves to and what it costs on that backend. The background memory writer's switches are on a backend's form. The ordered routing rules and the users with their keys, each key shown once, complete it. With no backend the page opens on Setup; on an engine without `--backends` it says how to turn the proxy on. A vLLM or SGLang backend's card shows its engine's own figures and whether its plugin reports ("L2 plugin" or "L1 only", Chapter 26).

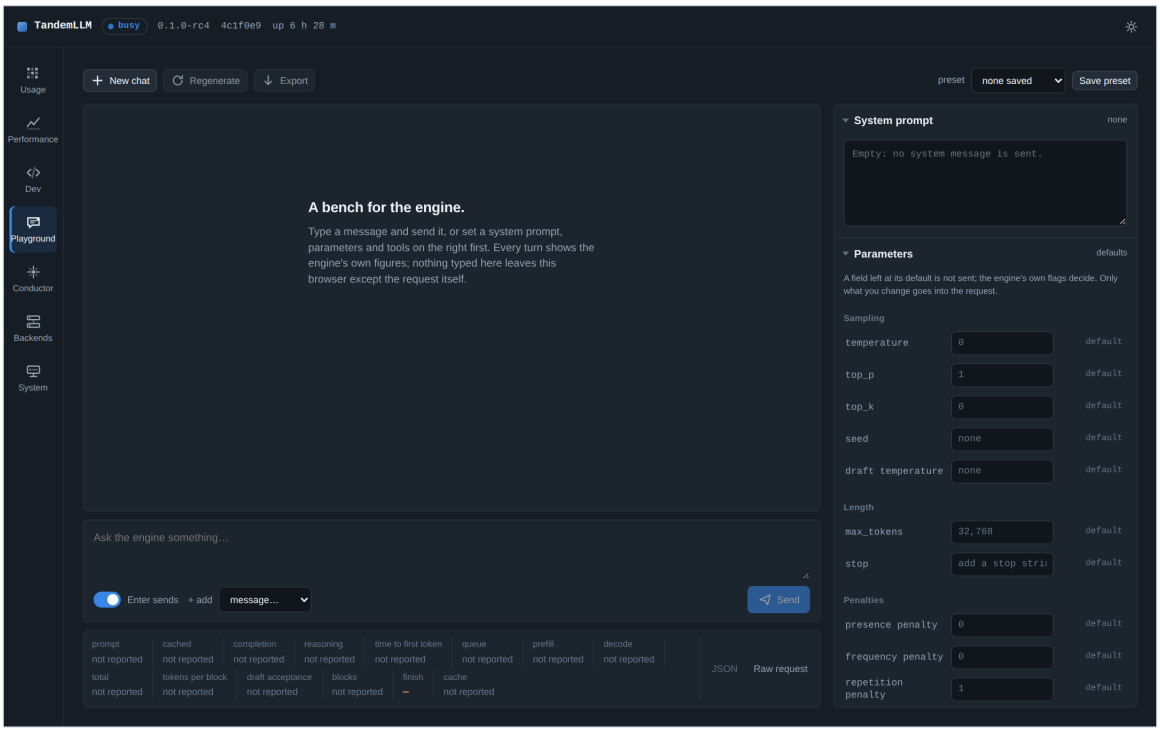


Figure 24.6: The Playground before the first message: the chat on the left, the system prompt and the sampling parameters on the right, the engine's own figures for each turn below. Demo data from the dashboard's mock server (`npm run preview:mock`), not a measurement.

Table 24.1: The Conductor view's screens.

screen	what it shows and edits
Overview	the live decision feed: each turn's class, confidence, effort, note and the monitor's events; tiles over the last 24 hours
Routing	the task classes and their policy (effort, effort by size, packages, rules), the classifier stages and thresholds, stickiness and the structural rules, a test box that classifies a text you type
Monitor	the session monitor's journal and rules: the tool checks' floors and project rules, the reasoning arms, the empty-turn nudges, the session tasks with their texts and patterns, and what they hold per session right now
Profiles	each profile's switches (mode, decisions, tasks and caps, thinking, what the user sees, outcomes, library, memory, shadow), the roles with their contracts, and the residency plan
Packages	every package, task skill, test skill, card and library skill, with tokens against limits and recent use; the editor; the Library tab; the Proposals tab
Deciders	the decision models: state, limits, latency, load and unload through the residency planner, the questions each answers, Laya for scripts
Sessions	the sessions, each a timeline of its turns and every hidden text the model saw, with an export
Memory	three tabs: the session transcripts kept on the machine, the memory notes (Chapter 22), and the disk tier's state and settings
Compare	the shadow precision of each task and question, per backend, from the labels a person gave decisions, and the evaluation reports

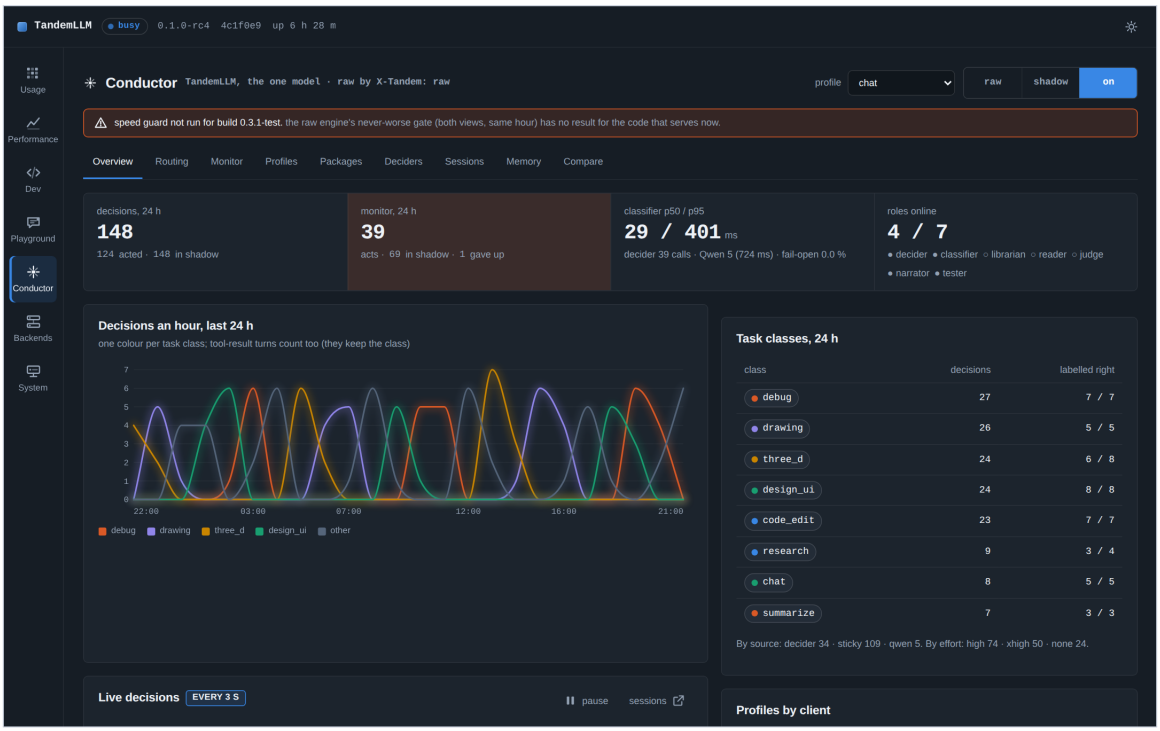


Figure 24.7: The Conductor view, Overview: decisions and the session monitor's acts over 24 hours, the classifier's latency, the roles online and the task classes. Demo data from the dashboard's mock server (`npm run preview:mock`), not a measurement.

24.8 System

The engine's settings (its command line and every `QWEN38_*` and `QSE_*` variable, the home directory written as ~, every token-like value redacted) and the machine's state: memory, GPU, caches, with 30 minutes of history kept in the browser. The memory plan shows the engine's own split (weights, KV pool, drafter caches, graphs, each read off the object that holds it), `MemAvailable`, every role with its claim and whether it fits now, the limits with what each holds, and the last refusal. Below them are the hosts: this machine with the engine's process and every backend on it, and each remote backend's machine, down to the GPU memory of its processes (Chapter 26).

The engine card carries one neutral line, the speed check: "passed for 0.4.0 · <date>" when the build has a speed guard report (the release ships its own, `release/speed-guard.json`), "not run for this build" otherwise; the numbers are in its tooltip and on the Conductor's Compare screen. It is no warning: the speed guard is the release process's check, not something to act on in the dashboard.

24.9 The contract

`docs/contract/dashboard-v1/` holds a JSON Schema for every JSON response of `/v1/dashboard/*` and a valid example of each; `docs/contract/conductor-v1/` does the same for the conductor's routes. Both sides test against them: the server's tests validate real responses, and the dashboard's

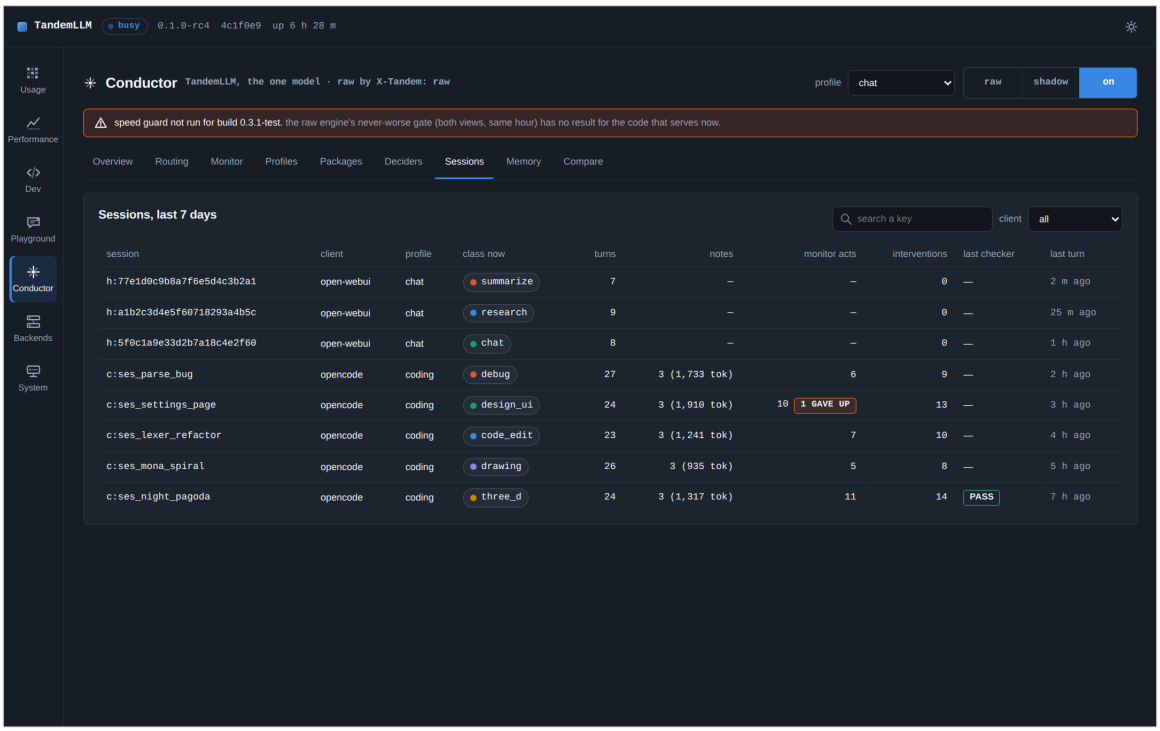


Figure 24.8: The Conductor view, Sessions: each session's client, profile, current class, turns and the hidden notes it carries. Demo data from the dashboard's mock server (`npm run preview:mock`), not a measurement.

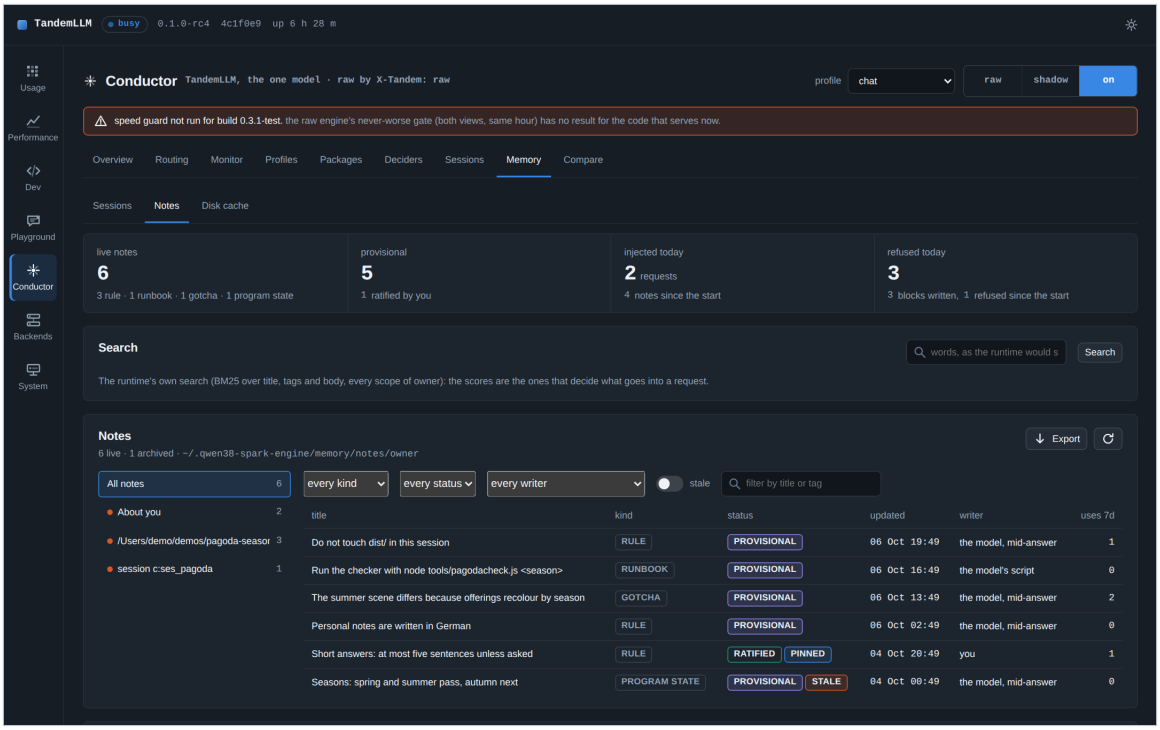


Figure 24.9: The Conductor view, Memory, the Notes tab: the live notes by scope, with kind, status, writer and use. Demo data from the dashboard's mock server (`npm run preview:mock`), not a measurement.

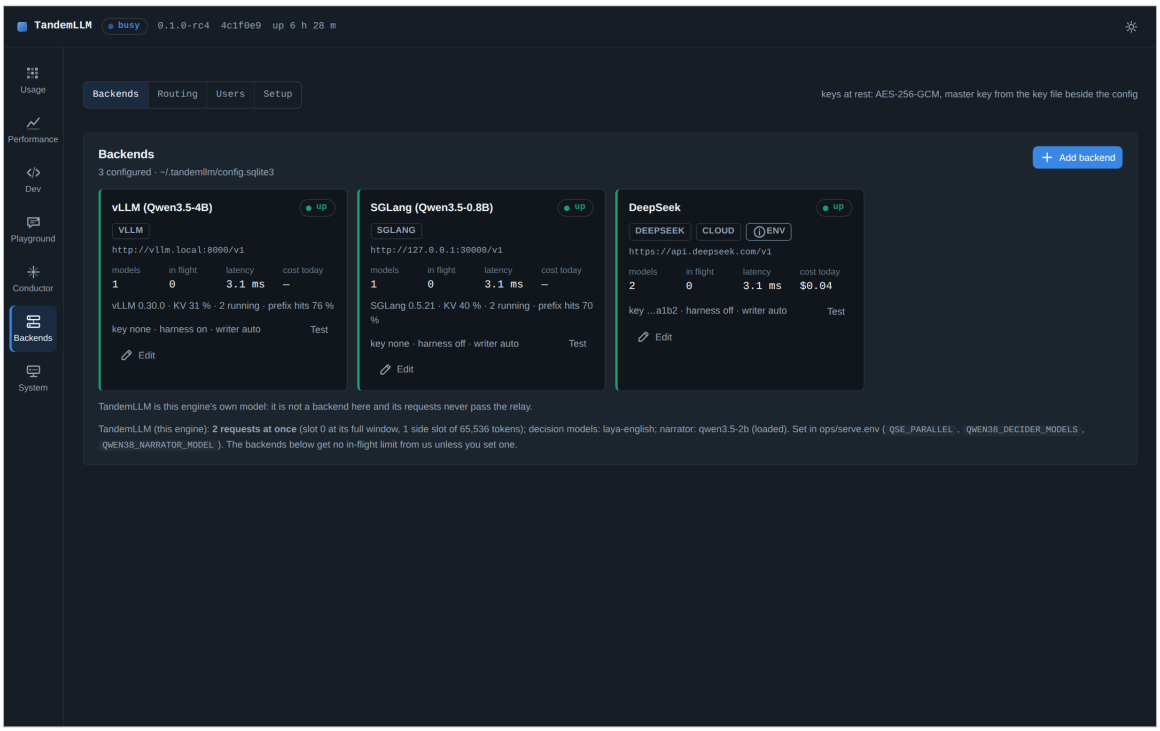


Figure 24.10: The Backends view with a vLLM, an SGLang and a DeepSeek backend, and the line for this engine's own settings. Demo data from the dashboard's mock server (`npm run preview:mock`), not a measurement.

tests validate the examples and every response of its mock server. A change to a field bumps `contract_version`, and the contract's README lists every change.

24.10 Building and testing the page

```
cd dashboard
npm ci
npm run dev:mock        # the app against a mock engine
npm test                # unit, contract and build tests
npm run build           # type-check and build dist/
npx playwright test     # end-to-end, Chromium and WebKit, desktop and phone width
```

The mock engine plays scenarios for the Live panel (an agent turn with a chunked prefill and a tool call, a client that leaves during a prefill, one request per stop reason) and failure modes (an expired session, server errors, dropped streams). The end-to-end suites also run against `server/app.py --fake-engine`, which serves the real API with a scripted engine and no GPU. Every view works at a phone's width of 390 pixels: the navigation becomes a bottom bar with short labels, and lanes and tables become cards.

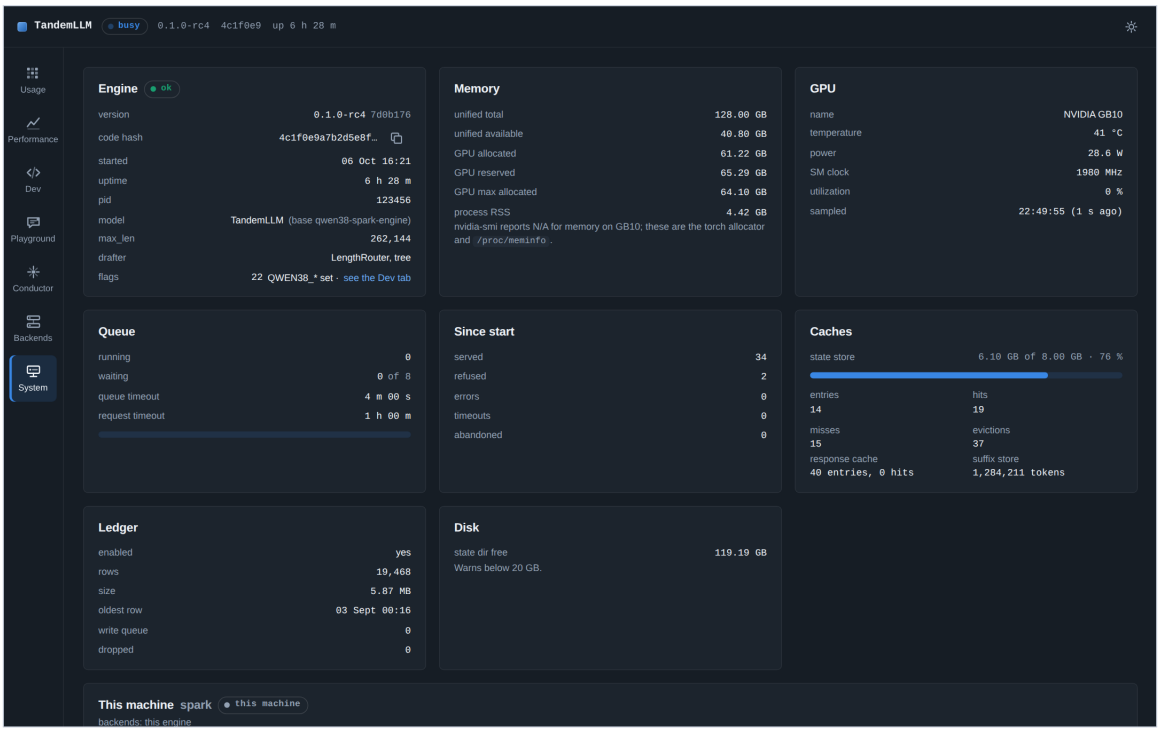


Figure 24.11: The System view: the engine, the memory, the GPU, the queue, the caches and the machine. Demo data from the dashboard's mock server (`npm run preview:mock`), not a measurement.

Chapter 25

The Chat tab

The dashboard's Chat tab (`#/chat`) is a chat whose runs live on the server. Close the browser in the middle of an answer, open the chat on a phone, and it shows the answer at the token the engine is writing now and keeps streaming. The tab is in beta in this release; a Beta pill beside it says so, and Section 25.5 lists what does not work yet. `docs/chat.md` has the full contract and every route.

25.1 What runs where

The browser talks to the engine server on port 8000, which passes `/v1/chat-app/*` to chatd after the dashboard's own auth check. chatd (`python -m chatd`, on `127.0.0.1:8016`) is a standard-library Python process. It holds the stream to the engine, so a run does not end when a browser closes, and it writes every change of a conversation to an event log with a sequence number. A device that attaches gets a snapshot and then follows the log from there. chatd reaches the model as any client does, through `POST /v1/chat/completions`, and reads what the model can do from its card in `/v1/models`, never from its name.

```
bash ops/make-secrets.sh            # once: adds QSE_CHAT_FORWARD_KEY to secrets.env
bash ops/chatd.sh start             # 127.0.0.1:8016, log in logs/chatd.log
bash ops/chatd.sh status
```

At most two runs talk to the engine at once (`QSE_CHAT_MAX_RUNS`); the others wait as queued. A run that waits for a permission prompt gives its slot back until the answer comes. An engine restart does not touch chatd: a run that loses the engine retries for 10 minutes, then ends `interrupted` with its text kept, and Continue resumes it.

25.2 The sandbox

Each chat's tools run in a container of its own, `tandem-chat-<id>`, from the image `tandem-sandbox:1`. Every container runs as uid 1000 with no capabilities, a read-only root and no network of its own, at most 4 GiB, 4 CPUs and 512 processes, and no GPU. Its internet goes through chatd's per-chat proxy, which refuses the LAN and the box itself. The engine comes first: a memory guard starts a sandbox only when the box keeps its floor, stops the least recently used idle one when memory runs short, and stops any sandbox that sat idle for 30 minutes. The chat's files stay.

A deleted chat goes to the trash for 7 days, and its sandbox stops at once. `POST /v1/chat-app/conversations/<id>/restore` brings the chat back, and its sandbox starts again on its next tool

call. When the 7 days end, or on `DELETE /v1/chat-app/conversations/<id>?forever=1`, chatd removes the container and the chat's folders under `workspaces/`, `homes/`, `egress/` and `snapshots/`. At its start chatd also removes every container whose chat is gone or in the trash, and logs each one.

25.3 Permission modes

Each chat has a permission mode, and the Tools settings page holds the default that a new chat gets. A pill in the composer's options row shows the chat's mode and opens a menu of the three, with one line each; YOLO is orange. A change applies from the next tool call. The modes came from a test run on a phone on 2026-10-13: six bash prompts for commands such as `grep` and `cp` took 20 s of tapping, and every one of those commands ran inside the chat's own sandbox.

Table 25.1: The Chat tab's permission modes (0.8.2).

mode	what asks first
Ask	every risky call, as before 0.8.2
Auto (default)	only a call that leaves the sandbox: `web_fetch` to a domain this chat has not fetched yet, an MCP tool its server does not mark `readOnlyHint`, an imported Open WebUI tool whose file imports a network or process module, and `rm` or `rmdir` aimed at `/`, `/workspace` or a home directory. Bash, files in `/workspace` and `render_page` run without a prompt.
YOLO	nothing

Some limits hold in every mode. A deny rule refuses the call, and so does a tool set to "never"; a tool set to "ask" asks. Plan mode offers only the read tools. The sandbox stays as locked down as before.

A call that ran without a prompt still leaves a record. Its tool card carries a small badge that names what allowed it ("auto", "yolo", "always"), and `tool.started` carries `approved: "auto"`, `"yolo"`, `"rule"` for a saved rule, or `"user"` after a prompt. The chat's event log gets a `tool.auto_approved` event with the call's one-line summary, and the run's mode is on `run.started` and on the assistant message. The default and each chat's mode have routes of their own (`GET` and `PUT /v1/chat-app/permissions/mode`, and `/v1/chat-app/conversations/<id>/permission-mode`). A chat from before 0.8.2 has no mode and runs in the default.

25.4 Rules and "Always allow"

A bash rule names a command's first word, or its first two for programs with subcommands (`git push`, `npm install`); `grep tilt` saves `grep`. A chained command (`&&`, `||`, `;`, `|`) runs without a prompt when every part has an allow rule, and is refused when any part hits a deny rule. "Always allow" on such a command saves a rule for every part no rule covers yet, and the prompt names them: "Always allow cd, node, echo, cat". Until 0.8.2 it offered only the first word, which never covered the whole command, so the prompt came back: one chat saved `cd` 11 times for `cd /workspace && node ... > run.log 2>&1; echo "exit:$?"; cat run.log`. Store schema 6

keeps one copy of each duplicate rule. A command with `$(...)` or backticks matches no rule and offers no “Always”. All of this matters in Ask mode; Auto runs bash in the sandbox without asking.

25.5 Known limitations

The agent loop does not yet keep its prompts prefix-stable from step to step (SRV-375), and old tool results and old thinking stay in the context until a compaction (SRV-365). A compaction writes a general summary, not the task’s state (SRV-373). Nothing notices when the model repeats the same failing approach with different calls (SRV-367), long tool output has no paging (SRV-371), and searches run in the main context with no sub-agents (SRV-381). The engine-side limits of 0.8.0 still hold (Section 34.4); the phone layout’s keyboard behaviour is still not verified in Safari on an iPhone.

Chapter 26

Watching several engines

With more than one backend, the dashboard answers three questions for all of them at once: what each request is doing right now, how each engine is doing inside, and how the machine behind it is doing. Each answer has its own source, and each source is optional. All of it is measured or null, never estimated: a figure a backend does not report reads "not reported by this backend".

Table 26.1: The four layers the proxy reads.

layer	where it reads	what it gives	needs
the relay	every answer the proxy relays	per request: tokens from `usage`, cached and reasoning tokens where reported, TTFT and decode rate at the proxy, cost	nothing
L1: the engine's page	vLLM's and SGLang's `/metrics`, every 15 s	per engine: requests running and waiting, KV use, decode rate, prefix-cache hits, TTFT, queue and prefill percentiles, preemptions, speculation	kind `vllm` or `sglang`; SGLang `--enable-metrics`
L2: the plugin	a plugin inside vLLM or SGLang, posting once a second	per request: the engine's own queue, prefill and decode time, cached tokens, preemptions, finish reason	the plugin and two variables
the host collector	the machine behind each backend	memory, CPU, GPUs, disks, network, the process behind each backend's port	nothing here; a collector or exporters on a remote machine

26.1 Every request at once

Performance, Parallel (`#/performance?tab=parallel`) shows every request in flight on every backend, the engine included. It reads `GET /v1/dashboard/parallel` once a second; the server builds the document at most every 0.4 s for all viewers, and a poll with `since` carries only the new samples.

- The aggregate decode rate now, and its last five minutes stacked by backend.
- The requests by state: waiting, prefilling, thinking, tool call, writing.

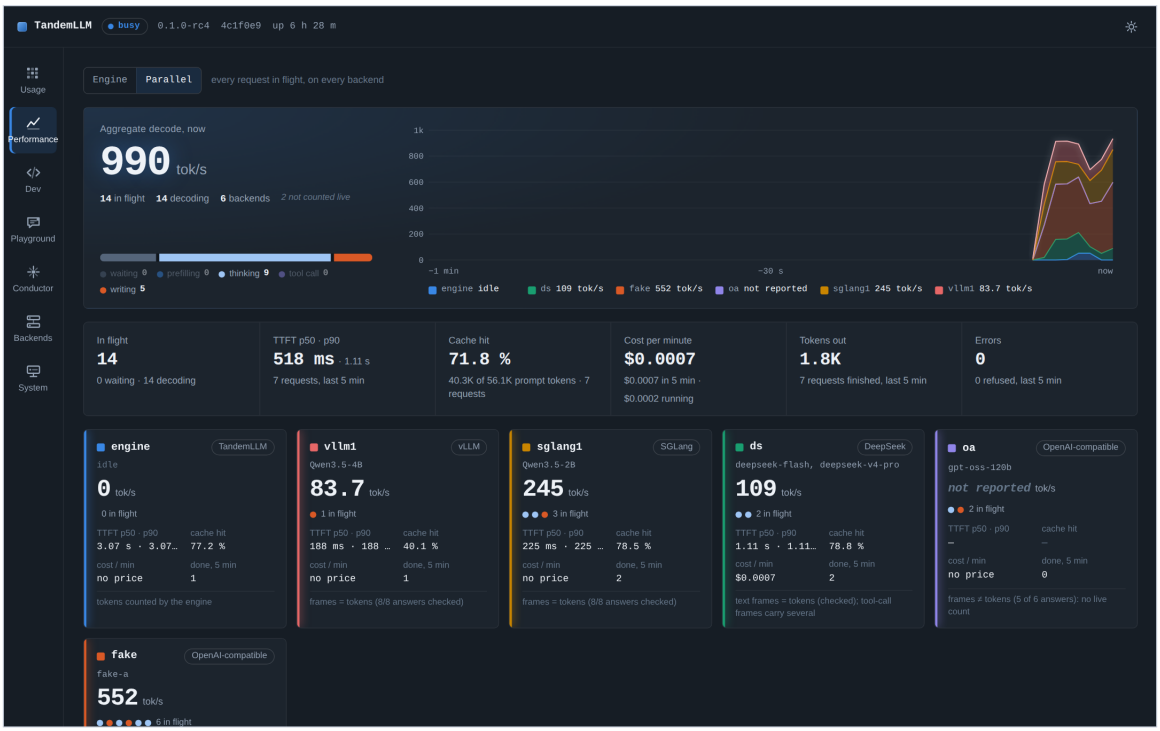


Figure 26.1: The Performance view, Parallel page: every request in flight on six backends, the aggregate decode rate, and one card per backend. Demo data from the dashboard's mock server (`npm run preview:mock`), not a measurement.

- The time to first token at p50 and p90, the cache hit and the cost per minute, over the requests finished in the last five minutes.
- One card per backend.
- One lane per request: backend and model, client, session, state, tokens with thinking apart, decode tok/s now and on average, TTFT, prefill rate, cache, cost so far, elapsed. A lane keeps its row until a few seconds after it ends.

A lane's live token count comes from the best source the backend allows. The engine's own requests use the engine's count. A TandemLLM backend behind a proxy is read from that engine's own board once a second, because the relay tags each request with `X-Tandem-Relay-Id`; the engine streams blocks of several tokens, so its frames are not a count. Any other backend's stream frames count only after they were checked: each finished streamed answer's frames are compared with its `usage.completion_tokens` (within 2 tokens and 1 %), answers with and without tool calls apart, and two passing answers of a kind with no failure among the last 8 make the frames a count. DeepSeek's reasoning and content frames are one token each; its tool-call arguments arrive several tokens a frame, so a DeepSeek lane has no live count while it writes a call. Without a count a lane shows its frames and "not reported", and its tokens arrive with the usage at the end.

`tools/parallel_load.py` drives the page on your own backends, for example `--mix deepseek-flash=10,TandemLLM=2 --duration 90 --budget-usd 0.5 --board`. It reports TTFT, decode rate,

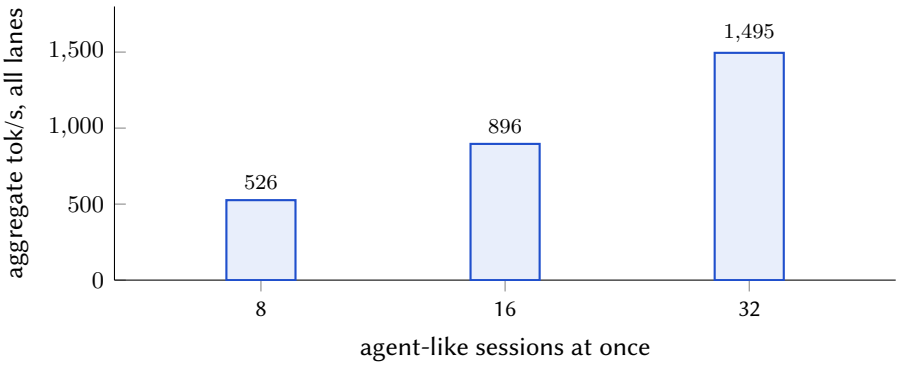


Figure 26.2: A load test of the Parallel view, not an engine benchmark: N agent-like sessions (a system prompt, two tools, up to three streamed turns) through a test proxy in front of a cloud API, the engine and two fake backends, 0 errors in each run. The aggregate is the sum over every backend. measured: `tools/parallel_load.py`, the 0.4.0 series. With 32 streams the page's document took about 1 ms to build and a poll was at most 55 KB.

cache hit, throughput and cost per model, stops starting turns at the budget, and with `--board` puts the page's per-second series beside the frames its own clients received.

26.2 Inside vLLM and SGLang: the engine's page (L1)

For a backend of kind `vllm` or `sglang` the proxy reads the engine's `/metrics` every 15 s through a name table checked against vLLM v0.30.0 and SGLang v0.5.21 (`proxy/engines.py`). Window figures come from the first and the newest reading of the last 5 minutes; with one reading, or no traffic in the window, they are null. A family a release renamed maps to nothing and its tile says "not reported by this backend": a new release can cost a tile, never a failed read. After a failed read the proxy waits 30 s, then 60, then 120 s.

Table 26.2: The engine figures and the families they come from.

figure	vLLM	SGLang
running, waiting	`vllm:num_requests_running`, `_waiting`	`sglang:num_running_reqs`, `num_queue_reqs`
KV in use	`vllm:kv_cache_usage_perc`	`sglang:token_usage`
decode rate	`vllm:generation_tokens_total` over the window	`sglang:generation_tokens_total`, `gen_throughput`
prefix-cache hits	`vllm:prefix_cache_hits_total` over `queries_total`	`sglang:cached_tokens_total` over `prompt_tokens_total`
queue, prefill, decode	`vllm:request_queue_time_seconds` and siblings	`sglang:queue_time_seconds` only
preemptions	`vllm:num_preemptions_total`	`sglang:num_retracted_requests_total`
speculation	drafts, draft tokens, accepted tokens, and acceptance per draft position	`sglang:spec_accept_rate`, `spec_accept_length`

The proxy's own `/metrics` carries the mapped families as `qse_backend_*` with a `backend` label, for a Prometheus of your own; they never enter the engine's own families. Mapping one page of about 500 lines takes about 2 ms on the poller's own thread, and a relay thread never waits for it.

26.3 Inside vLLM and SGLang: the plugins (L2)

The page gives totals. A single request's queue time, its prefill time and whether the scheduler preempted it are not on it. Two small packages in `plugins/` run inside the engine and send those numbers per request:

```
pip install plugins/vllm        # tandemllm-vllm-plugin, into vLLM's own Python
pip install plugins/sglang      # tandemllm-sglang-plugin, into SGLang's own Python
TANDEM_COLLECT_URL=http://proxy-host:8080/v1/collect/<backend id>
TANDEM_COLLECT_TOKEN=<the proxy's collect token>
```

vLLM loads its plugin from the `vllm.stat_logger_plugins` entry point, once per data-parallel engine in its API server process; SGLang loads its plugin from `sglang.srt.plugins` in every process it starts, with two hooks on its tokenizer manager and its scheduler's metrics collector. Neither package has a dependency, and neither reads a prompt, an answer or a token id: each item is counts and times. A plugin keeps the last 10,000 finished requests and 300 one-second scheduler samples in memory and posts what is new once a second, or an empty batch every 10 s when the engine is idle. The collect token is `TANDEM_COLLECT_TOKEN` in the proxy's environment, or one the proxy makes and stores in its config database at the first need; it lets its holder post numbers for a backend that exists, nothing else.

The proxy joins each item to the request's ledger row by an id it sends upstream: vLLM gets `X-Request-Id: px-<hex>` and reports `chatcmpl-px-<hex>`; SGLang gets `rid` in the body. A client that sets its own id keeps it, and the join uses it. The row then gets the engine's own `queue_ms`, `prompt_ms`, `decode_ms`, `decode_tps`, `prefill_tps`, `cached_tokens`, `preemptions` and `engine_finish`, and `l2_join` says `exact`. A request the proxy cannot tag (the Responses API, SGLang with `n` above 1) is matched by time and token counts when exactly one row fits, and says `approximate`. An item waits up to 10 minutes for its row, and a row for its item.

The engine always starts. If the plugin does not fit a release, vLLM's loader logs the import error and starts without it, SGLang skips a renamed hook target, and a failing hook is logged at most once a minute and switches itself off after 100 failures in a row. The backend's card says "L2 plugin" while batches arrive and "L1 only" after 60 s without one.

26.4 The machine behind each backend

The System view lists the machines: this one first, with the server's own process, then the machine behind each backend, down to the process behind the backend's port. For a vLLM on the same machine that is its API server and its engine core, with their RSS, CPU, threads, uptime and GPU memory. A backend's host setting says where to read it:

On a remote machine, run the collector next to the backend and give both sides the same token:

Table 26.3: A backend's host setting.

setting	reads	per process
`auto`	this machine when the backend's URL is loopback or one of its addresses; nothing for a cloud API	yes
`same`	this machine, whatever the URL says	yes
`collector`	`tandem-dash hostcollect` on the backend's machine	yes
`exporter`	a node_exporter page and a DCGM exporter page	no
`off`	nothing	

```
# on the backend's machine
TANDEM_COLLECTOR_TOKEN=$(openssl rand -hex 24) tandem-dash hostcollect --host 0.0.0.0 --
    ↪ port 9108
# on the proxy's machine
TANDEM_COLLECTOR_TOKEN=<the same token>
TANDEM_BACKEND_VLLM1_HOST=collector
TANDEM_BACKEND_VLLM1_HOST_URL=http://gpu-host:9108
```

The collector answers `GET /host` as JSON and `GET /metrics` as Prometheus text (`tandem_host_*`), both behind the bearer token. It refuses to bind beyond loopback without one, because its answer names processes and their command lines. Command lines show the home as ~, and the value after any flag whose name holds `key`, `token`, `secret`, `passw` or `auth` is masked. A remote read runs on its own thread, so a slow or dead host never holds the page.

On a GB10 the CPU and the GPU share one memory pool. The driver reports no memory for the GPU itself, so the view shows "unified" there and `MemAvailable` is the figure to watch. Nothing is sampled while nobody looks: when the System view asks, the collector reads at most every 5 s and scans the processes at most every 15 s. Measured on a DGX Spark against the running engine: one sample with a process scan cost 7.46 ms of CPU, 0.050 % of a core at that cadence; the memory figures equalled `/proc/meminfo` to the byte, and the engine's GPU memory equalled `nvidia-smi`'s within 49,152 bytes of 81.7 GB (measured, the 0.4.0 series).

26.5 Your own Prometheus

The proxy's `/metrics` (open while no token is set; else a bearer metrics or admin token, or a request from the machine itself) carries its own families (`qse_proxy_requests_total`, `qse_proxy_tokens_total`, `qse_proxy_cost_micros_total`, `qse_proxy_ttft_seconds`, `qse_backend_up`, `qse_backend_inflight` and more), the mapped engine families as `qse_backend_*`, and the plugin's age and joins. The collector's `/metrics` carries `tandem_host_*`. Table 30.1 lists every family registered in the code.



Part VI

Reference

The reference. Chapters 27 to 30 are generated from the code: every flag, variable, route, metric and ledger column, with the value the served profile sets. Then troubleshooting by symptom (31), questions and answers (32), the roadmap (33) and the changelog (34). The glossary and the index follow.

Chapter 27

Command-line reference

The tables in this part are generated from the code by `manual/tools/gen_reference.py`, which parses the argument parsers, the settings registry, the metric registrations and the ledger's schema with Python's `ast` module and never imports the project. The manual's build runs it first, so the tables follow the code they were built from. A description is the first sentence of the flag's own help text; where the code gives none, the generator carries a short one. A default is the code's; where the served profile passes another value, the cell says "served:" and that value, as `ops/start.sh` builds it from `ops/serve.env`.

27.1 The engine's server

`python server/app.py [flags]`, or `ops/start.sh`, which builds the flags from `serve.env`. `python server/app.py --help` prints the full help texts.

Table 27.1: Every flag of the engine's server, server/app.py, with the value the served profile passes where it differs

Flag	Default	What it does
`--model`	`none`	the checkpoint directory of the target model.
`--served-model`	`qwen3.8-27b-spark-engine; served: qwen38-spark-engine`	the raw engine's model name; with the conductor on it is a hidden alias of TandemLLM.
`--host`	`127.0.0.1; served: 0.0.0.0`	the address to bind.
`--port`	`8000`	the port to listen on.
`--max-len`	`8192; served: 262144`	context length. The KV buffer is allocated for all of it up front, so this is a memory decision as much as a capability one: 64 KiB a token of engine KV plus 20 KiB a token per drafter arm, 27.9 GB of KV and 53.4 GB of whole ...
`--rep-penalty`	`1`	deterministic repetition penalty on the target's logits, before the argmax (1.0 = off, today's output).
`--presence-penalty`	`0`	flat subtraction from tokens already in the history (0 = off).
`--frequency-penalty`	`0`	subtraction per occurrence, on tokens already in the history (0 = off).
`--temperature`	`0`	server default sampling temperature (0 = greedy, the exact path).

continued

Flag	Default	What it does
`--top-p`	`1`	server default for top_p.
`--top-k`	`0`	server default for top_k (0 is off).
`--no-repeat-ngram`	`0`	forbid the token that would complete an n-gram already seen (HF's no_repeat_ngram_size; 0 = off, must be >= 2).
`--pattern-stop`	`none`	end a generation that repeats one short pattern forever, as MAX:MIN:COUNT (e.g. 64:1:8, a block of 1..64 tokens repeated 8 times ends the generation with finish_reason stop).
`--argloop`	`on`	the tool-argument loop guard (server/argloop.py): on stops a held tool call whose arguments loop and tells the model on its next turn. One of: on, log, off.
`--default-max-tokens`	`8192; served: 32768`	what a request that does not send max_tokens gets.
`--reasoning-format`	`tags`	how the reasoning block reaches the client: inside <think></think> in content (default, and what Open WebUI folds on), as OpenAI-style reasoning_content deltas, or both. One of: tags, reasoning_content, both.
`--drafter`	`mtp`	the drafter set. lenrouter, the served one, holds both DFlash2 widths beside the lookup drafter; none decodes one token a step. One of: mtp, router, merged, dflash2, lenrouter, none.
`--tree`	`off (a switch)`	verify a draft TREE per step instead of a chain, where the drafter builds one; see docs/speculative-decoding.md.
`--regenerate`	`sample (QWEN38_REGENERATE)`	: a regenerate (the same conversation again after it was answered, under a TandemLLM plan; or X-Tandem-Regenerate: 1) samples with a fresh seed; every other request keeps its own sampling (greedy by default). One of: sample, off.
`--regenerate-temperature`	`0.7 (QWEN38_REGEN_TEMPERATURE)`	a regenerate's temperature (QWEN38_REGEN_TEMPERATURE).
`--regenerate-top-p`	`0.95 (QWEN38_REGEN_TOP_P)`	a regenerate's top-p (QWEN38_REGEN_TOP_P).
`--regenerate-top-k`	`20 (QWEN38_REGEN_TOP_K)`	a regenerate's top-k (QWEN38_REGEN_TOP_K).
`--sampled-tree`	`none; served: det`	let a SAMPLED request keep the tree verify. One of: det, mixed.
`--budget`	`16`	nodes per tree, anchor included.
`--df2-temp`	`1`	the block drafter's temperature for its own proposals.
`--corpus`	`none (QWEN38_CORPUS)`	the lookup drafter's corpus: a directory of token ids with a suffix array (tools/build_corpus.py).
`--depth`	`3`	the drafter's depth for the older chain drafters.
`--k`	`0`	verify block size; 0 = the drafter's depth.
`--draft-head`	`none`	an optional separate head file for the drafter.
`--nvfp4`	`none`	the NVFP4 weight files to load, one per target (mlp, gdn, attn); empty means the checkpoint's FP8.
`--price-table`	`none (QSE_PRICE_TABLE)`	FILE[:NAME], the routers' verify/draft/rollback prices for this weight set (engine/prices.py); empty = the NVFP4 constants in the code.

continued

Flag	Default	What it does
`--fp8-head`	`none`	e4m3 lm_head from tools/quant_head.py; halves the head's 2.54 GB in both the verify pass and the block drafter's top-k read.
`--dflash2-blocks`	`1`	chained 8-wide draft blocks; 1 proposes 7 tokens, 2 proposes 14.
`--dflash2-path`	`greedy`	how the block drafter walks its lattice: the greedy path or Viterbi. One of: greedy, viterbi.
`--dflash2-ckpt`	`none`	the narrow (block 8) DFlash2 drafter checkpoint.
`--dflash2-ckpt16`	`none`	the sixteen-wide drafter, for --drafter lenrouter.
`--len-fixed`	`0`	pin --drafter lenrouter to one block width (8 or 16). 0 lets the policy choose, which means --len-latch: one decision a request. 8 and 16 measure the fixed baselines through the same code.
`--len-latch`	`on (--no-len-latch turns it off)`	decide the block width ONCE a request, four wide blocks, up to four narrow probes, then no more switching.
`--learn-cost`	`on (--no-learn-cost turns it off)`	the length router learns its verify and draft costs from the blocks it times, across requests (default). --no-learn-cost keeps the served price table; timings inside a request still size its blocks, so two runs can part at a near tie.
`--len-explore`	`32`	blocks between forced wide probes when nothing suggests one.
`--deep`	`0 (QWEN38_DEEP)`	with --drafter lenrouter --tree: after --deep-after wide blocks in a row commit their whole width, propose the lookup drafter's long exact continuation of this request's text as one chain of up to this many rows. 0 = off.
`--deep-after`	`2 (QWEN38_DEEP_AFTER)`	full wide blocks in a row before a deep chain (2 keeps it off new text).
`--narrow-arm`	`on`	with --drafter lenrouter: build the block-8 drafter arm (on, the served default) or run the block-16 arm alone. One of: on, off.
`--drop-idle`	`off (--drop-idle turns it on); served: on`	release the arm that LOSES the latch for the rest of the request: no tap, no sync, and nothing of it in the state snapshot.
`--relax-tau`	`1`	Breaks the agreement rule: accept a drafted token whose probability is at least this fraction of the argmax's. 1.0, the default, keeps the rule.
`--relax-rank`	`1`	Breaks the agreement rule: accept a drafted token among the target's top r. 1, the default, keeps the rule.
`--think-stall`	`on`	with --conductor shadow or on, run the reasoning-loop arms of the policy's reasoning_loop task (a pattern repeated over 128+ tokens, the history-wide 16-gram repeat, novelty collapse); the monitor decides whether a hit closes the ...
`--no-think-stall`	`sets think_stall off`	never run the reasoning-loop arms (the budget still applies).

continued

Flag	Default	What it does
`--conductor`	`raw; served: on`	the second-level harness (server/conductor/monitor/). raw, the default (off is the same): nothing of it runs and the output is the raw engine's. shadow: every rule runs and journals, nothing is done. on: each task acts in its ... One of: raw, off, shadow, on.
`--conductor-policy`	`none`	the monitor's policy file (default ops/supervisor.yaml), validated at start against ops/supervisor.schema.json.
`--conductor-watch-s`	`30`	with --conductor shadow or on, how often the stall watchdog's watch looks for sessions whose client has gone.
`--think-budget`	`0`	default cap on reasoning tokens per request, 0 = uncapped.
`--answer-reserve`	`auto (QSE_ANSWER_RESERVE)`	tokens of the output limit kept for the answer when a client asks for a thinking budget in any shape (server/thinking.py): its reasoning closes early enough to leave them. auto = min(2048, a quarter of the limit), 0 = none ...
`--reasoning-effort`	`none; served: medium`	default `reasoning_effort` for the chat template. One of: low, medium, xhigh.
`--cache-budget-gb`	`2`	RAM budget for the state cache, in GiB (CACHE_GB).
`--store-entry-gb`	`1`	GiB one state-store snapshot may take (STORE_ENTRY_GB), whatever budget is free: 1 GiB is about 7k tokens.
`--no-session-cache`	`off (a switch)`	do not keep a conversation's state after its turn.
`--no-prefix-cache`	`off (a switch)`	do not checkpoint a prefill at chunk boundaries.
`--resident-gb`	`2.5`	GiB for the resident prefix's anchors (RESIDENT_GB), the recurrent state (146.8 MiB) at chunk boundaries of the prompt the KV buffer holds, so the next turn of a growing conversation resumes in place, KV not copied, at any length ...
`--resident-stash-gb`	`1`	GiB for the rows a short unrelated request between two turns may overwrite, copied aside and back (RESIDENT_STASH_GB; ~104 kB a row: 1 GiB holds a --resident-guest-max guest and its 512-row slack).
`--resident-guest-max`	`8192`	the longest request that is a guest (RESIDENT_GUEST_MAX): opencode's title call is 1-3k tokens; a longer one replaces the resident prompt with its head pinned.
`--resident-tail`	`4`	the newest anchors never evicted (the next turn resumes near the end).
`--resident-pin-gb`	`4`	GiB for the head of the resident prompt a long unrelated request would otherwise cut (RESIDENT_PIN_GB): its KV rows and the anchors in it are copied aside, and a later request that shares more with that head than with the rows in ...
`--resident-park-gb`	`4`	GiB for one other session's rows (RESIDENT_PARK_GB): when a request of another session (x-session-id) replaces the resident prompt, the rows past their shared head are copied aside with its newest anchors, and that session's next ...

continued

Flag	Default	What it does
`--end-anchor`	`on`	the resident prefix's end anchor (END_ANCHOR): at the end of every generation the recurrent state at its last position (147 MiB, one per resident prompt), so the next request that starts with the prompt and every token the engine ... One of: on, off.
`--compaction`	`continue (QSE_COMPACTION)`	a client's compaction request (QSE_COMPACTION, server/compaction.py): continue = served as the continuation of the conversation the engine holds for its session (the prefill reads only the instructions); client = as the client ... One of: continue, client.
`--compaction-prepare`	`on (QSE_COMPACTION_PREPARE)`	write an idle opencode session's summary ahead, so its /compact is answered without the engine (QSE_COMPACTION_PREPARE); a request that waits stops it at the next chunk or block. One of: on, off.
`--compaction-idle-s`	`30 (QSE_COMPACTION_IDLE_S)`	seconds without a request before a summary is prepared.
`--compaction-min-tokens`	`32768 (QSE_COMPACTION_MIN_TOKENS)`	the shortest conversation a summary is prepared for.
`--end-align`	`on`	the end anchor for a copy whose whitespace is not the rows' (END_ALIGN, server/endalign.py): a follow-up whose copy of an earlier generation differs from what the engine wrote in whitespace alone (a tool call's trimmed parameter ... One of: on, off.
`--mem-ceiling-gib`	`70`	GiB the engine may hold between requests, allocated plus the allocator's reserve (MEM_CEILING_GIB): a cache grows only while the engine stays under it less --reserve-slack-gib, and over it the caches are given back, cheapest ...
`--mem-peak-gib`	`77`	GiB the engine may hold during a prefill (MEM_PEAK_GIB): the ceiling plus the workspace of a 262,144-token prompt.
`--reserve-slack-gib`	`2`	GiB of free blocks the allocator may keep between requests (RESERVE_SLACK_GIB); over it they go back to the driver once a request has finished, never inside one.
`--prompt-token-cache`	`4`	millions of tokens of tokenised chat messages kept, so a growing conversation's prompt is tokenised only where it is new (server/tokcache.py). 0 = tokenise every prompt whole.
`--session-trim`	`off`	cut the engine back to the committed tokens before the session snapshot when the last verified block ran past them (end token, stop, budget) and its commit is still pending, so the put is not declined. One of: on, off.
`--store-boundaries`	`1`	chunk boundaries of one prefill the state store keeps: its longest ones under the per-entry cap. 0 = every boundary, where one cold prefill's nested snapshots fill the budget and evict everything else.

continued

Flag	Default	What it does
`--prefill-heartbeat-s`	`5`	a streamed request whose prefill has run this long gets an SSE comment (': prefill done/total') after every chunk, at most once a second, so a client and every proxy between see the stream is alive.
`--live-activity`	`on`	the live activity on /v1/dashboard/live (states, prefill progress, timeline, the last 20 stops) at QSE_LIVE_HZ events a second while busy. off = contract 1.1 with those fields null at one event a second: the kill switch, and the ... One of: on, off.
`--max-prefill-rows`	`8192`	with the prefix cache off, forward a long prompt in chunks of this many rows instead of one call; a 131k-token single forward exhausted the board. 0 = one call.
`--prefix-chunk`	`1024`	tokens between prefill checkpoints, and the forward size of EVERY prefill while the prefix cache is on, the two have to agree or a warm prefill is not the same arithmetic as a cold one.
`--response-cache`	`off (a switch)`	OPT-IN. Answer an identical (prompt, params) request from memory.
`--response-cache-mb`	`256`	the response cache's budget in MiB.
`--response-cache-ttl`	`3600`	seconds a response cache entry is kept.
`--suffix-store`	`~/.qwen38-spark-engine/suffix (QWEN38_SUFFIX_STORE)`	directory for the persistent suffix store of what this engine has read and written, which the lookup drafter reads as a second corpus.
`--suffix-store-readonly`	`off (a switch)`	read the suffix store and never append to it: a fixed benchmark measured against a store of real traffic must not write itself into it.
`--suffix-store-mb`	`192`	cap on the store, in MiB of int32 token ids (192 MiB = 48 M tokens).
`--disk-cache`	`off; served: (a path)`	the disk tier: a directory on the NVMe for prompt-prefix state, prefixes prefilled more than once, a session's first long request, under the caches in RAM, or off (the default).
`--disk-cache-min-tokens`	`4096`	the shortest prefix the disk tier writes an anchor for.
`--conductor-defaults`	`builtin; served: shipped`	shipped, the live documents are stored as version 1 for every document the conductor's directory has no version of; builtin, the minimal built-ins stay version 0. One of: builtin, shipped.
`--session-spill`	`on`	a live conversation on the NVMe, under the disk tier's directory: written whole at a planned stop and read back at the next start, written before another client's request evicts it, read back at its next turn. One of: on, off.
`--session-spill-gb`	`64`	the session spill's own budget on the NVMe (a 256k conversation is about 27 GB as it is in the engine, about 16 GB compressed); the disk tier's free-space floor holds for both; settings.memory.session_spill overrides it live.

continued

Flag	Default	What it does
`--session-spill-codec`	`zstd`	new spill blocks as byte planes compressed with zstd (exact under the agreement rule, 1.71x on a real entry; blocks on the disk are read either way). One of: zstd, off.
`--disk-write-budget-gb`	`200`	the engine's NVMe writes a local day past which optional writes stop (the tier's, and the spill's but a drain and evictions of 32k+ tokens; past 1.5x only the drain); 0 = no budget; settings.memory.disk_writes.budget_gb overrides ...
`--session-spill-min-tokens`	`16384`	the shortest resident conversation an eviction writes to the NVMe (shorter ones are the RAM park's).
`--disk-cache-head-anchors`	`4`	anchors a session's first long request gets, at the grid lines below where its last user message starts (where the next session parts).
`--suffix-store-scope`	`all`	`all` remembers prompts and answers, `outputs` only what the engine wrote. One of: all, outputs.
`--request-timeout`	`900; served: 3600`	wall-clock cap on one generation, checked between blocks; 0 disables.
`--max-queue`	`8`	requests allowed to wait for the engine before new ones get a 503 with Retry-After.
`--queue-timeout`	`120; served: 240`	how long a request waits for the engine before a 429.
`--side-lane`	`on`	a short request, a client's side request or a raw one, thinking off, within the two caps below, is served at the next safe point of the generation that holds the engine (on: between decode blocks and between prefill chunks ... One of: on, decode, off.
`--side-lane-prompt-max`	`2048`	the longest prompt the side lane serves, in tokens.
`--side-lane-new-max`	`64`	the most output tokens the side lane serves.
`--parallel`	`1 (QSE_PARALLEL)`	sequence slots: 1 (the default) is one sequence, as always.
`--side-max-len`	`65536 (QSE_SIDE_MAX_LEN)`	a side slot's window in tokens; slot 0 keeps --max-len (262,144).
`--parallel-floor-gb`	`15 (QSE_PARALLEL_FLOOR_GB)`	admission: MemAvailable less the caches' growth, the requests in flight and this request's worst case must stay at or above this; a side slot is built only if it does with the slot's bytes added.
`--parallel-batch`	`on (QSE_PARALLEL_BATCH)`	on: decoding slots verify their trees in one joint pass (lockstep rounds); off: they take turns, one verify each. One of: on, off.
`--parallel-draft`	`on (QSE_PARALLEL_DRAFT)`	with the joint verify on: the decoding slots' draft calls in one packed pass over the drafter (on), or one after the other (off). One of: on, off.
`--parallel-switch-ms`	`0 (QSE_PARALLEL_SWITCH_MS)`	with side slots: the interpreter's thread switch interval in ms (0 keeps Python's 5 ms); a woken slot waits up to this long for the GIL.
`--parallel-cut`	`share (QSE_PARALLEL_CUT)`	how a joint pass's rows are shared: share (each slot cuts its tree to a fixed share before it submits: 16 at two slots, 10 at three, 8 at four) or joint (the round cuts every slot's whole tree together, best nodes first). One of: share, joint.

continued

Flag	Default	What it does
`--parallel-max-rows`	`32`	rows of a joint pass at most (32: the row-invariant kernels' range).
`--batch-timeout-ms`	`2`	a joint round runs without a missing slot once the GPU turn has been idle this long.
`--turn-quantum-ms`	`50`	the GPU turn goes to a waiting slot this far behind in GPU time.
`--parallel-settle`	`on`	synchronize the device before the turn changes hands (the routers' timings then hold only their own slot's work). One of: on, off.
`--side-precapture`	`off`	capture a side slot's verify graphs at start (on, about a minute a slot) or at their first use (off: the build takes seconds; each width's first lone round on a side slot pays its capture, a fraction of a second). One of: on, off.
`--side-graphs`	`on (QSE_SIDE_GRAPHS)`	a side slot's own verify graphs: on (captured as --side-precapture says; most of a slot's constant bytes) or off (the slot verifies eager when it decodes alone and holds no graph pool; a joint pass is eager either way). One of: on, off.
`--side-slot-const-gb`	`0 (QSE_SIDE_SLOT_CONST_GB)`	a side slot's bytes beyond its window's, in the build's estimate; 0 (the default) follows --side-graphs: 7.5 GiB with it on (6.9 to 7.5 GiB allocated measured for 8,192-token slots, most of it the verify graphs' pools: 121 ...
`--kv-pool`	`off (QSE_KV_POOL)`	on: every slot's KV window (engine attention KV and the drafters' context KV) is reserved as virtual addresses and its pages are committed as a conversation's rows are written and given back when it ends, from one pool for every ... One of: on, off.
`--kv-hard-floor-gb`	`10 (QSE_KV_HARD_FLOOR_GB)`	with --kv-pool on: a commit never takes MemAvailable under this (GiB); the pool gives back caches and idle conversations first, else the request is not run (admission keeps requests out at --parallel-floor-gb).
`--kv-margin-rows`	`256 (QSE_KV_MARGIN_ROWS)`	with --kv-pool on: rows committed past every write (a masked tile's over-read stays on mapped pages).
`--vision`	`on`	load the checkpoint's vision tower (0.92 GB bf16) and accept image_url parts in chat requests; off refuses them with a 400. One of: on, off.
`--image-max-mb`	`20`	largest image a request may send or link, after base64 decoding.
`--image-max-decode-pixels`	`64000000`	largest canvas an image may declare, checked before its pixels are decoded; the processor then resizes to its own limits.
`--image-max-pixels`	`0`	the processor's resize target cap in pixels (0: the checkpoint's preprocessor_config.json, 16,777,216 for Qwen3.8 = 16,384 rows an image).

continued

Flag	Default	What it does
`--image-backend`	`torchvision (QSE_IMAGE_BACKEND)`	the image processor's backend, pinned so the patches do not depend on what is installed (the two differ by up to two 8-bit levels after a resize). torchvision (the default, what every served measurement ran) needs torchvision ... One of: torchvision, pil.
`--max-images`	`16`	images one request may send.
`--max-image-rows`	`65536`	prompt rows all of a request's images may take together (one row per 2x2 block of 16-pixel patches; a 1024x1024 image is 1024 rows).
`--image-https`	`on`	fetch https:// image URLs (off: data: URLs only). One of: on, off.
`--image-https-private`	`off`	let https image URLs reach loopback, private and link-local addresses (off: public hosts only). One of: on, off.
`--image-fetch-timeout`	`15`	seconds for a whole https image download, redirects included.
`--image-cache-mb`	`512`	encoded images kept on the device by content, so a conversation that sends an image again does not encode it again.
`--image-act-gb`	`6`	admission: an image whose encode would need more than this (the tower's byte math, engine/vision.py) is refused with a 400.
`--usage-default`	`on (QSE_USAGE_DEFAULT)`	a streamed request that sends no stream_options gets usage, timings and metrics on its finish chunk (on, the default: Open WebUI asks for include_usage only when a model's Usage capability is ticked), or nothing, as before (off). ... One of: on, off.
`--usage-ledger`	`off; served: (a path)`	the SQLite usage ledger's path, or off (the default).
`--conductor-config`	`off; served: (a path)`	the conductor's config directory (versioned profiles, router, roles, packages and its journal, server/conductor/), or off (the default).
`--backends`	`off`	the proxy's config database (proxy/, docs/proxy.md), or off (the default).
`--host-collector`	`on`	the System view's hosts (proxy/hostcollect.py): this machine's memory, GPU, CPU, disks and network, and the process behind each backend on it. One of: on, off.
`--usage-retention-days`	`400`	days the usage ledger keeps its rows (at least 400).
`--trust-loopback`	`on (--no-trust-loopback turns it off)`	a request from loopback with no X-Forwarded-For / X-Real-IP header (the box's own watchdog, row3, gate, soak) needs no token for /metrics, the full /health and the cache routes once an admin password or metrics token is set.
`--dashboard-login`	`off (QSE_DASHBOARD_LOGIN)`	on, with an admin password (QSE_ADMIN_TOKEN) set: /v1/dashboard/* needs the admin token or a session too (the page has no sign-in screen yet). off, the default: the dashboard's read API answers without a token. One of: on, off.

continued

Flag	Default	What it does
`--dashboard-dir`	`none`	where the dashboard's built files are (default: dashboard/dist in this repository); /dashboard/ serves them, or a placeholder when missing.
`--setup`	`off (a switch)`	setup mode (docs/setup.md, `ops/start.sh --setup`): no model of Qwen's is loaded; the dashboard, /v1/setup/* and Wren, the setup guide's assistant, alone.
`--fake-engine`	`off (a switch)`	the e2e: no weights, no GPU, a deterministic CPU token source (server/fake_engine.py) behind the real handler, auth, ledger, metrics and logs.
`--fake-tps`	`200`	with --fake-engine: tokens a second it writes.
`--fake-prefill-tps`	`0`	with --fake-engine: the prefill rate of a FAKE_SLOW_PREFILL prompt (0 = 2,000 tokens a second).
`--structured-outputs`	`on (--no-structured-outputs turns it off)`	serve response_format json_object / json_schema and structured_outputs (regex, choice, json) as masks over the target's rows, exact under speculation.
`--log-content`	`off (a switch)`	let exception messages that quote a request into the log.
`--log-request-keys`	`off (a switch)`	one [body] line per request with the parameter names and the scalar values of the non-content ones (model, stream, max_tokens, temperature,...); never messages, prompt, tools or stop strings.
`--verbose`	`off (a switch); served: on`	more log lines.

27.2 The dashboard and proxy

`tandem-dash serve` and `python -m proxy` take the same flags; `tandem-dash check`, `secrets` and backends take them too and add their own action (Table 4.1).

Table 27.2: tandem-dash serve and python -m proxy

Flag	Default	What it does
`--config-db`	`~/.tandemllm/config.sqlite3 (TANDEM_CONFIG_DB)`	the config database (created when missing).
`--env-file`	`none`	KEY=VALUE lines read before the start; the process environment wins.
`--host`	`127.0.0.1 (TANDEM_HOST)`	the address to bind.
`--port`	`8080 (TANDEM_PORT)`	the port to listen on.
`--usage-ledger`	`~/.tandemllm/ledger.sqlite3 (TANDEM_USAGE_LEDGER)`	the usage ledger (one row per request, no text); 'off' for none.
`--usage-retention-days`	`400`	days the usage ledger keeps its rows (at least 400).

continued

Flag	Default	What it does
`--dashboard-login`	`off (QSE_DASHBOARD_LOGIN) (TANDEM_DASHBOARD_LOGIN)`	on, with TANDEM_ADMIN_TOKEN (the admin password) set: the dashboard API needs it or a session too; with no password set every route is open. One of: on, off.
`--dashboard-dir`	`none`	default: dashboard/dist.
`--trust-loopback`	`on`	a request from this machine without proxy headers counts as the admin. One of: on, off.
`--no-keychain`	`off (a switch)`	do not ask the OS keychain for the master key.
`--conductor-config`	`~/.tandemllm/conductor (TANDEM_CONDUCTOR)`	the conductor's documents (profiles, router, packages) for the harness in front of the backends; 'off' relays every backend as does.
`--conductor-defaults`	`shipped`	what a new conductor directory starts with. One of: shipped, builtin.
`--verbose`	`off (a switch)`	more log lines.

Table 27.3: tandem-dash launch

Flag	Default	What it does
`ENGINE`		the engine to start: vllm or sglang.
`--model`	`required`	the model id or path the engine serves.
`--port`	`none`	the engine's port.
`--host`	`127.0.0.1`	the engine's bind address.
`--id`	`none`	the backend id (default: the engine's name and port, e.g. vllm8100).
`--label`	`none`	the backend's label in the dashboard.
`--config-db`	`~/.tandemllm/config.sqlite3 (TANDEM_CONFIG_DB)`	the proxy's config database the backend is registered in.
`--collect-url`	`none (TANDEM_COLLECT_BASE)`	the proxy address the plugin posts to; default: this machine's proxy on its port.
`--engine-cmd`	`none`	the engine's command instead of `vllm serve` / `sglang serve`.
`--python`	`none`	the Python of the engine's environment, for the plugin check (default: the one behind the engine's command).
`--no-register`	`off (a switch)`	do not create or update the backend.
`--dry-run`	`off (a switch)`	print, start nothing.

Table 27.4: tandem-dash hostcollect

Flag	Default	What it does
`--host`	`127.0.0.1`	the address to bind; beyond loopback it needs a token.
`--port`	`9108`	the collector's port.
`--token-file`	`none`	a file whose first line is the bearer token; else TANDEM_COLLECTOR_TOKEN.
`--watch`	`none`	ports to report always, as id:port,id:port (a request may add more).

continued

Flag	Default	What it does
`--path`	`[]`	a path whose filesystem to report (repeatable).

27.3 The service scripts

Table 27.5: The scripts in `ops/` and the installed control script.

command	what it does
`bin/tandem start|stop|restart|status|smoke|logs`	the installed engine's control script (Chapter 3)
`ops/start.sh`	starts the engine from `serve.env`, waits for `/health`, drops the weights' page cache, loads the roles
`ops/stop.sh [seconds]`	SIGTERM, the drain, then a kill only after the wait
`ops/watchdog.sh`	one health check, for cron every minute
`ops/hold.sh <minutes> <command>`	a command with the board to itself, then the service back
`ops/gate.sh <label> [--profile F]`	the release gate
`ops/make-secrets.sh [--admin] [--rotate]`	the metrics token; `--admin` adds the admin password
`ops/roles.sh load|up|down|settle`	roles beside the engine, through the residency planner
`. ops/engines.sh; engine_pids`	every engine process on the board

27.4 Measurement and maintenance tools

The `tools/` directory holds about 180 scripts: kernels, quantisers, probes, checks and benches. The ones this manual refers to are listed here; each prints its own `--help`.

Table 27.6: The tools this manual refers to.

tool	what it does
`tools/row3.py`	the benchmark row, N runs through one server, with `--compare`
`tools/block_ab.py`	the in-process A/B for changes that keep the bits
`tools/forced_bench.py`	the teacher-forced bench
`tools/verify_spec.py`	the agreement gate with its near-tie rule
`tools/flagoff_identity.py`	the identity gate with the new flags off
`tools/api_text_check.py`	13 plain requests against the current release
`tools/client_smoke.py`	the request shapes real clients send
`tools/verify_curve.py`	the verify prices StairCut cuts on
`tools/quant_nvfp4.py`, `quant_head.py`	the quantisers
`tools/quality_gate.py`	the quality gate
`tools/calib_corpus.py`, `build_corpus.py`	the calibration text and the lookup corpus
`tools/mem_audit.py`	the memory a configuration holds
`tools/disk_cache_live.py`, `disk_restore_check.py`	the disk tier's live and bit-exact checks
`tools/session_spill_check.py`	the session spill's bit-exact check
`tools/lane_check.py`	the side lane on the real engine
`tools/tandem_eval.py`	the conductor's evaluation suites
`tools/conductor_policy.py`	the monitor's policy schema and its check
`tools/shadow_ab.py`, `shadow_measure.py`	the shadow reader's cost and read rate
`tools/voice_replay.py`	a recorded session's lines through the voice
`tools/proxy_bench.py`, `proxy_gate.py`	the proxy's overhead and its decode check
`tools/parallel_load.py`	agent-like sessions over several backends
`tools/package_check.sh`	the wheel builds and installs without torch
`manual/tools/gen_reference.py`	this part's tables

Chapter 28

Environment and configuration reference

TandemLLM reads four families of settings. Engine switches are `QWEN38_*` variables, read through one registry. Server settings are command-line flags (Chapter 27); `ops/start.sh` builds them from `ops/serve.env`, and a few `QSE_*` variables set flags' defaults or serve the scripts. The proxy, the collector and the plugins read `TANDEM_*` variables. And the conductor's behaviour lives in its versioned documents, which the dashboard edits.

28.1 The served profile

`ops/serve.env` is the served configuration. On an installed engine it is `~/TandemLLM/run/ops/serve.env`, rewritten by every run of the installer, and `local.env` beside it is read last for your own values.

Table 28.1: The keys ops/serve.env sets, in file order (paths not shown)

Key	Value	Kind
HOME_DIR	(a path)	read by ops/start.sh
REPO	(a path)	read by ops/start.sh
PY	(a path)	read by ops/start.sh
PYTHONPATH	(a path)	read by ops/start.sh
HOST	0.0.0.0	read by ops/start.sh
PORT	8000	read by ops/start.sh
SERVED_MODEL	qwen38-spark-engine	read by ops/start.sh
NV	(a path)	read by ops/start.sh
HEAD	(a path)	read by ops/start.sh
CKPT8	(a path)	read by ops/start.sh
CKPT16	(a path)	read by ops/start.sh
CORPUS	(a path)	read by ops/start.sh
MAX_LEN	262144	read by ops/start.sh
DEFAULT_MAX_TOKENS	32768	read by ops/start.sh
PATTERN_STOP	(empty)	read by ops/start.sh
CONDUCTOR	on	read by ops/start.sh
REASONING_FORMAT	tags	read by ops/start.sh
REASONING_EFFORT	medium	read by ops/start.sh
LEN_FIXED	0	read by ops/start.sh
LEN_LATCH	1	read by ops/start.sh

continued

Key	Value	Kind
BUDGET	16	read by ops/start.sh
MEM_CEILING_GIB	70	read by ops/start.sh
MEM_PEAK_GIB	77	read by ops/start.sh
RESERVE_SLACK_GIB	2	read by ops/start.sh
RESIDENT_GB	2.5	read by ops/start.sh
RESIDENT_STASH_GB	1.0	read by ops/start.sh
RESIDENT_GUEST_MAX	8192	read by ops/start.sh
RESIDENT_PIN_GB	4.0	read by ops/start.sh
RESIDENT_PARK_GB	4	read by ops/start.sh
CACHE_GB	2	read by ops/start.sh
STORE_ENTRY_GB	1	read by ops/start.sh
NARROW_ARM	1	read by ops/start.sh
END_ANCHOR	on	read by ops/start.sh
END_ALIGN	on	read by ops/start.sh
DROP_PAGE_CACHE	1	read by ops/start.sh
REQUEST_TIMEOUT	3600	read by ops/start.sh
MAX_QUEUE	8	read by ops/start.sh
QUEUE_TIMEOUT	240	read by ops/start.sh
FUSE_PROJ	0	read by ops/start.sh
QWEN38_DF2_TREE_MODE	nodes	engine switch
QWEN38_TREE_ALIAS_STATE	0	engine switch
DROP_IDLE	1	read by ops/start.sh
QWEN38_DRAFT_TEMP	0.3	engine switch
SAMPLED_TREE	det	read by ops/start.sh
QWEN38_FUSED_GDNPREFILL	1	engine switch
QWEN38_DECODE_ATTN	1	engine switch
QWEN38_DRAFT_NVFP4	1	engine switch
QWEN38_NVFP4_PREFILL_V2	1	engine switch
QWEN38_NVFP4_PREFILL_V2_UNTIL	1025	engine switch
QWEN38_NVFP4_SKINNY	1	engine switch
QWEN38_SKINNY_TILES	(a path)	engine switch
QWEN38_FUSED_COMMIT	1	engine switch
QWEN38_FUSED_GDNVERIFY	1	engine switch
QWEN38_DRAFT_FC_NVFP4	1	engine switch
QWEN38_FUSED_ADDNORM	1	engine switch
QWEN38_TREE_HOST_DEPTH	1	engine switch
QWEN38_HOST_WALK	1	engine switch
QWEN38_VERIFY_GRAPH	1	engine switch
QWEN38_COMMIT_IN_VERIFY	1	engine switch
QWEN38_DRAFT_GRAPH	1	engine switch
QWEN38_GDNV_ONE_WARP	1	engine switch
QWEN38_TREE_NODES_NARROW	16	engine switch
QWEN38_TREE_MS	8:77.9,16:78.9,24:87.0,32:97.0	engine switch
QWEN38_VERIFY_ROWS	32	engine switch
QWEN38_SKINNY_TILES_WIDE	(a path)	engine switch

continued

Key	Value	Kind
QWEN38_TREE_NODES	24	engine switch
QWEN38_TREE_WIDE_AFTER	32	engine switch
QWEN38_DEEP	32	engine switch
QWEN38_DEEP_AFTER	2	engine switch
QWEN38_LEN_SWITCH	1	engine switch
QWEN38_LEN_MODE	wide	engine switch
QWEN38_STAIR_TABLES	(a path)	engine switch
QWEN38_STAIR_RHO	(a path)	engine switch
QWEN38_STAIR_SKIP	1	engine switch
QWEN38_SKINNY_SRUN	1	engine switch
QSE_USAGE_LEDGER	(a path)	server setting
QSE_CONDUCTOR_CONFIG	(a path)	server setting
QSE_BACKENDS	(empty)	server setting
QSE_PARALLEL	1	server setting
QSE_SIDE_MAX_LEN	65536	server setting
QSE_PARALLEL_FLOOR_GB	15	server setting
QWEN38_GDNV_CONV_BLOCK	32	engine switch
QWEN38_GDNV_CONV_WARPS	1	engine switch
QWEN38_GDNV_WY	1	engine switch
QWEN38_GDNV_WY_FUSED	1	engine switch
QWEN38_GDNV_WY_MAXT	16	engine switch
QWEN38_GDNV_WY_CHAIN_MAXT	16	engine switch
QWEN38_GDNV_WY_FUSED_MAXT	16	engine switch
LIVE_ACTIVITY	on	read by ops/start.sh
QSE_DASHBOARD_LOGIN	off	server setting
ROLES_AUTOLOAD	(empty)	read by ops/start.sh
QWEN38_DECIDER_MODELS	(empty)	engine switch
QWEN38_DECIDER_DIR	(a path)	engine switch
QWEN38_NARRATOR_MODEL	(empty)	engine switch
QWEN38_NARRATOR_DIR	(empty)	engine switch
QWEN38_SHADOW_MODEL	(empty)	engine switch
QWEN38_SHADOW_DIR	(empty)	engine switch
QWEN38_SHADOW_GB	4	engine switch
QWEN38_SHADOW_GRID	512	engine switch
QWEN38_SHADOW_DISK	auto	engine switch
QWEN38_SHADOW_DISK_GB	20	engine switch
QWEN38_PROBE_SNAPSHOTS	off	engine switch
QWEN38_PROBE_DIR	(a path)	engine switch
QSE_DISK_CACHE	(a path)	server setting
SESSION_SPILL	on	read by ops/start.sh
SESSION_SPILL_GB	64	read by ops/start.sh
SESSION_SPILL_CODEC	zstd	read by ops/start.sh
DISK_WRITE_BUDGET_GB	200	read by ops/start.sh
VISION	on	read by ops/start.sh
RESPONSE_CACHE	off	read by ops/start.sh
QSE_STATUS_PORT	(empty)	server setting

continued

Key	Value	Kind
QSE_PROFILE_NAME	(a path)	server setting
QSE_SETUP_ENV_FILE	(a path)	server setting

28.2 Engine switches

engine/settings.py lists every engine switch with its default and the module that reads it; tests/test_settings.py fails when a module reads one another way. The table shows each switch, its code default, and the value the served profile sets when it sets one.

Table 28.2: Every QWEN38_* switch in engine/settings.py, with the value ops/serve.env sets

Variable	Code default	Served value • read in
QWEN38_ATTN_STAGES3	1	tools/attn_kernels.py
QWEN38_BLOCKING_SYNC	0	server/app.py
QWEN38_CLIENT_CWD	note	server/conductor/router.py
QWEN38_CLIENT_REASONING	*=reasoning_content, hermes=tags	server/compat.py
QWEN38_COMMIT_IN_VERIFY	0	1 • engine/model.py
QWEN38_CORPUS	none	engine/drafters/ngram.py
QWEN38_DECIDER_COLLATE	call	server/decider_api.py
QWEN38_DECIDER_DEVICE	auto	server/decider_api.py
QWEN38_DECIDER_DIR	~/.qwen38-spark-engine/deciders/laya	(a path) • server/decider_api.py
QWEN38_DECIDER_MAX_BATCH	64	server/decider_api.py
QWEN38_DECIDER_MAX_QUEUE	256	server/decider_api.py
QWEN38_DECIDER_MODELS	none	server/decider_api.py
QWEN38_DECIDER_WAIT_MS	2	server/decider_api.py
QWEN38_DECODE_ATTN	0	1 • engine/model.py
QWEN38_DEEP	0	32 • server/app.py
QWEN38_DEEP_AFTER	2	2 • server/app.py
QWEN38_DF2_TEMP	1.0	engine/drafters/dflash2.py
QWEN38_DF2_TREE_MODE	paths	nodes • engine/drafters/dflash2.py
QWEN38_DFLASH2	none	engine/drafters/dflash2.py
QWEN38_DRAFT_EXL3	none	engine/drafters/dflash2.py
QWEN38_DRAFT_FC_NVFP4	0	1 • engine/drafters/dflash2.py
QWEN38_DRAFT_GRAPH	0	1 • engine/drafters/dflash2.py
QWEN38_DRAFT_HEAD	none	engine/drafters/dflash2.py
QWEN38_DRAFT_HEAD_EXL3	none	engine/drafters/dflash2.py
QWEN38_DRAFT_HEAD_NVFP4	0	engine/drafters/dflash2.py
QWEN38_DRAFT_NVFP4	0	1 • engine/drafters/dflash2.py
QWEN38_DRAFT_TEMP	1.0	0.3 • engine/drafters/dflash2.py
QWEN38_DSPARK	none	engine/drafters/dspark.py
QWEN38_DSPARK_NO_YARN	none	engine/drafters/dspark.py

continued

Variable	Code default	Served value • read in
QWEN38_END_ALIGN_MOVED	on	server/endalign.py
QWEN38_END_ALIGN_REASONING	on	server/endalign.py
QWEN38_EXL3	none	engine/loader.py
QWEN38_EXL3_CHUNK_UNTIL	128	tools/exl3_linear.py
QWEN38_EXL3_FUSE	1	engine/loader.py
QWEN38_EXL3_MLP_BRIDGE	1	tools/exl3_linear.py
QWEN38_EXL3_ROWGRID	1	tools/exl3_linear.py
QWEN38_EXL3_SILU_FUSED	0	tools/exl3_linear.py
QWEN38_EXL3_STAGES	3	tools/exl3_linear.py
QWEN38_EXL3_TILES	none	tools/exl3_linear.py
QWEN38_FN_DENSE	0	engine/models/flashnext/engine.py
QWEN38_FN_DRAFT_HOT	32768	engine/models/flashnext/serving.py
QWEN38_FN_DROP_SHARD_CACHE	1	engine/models/flashnext/load.py
QWEN38_FN_FOLD_COMMIT	1	engine/models/flashnext/engine.py
QWEN38_FN_GEMV_SPLIT	1	tools/fn_kernels.py
QWEN38_FN_LOOKUP	0	engine/models/flashnext/serving.py
QWEN38_FN_MIX	int8	engine/models/flashnext/pack.py
QWEN38_FN_MOE_ST	3	tools/fn_kernels.py
QWEN38_FN_MOE_WARPS	4	tools/fn_kernels.py
QWEN38_FN_PDL	1	tools/fn_kernels.py
QWEN38_FN_PREFILL	kernels	engine/models/flashnext/engine.py
QWEN38_FN_PREFILL_CHUNK	4096	engine/models/flashnext/engine.py
QWEN38_FN_PREFILL_MM	fp32	tools/fn_prefill_kernels.py
QWEN38_FN_TABLE	warm	engine/models/flashnext/load.py
QWEN38_FN_TABLE_CACHE_GB	2	engine/models/flashnext/load.py
QWEN38_FN_WARM_TABLE	1	engine/models/flashnext/load.py
QWEN38_FP8_HEAD	none	engine/loader.py
QWEN38_FP8_TILES	none	tools/fp8_linear.py
QWEN38_FUSED_ADDNORM	0	1 • engine/model.py
QWEN38_FUSED_ATTN	0	engine/model.py
QWEN38_FUSED_ATTN_PREP	0	engine/model.py
QWEN38_FUSED_COMMIT	0	1 • engine/model.py
QWEN38_FUSED_GDN	1	engine/model.py
QWEN38_FUSED_GDNBLOCK	1	engine/model.py
QWEN38_FUSED_GDNPRE	1	engine/model.py
QWEN38_FUSED_GDNPREFILL	0	1 • engine/model.py
QWEN38_FUSED_GDNTREE	1	engine/model.py
QWEN38_FUSED_GDNVERIFY	0	1 • engine/model.py
QWEN38_FUSED_HEAD	1	engine/model.py
QWEN38_FUSED_NORM	1	engine/model.py
QWEN38_FUSE_PROJ	0	tools/nvfp4_linear_v2.py
QWEN38_GDNV_BV	16	tools/gdn_verify_kernels.py
QWEN38_GDNV_CONV_BLOCK	256	32 • tools/gdn_verify_kernels.py
QWEN38_GDNV_CONV_WARPS	4	1 • tools/gdn_verify_kernels.py
QWEN38_GDNV_ONE_WARP	0	1 • tools/gdn_verify_kernels.py
QWEN38_GDNV_TREE_PF	0	tools/gdn_verify_kernels.py

continued

Variable	Code default	Served value • read in
QWEN38_GDNV_WARPS	4	tools/gdn_verify_kernels.py
QWEN38_GDNV_WY	0	1 • tools/gdn_verify_kernels.py
QWEN38_GDNV_WY_CHAIN_MAXT	32	16 • tools/gdn_verify_kernels.py
QWEN38_GDNV_WY_FUSED	0	1 • tools/gdn_verify_kernels.py
QWEN38_GDNV_WY_FUSED_MAXT	32	16 • tools/gdn_verify_kernels.py
QWEN38_GDNV_WY_KC	0	tools/gdn_verify_kernels.py
QWEN38_GDNV_WY_MAXT	32	16 • tools/gdn_verify_kernels.py
QWEN38_GDN_AB	0	engine/model.py
QWEN38_GDN_CHUNK	64	engine/model.py
QWEN38_GDN_MM	fp32	engine/gdn.py
QWEN38_GDN_MM_FROM	64	engine/gdn.py
QWEN38_GDN_PREFILL_BV	32	tools/gdn_prefill_kernels.py
QWEN38_GDN_PREFILL_NOREP	0	engine/model.py
QWEN38_GDN_PREFILL_PREC	bf16x3	tools/gdn_prefill_kernels.py
QWEN38_GDN_PREFILL_PREC_A	tf32	tools/gdn_prefill_kernels.py
QWEN38_GDN_PREFILL_PREC_SCAN	none	tools/gdn_prefill_kernels.py
QWEN38_GDN_PREFILL_SERIES	0	tools/gdn_prefill_kernels.py
QWEN38_GDN_PREFILL_STAGES	1	tools/gdn_prefill_kernels.py
QWEN38_GDN_PREFILL_WARPS_INTRA	8	tools/gdn_prefill_kernels.py
QWEN38_GDN_PREFILL_WARPS_SCAN	8	tools/gdn_prefill_kernels.py
QWEN38_GQA_FROM	64	engine/model.py
QWEN38_GRAPH_MAX_CTX	32768	engine/verify_graph.py
QWEN38_HEAD_GEMM	none	tools/head_gemv.py
QWEN38_HOST_ASYNC	0	engine/model.py
QWEN38_HOST_WALK	0	1 • engine/drafters/dflash2.py
QWEN38_KV_FP8	0	engine/model.py
QWEN38_LATCH_PRICE	0	engine/lenrouter.py
QWEN38_LAUNCH_FIRST	0	server/app.py
QWEN38_LEN_MODE	none	wide • engine/lenrouter.py
QWEN38_LEN_SWITCH	0	1 • engine/lenrouter.py
QWEN38_LOWER_RIGHT_FROM	64	engine/model.py
QWEN38_MEMORY_PROXIES	none	server/app.py
QWEN38_MEMORY_WRITER_TEXT	none	server/app.py
QWEN38_MODEL	none	engine/config.py
QWEN38_MODEL_NAME	none	server/conductor/schema.py
QWEN38_NARRATOR_DEVICE	auto	server/conductor/narrator.py
QWEN38_NARRATOR_DIR	none	server/conductor/narrator.py
QWEN38_NARRATOR_MODEL	none	server/conductor/narrator.py
QWEN38_NOTE_ANCHOR	on	server/conductor/router.py
QWEN38_NOTE_DEDUP	parts	server/conductor/router.py
QWEN38_NVFP4	none	engine/loader.py
QWEN38_NVFP4_BLOCK_TILES	0	tools/nvfp4_linear.py
QWEN38_NVFP4_DEQUANT_FROM	512	tools/nvfp4_linear.py
QWEN38_NVFP4_PREFILL_V2	0	1 • tools/nvfp4_linear.py
QWEN38_NVFP4_PREFILL_V2_UNTIL	1024	1025 • tools/nvfp4_linear.py

continued

Variable	Code default	Served value • read in
QWEN38_NVFP4_SKINNY	0	1 • tools/nvfp4_skinny.py
QWEN38_NVFP4_SPLITK	0	tools/nvfp4_linear.py
QWEN38_NVFP4_V2	1	tools/nvfp4_linear_v2.py
QWEN38_NVFP4_V2_BN	0	tools/nvfp4_linear_v2.py
QWEN38_NVFP4_V2_DOTS	1	tools/nvfp4_linear_v2.py
QWEN38_NVFP4_V2_MAX	32	tools/nvfp4_linear_v2.py
QWEN38_NVFP4_V2_MIN	1	tools/nvfp4_linear_v2.py
QWEN38_NVFP4_V2_PREFETCH	0	tools/nvfp4_linear_v2.py
QWEN38_NVFP4_V2_W	0	tools/nvfp4_linear_v2.py
QWEN38_PREFILL_CAUSAL	1	engine/model.py
QWEN38_PREFILL_WINDOW	8192	engine/cache.py
QWEN38_PREFILL_WINDOW_FREE	32768	engine/cache.py
QWEN38_PROBE_DAY_MB	100	server/probe.py
QWEN38_PROBE_DIR	~/.qwen38-spark-engine/probe	(a path) • server/probe.py
QWEN38_PROBE_EVERY	1024	server/probe.py
QWEN38_PROBE_SNAPSHOTS	off	off • server/probe.py
QWEN38_RANKK	1	engine/model.py
QWEN38_REGENERATE	sample	server/app.py
QWEN38_REGEN_TEMPERATURE	0.7	server/app.py
QWEN38_REGEN_TOP_K	20	server/app.py
QWEN38_REGEN_TOP_P	0.95	server/app.py
QWEN38_REREAD_SAY_TOKENS	8192	server/app.py
QWEN38_SAMPLED_CUT	0	engine/router.py
QWEN38_SAMPLE_BATCH	1	engine/sample.py
QWEN38_SAMPLE_NUCLEUS	1	engine/sample.py
QWEN38_SAMPLE_TOPP_K	1024	engine/sample.py
QWEN38_SCALE_ON	weight	tools/fp8_linear.py
QWEN38_SESSION_GENERATIONS	on	server/conductor/router.py
QWEN38_SHADOW_DIR	none	server/conductor/shadow.py
QWEN38_SHADOW_DISK	none	auto • server/conductor/shadow.py
QWEN38_SHADOW_DISK_GB	20	20 • server/conductor/shadow.py
QWEN38_SHADOW_GB	4	4 • server/conductor/shadow.py
QWEN38_SHADOW_GRID	512	512 • server/conductor/shadow.py
QWEN38_SHADOW_MODEL	none	server/conductor/shadow.py
QWEN38_SHADOW_POOL_SLACK_GIB	2	server/conductor/shadow.py
QWEN38_SIDE_CARRY	on	server/app.py
QWEN38_SIDE_CONTINUE	view	server/app.py
QWEN38_SIDE_CONV	on	server/app.py
QWEN38_SIDE_DISK	read	server/app.py
QWEN38_SIDE_GUESTS	on	server/app.py
QWEN38_SIDE_HOME_WAIT_S	10	server/app.py
QWEN38_SIDE_MIN_ROOM	16384	server/app.py
QWEN38_SIDE_NEW_FIRST	on	server/app.py
QWEN38_SIDE_NEW_WAIT_S	20	server/app.py
QWEN38_SIDE_RESIDENT_GB	0.5	server/app.py

continued

Variable	Code default	Served value • read in
QWEN38_SIDE_SLOT0_WAIT_S	3	server/app.py
QWEN38_SIDE_STASH_GB	1.0	server/app.py
QWEN38_SKINNY_LDW	0	tools/nvfp4_skinny.py
QWEN38_SKINNY_PDL	0	tools/nvfp4_skinny.py
QWEN38_SKINNY_SRUN	0	1 • tools/nvfp4_skinny.py
QWEN38_SKINNY_SSTUB	0	tools/nvfp4_skinny.py
QWEN38_SKINNY_TILES	none	(a path) • tools/nvfp4_skinny.py
QWEN38_SKINNY_TILES_B	none	tools/nvfp4_skinny.py
QWEN38_SKINNY_TILES_C	none	tools/nvfp4_skinny.py
QWEN38_SKINNY_TILES_WIDE	none	(a path) • tools/nvfp4_skinny.py
QWEN38_SKINNY_TILES_WIDE_B	none	tools/nvfp4_skinny.py
QWEN38_SKINNY_XSTUB	0	tools/nvfp4_skinny.py
QWEN38_SPILL_PARK	on	engine/cache.py
QWEN38_STAIR_MAX_NODES	none	engine/lenrouter.py
QWEN38_STAIR_RHO	none	(a path) • engine/lenrouter.py
QWEN38_STAIR_SKIP	0	1 • engine/lenrouter.py
QWEN38_STAIR_TABLES	none	(a path) • engine/lenrouter.py
QWEN38_STATUS_CLIENTS	open-webui	server/conductor/states.py
QWEN38_SUFFIX_STORE	~/.qwen38-spark-engine/suffix	server/app.py
QWEN38_TANDEM_SCRATCH	none	server/conductor/packages.py
QWEN38_TANDEM_URL	none	server/conductor/packages.py
QWEN38_TOOL_FIT	on	server/conductor/router.py
QWEN38_TREE_ALIAS_STATE	0	0 • engine/model.py
QWEN38_TREE_CHAIN_DELEGATE	1	engine/model.py
QWEN38_TREE_HOST_DEPTH	0	1 • engine/model.py
QWEN38_TREE_MS	none	8:77.9,16:78.9,24:87.0,32:97.0 • engine/router.py
QWEN38_TREE_NODES	0	24 • engine/router.py
QWEN38_TREE_NODES_NARROW	0	16 • engine/router.py
QWEN38_TREE_WIDE_AFTER	0	32 • engine/lenrouter.py
QWEN38_TWO_STREAM	0	engine/model.py
QWEN38_UT_INVERSE	1	engine/gdn.py
QWEN38_VERIFY_GRAPH	0	1 • engine/model.py
QWEN38_VERIFY_ROWS	16	32 • engine/model.py
QWEN38_VERIFY_TILES	1	tools/nvfp4_verify_tiles.py
QWEN38_WARM_PREFILL_TOKENS	2600	server/app.py
QWEN38_WY_BLOCK	8	tools/gdn_wy_kernels.py
QWEN38_WY_BV	32	tools/gdn_wy_kernels.py
QWEN38_WY_DBG	0	tools/gdn_wy_kernels.py
QWEN38_WY_PREC	ieee	tools/gdn_wy_kernels.py
QWEN38_WY_WARPS	4	tools/gdn_wy_kernels.py
QWEN38_WY_WARPS_PREP	4	tools/gdn_wy_kernels.py

28.3 Server variables

Table 28.3: QSE_* variables: the server's settings from the environment or serve.env

Variable	Read in
QSE_ADMIN_TOKEN	ops/make-secrets.sh, ops/serve.env, ops/start.sh and 7 more
QSE_ANSWER_RESERVE	server/app.py
QSE_ASSETS	ops/profiles/_flashnext.env, ops/use-profile.sh, server/setup/community.py
QSE_BACKENDS	ops/serve.env, ops/start.sh, server/app.py and 1 more
QSE_BARRIER_IDLE_OWNER	ops/serve.env, server/slots.py
QSE_CHAT_DATA	ops/chatd.sh, ops/sandbox/uninstall.sh, server/chat_forward.py
QSE_CHAT_FORWARD_KEY	ops/chatd.sh, ops/make-secrets.sh, server/chat_forward.py
QSE_CHAT_URL	server/chat_forward.py
QSE_COMMUNITY_DIR	server/setup/community.py
QSE_COMMUNITY_ENV_DIR	ops/serve.env, ops/use-profile.sh
QSE_COMMUNITY_URL	ops/serve.env, server/setup/community.py
QSE_COMPACTION	server/app.py
QSE_COMPACTION_IDLE_S	server/app.py
QSE_COMPACTION_MIN_TOKENS	server/app.py
QSE_COMPACTION_PREPARE	server/app.py
QSE_CONDUCTOR_CONFIG	ops/serve.env, ops/start.sh, ops/use-profile.sh and 3 more
QSE_CONDUCTOR_ROOT	ops/roles.sh, server/conductor/residency.py
QSE_DASHBOARD_LOGIN	ops/serve.env, ops/start.sh, proxy/__main__.py and 2 more
QSE_DISK_ADOPT_FROM	ops/serve.env, server/diskcache.py
QSE_DISK_CACHE	ops/profiles/_exl3.env, ops/profiles/_flashnext.env, ops/profiles/nvfp4.env and 5 more
QSE_ENGINE_FLAGS	ops/profiles/_flashnext.env, ops/serve.env, ops/start.sh and 1 more
QSE_ENGINE_PORT	ops/engine_status.py
QSE_EXL3_BPW	ops/profiles/_exl3.env, ops/profiles/exl3-2.5.env, ops/profiles/exl3-3.0.env and 4 more
QSE_FN_BPW	ops/profiles/_flashnext.env, ops/profiles/flashnext-2.05.env, ops/profiles/flashnext-3.05.env and 3 more
QSE_FN_TABLE	ops/profiles/_flashnext.env, ops/profiles/flashnext-2.05.env, ops/profiles/flashnext-3.05.env and 3 more
QSE_FN_VISION_FILE	ops/profiles/_flashnext.env, ops/profiles/flashnext-4.05.env
QSE_IMAGE_BACKEND	server/app.py
QSE_KV_HARD_FLOOR_GB	ops/profiles/_flashnext.env, ops/serve.env, server/app.py
QSE_KV_MARGIN_ROWS	server/app.py
QSE_KV_POOL	engine/kvpool.py, ops/profiles/_flashnext.env, ops/serve.env and 2 more
QSE_KV_POOL_MIN_ROWS	engine/kvpool.py
QSE_LIVE_DECODE_WARMUP_TOKENS	ops/serve.env, server/live.py
QSE_LIVE_HZ	ops/serve.env, ops/start.sh, server/app.py and 1 more
QSE_LIVE_MOSTLY_CACHED_PCT	ops/serve.env, server/live.py
QSE_LIVE_MOSTLY_CACHED_TOKENS	ops/serve.env, server/live.py
QSE_LIVE_NOW_WINDOW_S	ops/serve.env, server/live.py
QSE_LIVE_PREFILL_WARMUP_CHUNKS	ops/serve.env, server/live.py

continued

Variable	Read in
QSE_MEM_NEED_GIB	ops/profiles/flashnext-2.05.env, ops/profiles/flashnext-3.05.env, ops/profiles/flashnext-4.05.env and 3 more
QSE_METRICS_TOKEN	ops/make-secrets.sh, ops/start.sh, proxy/app.py and 1 more
QSE_NO_ROLLBACK	ops/use-profile.sh, server/setup/apply.py
QSE_OPS_DIR	ops/start.sh, server/setup/apply.py, server/setup/device.py
QSE_PAGE_CACHE_DROP	ops/profiles/_flashnext.env, ops/serve.env
QSE_PAGE_CACHE_KEEP	ops/profiles/_flashnext.env, ops/serve.env
QSE_PARALLEL	ops/profiles/_flashnext.env, ops/serve.env, server/app.py and 1 more
QSE_PARALLEL_BATCH	server/app.py
QSE_PARALLEL_CUT	server/app.py
QSE_PARALLEL_DRAFT	server/app.py
QSE_PARALLEL_FLOOR_GB	engine/kvpool.py, ops/profiles/_flashnext.env, ops/serve.env and 1 more
QSE_PARALLEL_LAYER_YIELD	server/app.py
QSE_PARALLEL_SWITCH_MS	server/app.py
QSE_PARALLEL_TIMING	server/app.py
QSE_PARALLEL_WINDOW_ROWS	server/app.py
QSE_PRICE_TABLE	server/app.py
QSE_PROFILE_NAME	ops/serve.env
QSE_PROFILE_OVERRIDE	ops/serve.env, ops/use-profile.sh, server/setup/apply.py and 1 more
QSE_ROLES_DYNAMIC	ops/serve.env, server/rolemem.py
QSE_ROLE_MAX_HOLD_S	ops/serve.env, server/rolemem.py
QSE_ROLE_RELOAD_MARGIN_GB	ops/serve.env, server/rolemem.py
QSE_ROLE_RELOAD_S	ops/serve.env, server/rolemem.py
QSE_ROLE_THRASH_S	ops/serve.env, server/rolemem.py
QSE_ROLE_TICK_FAKE	server/app.py
QSE_ROLE_TICK_S	server/rolemem.py
QSE_ROLE_UNLOAD_LAYA	ops/serve.env, server/app.py, server/rolemem.py
QSE_SANDBOX_SOCKET	ops/sandbox/install.sh
QSE_SECRETS	install.sh, ops/start.sh, server/setup/apply.py
QSE_SESSION_S	server/auth.py
QSE_SETUP_ENV	ops/serve.env, server/setup/apply.py
QSE_SETUP_ENV_FILE	ops/serve.env
QSE_SETUP_MODE	ops/start.sh
QSE_SIDE_GRAPHS	ops/serve.env, server/app.py
QSE_SIDE_MAX_LEN	ops/profiles/_flashnext.env, ops/serve.env, server/app.py
QSE_SIDE_RESIDENT_GB	server/app.py
QSE_SIDE_SLOT_CONST_GB	ops/serve.env, server/app.py
QSE_STATE_DIR	ops/chatd.sh, ops/make-secrets.sh
QSE_STATUS_FILE	ops/engine_status.py
QSE_STATUS_KEEP	ops/hold.sh, ops/stop.sh
QSE_STATUS_PORT	ops/engine_status.py, ops/hold.sh, ops/serve.env and 1 more
QSE_TEST	proxy/app.py, server/app.py, server/ledger.py
QSE_USAGE_DEFAULT	server/app.py

continued

Variable	Read in
QSE_USAGE_LEDGER	install.sh, ops/serve.env, ops/start.sh and 3 more
QSE_WEIGHTS	engine/models/flashnext/serving.py, ops/engine_status.py, ops/profiles/_exl3.env and 7 more
QSE_WEIGHTS_ALSO	ops/profiles/_flashnext.env, ops/use-profile.sh
QSE_WEIGHTS_DIR	ops/profiles/_exl3.env, ops/profiles/_flashnext.env, ops/profiles/nvfp4.env and 4 more
QSE_WEIGHTS_FILES	ops/profiles/_exl3.env, ops/profiles/_flashnext.env, ops/profiles/nvfp4.env and 3 more
QSE_WEIGHTS_GB	ops/profiles/exl3-2.5.env, ops/profiles/exl3-3.0.env, ops/profiles/exl3-3.5.env and 11 more
QSE_WEIGHTS_HF	ops/profiles/_exl3.env, ops/profiles/_flashnext.env, ops/profiles/nvfp4.env and 3 more
QSE_WEIGHTS_REVISION	ops/profiles/_flashnext.env, ops/profiles/flashnext-2.05.env, ops/profiles/flashnext-3.05.env and 6 more
QSE_WREN_MODEL	server/setup/wren.py

The most used: QSE_CONDUCTOR_CONFIG, QSE_USAGE_LEDGER, QSE_DISK_CACHE and QSE_BACKENDS name the conductor's directory, the ledger, the disk tier's directory and the proxy's config database; ops/start.sh passes them to the service only, so a benchmark server started with serve.env exported never writes them. QSE_DASHBOARD_LOGIN turns the dashboard login on. QSE_ADMIN_TOKEN and QSE_METRICS_TOKEN live in secrets.env, which ops/make-secrets.sh writes.

28.4 Proxy, collector and plugin variables

Table 28.4: TANDEM_* variables: the proxy, the collector, the plugins and the installer

Variable	Example	What it sets
TANDEM_BACKENDS	deepseek,vllm1	the backends the environment declares
TANDEM_BACKEND_<ID>_KIND	deepseek	generic | tandemllm | vllm | sglang | openai | deepseek
TANDEM_BACKEND_<ID>_URL	https://api.deepseek.com/v1	
TANDEM_BACKEND_<ID>_KEY	(a key)	held in memory, never stored
TANDEM_BACKEND_<ID>_LABEL	DeepSeek	
TANDEM_BACKEND_<ID>_MODELS	deepseek-flash, deepseek-v4-pro	
TANDEM_BACKEND_<ID>_MODELS_MODE	discover	discover (GET /models, plus MODELS) | list
TANDEM_BACKEND_<ID>_FALLBACK	openrouter	backends tried after this one, in order
TANDEM_BACKEND_<ID>_MAX_INFLIGHT	none	requests at once, past it a fallback or a 503; none (the default): no limit from the proxy, the backend takes what it accepts
TANDEM_BACKEND_<ID>_ENABLED	1	
TANDEM_BACKEND_<ID>_INJECT_USAGE	1	ask a stream for its usage frame
TANDEM_BACKEND_<ID>_HARNESS	off	off | shadow | on

continued

Variable	Example	What it sets
TANDEM_BACKEND_<ID>_HARNESS_PROFILE	coding	the conductor profile of its requests
TANDEM_BACKEND_<ID>_WRITER	auto	the background memory writer on this backend (proxy/memwriter.py): auto (on, off for a paid cloud model) | off | shadow | on
TANDEM_BACKEND_<ID>_WRITER_USD_DAY	0.25	its spend cap a day on a paid model
TANDEM_BACKEND_<ID>_WRITER_RUNS_DAY	48	its runs a day
TANDEM_BACKEND_<ID>_WRITER_PROMPT_TOKENS	32000	its prompt at most (older messages dropped)
TANDEM_BACKEND_<ID>_TIMEOUT_CONNECT	10	seconds; also _TIMEOUT_FIRST_BYTE, _TIMEOUT_IDLE
TANDEM_BACKEND_<ID>_HOST	auto	where the System view reads the backend's machine: auto (this machine when the URL is), same, collector, exporter, off
TANDEM_BACKEND_<ID>_HOST_URL	(a URL)	the remote collector (python -m proxy.hostcollect), or a node_exporter page
TANDEM_BACKEND_<ID>_HOST_DCGM_URL	(a URL)	a DCGM exporter page (mode exporter)
TANDEM_COLLECTOR_TOKEN	...	the remote collectors' bearer token, in memory only (proxy/hostcollect.py)
TANDEM_ROUTE_<n>	gpt-*=openai	routing rules, ordered by n
TANDEM_PRICE_<n>	deepseek/deepseek-flash=0.30,0.006,1.2	USD per million: input, cached input, output, and optionally cache write
TANDEM_PRICE_<n>_OFFPEAK	0.15,0.003,0.6	the same model outside the provider's peak hours (DeepSeek)
TANDEM_API_KEYS	(a key)	the proxy's own keys, in memory only
TANDEM_SECRETS_KEY	<base64	of 32 bytes> the master key (proxy/secrets.py)
TANDEM_ADMIN_TOKEN		read in ops/dash.env.example, proxy/__main__.py
TANDEM_BACKEND_DEEPSEEK_KEY		read in ops/dash.env.example
TANDEM_BACKEND_DEEPSEEK_KIND		read in ops/dash.env.example
TANDEM_BACKEND_DEEPSEEK_LABEL		read in ops/dash.env.example
TANDEM_BACKEND_VLLM1_FALLBACK		read in ops/dash.env.example
TANDEM_BACKEND_VLLM1_HOST		read in ops/dash.env.example
TANDEM_BACKEND_VLLM1_HOST_DCGM_URL		read in ops/dash.env.example
TANDEM_BACKEND_VLLM1_HOST_URL		read in ops/dash.env.example
TANDEM_BACKEND_VLLM1_KIND		read in ops/dash.env.example
TANDEM_BACKEND_VLLM1_MAX_INFLIGHT		read in ops/dash.env.example
TANDEM_BACKEND_VLLM1_TIMEOUT_FIRST_BYTE		read in ops/dash.env.example
TANDEM_BACKEND_VLLM1_URL		read in ops/dash.env.example
TANDEM_COLLECT		read in proxy/launch.py
TANDEM_COLLECT_BASE		read in proxy/launch.py

continued

Variable	Example	What it sets
TANDEM_COLLECT_TOKEN		read in plugins/sglang/tandem_sglang_plugin/_push.py, plugins/vllm/tandem_vllm_plugin/__init__.py
TANDEM_COLLECT_URL		read in plugins/sglang/tandem_sglang_plugin/_push.py, plugins/vllm/tandem_vllm_plugin/__init__.py
TANDEM_CONDUCTOR		read in proxy/__main__.py
TANDEM_CONFIG_DB		read in ops/dash.env.example, proxy/__main__.py
TANDEM_DASHBOARD_LOGIN		read in ops/dash.env.example, proxy/__main__.py
TANDEM_DIR		read in install.sh
TANDEM_HEALTH_TIMEOUT		read in install.sh
TANDEM_HOST		read in install.sh, ops/dash.env.example
TANDEM_MIN_FREE_GIB		read in install.sh
TANDEM_MODEL_NAME		read in install.sh
TANDEM_NOTIFY_URL		read in server/conductor/monitor/notify.py, server/conductor/monitor/sessiontasks.py
TANDEM_PORT		read in install.sh, ops/dash.env.example
TANDEM_PRICE_1		read in ops/dash.env.example
TANDEM_PRICE_1_OFFPEAK		read in ops/dash.env.example
TANDEM_REF		read in install.sh
TANDEM_REPO		read in install.sh
TANDEM_ROUTE_1		read in ops/dash.env.example
TANDEM_SCRATCH		read in server/conductor/packages.py
TANDEM_SKIP_MEMCHECK		read in install.sh
TANDEM_URL		read in server/conductor/packages.py, server/laya_kit/laya_drive.py
TANDEM_USAGE_LEDGER		read in ops/dash.env.example, proxy/__main__.py

28.5 The conductor's documents

A document is JSON, checked against the conductor's schema (`GET /v1/conductor/meta` serves it, and `docs/contract/conductor-v1/` holds a copy). A version stored by an older release loads with defaults for every field it lacks, and the journal says so once.

Table 28.5: The conductor's versioned documents.

document	what it holds	edited on
`profiles`	one document per profile: mode, decisions, task modes and caps, thinking, `ui`, outcomes, library, memory, shadow	Profiles
`router`	the classes and their policy, the classifier stages and thresholds, the structural rules, side shapes, outcomes, library	Routing
`packages`	every package: source, switch, limits, flags	Packages
`roles`	every role's contract	Profiles, Roles
`settings`	the default profile, transcripts, the disk tier, memory notes and the writer	Memory, Profiles
`deciders`	the decision models, their questions, Laya for scripts	Deciders
`residency`	the floor, the margin, the settle time, the load order, the claims	Profiles, Residency
`monitor`	the session monitor's policy (version 0 is `ops/supervisor.yaml`)	Monitor
`harness`	the harness's settings per backend, behind the proxy	Backends

Chapter 29

HTTP API reference

The engine's server and the standalone proxy speak the same API for clients, and each adds its own routes for the dashboard and the conductor. This chapter lists every route, every header and every error they use. The tables are written in `manual/tools/http_ref.py`, and `gen_reference.py --check` compares them with the code on every build: a route a handler tests, a header name or an error type the code uses that is missing here, or a row the code no longer has, fails the check.

29.1 Who may call what

The admin password is the admin token, `QSE_ADMIN_TOKEN` in `secrets.env` (`TANDEM_ADMIN_TOKEN` for the standalone proxy). It is not set by default. Then every route below answers anyone who reaches the port, and only `/metrics` keeps the metrics token when one is set (Section 3.8). With a password set, an admin route needs it as a bearer header or a dashboard session (a cookie the token buys). A request from the machine itself (loopback, no proxy headers) needs neither, because the watchdog and the measurement tools rely on that (`--trust-loopback`).

The OpenAI routes are open on the engine, with or without a password. Behind the proxy they are open until the first user key exists; from then on every request from another machine needs a key. Table 29.1 names the labels the route tables use.

Table 29.1: Who may call a route: the labels of the routes' who column

label	who may call
`open`	anyone who reaches the port
`admin`	the admin rule: anyone while no admin password is set; with one, the admin token, a dashboard session, or a tool on the machine itself
`admin-bearer`	as admin, but the token only as a bearer header, never the session cookie
`dash`	the dashboard rule: anyone unless a password is set and the dashboard login is on
`metrics`	the metrics token when one is set, else as admin
`session`	only under a session address, `/s/<token>/...`
`key`	anyone until the first user key exists; then a user key, except from the machine itself
`collect`	the collect token, from every address

29.2 The engine's routes

Table 29.2 lists the engine server's routes. A route the server does not know answers 404. A route whose part is off on this server also answers 404: the conductor's routes without `--conductor-config` and the proxy's without `--backends`, each with a message that names the flag, and the decision and Laya routes without decision models, as an unknown route. With `--backends` the engine's server also serves the proxy's routes.

Table 29.2: The engine server's routes

route	who	what, body, answer
`POST /v1/chat/completions`	open	A chat completion, streamed as server-sent events or not. With `--backends` a model of another backend is relayed there. *Body:* OpenAI's chat body: model, messages, tools, stream, `max_tokens` and the other fields of Section 14.1. *Answer:* chat.completion, or chat.completion.chunk events and `[DONE]`; usage carries `prompt_tokens_details.hidden_tokens` for the conductor's notes.
`POST /v1/completions`	open	A plain text completion. *Body:* prompt, `max_tokens` and the sampling fields. *Answer:* `text_completion`, or its chunks and `[DONE]`.
`POST /v1/embeddings`	key	Relayed to the backend that serves the model; only with `--backends`. *Body:* OpenAI's embeddings body. *Answer:* the backend's answer, unchanged.
`POST /v1/responses`	key	OpenAI's Responses API, relayed to an openai backend; only with `--backends`. *Body:* the client's body, sent on with the backend's model id. *Answer:* the backend's answer, unchanged.
`GET /v1/models`	open	The model list: TandemLLM alone with the conductor on, else the served name; with `--backends` every backend's models after it, behind the users' keys. *Answer:* `{object: list, data: [...]}`, and `decision_models` beside data when decision models are served.
`GET /v1/models/{id}`	open	One model card: the engine's, a decision model's, or a backend's. *Answer:* the card, or 404 `model_not_found`.
`GET /health`	open	Whether the server takes work. The full body follows the admin rule; anyone else gets the status alone. Also `/healthz` and `/v1/health`. *Answer:* `{status: ok|draining}`; the full body adds model, uptime, inflight, the queue, limits, sampling, cache, memory and parallel; 503 while draining.
`GET /metrics`	metrics	The Prometheus families of Chapter 30. *Answer:* Prometheus text format.
`GET /metrics/up`	open	One gauge, `qse_up`, so a scraper can tell an engine that is down from a wrong token. *Answer:* `qse_up` 1, or 0 while draining.
`GET /v1/cache/stats`	admin	Every cache's counters. *Answer:* the cache report `/health` carries.
`POST /v1/cache/peek`	admin	Where a chat's rows are before it is sent: the body is rendered as a request would be, the session's router notes are replayed, and the cache tiers are asked without running the model (SRV-349). *Body:* a chat body (messages, tools, the effort fields) and session. *Answer:* `{tier: resident|ram_park|ram_snapshot|nvme_spill|disk_tier|none, cached_tokens, reread_tokens, reread_eta_ms, tokens, restore_ms, session}`; 409 while a request holds the engine.

continued

route	who	what, body, answer
`POST /v1/cache/prewarm`	admin	Reads a conversation the peek finds on the NVMe back into the engine while nothing runs or waits, so its next request resumes at the end anchor. *Body:* as `/v1/cache/peek`. *Answer:* `{prewarmed, why?, restore?}` and the peek's fields; 409 while busy.
`POST /v1/tokenize`	open	Exact token counts with the model's own chat template, split by part; nothing runs on the GPU (SRV-347). 240 a minute from one address. *Body:* `{messages, tools?}` or `{text}`. *Answer:* `{tokens, parts: {system, tools, messages}}`.
`POST /v1/cache/clear?tier= ram|disk|all`	admin-bearer	Empties the caches in memory (ram, the default), the disk tier and the session spill (disk), or both (all). Nothing else is reset. *Answer:* the cache report after the clear; 400 for another tier.
`GET /dashboard/`	open	The dashboard's static files. They hold no data. *Answer:* the page.
`GET /v1/dashboard/summary?from=&to= &tz=&model=&client=&backend= &user=&internal=`	dash	The Usage view's totals for a range. *Answer:* `summary.schema.json` in docs/contract/dashboard-v1.
`GET /v1/dashboard/usage?bucket= hour|day&from=&to=&tz=&model= &client=&backend=&user= &internal=`	dash	Tokens, requests and cost per hour or day. *Answer:* `usage.schema.json`.
`GET /v1/dashboard/requests?limit= &before=&finish=&model=&client= &backend=&user=&internal=`	dash	Recent request rows of the usage ledger, newest first. *Answer:* `requests.schema.json`.
`GET /v1/dashboard/system`	dash	The machine, the engine, the memory plan, the slots and the proxy's mode. *Answer:* `system.schema.json`.
`GET /v1/dashboard/live`	dash	What each request is doing now, as server-sent events. *Answer:* `live.schema.json` events.
`GET /v1/dashboard/logs?level=&grep= &backlog=&since=&follow=`	dash	The log: a backlog of up to 5,000 lines, then live lines as server-sent events; Last-Event-ID resumes; follow=0 answers JSON once. *Answer:* `logs-line.schema.json` events, or `logs-json.schema.json`; 429 past four streams at once.
`GET /v1/dashboard/metrics`	dash	The `/metrics` page under the dashboard's rule, for the Performance view. *Answer:* Prometheus text format.
`GET /v1/dashboard/parallel?since=`	dash	Every request in flight, the engine's and the relayed ones. *Answer:* `parallel.schema.json`.
`GET /v1/dashboard/session`	open	Whether a sign-in exists and is needed. *Answer:* `{authenticated, login, password_set, expires_at}`.
`POST /v1/dashboard/session`	open	The sign-in, with a password set and the login on: sets the `qse_dash` cookie. Five failures a minute from one address and the sixth gets a 429. *Body:* `{token}`. *Answer:* the session; 401 for a wrong token.
`DELETE /v1/dashboard/session`	open	Signs this session out. *Answer:* the session, signed out.
`* /v1/conductor/*`	admin	The conductor's documents, journal, sessions and memory (Table 29.5). Reads follow the dashboard rule, writes and every memory route the admin rule. 404 without `--conductor-config`.

continued

route	who	what, body, answer
* /v1/setup/*	admin	The setup guide (docs/setup.md): the model registry, the box, the memory estimate, the review plan, the apply and reset jobs and Wren. Every POST must also be same-origin JSON: 415 for another content type, 403 for another origin. In setup mode (--setup) these, the dashboard and /health are the only routes that answer.
* /v1/config/*	admin	The Backends and Setup screens (Table 29.6). Reads follow the dashboard rule, writes the admin rule. 404 without --backends.
POST /v1/collect/{backend}	collect	A batch from the vLLM or SGLang plugin, joined to the ledger rows; only with --backends. *Body:* the plugin's batch: plugin, engine, requests, steps. *Answer:* {accepted: {requests, steps}, joined}.
POST /v1/systemone	admin	Typed decisions from the decision models, in the engine's idle window. /v1/predict is the same route. Only with QWEN38_DECIDER_MODELS. *Body:* {state, model?, questions: {id: {type, instructions, criteria?}}, task?, lang?}. *Answer:* {model, answers, usage, routing, latency_ms, path}.
POST /v1/predict	admin	The same as /v1/systemone. *Body:* as /v1/systemone. *Answer:* as /v1/systemone.
GET /readyz	open	Whether a decision model is loaded and warm. *Answer:* 200, or 503 with the reason.
GET /v1/laya	open	What the Laya route takes and answers, its limits, and whether it is on. *Answer:* a description of the route.
POST /v1/laya/ask	open	A typed answer from Laya on one text or many, for the model's own scripts; rate-limited. *Body:* {question, choices | scale | neither, text | texts, threshold?, model?, deadline_ms?}. *Answer:* {label, p, sure, probs, model, ms, ...}, or {answers: [...]} for texts.
POST /v1/laya/drive/end	open	Ends a playtest driver session and writes its journal row. *Body:* the session's summary. *Answer:* the row.
GET /v1/laya/kit/{file}	open	A file of the playtest kit the model copies into a project. *Answer:* the file.
OPTIONS /v1/laya/*	open	A browser's CORS preflight for the Laya routes. Every other path answers OPTIONS with 501. *Answer:* 204 with the CORS headers.
GET /v1/library	open	The package library: every enabled skill's name, description and classes. *Answer:* the list and the two routes below.
GET /v1/library/search?q=&k=&class=	open	BM25 search over the skills, best first (k from 1 to 20, default 5). *Answer:* [{name, description, score}].
GET /v1/library/{name}	open	One skill's text and its reference. *Answer:* {name, description, classes, body, reference}.
GET /s/{token}/v1/memory	session	The memory index this session sees. The /s/{token}/ prefix also serves the library and Laya routes, filed to the session. *Answer:* the notes' index lines.
GET /s/{token}/v1/memory/search?q= &k=	session	Memory notes that match, best first (k from 1 to 10). *Answer:* [{id, kind, scope, status, title, score}].
GET /s/{token}/v1/memory/{id}	session	One note. *Answer:* {id, title, kind, scope, status, body, ...}.

continued

route	who	what, body, answer
`POST /s/{token}/v1/memory`	session	Writes, changes or archives a note, with the fields of the hidden memory block. *Body:* `{op, scope, kind, title, tags, body, id, reason}`. *Answer:* `{op, id, title, ...}`, or 422 with the reason as the type.

29.3 The proxy's own routes

Table 29.3 lists the routes of `tandem-dash`, the standalone proxy. It serves no model of its own: every completion goes to a backend, and the answer's bytes go back to the client unchanged. Table 29.4 adds the two small servers that run beside the engine or the proxy: the remote host collector and the status responder that answers while the engine is stopped.

Table 29.3: The standalone proxy's routes (`tandem-dash`)

route	who	what, body, answer
`POST /v1/chat/completions`	key	Relayed to the backend the model id routes to (Section 5.10); the answer's bytes go back unchanged. The same for `/v1/completions`, `/v1/embeddings` and `/v1/responses`. *Body:* the client's body. *Answer:* the backend's answer, with X-Tandem-Backend.
`POST /v1/completions`	key	As `/v1/chat/completions`.
`POST /v1/embeddings`	key	As `/v1/chat/completions`; not for deepseek backends.
`POST /v1/responses`	key	As `/v1/chat/completions`; openai backends only.
`GET /v1/models`	key	Every backend's models under their real ids, `owned_by` the backend. *Answer:* `{object: list, data: [... with a tandem block]}`.
`GET /v1/models/{id}`	key	One model's card. *Answer:* the card, or 404 `model_not_found`.
`GET /`	open	Redirects to the dashboard. *Answer:* 302 to `/dashboard/`.
`GET /dashboard/`	open	The dashboard's static files. *Answer:* the page.
`GET /v1/dashboard/{view}`	dash	summary, usage, requests, system, live, logs, metrics, parallel and session, as on the engine; the engine-only panels answer 404. *Answer:* as on the engine.
`POST /v1/dashboard/session`	open	As on the engine. *Body:* `{token}`. *Answer:* the session.
`* /v1/config/*`	admin	The Backends and Setup screens (Table 29.6).
`POST /v1/collect/{backend}`	collect	A plugin's batch (as on the engine).
`GET /metrics`	metrics	The proxy's own Prometheus families. *Answer:* Prometheus text format.

continued

route	who	what, body, answer
`GET /health`	open	The proxy's health, for a container's check; the full body for the admin or the machine. Also `/healthz` and `/v1/health`. *Answer:* `{status}`.
`* /v1/conductor/*`	admin	The conductor in front of the backends, with `--conductor-config`; the engine-only screens answer 404.
`GET /v1/library`	open	The package library, as on the engine, also under `/v1/library/*`.
`GET /v1/laya`	open	Laya, when the decider role names an endpoint.
`POST /v1/laya/ask`	open	As on the engine, relayed to the decider role's endpoint.

Table 29.4: The two small servers beside them

server	route	answer
`tandem-dash hostcollect`	`GET /host?watch=`	the machine's sample as JSON (the bearer token when `--token` is set)
`tandem-dash hostcollect`	`GET /metrics`	the same sample in the Prometheus text format
`tandem-dash hostcollect`	`GET /health`	`{status: ok}`, no token
`ops/engine_status.py`	`GET /v1/models`	200 with the engine's last model list and its state, while the engine is away
`ops/engine_status.py`	`GET /dashboard/`	503 with a page that says why the engine is away and opens the dashboard when it is back
`ops/engine_status.py`	`* any other path`	503 `provider_unavailable` with Retry-After and the state as engine_<state>

29.4 The conductor's and the configuration routes

The conductor's routes and the proxy's configuration routes are generated from the route tables at the top of `server/conductor/routes.py` and `proxy/config_api.py`.

Table 29.5: The conductor's routes, `/v1/conductor/*`

Method	Path	What
`GET`	`/v1/conductor/config`	loaded versions, load errors, journal, roles
`GET`	`/v1/conductor/meta`	enums, the task catalogue, the schemas
`GET`	`/v1/conductor/profiles`	tiles
`GET`	`/v1/conductor/{kind}[/{name}]`	settings, router, roles, packages; profiles/{name}
`PUT`	`/v1/conductor/{kind}[/{name}]`	`{doc, note, base_version?}`: a new version
`GET`	`.../versions .../versions/{v} .../diff?from=&to=`	a document's versions, one version, and the diff between two

continued

Method	Path	What
POST	`.../restore`	`{version, note?}` a new version equal to an old one
POST	`/v1/conductor/profiles/{name}/mode`	`{mode, note?}`: the three-way switch
GET	`/v1/conductor/roles`	the document, health, and what each role's tokenizer hash allows it to share
POST	`/v1/conductor/roles/{role}/test`	one question, its latency and answer
GET	`/v1/conductor/packages/scan`	skills found in skill_dirs on the box
GET	`/v1/conductor/packages/catalog`	every package: category, tokens on the engine's tokenizer, limit, classes, library switches
GET	`/v1/conductor/packages/item/{name}`	one package's texts, tokens, BM25 neighbours
POST	`/v1/conductor/packages/tokens`	`{texts}` the tokens of up to 20 texts (a read)
GET	`/v1/conductor/packages/usage?name=&limit=`	injections, shadow notes and library picks per package in the last day and week (journal)
POST	`/v1/conductor/classify`	`{text, profile?, tools?}`: the test box
GET	`/v1/conductor/decisions[/{id}]`	the live feed (router rows, each with the monitor's `events` of its request) and one row
POST	`/v1/conductor/decisions/{id}/label`	`{label: right|wrong|clear, note?}` (any row)
GET	`/v1/conductor/events?task=&mode=&acted=&session=&request=&since=&limit=`	the session monitor's decisions
GET	`/v1/conductor/events/{id}`	one of them
GET	`/v1/conductor/stats?hours=24`	the Overview's tiles and ribbon, `monitor` too
GET	`/v1/conductor/sessions[/{key}[/export?format=json|md]]`	sessions, one timeline, the honest record: every hidden text of router and monitor alike
GET	`/v1/conductor/evals`	classifier and suite reports on the box
POST	`/v1/conductor/evals`	`{report}`: a suite report (tools/tandem_eval.py)
GET	`/v1/conductor/residency/plan`	the residency planner: limits, what fits now, the engine's split, the last refusal
GET	`/v1/conductor/watch`	the session tasks' state per session
GET	`/v1/conductor/deciders/status`	the decision models this engine serves: states, limits, numbers, the planner's answer for a load now, the model behind each question

continued

Method	Path	What
POST	/v1/conductor/deciders/models/{name}/load|unload	through the residency planner (write)
POST	/v1/conductor/deciders/autoload	{service_start?, hold_restart?}: the on_start models, one after the other (ops/start.sh; write)
POST	/v1/conductor/deciders/test	{state, questions, model?}: the test box
GET|PUT	/v1/conductor/deciders	the deciders document, like the others
GET	/v1/conductor/narrator/status	the narrator's slot: its model, state, contract, calls and latency
POST	/v1/conductor/narrator/try	{events: [{kind, facts, extra?}], budget_ms?, max_tokens?}: each event's template and the narrator's line, in the idle window (write)
POST	/v1/conductor/narrator/load|unload	through the residency planner (write)
GET	/v1/conductor/shadow/status	the shadow reader (SR1): its slot, the sessions' states (read, behind, bytes), the router's agreement, the last answers
POST	/v1/conductor/shadow/ask	{session, question, choices: {id: text}, budget_ms?}: a typed decision of the small model over the session's whole context, in the idle window (write)
GET	/v1/conductor/memory/sessions	the session transcripts on the box: one row per session, the writer's counters and settings
GET	/v1/conductor/memory/sessions/{key}?page=N	its blocks, 200 a page (default: the newest page), to the last complete block
GET	.../memory/sessions/{key}/export?format=md|json	the transcript itself
PATCH	.../memory/sessions/{key}/keep	{keep}: meta.json's keep, which the prune honours
GET	/v1/conductor/memory/disk	the disk tier: running or not, its settings and their version, bytes, budget, free space, hit rate, the most-hit entries (a preview for opencode heads and packages ...
POST	/v1/conductor/memory/disk/clear	every entry and count gone (write)
GET	/v1/conductor/memory/notes?user=	the memory notes: the users, the tree (scopes with counts), every live note's frontmatter with its uses in 7 days, the archived ones, the duplicates, the store's counts
GET	.../memory/notes/search?user=&q=&k=	the runtime's search over every scope, with the scores
GET	.../memory/notes/injected?user=&limit=	the last requests' injected note ids (journal)

continued

Method	Path	What
GET	`.../memory/notes/writes?limit=`	the last writes and refusals (journal)
GET	`.../memory/notes/export?user=`	a zip of the user's directory
GET	`.../memory/notes/{id}?user=`	one note: frontmatter, body, changelog, the revisions, its uses by day
PUT	`.../memory/notes/{id}?user=`	`{title, body, kind, tags, verify_by}`: a person's edit (a revision; ratified, writer human)
PATCH	`.../memory/notes/{id}?user=`	`{status?, pinned?}`: ratify, back to provisional, pin a ratified rule
POST	`.../memory/notes/{id}/archive?user=`	`{reason}`
DELETE	`.../memory/notes/{id}?user=`	the note, its revisions and archive copy, at once
DELETE	`.../memory/notes/users/{user}`	`{confirm: <user>}`: the whole directory

Table 29.6: The proxy's configuration routes, `/v1/config/*`

Method	Path	What
GET	`/v1/config`	everything the screens show, in one answer
POST	`/v1/config/test`	`{kind, url, key?}`: test a backend not saved
POST	`/v1/config/backends`	`{id, kind, url, key?,...}`: add
GET	`/v1/config/backends/{id}`	one, with its models
PATCH	`/v1/config/backends/{id}`	change fields (`key: null` removes the key)
DELETE	`/v1/config/backends/{id}`	
POST	`/v1/config/backends/{id}/test`	test the saved backend, refresh its models
PUT	`/v1/config/backends/{id}/models/{model}`	public id, enabled, price, fallback, features (creates a manual model)
DELETE	`/v1/config/backends/{id}/models/{model}`	
PUT	`/v1/config/rules`	`{rules: [{id?, pattern, backend, enabled}]}`
POST	`/v1/config/users`	`{label}`: the answer holds the key, once
PATCH	`/v1/config/users/{id}`	`{enabled}`
DELETE	`/v1/config/users/{id}`	
GET	`/v1/config/prices/seed`	the public list prices shipped with this release, applied only by a click
GET	`/v1/config/audit`	the last 50 changes (field names only)

29.5 Headers

Table 29.7 lists every header the engine and the proxy read or write beyond the ones HTTP itself needs. *in* is a header the server reads, *out* one it writes.

Table 29.7: Every header the engine and the proxy read or write

header	direction, where	meaning
`X-Tandem`	both, both	raw, shadow or on: lowers the conductor's mode for one request; the response says which mode ran
`X-Tandem-Profile`	both, both	picks a conductor profile; the response names the profile that ran; an unknown name is a 400
`X-Tandem-Memory`	in, engine	off: no memory note is read or written; read: reads only, a block the model writes is hidden and dropped
`X-Tandem-Probe`	in, engine	off takes the request out of the probe snapshots
`X-Tandem-User`	in, engine	a proxy's user name for the memory notes, honoured from loopback or QWEN38_MEMORY_PROXIES
`X-Tandem-Relay-Id`	in, engine	a proxy's id for a relayed request, for the live counts; a relayed request without X-Tandem-User touches no memory
`X-Tandem-Parent`	in, engine	the request a call of TandemLLM to itself serves; the row becomes client kind internal
`X-Tandem-Regenerate`	in, engine	1 serves the request as a regenerate (sampled, a fresh seed); 0 turns the detection off for it
`X-Tandem-Images-Left-Out`	out, engine	how many earlier images were left out of the prompt
`X-Tandem-Forward`	out, engine	to chatd on a `/v1/chat-app` request: the forward key, the only proof chatd accepts
`X-Tandem-Principal`	out, engine	to chatd: who the request is for (owner)
`X-Tandem-Password-Set`	out, engine	to chatd: 1 when an admin password is set
`X-Tandem-Signed-In`	out, engine	to chatd: 1 when the request passed the dashboard's rule
`X-Tandem-Wait`	out, engine	memory, on a response that waited for memory before its prefill; its stream says so every 15 s
`X-Tandem-Backend`	out, proxy	the id of the backend that answered a relayed request
`X-Engine-Stop`	out, engine	pattern-stop(<label>) on a response without a stream that a stop pattern ended
`x-session-id`	in, both	opencode's session id: the router's session and the caches' session key
`X-Conversation-Id`	in, both	a conversation id for clients that send no session field; read after the body's own ids
`X-OpenWebUI-Chat-Id`	in, both	Open WebUI's chat id (a connection header with `{{CHAT_ID}}`): the router's session for that chat
`X-Request-Id`	both, proxy	sent to a vllm backend as px-<hex>, which the plugin reports back for the join; a client's own id is kept
`X-QSE-Client`	in, engine	dashboard marks the dashboard's own requests (client kind dashboard)
`X-Real-IP`	in, both	set by a reverse proxy; a request that carries it is never trusted as local; the sign-in's rate limit counts per address
`X-Forwarded-For`	in, both	as X-Real-IP: a request that carries it is never trusted as local

continued

header	direction, where	meaning
`X-Forwarded-Proto`	in, both	https makes the session cookie Secure, and the notes' `{tandem_url}` https
`X-Forwarded-Host`	in, engine	the host the notes' `{tandem_url}` names, unless QWEN38_TANDEM_URL sets it
`X-Decider-Deadline-Ms`	in, engine	how long a decision request waits for the idle window (default 30,000, at most 120,000)
`X-Decider-Window`	in, engine	`in_decode` is refused: decisions run in the idle window only
`X-Accel-Buffering`	out, both	no, on every stream, so nginx sends each event at once
`X-Content-Type-Options`	out, both	nosniff on the dashboard's static files
`x-ratelimit-*`	out, proxy	a cloud backend's rate-limit headers, passed on to the client
`OpenAI-Beta`	both, proxy	a client's OpenAI-Beta header, passed on to an openai backend
`Authorization`	in, both	Bearer: the admin token, the metrics token, the collect token or a proxy user key, by route; the engine's OpenAI routes ignore it and file the request under a hash of it
`User-Agent`	in, both	the client kind in the ledger: open-webui, opencode, openai-sdk, curl, other (a Hermes-shaped body: hermes)
`Last-Event-ID`	in, both	resumes the log stream after that line
`Retry-After`	out, both	with a 503 (queue full, memory gate, draining, a full backend), a 429, or a decision model that is loading
`WWW-Authenticate`	out, both	Bearer, with a 401
`Set-Cookie`	out, both	`qse_dash`, the dashboard session: HttpOnly, SameSite=Strict, Secure behind https
`Access-Control-Allow-Origin`	out, engine	* on the library and Laya routes, so a page the model builds can call them; with Access-Control-Allow-Methods, -Headers and -Max-Age on the preflight
`Access-Control-Allow-Methods`	out, engine	on the Laya preflight
`Access-Control-Allow-Headers`	out, engine	on the Laya preflight
`Access-Control-Max-Age`	out, engine	on the Laya preflight

29.6 Errors

Errors are OpenAI-shaped:

```
{"error": {"message": "max_tokens must be at least 1", "type": "invalid_request_error",
           "param": "max_tokens", "code": null}}
```

A 400 names the field or the message part it refuses in `param`. A refusal the client can retry (a full queue, the memory gate, a server that drains) has the type `server_busy`, the status in `code`, and a `Retry-After` header. A refusal after a stream opened (a queue timeout, a prompt too long, the memory gate, a backend that failed) ends the stream with a chunk whose `finish_reason` is

`"error"` and which carries the `error` object, then `[DONE]`. The decision routes keep their own body, `{"type": "error", "error": {"type", "message"}}`. Table 29.8 lists every status and type.

Table 29.8: Every error status and type

status, Retry-After	error.type or code	when
400	invalid_request_error	a request the engine will not serve as asked: error.param names the field (messages, a role, max_tokens, tools, an image part, a prompt too long, a body that is not JSON or not an object)
400	bad_request	a dashboard or conductor query that does not parse (a level, a range, a number)
400	confirm_required	a setup reset without the confirmation its level asks for (true, or the typed word)
401	unauthorized	a guarded route without the token or the session it needs, once an admin password is set; a wrong collect token
401	invalid_api_key	the proxy, once a user key exists: no key, or a key it does not know (code invalid_api_key)
403	forbidden	a memory route outside a known session address
404	not_found	no such route, or a route whose part is off on this server (the conductor, the proxy, the decision models)
404	unsupported_route	the proxy: a route the backend's kind does not serve, such as embeddings on deepseek (code unsupported_route)
404	model_not_found	a model id no backend serves, or one two backends serve (the message names the prefixed forms) (code model_not_found)
409	busy, not_loaded, not_running, closed, stale	a conductor action that needs a role or a decision model in another state; a package proposal already closed, or whose base text changed since
422	invalid_request_error, invalid_report	a value the schema refuses: a config field (error.field), a field an environment variable locks (error.variable), a Laya, library or memory call's fields, an eval report
422	schema, secret, cap, user, unknown_id, unknown_package, rule_from_tool, no_session, no_user, refused_root, no_proposals, propose	a memory note or a package proposal refused, with the reason as the type: a field (schema), a secret, a cap, an unknown note or package
429 with Retry-After	server_busy	a request without a stream that waited --queue-timeout for the engine
429 with Retry-After	too_many	a fifth log or live stream at once; a sixth failed sign-in in a minute
429 with Retry-After	rate_limited	the library, Laya or memory routes past their calls a minute
500	internal_error, error	a bug: the message is the exception's; the request is in the log
501	none	OPTIONS on any path but the Laya routes
502	connect_error, upstream_error, key_unreadable	the proxy could not reach the backend or read its stored key, after its retries

continued

status, Retry-After	error.type or code	when
200	upstream_closed, upstream_error	a stream that already began: the backend failed or closed early; the client gets an error event, then [DONE]
503 with Retry-After	server_busy	the queue is full, the server is draining, or the memory gate or the admission by bytes cannot make room
503 with Retry-After	memory_wait_timeout	a request without a stream that waited past --queue-timeout for memory to free (type server_busy, Retry-After: 30)
503 with Retry-After	setup_mode	any route but the dashboard, /health, /v1/models and /v1/setup/* on a box in setup mode (Retry-After: 60)
503 with Retry-After	max_inflight	every backend for the model is at the max_inflight you set (Retry-After: 1)
503 with Retry-After	warming, loading, refused, failed, not_loaded, busy_decoding	a decision request while the model loads, was refused memory, failed, or the engine kept decoding past the deadline
503 with Retry-After	provider_unavailable	the status responder while the engine is stopped, starting or held (code engine_<state>)
504	upstream_timeout, timeout	a backend that sent no first byte in time; a Laya call past its deadline
529	overloaded_error	a decision request the runtime could not take

Chapter 30

Metrics and the ledger

30.1 Prometheus metrics

`GET /metrics` serves the families below in the Prometheus text format, to anyone while no token is set, else behind the metrics or admin token, or to a request from the machine itself. `ops/monitoring/` has a scrape configuration, alert rules with their tests, and a Grafana dashboard generated by `tools/grafana_dashboard.py`. With the metrics token set, a scrape job needs it as a bearer token:

```
- job_name: tandemllm
  authorization: {type: Bearer, credentials_file: /etc/prometheus/tandem-metrics.token}
  static_configs: [{targets: ['gpu-host:8000']}]
```

The engine's families never count a relayed request, and the proxy's families never see the engine's own. The table lists every family registered with a literal name in the code; families built at scrape time from a table (the mapped `qse_backend_*` engine families, the cache families by `cache` label) are listed by their prefix in the help text of the page itself.

Table 30.1: Prometheus families registered in the code

Metric	Type, labels	What it counts
`qse_requests_total`	`counter, finish_reason`	generations that finished, by the reason they finished
`qse_request_success_total`	`counter`	generations that finished on the model's own stop token or the caller's token budget
`qse_prompt_tokens_total`	`counter`	prompt tokens accepted, including tokens served from a cache
`qse_generation_tokens_total`	`counter`	tokens written by the engine
`qse_errors_total`	`counter, type`	exceptions that ended a generation, by exception class
`qse_images_left_out_total`	`counter`	a conversation's earlier images left out of a request over the image caps (server/images.py)
`qse_time_to_first_token_seconds`	`histogram`	arrival of the request to its first token, queue wait included
`qse_time_per_output_token_seconds`	`histogram`	gap between two tokens of one generation.
`qse_e2e_request_latency_seconds`	`histogram`	arrival of the request to the end of its stream, queue wait included

continued

Metric	Type, labels	What it counts
qse_spec_decode_num_accepted_tokens_total	counter	drafted tokens the target agreed with. The token the target writes itself is not counted here, so accepted over drafted is the draft acceptance rate
qse_spec_decode_num_draft_tokens_total	counter	tokens proposed by the drafter, anchor excluded
qse_spec_decode_num_drafts_total	counter	blocks the drafter proposed
qse_spec_accept_per_block	histogram	tokens committed by one verified block, the target's own token included
qse_draft_width_chosen_total	counter, width	blocks proposed at each width, anchor included.
qse_verify_seconds	histogram	one verify pass over a draft block, as the decode loop paid for it
qse_draft_seconds	histogram	one call to the drafter, lookup and block drafter together
qse_conductor_decisions_total	counter, task, tier, mode, decision	conductor decisions, passes included: the monitor's tasks and the router's class (task route)
qse_conductor_actions_total	counter, task, action	decisions the conductor executed, by task and action
qse_conductor_hidden_tokens_total	counter	tokens the model saw that the client did not send or receive (hidden closes, hidden turns, reminders, router notes new and replayed)
qse_conductor_gave_up_total	counter	turns the conductor ended with its GIVE_UP sentence
qse_conductor_tier1_seconds	histogram	one tier 1 question (a classifier stage: the decider or Qwen), answered or not
qse_conductor_tier1_failures_total	counter, reason	tier 1 questions without a usable answer, by reason
qse_conductor_reasoning_tokens_cut_total	counter	reasoning tokens written before a hidden close cut the block (the size of the blocks the conductor closed)
qse_requests_running	gauge	generations the engine is working on
qse_requests_waiting	gauge	requests holding a socket and waiting for the engine
qse_requests_refused_total	counter, reason	requests turned away with a Retry-After before the engine saw them
qse_cache_misses_total	counter, cache	prefix lookups that found nothing
qse_cache_evictions_total	counter, cache	entries dropped to stay inside the byte budget
qse_cache_bytes	gauge, cache	bytes held by each cache
qse_memory_bytes	gauge, part	the engine's memory against its limits, by part
qse_memory_shed_total	counter, level	caches given back to keep the engine under its ceiling or its peak, by level
qse_cache_entries	gauge, cache	entries held by each cache
qse_disk_cache_bytes	gauge, part	the disk tier: what its entries hold, its budget, the file system's free space and the floor it never crosses, the host staging buffer
qse_disk_cache_entries	gauge, kind	the disk tier's files: anchors (one state each) and chunks (1,024 tokens of rows each)
qse_disk_cache_lookups_total	counter	prefills the disk tier could have shortened by a chunk or more
qse_disk_cache_hits_total	counter	prefills that started from the disk tier
qse_disk_cache_misses_total	counter	lookups the disk tier did not answer

continued

Metric	Type, labels	What it counts
qse_disk_cache_tokens_restored_total	counter	prompt tokens read back from the disk instead of prefilled
qse_disk_cache_read_bytes_total	counter	bytes a restore read
qse_disk_cache_written_bytes_total	counter	bytes the writer wrote
qse_disk_cache_writes_total	counter, kind	files the writer wrote, by kind
qse_disk_cache_evictions_total	counter, reason	entries the disk tier gave up, by reason (budget, floor, ttl, orphan)
qse_disk_cache_dropped_total	counter, reason	writes not done, by reason (window: the next prefill came first; budget, floor, staging, aborted, too_big)
qse_disk_cache_failed_restores_total	counter	restores that failed a check (checksum, layout) and took the cold path
qse_disk_cache_last_restore_seconds	gauge	the last restore's wall time
qse_cache_collisions_total	counter, cache	stored prefixes whose 64-bit hash matched and whose tokens did not
qse_prefill_tokens_reused_total	counter	prompt tokens restored from a snapshot instead of forwarded
qse_prefill_tokens_forwarded_total	counter	prompt tokens run through the 64 layers
qse_gpu_memory_used_bytes	gauge	allocator bytes held by the engine
qse_gpu_memory_reserved_bytes	gauge	bytes the allocator has taken from the driver and not given back
qse_unified_memory_free_bytes	gauge	free bytes in the board's one memory pool
qse_engine_uptime_seconds	gauge	seconds since the server started
qse_engine_info	gauge, version, model, drafter, width, tree, nvfp4, fp8_head, max_len, caches	always 1. The labels are the configuration the numbers above were produced by
qse_http_requests_total	counter, route, code	HTTP responses by route (a fixed set) and status code
qse_prompt_tokens_cached_total	counter	prompt tokens restored from the state store (all of them on a response-cache replay)
qse_reasoning_tokens_total	counter	committed tokens inside the reasoning block
qse_suffix_store_drafts_total	counter, outcome	proposed blocks for which the persistent suffix store matched the context (hit) or did not (miss); counted once a block, never once a lookup
qse_request_queue_seconds	histogram	arrival to the engine lock, per request
qse_request_prefill_seconds	histogram	the engine lock to the first token (template, tokenise, prefill), per request
qse_request_decode_tokens_per_second	histogram	per decoded request: (completion tokens - 1) / (last token - first token), the atlas row's convention, and the predicted_per_second a response carries
qse_request_prefill_tokens_per_second	histogram	per request with a forwarded prompt: forwarded tokens / prefill seconds
qse_request_prompt_tokens	histogram	prompt tokens per request, cached ones included
qse_request_completion_tokens	histogram	completion tokens per request

continued

Metric	Type, labels	What it counts
qse_suffix_store_tokens	gauge	token ids in the persistent suffix store
qse_state_store_budget_bytes	gauge	the state cache's byte budget
qse_process_resident_memory_bytes	gauge	resident set of the server process
qse_engine_start_time_seconds	gauge	when the server started, unix seconds
qse_log_subscribers	gauge	open /v1/dashboard/logs streams
qse_usage_ledger_rows_total	counter	rows the usage ledger has written
qse_usage_ledger_dropped_total	counter	rows the usage ledger lost: a full queue or a failed write
qse_build_info	gauge	always 1. version, the git commit, the code hash row3 records (tools/row3.py::code_hash) and a hash of the effective flags
qse_proxy_requests_total	counter, backend, kind, finish_reason	requests relayed to a backend, by its id, its kind and the finish reason
qse_proxy_tokens_total	counter, backend, type	tokens a backend reported in usage, by type (prompt, cached, completion, reasoning)
qse_proxy_cost_micros_total	counter, backend	cost at the model's price, micro-dollars
qse_upstream_retries_total	counter, backend, reason	tries again before any byte reached the client, by backend and reason (429, 502, 503, connect)
qse_proxy_overhead_ms	histogram	the proxy's own time per request: arrival to the upstream send, plus the first frame's delay through the proxy
qse_proxy_ttft_seconds	histogram	arrival at the proxy to the first token frame (the network and the backend's queue inside)
qse_backend_up	gauge, backend	1 when the backend's last health check passed
qse_backend_inflight	gauge, backend	requests relayed to the backend now
qse_backend_scrape_age_seconds	gauge, backend	seconds since the backend's own /metrics page was last read (vLLM, SGLang)
qse_backend_plugin_age_seconds	gauge, backend	seconds since the plugin inside the backend's engine last posted (vLLM, SGLang with tandemllm-*-plugin)
qse_backend_plugin_joins	gauge, backend, join	requests whose ledger row got the engine's own numbers, by how the plugin's item was matched (exact, approximate) or why it was not (unmatched, untagged)

30.2 The usage ledger

`server/ledger.py` writes one SQLite row per request: counts, timings, the finish reason, the cache source, a client id. Never the prompt, the answer, a tool name or argument, a conversation id, the `user` field, an IP address or a raw token. Refused requests (503, 429) and rejected ones (400) are rows too, with null token fields, so the request count is honest.

It never slows or fails a request. A row is put on a bounded queue (10,000), a writer thread commits in batches at most every second, and a disk error drops that batch, counts it and logs it at most once a minute. Rows older than `--usage-retention-days` (at least 400) are deleted once a day, and once an ISO week an online backup goes beside the file, the newest 8 kept. The server takes the

path from `--usage-ledger` only, never from the environment, so a benchmark server started with `serve.env` exported never writes the operator's ledger. Migrations are forward only and in place, and keep every row.

Table 30.2: The usage ledger's requests table (schema version 9)

Column	Type, since	What it holds
`ts_ms`	`INTEGER, v1`	when the request arrived, ms since the epoch
`request_id`	`TEXT, v1`	the response id
`model`	`TEXT, v1`	the model name the client sent
`client_id`	`TEXT, v1`	k: and 12 hex digits of the bearer token's sha256, or anon
`client_kind`	`TEXT, v1`	read from the User-Agent: open-webui, openai-sdk, opencode, hermes, curl, dashboard, other
`endpoint`	`TEXT, v1`	the route
`stream`	`INTEGER, v1`	1 for a streamed request
`status`	`INTEGER, v1`	the HTTP status
`finish_reason`	`TEXT, v1`	stop, length, tool_calls, timeout, error, abandoned
`prompt_tokens`	`INTEGER, v1`	prompt tokens as the client sent them
`cached_tokens`	`INTEGER, v1`	prompt tokens served from a cache
`completion_tokens`	`INTEGER, v1`	tokens streamed to the client
`reasoning_tokens`	`INTEGER, v1`	visible reasoning tokens
`queue_ms`	`REAL, v1`	time waiting for the engine
`prompt_ms`	`REAL, v1`	prefill time
`ttft_ms`	`REAL, v1`	time to the first token
`decode_ms`	`REAL, v1`	time after the first token
`total_ms`	`REAL, v1`	arrival to the end
`decode_tps`	`REAL, v1`	(completion tokens - 1) / decode time; null for a replay
`prefill_tps`	`REAL, v1`	forwarded prompt tokens a second
`blocks`	`INTEGER, v1`	verify rounds
`draft_tokens`	`INTEGER, v1`	tokens the drafters proposed
`draft_accepted`	`INTEGER, v1`	drafted tokens the model agreed with
`tool_calls`	`INTEGER, v1`	tool calls in the answer
`thinking`	`INTEGER, v1`	1 when the model thought
`cache_source`	`TEXT, v1`	where the prompt resumed: resident, end, store, disk, spill, continuation, prepared, response, none
`max_tokens`	`INTEGER, v1`	the output limit
`error_type`	`TEXT, v1`	the exception class or the refusal
`engine_version`	`TEXT, v1`	the VERSION file
`code_sha`	`TEXT, v1`	a hash over the code folders
`model_served`	`TEXT, v2`	TandemLLM with the conductor on
`profile`	`TEXT, v2`	the conductor profile
`profile_version`	`INTEGER, v2`	its version
`conductor_mode`	`TEXT, v2`	raw, shadow or on
`conductor_acts`	`INTEGER, v2`	the session monitor's executed acts
`conductor_hidden_tokens`	`INTEGER, v2`	tokens the model saw that the client did not send
`conductor_acted`	`INTEGER, v2`	1 when the conductor changed what the model saw or wrote

continued

Column	Type, since	What it holds
lane	TEXT, v2	decode, prefill or self for a side-lane request
backend	TEXT, v3	the backend id of a relayed request
backend_kind	TEXT, v3	its kind
backend_model	TEXT, v3	the id sent upstream
public_model	TEXT, v3	the id the client sent
usage_source	TEXT, v3	backend or none
cache_write_tokens	INTEGER, v3	tokens a provider charged as a cache write
cost_micros	INTEGER, v3	cost in micro-dollars at the model's price
proxy_ms	REAL, v3	the proxy's own time
retries	INTEGER, v3	tries again before the first byte
fallback_from	TEXT, v3	the backend tried first
upstream_status	INTEGER, v3	the backend's HTTP status
user	TEXT, v3	the label of the proxy key
preemptions	INTEGER, v4	preemptions (vLLM) or retractions (SGLang), from the plugin
engine_finish	TEXT, v4	the backend engine's own finish reason
corrupted	INTEGER, v4	vLLM's NaN check
l2_join	TEXT, v4	exact or approximate join of the plugin's numbers
wait_own_ms	REAL, v5	ms the request waited behind the engine's own work (the shadow reader, a summary written ahead, the conductor's question)
parent_request	TEXT, v6	the turn an internal row served (the router's question names the turn it routes)
slot	INTEGER, v7	the engine's sequence slot with --parallel N (0 the main slot, 1 and up a side slot); null without
engine_cached	INTEGER, v8	the prompt tokens the engine read from a cache, the hidden notes included (cached_tokens counts the client's among them)
forwarded_tokens	INTEGER, v8	what the prefill forwarded, hidden tokens included
wait_mem_ms	REAL, v8	ms the parallel memory admission held the request (inside queue_ms)
sampling	TEXT, v9	how the request sampled: null greedy, client (its own temperature), regenerate (TandemLLM sampled a regenerate)

The conductor's journal shares the file: conductor_events has 53 columns (id, ts_ms, request_id, session_key, kind, task, tier, mode, sp, rule, trigger, score, decision, action, cap, tier1_model, question_id, probabilities, tier1_ms, hidden_text_id, hidden_tokens, tokens_at_cut, position, outcome, next_checker, policy_version, client_kind, model, profile, profile_version, router_version, class_id, confidence, source, kept, stages, classifier_ms, effort, budget, budget_source, thinking, decisions, injected, replayed, shadow, digest, label, label_note, labelled_at, acted, changed, detail, extra), and conductor_texts keeps every hidden text once, by its hash.

A row's speed figures follow the benchmark row's formula, and a replay or a side-lane request has no decode rate. A relayed request has null queue, prefill, blocks, draft counts and cache source unless a plugin reported them, in which case `l2_join` says how the row was joined. A quick look:

```
sqlite3 ~/.qwen38-spark-engine/usage/ledger.sqlite3 \
  "select datetime(ts_ms/1000,'unixepoch','localtime'), finish_reason, prompt_tokens,
          completion_tokens, round(decode_tps,1), cache_source from requests order by id
    ↪ desc limit 10"
```

Chapter 31

Troubleshooting

Each entry is a symptom, what causes it, and what to do. The dashboard's Dev view shows the server log live, and `~/TandemLLM/bin/tandem logs` follows it on an installed engine.

31.1 Installing

The installer stops: no tag, branch or commit 'v0.8.3'. The repository it clones has no such tag. The installer clones the repository `--repo` names, the public one by default, at the tag `--ref` names, `v0.8.3` by default. To install from another repository, such as a fork, pass its address with `--repo`. The same missing tag makes `curl` answer 404 for the installer's own URL.

`install.sh --setup` refuses: the directory already has a configured engine. A box that switched to a profile counts as configured, and `--setup` would stop that engine. To update it, add `--force-setup`, then run `use-profile.sh` with the profile again; its files are already on disk.

`use-profile.sh` refuses the download: not enough disk. The fetcher keeps 60 GB free after every download, per filesystem, and counts the profile's weights and the files beside them first. `--dry-run` prints each file with its size. Free the space, or pick a smaller profile: `exl3-2.5` needs 12.0 GB of weights beside the 27B's shared 40 GB.

A download stopped halfway. Run the same `use-profile.sh` command again. It keeps every file that finished and checked, pins the same commit, and continues; a folder appears only when all its files are there.

The setup guide's community section says "no published list yet". GitHub answered 404 for `models/community-index.json` on the public repository's `main`: no release there carries the list yet. That is a note, not an error. The page shows the list from the box's last fetch, or the one that came with its release, and Refresh asks again (Section 7.2). A real failure shows one short line instead, such as "GitHub could not be reached", and the server's log has the URL.

31.2 The engine

The start refuses: an engine is already alive. One engine per board. Two engines loading side by side run a DGX Spark out of memory, and a board in that state stops answering until a power cycle. A loading engine counts although it answers nothing yet, because it binds its port only after the weights are in. `. ops/engines.sh; engine_pids` lists every engine process; stop the other one before starting. The same rule holds for a vLLM or SGLang on the same board: check the free memory first and keep at least 15 GiB of `MemAvailable` at the worst case.

A request gets a 503 with Retry-After, and the log names the peak. The memory gate (Section 14.10). Before every prefill the engine gives memory back cheapest first until the prompt's workspace fits under the declared peak (77 GiB); when it still does not fit, it refuses rather than wedge the board. The System view's memory plan shows what holds memory: a small model beside the engine holds its claim, and a 262,144-token context holds about 13 GiB more than a 131,072-token one. If the refusals go on after a role loaded, unload it in the Conductor view or lower `QWEN38_SHADOW_GB`.

A role does not load. The residency planner refused it: `refused_memory`, `refused_hold` or `refused_load_in_progress`. It loads one model at a time, never during a hold, and only when the board keeps its floor and margin (15 + 6 GiB) after the engine's growth to its peak and the role's claim. The Deciders screen and the System view say why, and `residency.jsonl` in the conductor's directory journals each decision. The narrator's keeper tries again in the next idle window.

The engine does not start: the image processor resolved to the pil backend. torchvision is missing or too old. Install the pair that matches torch (`pillow==11.3.0`, `torchvision==0.28.0+cu130` from the CUDA 13.0 index), or start with `--image-backend pil`, or `--vision off` for text only. The installer's venv has both.

A benchmark is slower than the paper, or its text differs. With the conductor on, every request runs through the router: a class, an effort cap and a hidden note. Send `X-Tandem: raw`, or the profile `bench`, for the raw path. Then check the lookup corpus: the installer builds a public one, and the paper's corpus is not published. And check the store view: a store that has seen the benchmark's prompts inflates it (Chapter 15).

A client asks for a model name the engine does not list. With the conductor on, `/v1/models` lists `TandemLLM` alone, and any model name is served as `TandemLLM`. Nothing to fix.

A turn waits a long time before its first token. With one sequence, the shipped profile, the engine serves one turn at a time. A streamed request behind a running turn gets an SSE comment every 10 s with its position and an estimate, and with the progress lines on a waiting line in the client; it waits as long as a request may run. A request without a stream gets a 429 after `--queue-timeout`. Short requests go through the side lane. A full queue (8) is a 503 at once.

The next turn re-reads the whole conversation. Look at the request's cache source in the Performance view or the Dev view's detail: `end`, `resident`, `prefix`, `session`, `disk`, `spill`, `continuation`, or `none` for a cold prefill. A client that rewrites its history (another system text, a changed earlier message) parts from the cached rows where it changed. A short conversation started while a longer one is resident is a guest of it and re-reads its first answer at its second turn. Chapter 13 lists each case. With `--parallel N`, a turn that ran on a side slot (the request row's `slot` 1 or more, the `[parallel]` log line) reads its prompt from the start: side slots keep no conversation. A turn goes there only while another client's request holds slot 0. A new user message in Open WebUI after the agent changed directory used to part at the tool block (its working directory is in `run_command`'s description); under a conductor plan it no longer does (`QWEN38_CLIENT_CWD`). If the turn's source is `none` right after short requests of TandemLLM's own (`json` rows of the router), check `refused_stash` in `/health`'s resident report and `cache.memory` against the ceiling.

An agent turn ends with no answer and no tool call. The model wrote its tool call inside its thinking and stopped there. With the session monitor acting and E1 on in the request's profile (a profile stored before this release may have it off: set it to act on the Monitor screen), the call goes out as the answer when it parses and names an offered tool, under the line *tool call written in the thinking, sending it*; otherwise the turn is continued, and after two tries it ends with one sentence. On the raw path the engine returns what the model wrote.

A long conversation shows X-Tandem-Images-Left-Out. A request carries at most 16 images and 65,536 image rows. When the excess is in earlier messages, the oldest images are left out in steps of 8 and the request is served; only the newest message's own images over a cap are a 400.

The dashboard says an admin password is set, and edits fail with 401. `QSE_ADMIN_TOKEN` is set in `secrets.env`, and the dashboard has no sign-in in this release. Remove the line (keep a copy of the file) and restart; API clients can keep sending the token as a bearer header.

The dashboard says the engine is stopped for maintenance or starting. The engine is away and the reverse proxy shows the status responder's page (Section 6.7): the reason, since when, about how long. It opens the dashboard by itself when `/health` answers.

A request answers 400 and `param` names a field. The request's shape is wrong: no user message, an unknown role, a late system message, `max_tokens` below 1, an empty prompt (Section 14.1).

Open WebUI shows a literal `</think>` while an answer streams. Open WebUI 0.11.3 shows the stream piece that carries `</think>` as message text until the round ends. Under a conductor plan the engine sends Open WebUI its reasoning in `reasoning_content`, which has no tag; a request with `reasoning_format: tags`, the raw path, or `QWEN38_CLIENT_REASONING` set empty bring the tags back. The saved chat is clean either way.

The model writes "Laya unavailable". Its scripts could not reach the engine from where they run. Set `QWEN38_TANDEM_URL` to an address that machine reaches.

31.3 The Chat tab

The Chat tab asks before every bash command. The chat runs in Ask mode. The pill in the composer's options row names the mode; pick Auto there, and calls that stay inside the chat's sandbox run without a prompt from the next call on. The default for new chats is at the top of the Tools settings page. A chat from before 0.8.2 has no mode of its own and runs in the default (Section 25.3).

Auto still asks. The call leaves the sandbox: `web_fetch` to a domain this chat has not fetched yet, an MCP tool its server does not mark read-only, an imported Open WebUI tool that imports a network or process module, or `rm` aimed at `/`, `/workspace` or a home directory. A tool set to "ask" in the tool settings asks in every mode, and a deny rule refuses in every mode, YOLO included.

"Always allow" does not stop the prompt. Before 0.8.2 it saved only the first word of a chained command, which never covered the whole command, so one chat saved `cd` 11 times. On 0.8.2 it saves a rule for every part and names them in the prompt. A command with `$(...)` or backticks still matches no rule and offers no "Always"; Auto runs it without asking when it stays in the sandbox.

31.4 vLLM and SGLang behind the proxy

vLLM answers 400 to every request with tools. vLLM refuses tools without a tool parser. Start it with `--enable-auto-tool-choice --tool-call-parser <parser>` (`qwen3_coder` for the Qwen3.5 and Qwen3.8 models). The memory writer sends the session's tools with `tool_choice: "none"`, so it needs the parser too.

SGLang returns tool calls as text. Start it with `--tool-call-parser <parser>`. SGLang does not hold the model to `tool_choice: "none"`; the memory writer cuts any call it writes.

The SGLang card says the metrics page is off. SGLang answers `/metrics` with 404 unless it runs with `--enable-metrics`, and the plugin's hooks need it too.

Cached tokens read "not reported by this backend". vLLM needs `--enable-prompt-tokens-details`, SGLang `--enable-cache-report`. A generic server fills `prompt_tokens_details.cached_tokens` or not; the proxy never estimates.

Reasoning tokens are null, or the reasoning shows inside the answer. Start vLLM or SGLang with `--reasoning-parser <parser>` (`qwen3` for Qwen).

The card says "L1 only". No batch from the plugin for 60 s. In the engine's own Python, check that the plugin is installed, that `TANDEM_COLLECT_URL` names this proxy and this backend's id and is reachable from the engine's machine, and that `TANDEM_COLLECT_TOKEN` is the proxy's token. The engine's log has one line when the plugin fails to load; the engine starts either way. `tandem-dash launch ... --dry-run` prints the variables it would set.

Rows say l2_join: approximate. The proxy could not tag the request (the Responses API, SGLang with `n` above 1) and matched it by time and token counts. A client that sends its own `X-Request-Id` (vLLM) or `rid` (SGLang) keeps it, and the join uses it.

31.5 The proxy

401: a proxy API key is needed. Once one user key exists, every request from another machine needs `Authorization: Bearer tdm-...`. Make a key under Backends, Users, or set `TANDEM_API_KEYS`.

A save in the Backends view answers 422 and names a variable. A `TANDEM_*` variable sets that field, and the environment wins. Change the variable, or remove it and restart.

A request fails with key_unreadable. The master key that encrypted the backend's key does not answer: `TANDEM_SECRETS_KEY` changed, the keychain entry or `secrets.key` is gone, or the database was copied without its key. `tandem-dash secrets status` says which source answers now. Type the backend's key in again.

The proxy refuses to start and names a variable. An unknown `TANDEM_*` variable, an unknown backend id, or a value the schema refuses. `tandem-dash check --env-file dash.env` prints the same message without starting.

404 model_not_found. No rule, prefix or backend serves the id. When two backends serve one id, use `<backend>/<model>`, give one a public id, or add a routing rule.

The cost column is empty. The model has no price. Set one on the model's row, or with `TANDEM_PRICE_<n>`.

The System view shows no processes for a backend. The host setting is `exporter`, the process belongs to another user, or the proxy runs in docker without `--pid=host`. For a remote machine, run `tandem-dash hostcollect` there with a token.

31.6 Memory notes

The model never writes a note. Every shipped profile has `memory: off`; set it to `shadow` or `on`. The background writer has its own switch, `settings.memory.notes.writer.mode`. The log says why a due session was not written: `hold`, `gpu`, `nothing`, `moved` or `no session`.

Behind the proxy the writer never runs on a cloud model. `auto` is off on a paid cloud model. Switch it on for that backend, give the model a price, and check the day's caps on the backend's page.

A note was refused. Caps refuse, never cut; a body that is mostly a secret is refused; a rule about the user needs the user's own message in the turn. The Notes tab lists the refusals with their reasons.

Chapter 32

Questions and answers

Does speculation change the text? Under greedy decoding the engine writes what plain decoding on the same weights writes, up to the first near tie of at most one ulp between the two best tokens, where the batched verify can round the other way. The paper measured 17 of 30 outputs identical to the end of the answer and 13 that diverged at such a near tie. Chapter 12 states it exactly.

Does the conductor change the text? It changes what the model reads (a hidden note, a thinking cap, a hidden close) and so what it writes, by design; the arithmetic is the same greedy decoding. `X-Tandem: raw` gives the release's raw path. Even there, two runs of one request can part at a near tie, because the length router sizes blocks from timings (Section 12.3).

Does it serve several requests at once? Yes. With `--parallel N` the engine decodes up to N requests in the same rounds: two at once gave 1.38× the total of one, four 1.42×, in single measured runs (Section 14.13). The shipped profile keeps one sequence, and the headline numbers in this manual are single-request numbers. With one sequence a short request still goes through the side lane, and the proxy serves other backends in parallel either way.

Will it run on another GPU? Best effort. The installer turns off the skinny kernel that is built for the GB10 only, and StairCut's prices were measured on the Spark, so the router starts from the wrong staircase and corrects its level online. Nothing about another GPU's speed is measured.

Why NVFP4 weights but 16-bit arithmetic? On this board the FP4 tensor-core path still has to move operands through shared memory, and a 4-bit activation path would need its own quantiser and gate to save at most the 8 % of traffic the activations are. NVFP4 saves bytes, and bytes are time on a memory-bound decode (Chapter 10).

Why are my speeds below the paper's? Check four things: the raw path (`X-Tandem: raw`), the lookup corpus (the paper's is not published), the store view (a store with the benchmark's text inflates it, a clean one does not), and the board's lock (another engine on the board halves the bandwidth).

Can I use the dashboard without the engine? Yes. `pip install '.[host]'` and `tandem-dash` run the dashboard and the proxy on any machine with Python 3.11, in front of vLLM, SGLang, llama.cpp, LM Studio, Ollama, OpenAI, DeepSeek or another TandemLLM (Chapter 4).

Does the proxy add latency? 0.33 ms to the time to first token at the median against a fake backend on one machine, and a decode rate within 0.8 % of the engine's own over the local network (measured, the 0.4.0 series).

Are my prompts stored anywhere? No log line, ledger row or metric holds a prompt or an answer. The suffix store holds token ids, never text. The conductor's transcripts keep a session's tool turns under the engine's home, mode 0600, secrets redacted, 30 days; memory notes are files you can read, edit and delete. Probe snapshots, off by default, hold activations, never text.

Is the dashboard safe to expose? No. It is meant for a trusted local network. No admin password is set by default, so anyone who reaches the port can read and change the configuration and the memory notes. This release has no sign-in screen: with a password set, the dashboard's edits and its Memory screen answer 401. If other people can reach the machine, keep the port on loopback and put a reverse proxy with its own sign-in, or a VPN, in front (Section 3.8).

Where are the docs this book draws on? In `docs/` of the repository, one page per part, under CC BY 4.0. This book adds the diagrams, the charts and the generated reference.

Chapter 33

Roadmap

Today the engine serves two main models, one at a time (Qwen3.8-27B and Qwen3.8-Flash-Next), with small models beside them, it is measured on one board (the DGX Spark, Section 1.1), and it decodes up to four requests at once. Each step below widens one of those limits, and each keeps the two rules that hold today: the output stays the model's greedy text, up to the documented near-tie exception, and no measured number gets worse. The steps are listed in the order they are planned. Nothing here is a promise of dates, and nothing here is measured unless it says so.

33.1 A mixture-of-experts model, and what is left

Qwen3.8-Flash-Next is served (Chapter 17), from turboderp's EXL3 packs and not from the NVFP4 checkpoint this chapter once planned. Three parts are open. The 5.05 pack codes its experts at 5 and 7 bits, and the MoE kernels take 2 to 6 and 8. No pack's quality against the BF16 model is measured, because the 360 GB release does not fit the board. And parallel requests are on with three slots, but three together take as long as one after the other; CUDA graphs for the joint pass are next. After it, other EXL3 models that fit one board, Qwen3.6-35B-A3B among them.

33.2 Parallel requests

Phase A ships, off in the 27B's shipped profile and on with three slots in Flash-Next's (Section 14.13): side slots, the GPU turn, a packed draft and a joint verify of at most 32 rows, admission by bytes. Two requests at once reach about 1.4× the total of one, not 2×: a joint pass of 32 rows costs more than one of 16 at today's kernels. Next: one engine thread for all slots, joint verify graphs and one graph pool, so a side slot costs under 3 GiB instead of 7. Later, multi-user serving with caches isolated per user, so no user can resume from another user's prefix; the proxy already gives each user a key of their own.

33.3 The NVMe tier

The disk tier shipped in 0.4.0, and its restore was measured on the board: a 10,701-token head read back in 226 ms, and after a restart the first process's files restored bit for bit to the second process's cold prefill (measured). What remains is a background backfill of the resident prompt's chunks while the engine is idle.

33.4 Profiles as data

Today a serving profile is a `serve.env` file. Part of this shipped: weight profiles that `ops/use-profile.sh` switches one engine at a time with a rollback (0.5.0), the model registry and the setup guide's estimate (0.7.0), and a fetch of every file a profile reads, each checked against the hub's sha256 (0.8.1). The plan is a small profile format that a validator checks: every weight file with its format and hash, all the settings, and a facts block with what was measured (memory and disk, speed at prompt lengths from 256 to 64k tokens, quality against the reference), each number labelled with how it was obtained. Still open: the format and its validator, and a signed catalogue of tested profiles in the repository that the setup guide installs from.

33.5 Other GPUs, then several

Other single GPUs first (H100, H200, RTX Pro 6000, L40S). Most have no FP4 tensor cores, so they need to decode the 4-bit weights in software or run an FP8 or BF16 profile. Each needs its own tile tables, and the router needs new prices: a GPU that reads memory ten times faster moves the best tree width and drafter size. Discrete GPUs also change the memory model, since admission must count the GPU's own memory and not the system's. Then several GPUs: tensor parallel inside one server, two DGX Sparks over their 200 Gb/s link, servers with 8 GPUs. A reduction across GPUs changes the order of a floating-point sum, so row invariance needs a fixed reduction order across devices.

33.6 Housekeeping

Runtime names still carry the old project name, from the `QWEN38_` and `QSE_` variable prefixes to the metric names. They change together in one release, and the old variable names keep working for one release after that. Engine settings move from import-time globals to a settings object handed to the engine, so two engines with different settings can share a process. The engine moves into an installable package, and the dashboard's package goes to PyPI.

Chapter 34

Changelog summary

`CHANGELOG.md` in the repository lists every release with its measurements. This chapter summarises it. Every release is held to the one before it on the raw path by the identity gate (the same logits and state, bit for bit, on a fixed scenario), except where an entry says otherwise, and each one's speed is checked on the benchmark row against the release before it, in the same hour. The gate fixes the arithmetic, not the tree shapes the length router picks from timings, so the served text can part at a near tie between two runs of either release (Section 12.3).

34.1 0.8.3, repository hygiene

Repository hygiene for the public release: no private hostnames, home paths or personal notes in the files; no code change. The engine's modules are the bytes of 0.8.2, and the text does not change. Comments, docs and tests no longer name a person who gave an instruction, and tests use `192.168.1.x` for a LAN address and `tandem.example.lan` for a host behind a proxy. `install.sh` installs v0.8.3 by default.

34.2 0.8.2, community weights and permission modes

0.8.2 adds community weight packs, permission modes in the Chat tab and standalone EXL3 packs of the 27B family, and fixes two bugs found on v0.8.0 in Open WebUI on a phone. The engine's modules are the bytes of 0.8.1.

Community weights. One JSON file per pack under `models/<model>/community/`, checked by the registry against `models/schema/community.schema.json` and gathered in `models/community-index.json`. The setup guide lists the packs after our profiles as `community/<id>`, with search and five sort orders. It fetches the index from the public repository when its weights step opens or on Refresh, never on a timer, and falls back to its last copy, then to the one in the release; a 404 shows as the note "no published list yet". `ops/use-profile.sh community/<id>` downloads a pack at its pinned commit, checks the hub's sha256 and runs it with our drafters. An issue form and a check on pull requests come with it. The first entry is ThinkingCap Qwen3.8-27B Kearuga by 0xWhiteMage: EXL3 at 4.0 bits a weight, standalone, 39.6 tok/s prose, 95.9 code and 51.3 chat on 0.8.2 (measured). Chapter 7.

Table 34.1: Releases of TandemLLM.

release	date	what it brought
0.1.0-rc1 to rc4	2026-09-18 to 09-25	the engine, the length router, release of the losing drafter, serving defaults for long work; rc4 is the first release measured in two store views
0.1.0-rc5 to rc7	09-26 to 09-27	sampled requests on the draft tree, the in-process block A/B, tool calls read in three forms and streamed as argument deltas
0.1.0-rc8 to rc10	09-27 to 09-28	the resident prefix, live activity on the dashboard, the FP8 and balanced profiles, the first seams for other models
`v0.2.0-staircut`	09-29	StairCut, on in the served profile: the paper's release
`v0.2.1`	09-29	the dashboard login off by default; no decode path changed
`v0.3.0`	09-30	image input; the text path unchanged
0.4.0	10-06	the conductor, memory notes, the caches for long agent sessions, parallel requests (off by default), and the dashboard as a standalone proxy in front of other engines and cloud APIs; not published, 0.4.7 replaces it
`v0.4.7`	10-08	0.4.0 plus the package library, Regenerate, long Open WebUI chats back from the NVMe, the compressed session spill and a daily NVMe write budget
`v0.4.8`	10-08	two Open WebUI chats beside each other (a side slot keeps its conversation), a chat whose retrieval block moved resumes at its end, the park keeps the end anchor, the router's notes stay live by their message
0.5.0	10-09	weight profiles: NVFP4 or EXL3 at 2.5 to 6.0 bits a weight, one at a time, switched with `ops/use-profile.sh`
0.5.1	10-10	two dashboard numbers corrected; four parallel requests on the reference box
0.6.0	10-10	one shared KV pool that maps pages as a conversation grows; a request that does not fit waits instead of failing
0.6.1	10-10	the small models give memory back when the pool runs short; the memory split counts committed pages
0.6.2	10-10	decoding no longer stops while another slot prefills; the Perf tab's live feed fixed
0.7.0	10-11	the setup guide at `#/setup` with Wren, its assistant; reset levels; setup mode
0.7.1	10-11	Wren drawn as the DGX Spark box; dashboard only
0.7.2	10-11	Wren answers a question in words again; setup guide only
0.8.0	10-13	a second model, Qwen3.8-Flash-Next, from turboderp's EXL3 packs; the Chat tab with chatd; the cache peek and prewarm
0.8.1	10-13	a new box downloads every file a weight profile needs, with no token; the EXL3 weights public; the engine unchanged
0.8.2	10-13	community weight packs; permission modes in the Chat tab; standalone EXL3 packs of the 27B family; a chat no longer waits minutes for slot 0 while a side slot is free; the model says it is TandemLLM when asked
0.8.3	10-13	repository hygiene for the public release; no code change

Permission modes. Each chat in the Chat tab runs in one of three modes, and Auto is the new default: a call that stays inside the chat's sandbox runs without a prompt, and one that leaves it still asks. Deny rules and plan mode's read-only list hold in every mode. A tool card that ran without a prompt carries a badge, and every such call is logged as `tool.auto_approved`. "Always allow" on a chained bash command now saves a rule for every part not yet allowed; before, one chat saved `cd` 11 times and still got the prompt. Chapter 25.

Standalone EXL3 packs. A complete BF16-origin `mul1` EXL3 pack of the 27B family loads as `--model`, with no separate base checkpoint (pull request #1 by 0xWhiteMage). The path is text only, and speculative parallel serving still needs our drafters (Section 9.8). The 27B's own profiles take the path they took before. `engine/loader.py`'s new bytes count as the old ones in the disk tier's fingerprint (`SAME_ROWS` in `engine/methods/manifest.py`), so the first 27B start adopts v0.8.0's saved files once, and the 27B's code fingerprint stays 0.7.2's.

The slot-0 wait. A new Open WebUI chat waited 437 s for slot 0 behind another chat's 496 s generation while both side slots were free. It shared only Open WebUI's system prompt and tool list with an entry on the NVMe, and the wait for slot 0 had no limit. Now the spill wait needs the request's own conversation, and every wait for slot 0 ends after 3 s (`QWEN38_SIDE_SLOT0_WAIT_S`) when a side slot the request fits in is free. `/health` `parallel.max_running` is the limit with its reason, not the high-water mark, and the Lanes view says why a request is queued. On the reference box a turn that slot 0's chat had held moved to slot 2 after its 3.0 s wait, while another chat ran 80 s on slot 0 (measured).

Identity. Asked who it was, Flash-Next named another assistant and its maker: the reference box served it with the conductor off, so the identity note never went in. The note now says the model is TandemLLM running the served model (Qwen3.8-Flash-Next or Qwen3.8-27B, from the pack's model card) on the user's own machine, for every client the conductor serves, and the reference box runs Flash-Next with the conductor on. With Open WebUI's own system prompt and 49 tools, 20 of 20 answers said so, against 0 of 20 on v0.8.0 (measured).

Smaller fixes. The Flash-Next profiles load Laya, so the memory features work there: the remember, forget and recall intent took 20.6 ms on 3.05. A deleted chat's sandbox stops at once; a restore brings the chat back, and the purge after 7 days or on `?forever=1` removes the container and the chat's folders. A 404 from the community sync is a note, not an error.

34.3 0.8.1, a new box fetches every file

A new DGX Spark runs every 27B EXL3 profile and Flash-Next from public files, with no Hugging Face token (OPS-95). `ops/use-profile.sh` downloads what a 27B profile reads besides its weights before it switches: the K3 drafters and the K4 draft head, the BF16 drafters into `train/`, and the FP8 base checkpoint into the Hugging Face cache. It pins each download to the hub's commit, checks each weight file's sha256, checks the disk first, and continues an interrupted run; `--dry-run` prints every URL and size. The EXL3 profiles read the BF16 drafters from `train/` on every box. A switch

from setup mode removes the marker first and puts it back if the profile does not start. `install.sh` installs v0.8.1 by default, and `--repo` names the repository it clones. The engine's modules are the bytes of 0.8.0. Section 3.3 has the steps for a new box.

Public weights. The six EXL3 sizes of the 27B and their drafters are public on Hugging Face: `0xBakeer/TandemLLM-Qwen3.8-27B-EXL3-<bpw>` for 2.5, 3.0, 3.5, 4.0, 4.5 and 6.0 bits a weight, and `0xBakeer/TandemLLM-Qwen3.8-27B-EXL3-drafters`, in one collection. Each card has the size's KL against BF16 and a speed table; KL runs from 0.1112 at 2.5 bits to 0.0018 at 6.0 (Table 9.2). The calibration repository stays private, and nothing that runs reads it.

34.4 0.8.0, Qwen3.8-Flash-Next and the Chat tab

A second model, Qwen3.8-Flash-Next, served from turboderp's EXL3 packs through the same server, caches, ops scripts and setup guide as the 27B (Chapter 17). The 27B's text does not change: its modules, flags and disk-tier fingerprint are as in 0.7.2, and every switch added for Flash-Next is off for it. On Flash-Next, greedy and seeded answers with speculation equal plain decoding token for token.

The Chat tab. A chat in the dashboard backed by chatd, its own service behind the engine's `/v1/chat-app`: runs that outlive the browser, tools in a sandbox per chat, the workspace panes, MCP servers, plan mode, the PWA. The Context panel reads the usage frame's split by part and the cache peek; a chat on the NVMe is read back while the engine is idle (`/v1/cache/prewarm`). Found by the live checks and fixed: a restart's restore that faulted the GPU on Flash-Next, the side lane on Flash-Next, its missing model card, a side slot's chat that came back cold after a restart, a peek that answered 409 right after a turn, a prewarm that held a request for its whole read (it now stops between blocks), a chat waiting for a permission prompt that held one of chatd's two run slots, and an iPhone test, where five screenshots counted by their base64 filled the window (319,136 tokens estimated against 16,980 real). The Chat tab carries a Beta pill in this release. On a phone the bottom bar has four slots with the rest behind More, and the composer shows every control.

Chat tab (beta): known limitations. The Chat tab is in beta in this release, and a Beta pill beside it says so. We know of these limits and have not fixed them yet:

- The agent loop does not yet work the way Claude Code does (epic VIS-E10). Its prompts are not kept prefix-stable from step to step (SRV-375). Old tool results and old thinking stay in the context until a compaction (SRV-365). A compaction writes a general summary, not the task's state (SRV-373). Every sandbox command asks for permission; there is no mode that runs them without asking (SRV-387). Nothing notices when the model repeats the same failing approach with different calls (SRV-367). `render_page` passes on Chromium's console noise (SRV-369). Long tool output has no "see more" paging (SRV-371). Searches run in the main context, with no sub-agents (SRV-381).
- The engine reads a short Flash-Next chat again on its next turn when the answer ended inside a verified block (ENG-235).

- A request that arrives during a prewarm still waits 0.5 to 1 s (SRV-349).
- When memory reaches its ceiling, the engine drops the parked chat without writing it to the NVMe, so its next turn starts cold.
- Images only: there is no video input (VIS-97).
- The 5.05 Flash-Next pack does not load yet.
- Parallel decoding does not raise throughput over one request at a time: three requests together took 27.18 s against 27.03 s one after another, though the later ones start sooner.
- The phone layout's keyboard behaviour is not verified in Safari on an iPhone.

Profiles and windows. Five weight profiles, `flashnext-2.05` to `flashnext-6.05`, and a second tile in the setup guide. Each pack has its largest window in the registry: 262,144 tokens for 2.05 and 3.05, 208,896 for 4.05 with the vision tower on; 5.05 does not load yet (its experts use 5 and 7 bits a weight), and 6.05 fits no window on one DGX Spark. 3.05 reads its n-gram table cold. A start with a longer `--max-len` is refused before the load.

Speculation. The pack's own MTP layer drafts, cut by the shared StairCut, with the lookup in the same tree. StairCut keeps its prices, the plain step's cost and a drafted round's cost per context class, takes the plain step where drafting does not pay, and captures missing graphs while the server is idle. One greedy request a length and its repeat, second round on a fresh server: 56.6 and 57.3 tok/s at 1k against 41.0 and 45.4 plain, 42.1 and 52.4 at 140k against 33.9 and 41.4; at 260,800 tokens the class takes plain turns (measured).

Caches and slots. Every cache tier holds Flash-Next's state on a 4,096-token grid. The KV pool, side slots and a joint verify pass work on it without changing any answer, and its profiles turn them on with three slots. Three requests arriving together take as long as one after the other (27.18 against 27.03 s), and the later first tokens come at 2.2 to 6.1 s instead of up to about 20 s (measured). The mixers are int8 with a scale a group of 128 weights: KL 0.0031 on code and 0.0188 on prose against the fp16 mixers.

Images and saved chats. Flash-Next reads images: the pack's vision tower runs on the EXL3 kernels, and its profiles turn it on (VIS-97). An image chat answers the same with the MTP drafter and without it, and the text after 140 image rows stays at KL 0.019 against transformers (measured). The disk tier now knows its files by their content (the `fp4` fingerprint), not by their folder. Saved chats of 0.7.2 carry over in place, or from the old folder that `QSE_DISK_ADOPT_FROM` names; on the reference box Flash-Next kept its 34 saved chats through the switch.

34.5 What 0.7 added

0.7.0: the setup guide. The dashboard's `#/setup` walks through an install in eight steps, with a memory bar that compares the selection's estimate with the box's free memory. One formula in `server/setup/estimate.py` gives the numbers, and the page's copy must pass the same cases. The models come from `models/<id>/model.yaml`. Wren, Qwen3.5-2B on the engine's own forward, answers from the docs, the manual and the registry and can fill fields, but cannot apply, reset, load or unload anything. Finish runs an install job of fixed steps that brings back the old profile when its check fails; reset has five levels; `install.sh --setup` starts the dashboard and Wren alone on a box with no engine yet. Every POST under `/v1/setup` must be same-origin JSON. The text does not change: with Wren loaded on the narrator's copy, three greedy prose requests gave the same 411 tokens as without it, at 148.6 against 147.5 tok/s (measured). The KV figure is corrected to 104.25 KiB a token.

0.7.1 and 0.7.2. Wren is drawn as the DGX Spark box, with eight states and a still face for reduced motion. It answers a question in words again: a tool call may name only what the page can highlight, an invalid change gets one repair, and the last round sends no tools. On ten change requests and five questions it was right 15 times of 15 (measured). Neither release touches the engine's modules.

34.6 What 0.6 added

0.6.0: one shared KV pool. Each slot reserves addresses for up to 262,144 tokens and maps 2 MiB pages as rows are written (`QSE_KV_POOL=on`); pages past what a slot keeps go back when its request ends. Four slots of 262,144 tokens fit beside Laya and the narrator, with 58 to 63 GiB of MemAvailable after warm-up against 26 with the old 32k side windows, and the 4-request load test went from 28.4 to 31.5 tok/s on 2 slots to 34.2 to 35.6 on 4 (measured). A request that does not fit waits, says so in its stream and gets a 503 `memory_wait_timeout` only after the queue timeout. The logits, every KV row and the recurrent state match the static buffers bit for bit.

0.6.1 and 0.6.2. When the pool runs short, the caches give memory back first, then the shadow reader, the narrator, idle side conversations and slot 0's idle conversation; the models come back through the residency planner once memory stays free. An EXL3 prefill offers the GPU between its layers when another slot waits: in a test of three streamed chats beside a 37k-token prefill, the seconds without a decoded token went from 42 of 147 to 0 of 145 (measured). The dashboard's memory split counts committed pages, and its Perf tab reads the 0.6 live feed again.

34.7 What 0.5 added

0.5.0: weight profiles. The engine serves the NVFP4 set or an EXL3 quant of Qwen3.8-27B from 2.5 to 6.0 bits a weight, one at a time, through `ops/use-profile.sh`, which fetches missing weights, starts the profile, runs a smoke request and brings the old profile back when it fails. EXL3 linear layers have one kernel per bit width from 2 to 8 for 1 to 32 verify rows, and the trellis decode matches exllamav3 bit for bit; the drafters run in EXL3 at 3 bits. Against BF16 on 16,376 held-out

positions, NVFP4 has a KL of 0.0582 and agrees on the top token 93.35 % of the time, EXL3 4.5 0.0217 and 95.78 % (measured). An EXL3 profile serves other weights, so its text differs from NVFP4's; speculation stays exact against whichever weights run.

0.5.1. The dashboard's cache hit counts only the client's own prompt tokens, and the live prefill rate only the rows the engine computed. The reference box runs four parallel requests.

34.8 What 0.4.8 adds

0.4.8 is 0.4.7 plus the 0.4.8-test builds of 2026-10-08; `CHANGELOG.md` has them with their live checks.

Parallel requests. A side slot takes a request whose window leaves it at least 16,384 rows of output, with `max_tokens` cut to the room (0.4.7 compared the window with the whole served limit, so a chat past about 24,000 tokens never took the side slot and two chats took turns on slot 0). A side slot has a resident prefix of its own: the next turn of the conversation it holds resumes at the end anchor there, beside slot 0's conversation; the router's question in between leaves it in place. A new conversation beside one a side slot could never hold takes the idle side slot first, and a short one waits up to 20 s for a busy side slot before it sends the long conversation to the NVMe.

Caches. The end alignment's third pass serves a block the client moved from an earlier user message to its last one (Open WebUI's retrieval block) from the rows where the model saw it, token-aligned on the bare bytes and only when the same bytes are in the last message; a message the user edited is served as sent. The router's notes stay live by their message when the prompt changed before them. The RAM park keeps the end anchor, so a parked Open WebUI chat comes back at its end. A long prompt the end anchor does not serve says where the copy parted and what was refused, in the log and the journal.

Speed. The speed guard's short gate on the 0.4.8 engine code: 50.64 against release 0.3.0's 49.75 tok/s (measured, one run a side, store off).

34.9 What 0.4.7 added

0.4.7 is 0.4.0 plus the test builds 0.4.1-test to 0.4.7-test; `CHANGELOG.md` has each one with its live checks.

Packages. The system set (`tandem-core`, `render-tools`, `eval-judge`), four library packages (`creative-coding`, `charts`, `voxel-worlds`, `explainer-video`) and 34 everyday packages with `everyday-core` and `reply-format`. Every prose class offers the whole everyday set; the package question runs once per class (`package_sticky: class`) on an idle side slot. Measured against no package, 91 pairs judged by the served model: a mean of +1.19 of 5, indicative only; five everyday packages ship off. Everyday answers draft first and never stop on a form.

Requests. Regenerate samples (temperature 0.7, top-p 0.95, top-k 20, a fresh seed; `X-Tandem-Regenerate`); every other answer stays greedy, and the ledger's `sampling` column (schema 9) says which. Continue Response continues the partial answer. A memory note the store refuses is never shown, and a project note without a working directory lands at user scope.

Caches. A long Open WebUI chat comes back from the NVMe after other chats ran (a 48k-token chat in 1.0 s, TTFT 2.3 s). The session spill is compressed without loss (1.71× on a real entry's rows), the spill budget is 64 GB and the disk tier 16 GB, and the NVMe writes at most 200 GB a day. A conversation that ran on a side slot is carried into slot 0. Open WebUI chats own their NVMe entries, and a deploy that only registers a knob keeps the disk's files.

Speed. The speed guard's short gate on the 0.4.7 engine code: 49.93 against release 0.3.0's 50.38 tok/s (measured, one run a side, store off).

34.10 What 0.4.0 brought

The conductor. The engine serves one model, `TandemLLM`, with a conductor in front (Part IV): profiles; a router that decides each new user message's task class with a decision model and then Qwen itself; effort levels that are real caps on the reasoning; hidden notes with packages and skills, replayed at the same position; thinking budgets in every shape clients send and an answer reserve; side requests; the side lane; waiting streams with an estimate; status lines and the voice; the session monitor with its loop guards, tool checks and claim check; the tool-argument loop guard; the shadow reader; memory notes; session transcripts, turn snapshots and outcomes; session addresses. `X-Tandem: raw` turns all of it off for a request. Every client is the one user whose memory notes the engine keeps; `X-Tandem-Memory: off` keeps a request out of them. The testing packages name the engine as `{tandem_url}` and keep scratch files in `{tandem_scratch}`, out of the deliverable, put the deliverable before any test tooling and stop a test that fails the same way twice, and check a page with the kit's own headless Chrome (`laya_drive.py page`) instead of a driver of the model's own. The turn budget (S5) ends an agent turn that loops on its own test, sets up before it writes, or keeps a chat waiting. Asked who it is, the model names its model and TandemLLM. A prose request is not routed to a code class on a weak guess. A class's core skill goes in on every turn of the class, and an unsure deliverable is built as picked and stated in one line instead of asked. Each Open WebUI chat is its own session (its chat id as a connection header, or its first turn without one), and a new folder never replaces the user's files (the `files-care` package).

Requests. A malformed request is a 400 that names the field; `developer` is the system prompt; `usage.prompt_tokens` counts what the client sent and the conductor's tokens are `prompt_tokens_details.hidden_tokens`; a JSON answer has its thinking in `reasoning_content`, and so has every client but the Hermes agent under a conductor plan (no literal `</think>` in Open WebUI's live view, none in opencode or an SDK's answer); stop strings apply to the answer only; a request without tools never gets `tool_calls`; `cached_tokens` counts the client's tokens below the point where the request resumed. A class's packages fit the request's tools, a reply-only instruction is chat, and

the follow-up of an answer that thought resumes at its end. While the engine is away, a status responder answers the reverse proxy's fallback with a page and a 503 that say what is going on.

Parallel requests. `--parallel N`, off by default: side slots beside the engine, a packed draft and a joint verify of at most 32 rows for the decoding requests, admission by bytes; two requests at once 1.38× the total of one, four 1.42× (Section 14.13); a turn that continues slot 0's conversation, judged on the client's own tokens, waits for slot 0 and resumes in place, so an agent session keeps its cache with parallel requests on; the side slots' bytes stay in the memory limits, and the router's short questions and a client's title tasks take an idle side slot, so a new user message resumes the conversation in place; Open WebUI's live working directory moves from the tool block into a note. A request that shares only its client's head with slot 0's conversation takes the side slot, and a side slot reads the disk tier.

Caches. The end anchor; the parked session; the engine's memory limits and the memory gate; the disk tier on the NVMe; live conversations on the NVMe through a restart or an eviction; compaction as a continuation with a summary written ahead; images left out of long conversations in steps; a canonical skills list for opencode; a tokenised-prompt cache.

The dashboard. The Conductor view with all its screens, the Parallel page, the hosts and the memory plan in the System view, the Backends and Setup views, filters by backend and user, and a phone layout for every view. TandemLLM's own calls are left out of Usage unless asked, and the Live panel's cached tokens never read above the prompt. It asks for no password: the admin routes are open until an admin password is set, so the engine belongs on a trusted network (Section 3.8).

The proxy. `tandem-dash`: the dashboard in front of any OpenAI-compatible engine or cloud API, six backend kinds, routing by model id, fallbacks, one config database overlaid by `TANDEM_*` variables, encrypted keys, per-user keys, cost at each model's price; vLLM's and SGLang's own metrics, the plugins inside them and `tandem-dash launch`; the host collector; the conductor in front of any backend, with the memory writer in the backend's idle time.

The raw path runs 0.3.0's arithmetic: every build of the series passed the identity gate against the one before it (`tools/flagoff_identity.py`, a fixed scenario of fixed shapes, logits and state bit for bit). With the same tree shapes it writes 0.3.0's text, with one exception: the raw path no longer closes a reasoning block that repeats itself (0.3.0's stall close, on by default); under a conductor plan the session monitor's reasoning-loop task (D1) does that job. The length router sizes blocks from timings, so two runs of either version can part at a near tie (Section 12.3). The last benchmark row recorded in the series, with `--no-learn-cost` and 50 requests, read 50.09 tok/s (measured); the paper reports 49.89. The release's speed check, 0.3.0 and 0.4.0 on the same box in the same hour with one run each, read 50.50 and 51.38 tok/s (measured, `release/speed-guard.json`). The upgrade notes are in Chapter 3.

Not verified in this release. OpenAI's adapter with a paid call (it is tested on frames written from OpenAI's documented shapes); the conductor's second requests on a real vLLM or SGLang (tested on a fake backend); the plugins' check against running servers; the per-process GPU memory on a discrete GPU; the docker image (the wheel it installs is built and tested).

34.11 Measured in the 0.4.0 series

Table 34.2 lists the measurements this manual cites from the series, all on the published weights on one DGX Spark.

Table 34.2: Measurements from the 0.4.0 series that this manual cites. All measured, on the published weights on one DGX Spark.

what	value
side lane: 10 short requests fired into a 5,000-token turn	9 served at 1.2 s median; turn +7.4 s; the turn's state comes back bit for bit, its text only while the tree shapes match
compaction of a 57,008-token request	115.5 s as sent, 40.7 s as a continuation, 0.07 s prepared
session spill: a 41,587-token conversation	written in 1.03 s (5.20 GB), read back in 0.94 s
disk tier: a 15,360-token head	restored in 334 ms; TTFT 23.1 s down to 2.7 s
the voice, Qwen3.5-2B	decode −1.90 % (63.47 to 62.26 tok/s)
the shadow reader, Qwen3.5-0.8B, 16,350-token prompt	decode −2.05 % (36.02 to 35.28 tok/s)
memory notes on and off	99.7 against 99.6 ms a block
the proxy against a fake backend	+0.33 ms to TTFT at the median
the host collector, one sample with a process scan	7.46 ms of CPU

Glossary

adapter The proxy's code for one backend kind: how it relays a route, where it reads cached and reasoning tokens, how it switches thinking. `proxy/adapters/`.

anchor A copy of the 48 recurrent states and their convolution tails at one position, 147 MiB. The resident prefix keeps one at every 1,024-token line of a prefill; the disk tier keeps them as files. A request resumes at the largest anchor its prompt shares.

backend An engine or a cloud API behind the proxy: another TandemLLM, vLLM, SGLang, any OpenAI-compatible server, OpenAI or DeepSeek. Each has a kind, a URL, models, an optional key, and switches.

benchmark row `serve-single-i256-o256-v1`: one request at a time, 256 prompt tokens, 256 generated, thinking off, greedy, 50 requests after 3 warm-ups. Every speed in this book that says "the benchmark row" is this.

block drafter The drafter that proposes a whole block of tokens in one pass: DFlash2, five decoder layers reading five of the target's hidden states, fine-tuned at block 8 and block 16.

board The DGX Spark the engine runs on: one GB10 GPU and 128 GB of memory shared by the CPU and the GPU. "One engine per board" is the first rule of operations.

cache source Where a request's prompt resumed: `resident`, `end`, `prefix`, `session`, `disk`, `spill`, `continuation`, `prepared` or `none`. In the ledger and on the dashboard.

claim A role's declared memory peak, which the residency planner checks before a load.

class The kind of task the router decides a turn is (`code_edit`, `chat` and eight more). It sets the effort, the packages and the rules.

commit The step that keeps the accepted path's KV rows and recurrent state after a verify. For the recurrent layers it is one low-rank update per layer, folded into the next verify.

compaction A client's request to summarise a long conversation. The engine serves it as a continuation of the conversation it holds, or from a summary written ahead.

conductor The harness in front of the model: profiles, the router, packages, the session monitor, status lines, memory notes. It makes the model `TandemLLM`.

decider A decision model that answers a typed question in one forward pass, such as Laya; the router's decider stage, off in the shipped router, asks one.

draft tree The guesses of one round, merged so that a shared prefix is a shared node, stored in depth-first pre-order, at most 32 nodes.

drafter A cheap model or procedure that guesses the next tokens for the target to verify: the block drafter and the lookup drafter.

effort A turn's thinking level: off, low, medium, high or xhigh, with a reasoning cap of 0, 1,024, 8,192, 16,384 or none below the backstop.

end anchor An anchor at the end of the last generation, so the next turn that sends the answer back resumes there instead of re-reading it.

feature group One switchable part of the harness in front of a backend: class, effort, packages, side, loops, e1, tools, compaction, states, memory.

grid The 1,024-token lines of a prefill. Anchors, chunk files and prefix snapshots are taken on the grid, and a resumed request forwards the rest of its prompt on the same grid, so its state equals a cold prefill's bit for bit.

guest A short request between two turns of a longer resident conversation; the rows it can reach are stashed and put back. Also the short request the side lane serves inside a running turn, which gets its engine state back bit for bit.

harness The layer around the model that plans each turn and watches it run; in TandemLLM it is the conductor.

hold Taking the board for a measurement or a second model: `ops/hold.sh` stops the service, runs the command, and restores the service.

host collector The code that reads a machine's memory, CPU, GPUs, disks and the process behind each backend's port; `tandem-dash hostcollect` on a remote machine.

journal The conductor's record of every decision, acted on or not, in the usage ledger's file, kept 30 days.

L1, L2 The two layers of engine figures behind the proxy: L1 is the engine's `/metrics` page, L2 the plugin's per-request numbers.

lookup drafter The drafter that copies text: the longest match of the last tokens in the request, a corpus and the suffix store.

memory gate The check before every prefill that gives caches back, cheapest first, until the prompt's workspace fits under the declared peak, else a 503.

memory note One fact the model keeps as a file, written from a block in its answer or by the background writer, read back in later turns.

narrator The small model that writes status lines: rewritten templates, thought summaries, or the voice.

near tie A position where the two best tokens score within one unit in the last place, where a batched verify and a one-token step may choose differently.

note The router's hidden text at the end of a turn's last message: the effort sentence, rules, packages. Replayed at the same position on later turns.

outcome The structural reading of a tool-result turn: progress, no visible change, regression, error, stuck and others.

package A text the router may put into a note: a task skill, a test skill, a card, a coding guide, or a library skill.

park Copying another session's rows aside when a request of a new session replaces the resident prompt, so that session's next turn swaps them back.

pin Copying the head of the resident prompt aside before a long unrelated request cuts it.

plugin A small package inside vLLM or SGLang that posts each request's own times to the proxy.

profile Two things. A conductor profile is a document that sets what the harness does for a client (`coding`, `chat` and others). A serving profile is a `serve.env` file that sets the weights.

proxy The router of requests by model id to backends, with one ledger and one dashboard; in the engine's server with `--backends`, or standalone as `tandem-dash`.

raw The engine without the conductor: the release's raw path. `X-Tandem: raw` or the `raw` and `bench` profiles.

residency planner The code that admits a role's load only when the board keeps its floor after the engine's growth to its peak.

resident prefix The last prompt's rows, left in the engine's own buffers, with anchors, an end anchor, a stash, a pin and a park.

role Anything beside the big model the conductor asks, with a contract: its kind, claim, window, latency budget, fallback and what it may write.

round One draft, verify, accept and commit: about 93 ms and 4.58 tokens on the benchmark row.

router Two things. The conductor's router decides a turn's class and plan. The speculation router (StairCut, the length router) decides each round's draft tree.

safe point One of the four moments the session monitor may act: before the prefill, after a block, at the end token, at the held tool calls.

session resume Continuing a conversation from state that verify rounds wrote (the end anchor, the session spill) rather than prefill chunks. It is held to the greedy gate (the same tokens), not to bit equality; the *taint* row marks where it begins.

session monitor The conductor's watch over a running turn: loop guards, the empty-turn continue, tool-call checks, the claim check, session tasks.

session spill Whole live conversations written to the NVMe at a planned stop or before an eviction, and read back.

shadow A mode in which a part decides and journals what it would do and changes nothing.

shadow reader A small model that keeps its own exact state of each session, read in idle time, and answers typed questions over the whole session.

side lane Serving a short request at a safe point inside the running generation, which then gets its engine state back bit for bit. Its text equals an uninterrupted run's only while the tree shapes are the same (Section 12.3).

side request A client's title or summary call, known by its shape and served small.

spill See session spill.

StairCut The router that cuts each round's draft tree to the node count with the most expected tokens per millisecond on the measured verify-cost staircase.

state store Whole state snapshots, for the session cache and the prefix cache.

status line One short line in the client for a step the conductor takes, stripped from whatever the client sends back.

suffix store An append-only log of the token ids the engine read and wrote, with a suffix array, for the lookup drafter.

taint The lowest row that a session resume wrote. Below it the caches are a cold prefill's bit for bit.

time to first token The time from a request's arrival to its first streamed token: the prefill, plus any wait in the queue. TTFT in the tables.

ulp A unit in the last place: one rounding step of BF16 at a value.

verify One forward pass of the target model over the whole draft tree, reading the model's own token at every node.

voice The narrator's live lines: facts from the engine and a short checked phrase from the small model.

writer The background turn that writes a session's facts as memory notes at its end or after a compaction.

Index

A

adapter, 227
admin password, 18
admission, 87
AES-256-GCM, 32
AGPL-3.0-only, iii
anchor, 9, 75, 227
`--answer-reserve`, 84
auth, 187

B

backend, 227
backend kind, 27
`--backends`, 8, 26, 137, 145, 188
Backends view, 145
benchmark row, 94, 227
block drafter, 60, 227
board, 227
byte budget, 46

C

cache exactness, 71
cache source, 227
cache tiers, 74
CC BY 4.0, iii
Chat tab, 151
chat template, 84
chatd, 151
claim, 227
claim check, 126
clip search, 50
commit, 60, 227
community weights, 41
    sync, 41
compaction, 80, 227
`--compaction`, 81
conductor, 111, 227
conductor documents, 112, 185
Conductor view, 145
`--conductor-config`, 21, 137
context length, 63
cost, 33
cron, 37
CUDA graphs, 58

D

dashboard, 24
dashboard contract, 147
dashboard login, 18
decider, 120, 227
decode loop, 47
DeepSeek, 31
deploy, 39
DFlash2, 60
DGX Spark, 3
disk tier, 78
`--disk-cache`, 21, 81
docker, 25
draft tree, 64, 228
drafter, 3, 60, 228
drain, 37
`--drop-idle`, 64

E

effort, 115, 228
end anchor, 9, 77, 228
`--end-anchor`, 77
`engine/cache.py`, 88
environment overlay, 28
errors, 197
EXL3, 53
    standalone pack, 53
    the 27B, 52
EXL3 pack, 53, 104

F
fallback, 31
feature group, 137, 228
Flash-Next, 103
  exactness, 72
  measurement, 97
  memory notes, 136
  operations, 35
  serving, 92
  state, 74
FP8, 3, 49

G
Gated DeltaNet, 45, 57
GB10, 3
GPTQ, 50
Grafana, 39
greedy agreement, 5, 68
grid, 75, 228
guest, 76, 228
Gumbel-max trick, 71

H
harness, 228
headers, 196
Hermes agent, 8
hold, 37, 228
host collector, 157, 228
Hugging Face, 20, 52

I
images, 86
install.sh, 15

J
journal, 117, 228

K
keychain, 32
keys at rest, 32
KV cache, *see* resident prefix, state store
KV rows, 46

L
L1, L2, 228
Laya, 2, 120
Laya API, 121
length router, 63
library, 119
license, iii
Live panel, 141
llama.cpp, 30
LM Studio, 30
logs, 38
lookup drafter, 61, 228
lookup store, *see* suffix store
loop guard, 124

M
master key, 32
`--max-image-rows`, 87
`--max-images`, 87
measured at the proxy, 33
`--mem-ceiling-gib`, 87
`--mem-peak-gib`, 87
memory gate, 87, 228
memory note, 133, 228
memory rules, 34
memory writer, 135, 230
metrics, 200
model adapter, 99
ModelState, 99
monitoring, 39
MTP drafter, 66

N
n-gram drafter, *see* lookup drafter
narrator, 2, 129, 228
near tie, 69, 229
never worse, 5, 95
`--no-learn-cost`, 63, 95, 130, 225
note, 115, 229
NVFP4, 3, 49

O
Ollama, 30
one engine per board, 14, 34
Open WebUI, 8
OpenAI, 30
OpenAI API, 82
OpenAI SDK, 8
opencode, 8
`ops/gate.sh`, 96, 102
`ops/hold.sh`, 37

ops/serve.env, 15, 36, 59, 62, 160, 173
ops/start.sh, 21, 34, 113, 160, 173
ops/supervisor.yaml, 123
outcome, 229

P
package, 9, 118, 229
Packages screen, 119
page cache, 34
--parallel, 90
parallel requests, 90
Parallel view, 154
park, 77, 229
permission modes, 152
pin, 76, 229
plugin, 157, 229
policy, 123
prefill, 73
--prefix-chunk, 81
privacy, 93
profile, 9, 112, 229
profile (serve.env), 22
Prometheus, 39, 200
proxy, 2, 24, 229

Q
QSE variables, 181
QSE_ADMIN_TOKEN, 187
QSE_BACKENDS, 20, 26, 183
QSE_CONDUCTOR_CONFIG, 21, 183
QSE_DASHBOARD_LOGIN, 183
QSE_DISK_CACHE, 21, 183
QSE_USAGE_LEDGER, 21, 183
quality gate, 51
quantiser, 50
queue, *see* admission
--queue-timeout, 89
Qwen3.8-27B, 45
QWEN38 variables, 176
QWEN38_DECIDER_MODELS, 120
QWEN38_DEEP, 62
QWEN38_NARRATOR_MODEL, 130
QWEN38_SHADOW_MODEL, 130
QWEN38_STAIR_RHO, 62
QWEN38_STAIR_TABLES, 62

R
raw, 229
reading line, 114
reasoning budget, 66
reasoning format, 83
--reasoning-format, 83
recurrent state, 46
release gate, 96
release identity, 70
replay, 115
--request-timeout, 89
requirements, 13
--reserve-slack-gib, 87
residency planner, 20, 38, 229
resident prefix, 9, 75, 229
--resident-gb, 76
--resident-guest-max, 81
--resident-park-gb, 81
--resident-pin-gb, 81
--response-cache, 71
Responses API, 30
role, 38, 120, 229
rollback, 39
round, 60, 229
router, 113, 229
routing by model id, 31
row invariance, 56
run-to-run text, 69

S
safe point, 123, 229
sampling, 66
sandbox, 151
seed, 71
self-check, 105
serve.env, 173
server flags, 160
server/app.py, 82
session address, 116
session monitor, 10, 123, 229
session resume, 71, 229
session spill, 79, 230
--session-spill, 81
settings registry, 39
setup mode, 14
Setup screen, 24

setups, 12
SGLang, 29
shadow mode, 230
shadow reader, 2, 130, 230
side lane, 88, 230
side request, 116, 230
`--side-graphs`, 91
sign-in, 18
skinny kernel, 56
spill, 230
SSE comment, 89
staircase, 62
StairCut, 4, 62, 230
    Flash-Next, 66
StairCut paper, 6
start.sh, 36
state store, 78, 230
status line, 128, 230
status page, 38
stop.sh, 36
store view, 94
streaming, 85
structured outputs, 84
suffix store, 61, 80, 230
System view, 147
systemd, 37

T

taint, 230
TANDEM variables, 183
tandem-dash, 24, 169
`tandem-dash hostcollect`, 26
tandem-dash launch, 30
`tandem-dash launch`, 30
`TANDEM_API_KEYS`, 32
`TANDEM_COLLECT_TOKEN`, 157
TandemLLM, 2
TandemLLM (model name), 9, 111
`TANDEM_SECRETS_KEY`, 32
task class, 113, 227
teacher-forced bench, 96
test environment, 3
thinking budget, 83
time to first token, 73, 230
tokens per round, 47
tool calls, 84
tool-argument loop guard, 92
tool-call checks, 124
`tools/block_ab.py`, 95
`tools/forced_bench.py`, 96
`tools/row3.py`, 95
`tools/verify_spec.py`, 69, 96, 102
tree verify, 64
`--trust-loopback`, 187
turn outcome, 126

U

ulp, 69, 230
uninstall, 22
upgrade, 15, 22
usage ledger, 203
Usage view, 144
`--usage-ledger`, 21, 204
`use-profile.sh`, 14
users, 32

V

verify, 60, 230
vision tower, 86
vLLM, 29
voice, 129, 230

W

W4A16, 50
waiting line, 89
watchdog, 37
weight profile, 35

X

`X-OpenWebUI-Chat-Id`, 116
`x-session-id`, 116
`X-Tandem`, 22, 111, 123
X-Tandem header, 18
`X-Tandem-Profile`, 22, 111

Z

Zenodo, 6